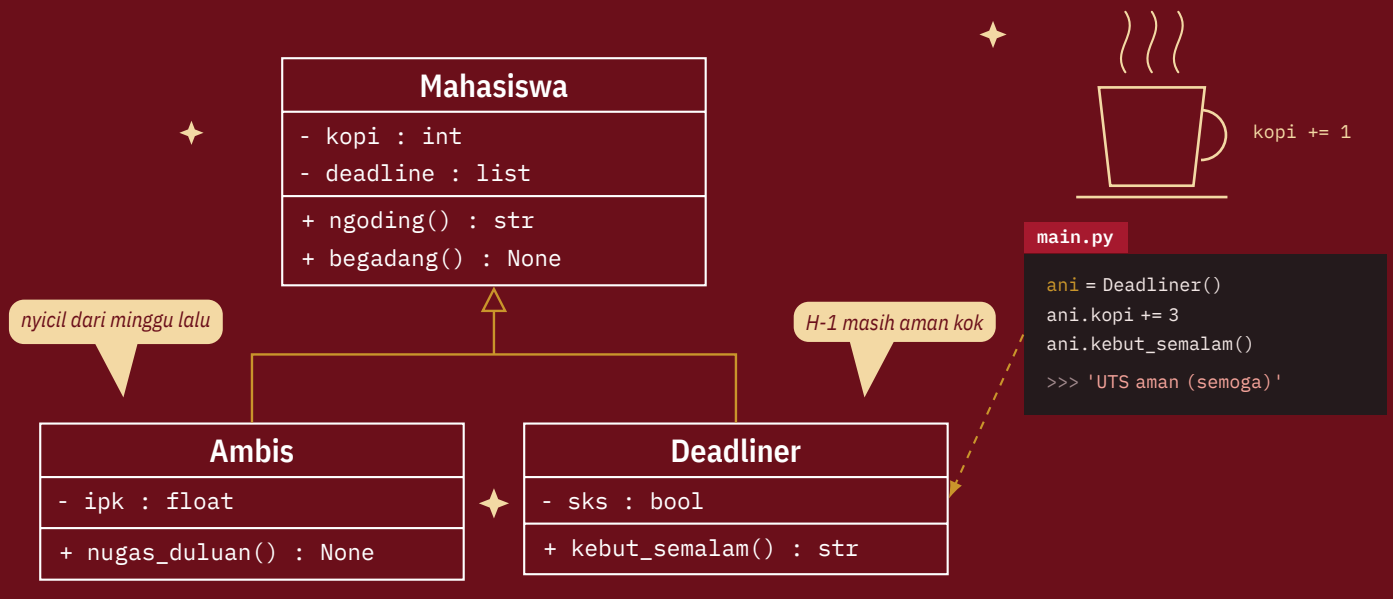
Riskyana Dewi Instan Puspitasari, M. Kom
Muhammad Fariq Faqih
Mohammad Ariffansyah
Abdullah Al-Firdaus Nuzula

BUKU AJAR

# Pemrograman Berorientasi Objek dengan Python

*Konsep Dasar, Prinsip Desain, dan Implementasi*




2026

# Kata Pengantar

Buku ini disusun sebagai bahan ajar pendamping mata kuliah Pemrograman Berorientasi Objek. Seluruh contoh program menggunakan bahasa Python dan dirancang untuk membantu pembaca memahami serta mempraktikkan konsep pemrograman berorientasi objek.

Materi disajikan secara bertahap, mulai dari pengenalan paradigma berorientasi objek, class dan object, encapsulation, decorator, inheritance, abstraksi, polymorphism, dan relasi antarkelas. Pembahasan dilanjutkan dengan modular programming, exception handling, file handling, serta pembuatan antarmuka grafis (Graphical User Interface atau GUI). Pada bagian akhir, tersedia panduan pengembangan proyek aplikasi yang mengintegrasikan berbagai konsep yang telah dipelajari.

Setiap bagian dilengkapi dengan penjelasan konsep, contoh kode beserta hasil eksekusinya, studi kasus, dan latihan. Buku ini diharapkan dapat membantu pembaca memahami setiap konsep secara bertahap, melihat penerapannya dalam pemrograman, serta mengembangkan keterampilan dalam merancang dan membangun aplikasi menggunakan prinsip-prinsip pemrograman berorientasi objek.

Penyusun menyadari bahwa modul ini masih memiliki kekurangan. Oleh karena itu, saran dan masukan dari pembaca diharapkan dapat menjadi bahan evaluasi dan penyempurnaan untuk edisi berikutnya.

2026

**Penyusun**

# Daftar Isi

**Kata Pengantar** . . . . . ii

**1 Konsep dan Pengantar OOP** . . . . . 1

1.1 Konsep Class . . . . . 2

1.2 Konsep Object/Instance . . . . . 4

1.3 Struktur Class di Python . . . . . 6

1.4 Mengakses Attribute dan Method . . . . . 8

1.5 Konsep Attribute . . . . . 12

1.6 Instance Attribute dan Class Attribute . . . . . 13

1.7 Konsep Method . . . . . 18

1.8 Instance Method . . . . . 20

1.9 Membuat Object . . . . . 23

1.10 Pemodelan Objek dari Permasalahan Sederhana . . . . . 26

1.11 Rangkuman . . . . . 30

1.12 Kuis Teoritis . . . . . 30

1.13 Praktik Mandiri . . . . . 33

**2 Visibility dan Encapsulation . . . . . 35**

2.1 Visibility Property . . . . . 36

2.2 Konsep Encapsulation . . . . . 37

2.3 Public, Protected, dan Private secara Konseptual . . . . . 39

2.4 Name Mangling . . . . . 40

2.5 Pengendalian Akses Attribute . . . . . 43

2.6 Validasi Data melalui Property . . . . . 45

2.7 Rangkuman . . . . . 51

2.8 Kuis Teoritis . . . . . 51

2.9 Praktik Mandiri . . . . . 54

**3 Class Method, Static Method, dan Decorator . . . . . 56**

3.1 Class Method . . . . . 57

3.2 Static Method . . . . . 59

3.3 Konsep Decorator . . . . . 62

3.4 Custom Decorator . . . . . 65

3.5 Chaining Decorator . . . . . 67

3.6 `*args` dan `**kwargs` pada Decorator . . . . . 69

3.7 Rangkuman . . . . . 74

3.8 Kuis Teoritis . . . . . 75

3.9 Praktik Mandiri . . . . . 78

**4 Inheritance . . . . . 81**

4.1 Konsep Inheritance . . . . . 82

4.2 Parent/Base Class . . . . . 84

4.3 Child/Derived Class . . . . . 85

4.4 Konsep Overriding . . . . . 88

4.5 Nested Inheritance . . . . . 91

4.6 Multiple Inheritance . . . . . 93

4.7 Rangkuman . . . . . 96

4.8 Kuis Teoritis . . . . . 96

4.9 Praktik Mandiri . . . 99

**5 Abstract Method, Data Class, dan Typing . . . 102**

5.1 Abstract Method . . . 103

5.2 Data Class . . . 107

5.3 Attribute Mutability . . . 111

5.4 Duck Typing . . . 113

5.5 Structural Typing . . . 116

5.6 Rangkuman . . . 119

5.7 Kuis Teoritis . . . 120

5.8 Praktik Mandiri . . . 122

**6 Polymorphism dan Class Relationship** . . . . . **125**

6.1 Konsep Polymorphism . . . . . 126

6.2 Method Overriding . . . . . 127

6.3 Method Overloading secara Konseptual . . . . . 129

6.4 Association . . . . . 133

6.5 Aggregation . . . . . 134

6.6 Composition . . . . . 135

6.7 Class Relationship . . . . . 137

6.8 Pemilihan Relationship Berdasarkan Kebutuhan Sistem . . . . . 139

6.9 Rangkuman . . . . . 142

6.10 Kuis Teoritis . . . . . 143

6.11 Praktik Mandiri . . . . . 145

**7 Modular Programming . . . . . 148**

7.1 Konsep Modular Programming . . . . . 149

7.2 Module . . . . . 150

7.3 Import . . . . . 154

7.4 Package . . . . . 156

7.5 Struktur Folder Project . . . . . 159

7.6 Pemisahan Class ke dalam Module . . . . . 160

7.7 Reusability . . . . . 165

7.8 Maintainability . . . . . 167

7.9 Rangkuman . . . . . 168

7.10 Kuis Teoritis . . . 168

7.11 Praktik Mandiri . . . 171

**8 Exception Handling . . . 174**

8.1 Error dan Exception . . . 175

8.2 Jenis Exception . . . 177

8.3 `try–except` . . . 178

8.4 Multiple Exception . . . 182

8.5 `else`, `finally`, dan `raise` . . . 183

8.6 Custom Exception . . . 186

8.7 Exception Handling dalam Class . . . 189

8.8 Rangkuman . . . 192

8.9 Kuis Teoritis . . . 193

8.10 Praktik Mandiri . . . . . 195

**9 File Handling . . . . . 198**

9.1 Konsep File Handling . . . . . 199

9.2 Read dan Write File dengan `open()` . . . . . 200

9.3 Context Manager . . . . . 204

9.4 File CSV . . . . . 206

9.5 File JSON . . . . . 209

9.6 Serialisasi Data Sederhana . . . . . 211

9.7 Integrasi File Handling dengan Object . . . . . 214

9.8 Rangkuman . . . . . 219

9.9 Kuis Teoritis . . . . . 220

9.10 Praktik Mandiri . . . . . 222

**10 Dasar GUI** . . . . . **225**

10.1 Prinsip Dasar GUI . . . . . 226

10.2 Komponen GUI . . . . . 229

10.3 Event Handling . . . . . 233

10.4 Integrasi GUI dengan Class OOP . . . . . 238

10.5 GUI Berbasis Web dengan Streamlit . . . . . 243

10.6 Rangkuman . . . . . 247

10.7 Kuis Teoritis . . . . . 247

10.8 Praktik Mandiri . . . . . 250

**11 Analisis dan Perancangan Aplikasi** . . . . . **252**

11.1 Analisis Kebutuhan Aplikasi . . . . . 253

11.2 Identifikasi Object . . . . . 257

11.3 Identifikasi Class, Attribute, dan Method . . . . . 260

11.4 Class Relationship . . . . . 264

11.5 Class Diagram Sederhana . . . . . 266

11.6 Rangkuman . . . . . 270

11.7 Kuis Teoritis . . . . . 271

11.8 Praktik Mandiri . . . . . 273

**12 Implementasi Aplikasi** . . . . . **275**

12.1 Implementasi Class . . . . . 277

12.2 Encapsulation . . . . . 282

12.3 Inheritance/Composition sesuai Rancangan . . . . . 284

12.4 Polymorphism bila Diperlukan . . . . . 286

12.5 Module/Package . . . . . 287

12.6 File Handling . . . . . 288

12.7 Integrasi Antar-Class . . . . . 291

12.8 Rangkuman . . . . . 301

12.9 Kuis Teoritis . . . . . 302

12.10 Praktik Mandiri . . . . . 304

**13 Implementasi GUI** . . . . . **306**

13.1 Implementasi GUI . . . . . 307

13.2 Komponen GUI . . . . . 307

13.3 Event Handling . . . . . 307

13.4 Integrasi GUI dengan Class OOP . . . . . 307

**14 Penutup** . . . . . **308**

14.1 Rangkuman Materi . . . . . 309

14.2 Langkah Selanjutnya . . . . . 309

**Daftar Pustaka** . . . . . **310**

Bab 01

# Konsep dan Pengantar OOP

*Mengenal class dan object sebagai bahan dasar program berorientasi objek.*

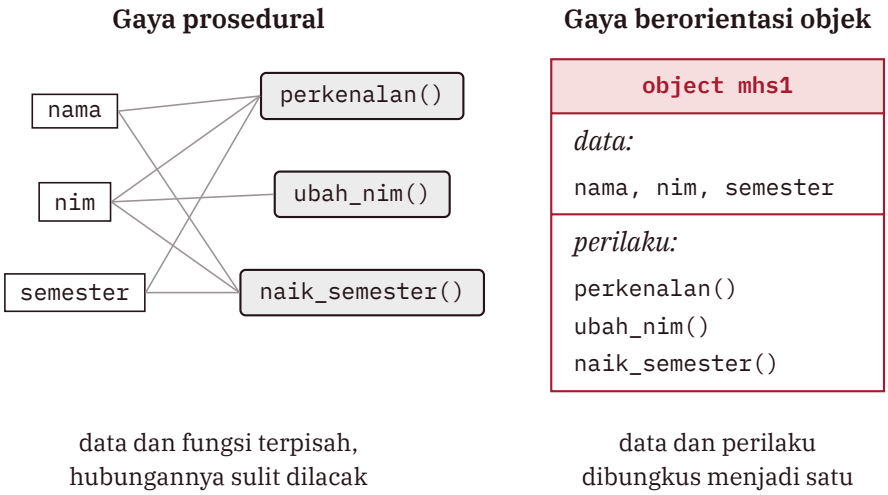
01

**DI BAB INI**

- Class dan object
- Struktur class di Python
- Attribute dan method
- Pemodelan objek sederhana

Selama ini kita banyak menulis program dengan gaya *prosedural*: data disimpan dalam variabel, lalu diolah oleh fungsi-fungsi yang terpisah dari data tersebut. Gaya ini cukup untuk program kecil, tetapi mulai merepotkan ketika program membesar. Data dan fungsi yang mengolahnya tercecer di banyak tempat, sehingga sulit dilacak siapa mengubah apa.

Pemrograman Berorientasi Objek (*Object-Oriented Programming*, OOP) menawarkan cara lain: data dan perilaku yang saling berkaitan dibungkus menjadi satu kesatuan yang disebut *object*. Cara berpikir ini dekat dengan dunia nyata. Seorang mahasiswa punya nama dan NIM (data), dan bisa mengisi KRS atau mengumpulkan tugas (perilaku). Dalam OOP, keduanya kita satukan. Perbedaan kedua gaya ini diperlihatkan pada Gambar 1.1.




**Gambar 1.1.** Gaya prosedural memisahkan data dan fungsi, sedangkan OOP menyatukan keduanya di dalam object.

Bab ini membahas dua konsep paling dasar dalam OOP, yaitu *class* dan *object*, beserta isi keduanya: *attribute* dan *method*. Di akhir bab, kita akan berlatih mengubah sebuah permasalahan sederhana menjadi rancangan class lalu menuliskannya dalam Python.

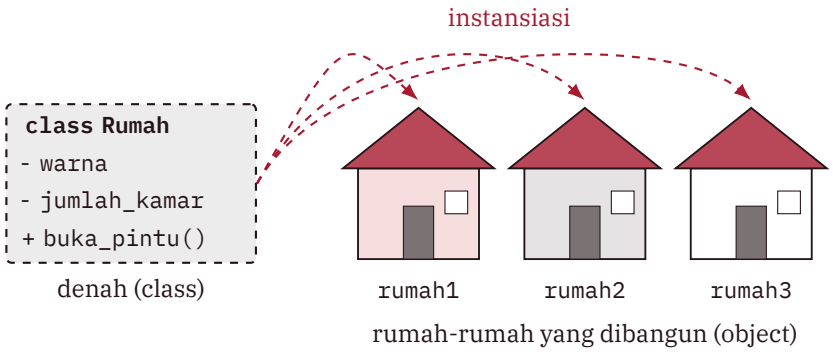
## 1.1 Konsep Class

*Class* adalah cetak biru (*blueprint*) atau templat untuk membuat object. Class mendefinisikan dua hal:

- **data apa** yang dimiliki setiap object (disebut *attribute*), dan
- **apa yang bisa dilakukan** oleh setiap object (disebut *method*).

Bayangkan denah rumah yang dibuat arsitek. Denah itu sendiri bukan rumah; kita

tidak bisa tinggal di dalamnya. Namun dari satu denah, pengembang bisa membangun puluhan rumah dengan bentuk yang sama. Class berperan seperti denah tersebut, sedangkan rumah-rumah yang dibangun adalah object-nya (Gambar 1.2).




**Gambar 1.2.** Class ibarat denah: satu denah dipakai untuk membangun banyak rumah (object). Setiap rumah berdiri sendiri dan boleh memiliki warna yang berbeda.

**ANALOGI CLASS**

- Cetakan kue → class; kue yang dihasilkan → object.
- Formulir pendaftaran kosong → class; formulir yang sudah diisi tiap pendaftar → object.
- Spesifikasi "Mobil" dari pabrik → class; mobil dengan nomor polisi L 1234 AB → object.

Di Python, class dibuat dengan kata kunci `class` diikuti nama class dan tanda titik dua. Class paling sederhana bahkan boleh tidak berisi apa pun; kita cukup menuliskan `pass` sebagai penanda badan class yang kosong.

**Class paling sederhana**

```
class Mahasiswa:
    pass

print(Mahasiswa)
```

**Keluaran**

```
<class '__main__.Mahasiswa'>
```

Keluaran tersebut menunjukkan bahwa `Mahasiswa` sudah dikenali Python sebagai sebuah class yang didefinisikan di modul utama (`__main__`).

**ATURAN PENAMAAN**

Menurut panduan gaya Python (PEP 8), nama class ditulis dengan gaya *PascalCase*: setiap kata diawali huruf kapital tanpa garis bawah, misalnya `Mahasiswa`, `RekeningBank`, atau `MataKuliah`.

Gunakan kata benda tunggal, karena satu class menggambarkan satu jenis benda.

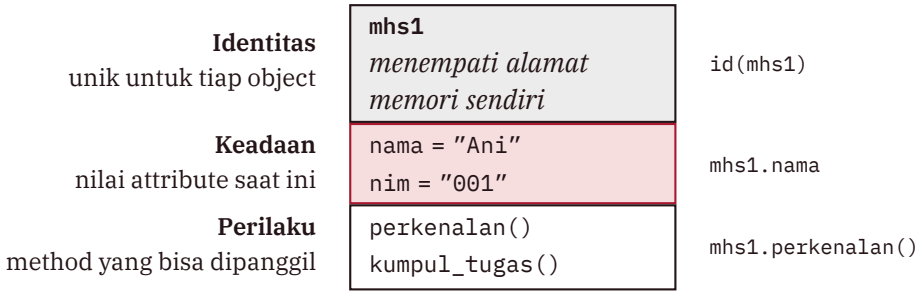
## 1.2 Konsep Object/Instance

*Object* adalah wujud nyata yang dibuat dari sebuah class. Istilah lain yang sering dipakai adalah *instance*; "object `mhs1` adalah instance dari class `Mahasiswa`". Proses membuat object dari class disebut *instansiasi*.

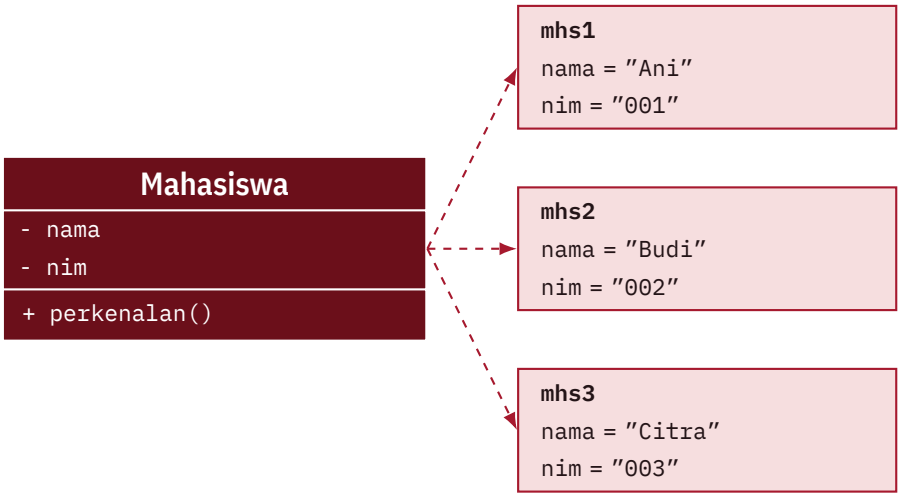
Setiap object memiliki tiga ciri:

- **Identitas**: setiap object unik dan menempati lokasi memori sendiri, walaupun isinya sama dengan object lain.
- **Keadaan** (*state*): nilai-nilai attribute yang dimiliki object pada suatu saat.
- **Perilaku** (*behavior*): tindakan yang bisa dilakukan object melalui method-nya.

Gambar 1.3 memperlihatkan ketiga ciri tersebut pada satu object, lengkap dengan cara mengaksesnya di Python.




**Gambar 1.3.** Tiga ciri sebuah object: identitas, keadaan, dan perilaku.




**Gambar 1.4.** Satu class dapat menghasilkan banyak object dengan keadaan masing-masing.

Untuk membuat object, kita "memanggil" class seolah-olah ia sebuah fungsi, yaitu dengan menuliskan nama class diikuti tanda kurung.

**Membuat dua object**

```
class Mahasiswa:
    pass

mhs1 = Mahasiswa()
mhs2 = Mahasiswa()

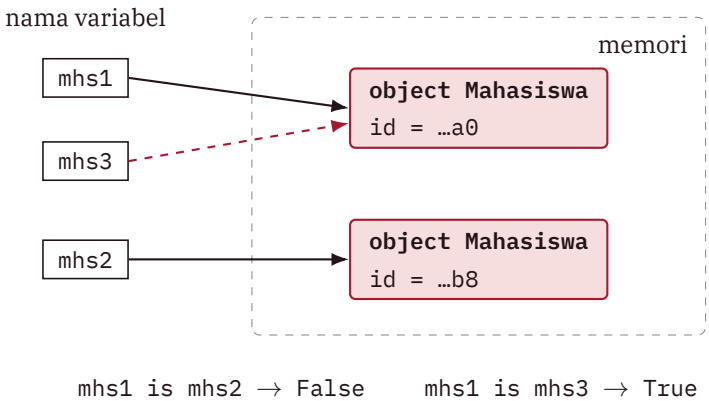
print(type(mhs1))
print(isinstance(mhs1, Mahasiswa))
print(mhs1 is mhs2)
print(id(mhs1) == id(mhs2))
```

**Keluaran**

```
<class '__main__.Mahasiswa'>
True
False
False
```

Fungsi `type()` memberi tahu dari class mana sebuah object dibuat, sedangkan `isinstance()` memeriksa apakah object merupakan instance dari class tertentu. Operator `is` dan fungsi `id()` membuktikan bahwa `mhs1` dan `mhs2` adalah dua object berbeda meskipun dibuat dari class yang sama.

Perlu dipahami juga bahwa variabel seperti `mhs1` sebenarnya hanyalah *nama* yang menunjuk ke object di memori. Jika kita menulis `mhs3 = mhs1`, Python tidak membuat object baru; `mhs3` hanya menjadi nama lain untuk object yang sama, sehingga `mhs1 is mhs3` bernilai `True` (Gambar 1.5).




**Gambar 1.5.** Variabel menunjuk ke object di memori. `mhs1` dan `mhs2` menunjuk dua object berbeda, sedangkan `mhs3 = mhs1` hanya menambah nama baru untuk object yang sama.

### SEMUA HAL DI PYTHON ADALAH OBJECT

Tanpa disadari, kita sudah memakai object sejak awal belajar Python. Angka `5` adalah object dari

class `int`, teks `"halo"` adalah object dari class `str`, dan `[1, 2, 3]` adalah object dari class `list`. Coba jalankan `type(5)` atau `type("halo")`. Bahkan pemanggilan seperti `"halo".upper()` sebenarnya adalah pemanggilan method milik object string.

## 1.3 Struktur Class di Python

Struktur sebuah class di Python dapat dipahami melalui tiga bagian utama: **deklarasi class**, **attribute**, dan **method**. Ketiganya membantu kita mengenali jenis object yang dimodelkan, data yang dimilikinya, serta perilaku yang dapat dilakukannya.

1. **Deklarasi class** ditulis dengan kata kunci `class`, nama class, dan tanda titik dua, misalnya `class Mahasiswa:`. Baris-baris di dalam badan class ditulis dengan indentasi yang konsisten; buku ini menggunakan empat spasi.
2. **Attribute** menyimpan data atau keadaan yang berkaitan dengan class dan object. Pada contoh `Mahasiswa`, `kampus` merupakan *class attribute* yang disimpan pada class, sedangkan `nama` dan `nim` merupakan *instance attribute* milik setiap object mahasiswa. Karena itu, dua object mahasiswa dapat memiliki nama dan NIM yang berbeda.
3. **Method** adalah fungsi yang didefinisikan di dalam class untuk menyediakan perilaku. Method `perkenalan()` pada contoh berikut membaca attribute mahasiswa lalu mengembalikan teks perkenalan. Sebagai *instance method*, method ini menerima `self` sebagai rujukan ke object yang sedang digunakan.

Class juga dapat memiliki *docstring*, yaitu dokumentasi singkat tentang kegunaannya. Sementara itu, method khusus `__init__` bertugas menyiapkan keadaan awal object. Pada contoh berikut, method tersebut menerima nama dan NIM, lalu menyimpannya ke `self.nama` dan `self.nim`. Attribute dan method akan dibahas lebih rinci pada subbab berikutnya; untuk saat ini, perhatikan letak dan peran masing-masing dalam satu class.

**Anatomi sebuah class**

```python
class Mahasiswa:
    """Merepresentasikan seorang mahasiswa."""   # docstring

    kampus = "UNESA"                              # class attribute

    def __init__(self, nama, nim):                # inisialisasi object
        self.nama = nama                          # instance attribute
        self.nim = nim                            # instance attribute

    def perkenalan(self):                         # instance method
        return f"Halo, saya {self.nama} ({self.nim}) dari {self.kampus}"
```

Perhatikan perbedaan `nama` dan `self.nama` pada baris `self.nama = nama`. Nama di sebelah kanan adalah parameter yang menerima masukan, sedangkan `self.nama` di sebelah

**Tabel 1.1.** Bagian-bagian class `Mahasiswa`.

| Bagian | Penjelasan |
|---|---|
| `class Mahasiswa:` | Header class: kata kunci `class`, nama class, lalu titik dua. Seluruh isi class ditulis menjorok ke dalam (indentasi 4 spasi). |
| `"""..."""` | *Docstring*, yaitu dokumentasi singkat tentang kegunaan class. Bisa dibaca lewat `help(Mahasiswa)`. |
| `kampus = "UNESA"` | *Class attribute*: data yang dimiliki bersama oleh semua object. |
| `__init__` | Method inisialisasi: menyiapkan keadaan awal object dengan menyimpan nilai parameter ke instance attribute. |
| `self.nama, self.nim` | *Instance attribute*: data milik masing-masing object. |
| `perkenalan` | *Instance method*: perilaku yang bisa dilakukan setiap object. |

kiri adalah attribute tempat nilai itu disimpan pada object. Setelah object dibuat, data dibaca dengan notasi titik, sedangkan method dijalankan dengan menambahkan tanda kurung.

**Mengakses attribute dan menjalankan method**

```
mhs = Mahasiswa("Ani", "001")
print(Mahasiswa.kampus)   # membaca class attribute
print(mhs.nama)           # membaca instance attribute
print(mhs.nim)            # membaca instance attribute
print(mhs.perkenalan())   # menjalankan instance method
```

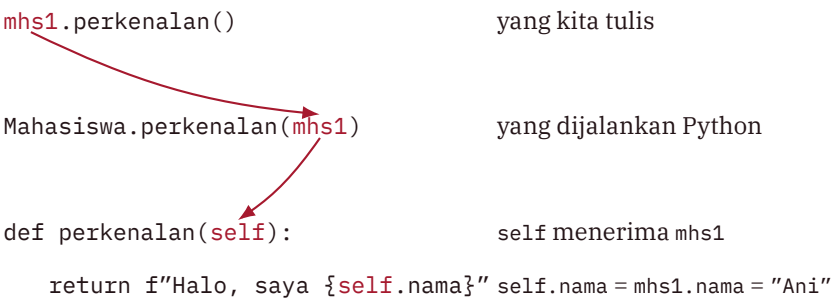
**Keluaran**

```
UNESA
Ani
001
Halo, saya Ani (001) dari UNESA
```

Dengan demikian, attribute menjawab pertanyaan "data apa yang dimiliki?", sedangkan method menjawab "apa yang dapat dilakukan?". Pada contoh ini, `perkenalan()` memakai data dari attribute untuk menghasilkan informasi tentang mahasiswa yang memanggilnya.

### Peran Parameter `self`

Perhatikan bahwa instance method pada contoh di atas menerima `self` sebagai parameter pertama. `self` adalah rujukan ke *object yang sedang dipakai*. Ketika kita menulis `mhs1.perkenalan()`, Python secara otomatis mengirim `mhs1` sebagai nilai `self`. Dengan begitu, di dalam method kita bisa mengakses data milik object tersebut melalui `self.nama`, `self.nim`, dan seterusnya. Alur ini digambarkan pada Gambar 1.6.




**Gambar 1.6.** Saat `mhs1.perkenalan()` dipanggil, Python mengirim `mhs1` ke parameter `self`, sehingga `self.nama` berarti `mhs1.nama`.

**JANGAN LUPA SELF**

`self` wajib ditulis sebagai parameter pertama setiap instance method, tetapi **tidak** ditulis saat method dipanggil. Lupa menuliskan `self` di definisi method adalah kesalahan yang sangat umum dan akan menghasilkan `TypeError` seperti *"takes 0 positional arguments but 1 was given"*.

### Constructor `__init__`

Nama `__init__` diawali dan diakhiri dua garis bawah. Method dengan pola nama seperti ini disebut *dunder method* (*double underscore*) atau *magic method*; Python memanggilnya secara otomatis pada momen tertentu. Untuk `__init__`, momennya adalah saat object baru dibuat. Parameter selain `self` pada `__init__` menjadi data yang wajib diberikan saat instansiasi.

**KONVENSI PENULISAN**

- Nama class: *PascalCase* (`RekeningBank`).
- Nama attribute dan method: *snake_case* (`saldo_awal`, `hitung_total()`).
- Pisahkan tiap method dengan satu baris kosong agar class mudah dibaca.

## 1.4 Mengakses Attribute dan Method

Setelah mengenal struktur class, kita perlu mengetahui cara memakai attribute dan method pada object yang sudah dibuat. Keduanya diakses dengan *notasi titik*: nama object, titik, lalu nama attribute atau method. Melalui notasi ini, kita bisa membaca nilai, mengubah nilai, dan memanggil method sebelum mempelajari jenis-jenisnya lebih rinci.

**Membaca, mengubah, dan memanggil**

```
1  class Mahasiswa:
2      def __init__(self, nama):
3          self.nama = nama
```

```
        self.semester = 1

    def naik_semester(self):
        self.semester += 1

    def info(self):
        return f"{self.nama} - semester {self.semester}"

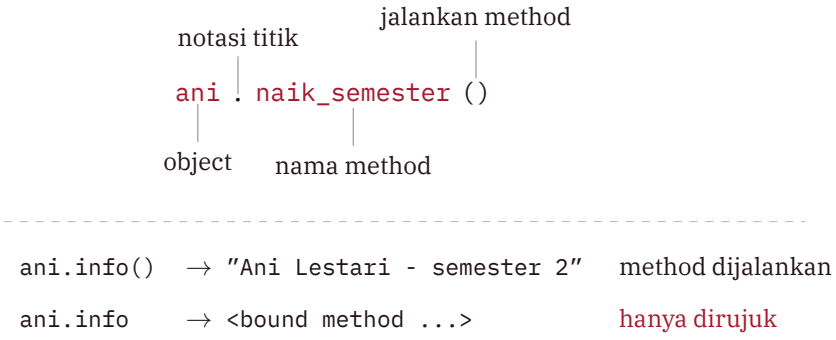
ani = Mahasiswa("Ani")

print(ani.nama)          # membaca attribute
ani.nama = "Ani Lestari" # mengubah attribute
ani.naik_semester()      # memanggil method
print(ani.info())
print(ani.info)          # tanpa tanda kurung: method tidak dijalankan
```

**Keluaran**

```
Ani
Ani Lestari - semester 2
<bound method Mahasiswa.info of <__main__.Mahasiswa object at 0x...>>
```

Perhatikan baris terakhir. Tanpa tanda kurung, `ani.info` hanya merujuk ke method tersebut (disebut *bound method*) tanpa menjalankannya. Ini kesalahan kecil yang sering membuat bingung pemula. Gambar 1.7 merangkum bagian-bagian notasi titik dan perbedaan pemanggilan dengan dan tanpa tanda kurung.




**Gambar 1.7.** Bagian-bagian notasi titik. Tanda kurung `()` yang membuat method benar-benar dijalankan.

## Mengakses Attribute yang Tidak Ada

Pada object `ani` di atas, attribute `ipk` belum dibuat. Jika kita mencoba membacanya dengan notasi titik, Python menghasilkan `AttributeError`. Contoh berikut sengaja menunjukkan kesalahan tersebut dan dijalankan terpisah dari contoh selanjutnya.

AttributeError

```
print(ani.ipk)
```

Keluaran

```
Traceback (most recent call last):
  ...
AttributeError: 'Mahasiswa' object has no attribute 'ipk'
```

### Akses Dinamis dengan `getattr`, `setattr`, dan `hasattr`

Notasi titik cocok ketika nama attribute sudah ditulis langsung dalam kode, misalnya `ani.semester`. Namun, kadang nama attribute disimpan dalam variabel dan baru ditentukan saat program berjalan. Untuk keperluan ini, Python menyediakan fungsi bawaan `getattr`, `setattr`, dan `hasattr`. Ketiganya menerima nama attribute sebagai *string*, tetapi memiliki tugas dan hasil yang berbeda.

1. **`getattr(objek, nama)`** digunakan untuk **membaca nilai** attribute. Sebagai contoh, `getattr(ani, "semester")` membaca nilai yang sama dengan `ani.semester`. Jika attribute tidak ditemukan, bentuk ini menghasilkan `AttributeError`. Kita dapat memberikan argumen ketiga sebagai nilai cadangan, misalnya `getattr(ani, "ipk", 0.0)`. Jika `ipk` belum ada, hasilnya `0.0`; pembacaan dengan nilai cadangan ini tidak membuat attribute baru.
2. **`setattr(objek, nama, nilai)`** digunakan untuk **menetapkan nilai** attribute. Pada contoh ini, `setattr(ani, "semester", 5)` setara dengan `ani.semester = 5`. Jika attribute belum ada dan object mengizinkannya, fungsi ini juga dapat membuat attribute baru. Nilai kembaliannya adalah `None`, sehingga hasil perubahan perlu dibaca kembali dari attribute object.
3. **`hasattr(objek, nama)`** digunakan untuk **memeriksa apakah attribute dapat ditemukan**. Hasilnya `True` jika akses ke attribute berhasil, atau `False` jika akses tersebut menghasilkan `AttributeError`. Fungsi ini tidak mengembalikan isi attribute dan tidak memastikan bahwa isinya memenuhi aturan tertentu; misalnya, keberadaan `ipk` belum membuktikan bahwa nilainya berada dalam rentang 0 sampai 4.

Contoh berikut melanjutkan object `ani` dari contoh "Membaca, mengubah, dan memanggil". Pada awal contoh ini, `ani.semester` bernilai `2` dan attribute `ipk` belum ada. Variabel `nama_attribute` menunjukkan bahwa nama attribute yang dibaca tidak harus ditulis langsung setelah titik.

Akses attribute secara dinamis

```
nama_attribute = "semester"

```

**Tabel 1.2.** Perbedaan fungsi akses attribute secara dinamis.

| Fungsi | Tujuan | Nilai kembalian |
|---|---|---|
| `getattr` | Membaca nilai attribute | Nilai attribute atau nilai cadangan |
| `setattr` | Mengisi atau mengubah nilai attribute | `None` |
| `hasattr` | Memeriksa apakah attribute dapat ditemukan | `True` atau `False` |

```
print(hasattr(ani, "nama"))          # memeriksa keberadaan attribute
print(hasattr(ani, "ipk"))
print(getattr(ani, nama_attribute)) # membaca semester melalui variabel
print(getattr(ani, "ipk", 0.0))     # nilai cadangan; tidak membuat ipk
print(hasattr(ani, "ipk"))          # ipk masih belum ada

setattr(ani, "semester", 5)         # mengubah attribute yang sudah ada
print(ani.semester)

setattr(ani, "ipk", 3.75)           # membuat attribute baru pada contoh ini
print(getattr(ani, "ipk"))
print(hasattr(ani, "ipk"))
```

**Keluaran**

```
True
False
2
0.0
False
5
3.75
True
```

Perhatikan dua hasil pemeriksaan `ipk` sebelum `setattr` dijalankan: keduanya masih `False`, walaupun `getattr` sudah mengembalikan nilai cadangan `0.0`. Attribute `ipk` baru tersedia setelah `setattr(ani, "ipk", 3.75)` dijalankan. Sesudah itu, `getattr` membaca `3.75` dan `hasattr` menghasilkan `True`.

Nama method juga dapat diakses dengan `getattr`. Bentuk `getattr(ani, "info")` memperoleh method yang terikat pada `ani`, sedangkan `getattr(ani, "info")()` menjalankannya. Seperti pada notasi titik, tanda kurung tetap diperlukan untuk memanggil method.

## 1.5 Konsep Attribute

*Attribute* adalah variabel yang "menempel" pada sebuah object atau class. Attribute menyimpan keadaan object: nama seorang mahasiswa, saldo sebuah rekening, warna sebuah mobil, atau status sebuah lampu (menyala atau mati).

Attribute dibuat dengan cara memberi nilai melalui notasi titik, umumnya di dalam `__init__` menggunakan `self`. Nilai attribute boleh bertipe apa saja: angka, string, boolean, list, dictionary, bahkan object lain.

**Attribute dengan berbagai tipe data**

```
class Mahasiswa:
    def __init__(self, nama, nim):
        self.nama = nama          # str
        self.nim = nim            # str
        self.semester = 1         # int, nilai awal tetap
        self.aktif = True         # bool
        self.daftar_nilai = []    # list kosong

mhs = Mahasiswa("Ani", "24051204001")
print(mhs.nama)
print(mhs.semester)
print(mhs.__dict__)
```

**Keluaran**

```
Ani
1
{'nama': 'Ani', 'nim': '24051204001', 'semester': 1, 'aktif': True, 'daftar_nilai':
    []}
```

## Perbedaan Attribute dan Parameter

Attribute dan parameter dapat memiliki nama yang sama, tetapi keduanya mempunyai peran berbeda. **Parameter** adalah nama yang ditulis pada definisi fungsi atau method untuk menerima masukan saat pemanggilan. Pada `def __init__(self, nama, nim)`, `nama` dan `nim` adalah parameter masukan, sedangkan `self` adalah parameter yang menerima rujukan ke object secara otomatis.

**Attribute** adalah data yang disimpan pada object atau class. Dalam contoh ini, `self.nama` dan `self.nim` merupakan attribute object. Pada baris `self.nama = nama`, parameter `nama` di sebelah kanan menyediakan nilai masukan, lalu nilai tersebut disimpan melalui attribute `self.nama` di sebelah kiri. Parameter tidak otomatis menjadi attribute; penugasan itulah yang membuat data nama tersedia pada object.

Saat kita menulis `Mahasiswa("Ani", "24051204001")`, nilai `"Ani"` dan `"24051204001"` disebut *argumen*, yaitu nilai yang diberikan saat pemanggilan. Di dalam `__init__`, kedua nilai itu diterima melalui parameter `nama` dan `nim`.

Setelah `__init__` selesai, kita tetap dapat membaca nama mahasiswa melalui `mhs.nama`. Hal ini terjadi karena nama sudah disimpan sebagai attribute, bukan karena parame-

**Tabel 1.3.** Perbedaan parameter dan attribute pada contoh `Mahasiswa`.

| Aspek | Parameter | Attribute object |
|---|---|---|
| Letak penulisan | Dalam daftar parameter definisi method, misalnya `nama` | Diakses melalui object, misalnya `self.nama` atau `mhs.nama` |
| Peran | Menerima masukan untuk diproses oleh method | Menyimpan data atau keadaan object |
| Ruang lingkup | Nama lokal pada pemanggilan method tersebut | Melekat pada object dan dapat diakses melalui rujukan ke object |
| Sumber nilai | Argumen saat pemanggilan atau nilai default parameter | Dapat berasal dari parameter, nilai tetap, atau hasil perhitungan |

ter `nama` menjadi variabel yang dapat diakses dari luar method. Nama parameter dan attribute juga tidak wajib sama; misalnya, parameter bernama `nama_lengkap` dapat disimpan dengan penugasan `self.nama = nama_lengkap`.

Dengan demikian, **attribute tidak harus berasal dari parameter**. Pada contoh di atas, `self.nama` dan `self.nim` memperoleh nilai dari parameter, sedangkan `self.semester = 1`, `self.aktif = True`, dan `self.daftar_nilai = []` menetapkan nilai awal langsung di dalam method. List kosong tersebut dibuat baru untuk setiap object mahasiswa, sehingga daftar nilai antarmahasiswa tidak dipakai bersama.

Pada contoh ini, attribute khusus `__dict__` menampilkan instance attribute yang tersimpan pada object dalam bentuk dictionary. Isinya membantu kita memeriksa data object saat *debugging*.

**DEFINISIKAN SEMUA ATTRIBUTE DI `__INIT__`**

Python mengizinkan kita menambah attribute baru dari luar class, misalnya `mhs.hobi = "catur"`. Namun cara ini membuat object dari class yang sama memiliki isi berbeda-beda dan menyulitkan pembacaan kode. Biasakan mendefinisikan seluruh attribute di dalam `__init__`, walaupun nilai awalnya hanya `None` atau list kosong.

## 1.6 Instance Attribute dan Class Attribute

Python membedakan dua jenis attribute berdasarkan siapa pemiliknya.

- **Instance attribute** dimiliki oleh masing-masing object. Biasanya nilai awalnya ditetapkan melalui `self.nama = ...` di dalam `__init__`, tetapi nilainya juga dapat diubah setelah object dibuat: secara langsung melalui `objek.nama = nilai_baru` atau melalui method lain yang menulis `self.nama = nilai_baru`. Pada dua object yang

berbeda, pemberian nilai baru pada attribute milik satu object tidak mengubah attribute milik object lainnya.

- **Class attribute** dimiliki oleh class dan dipakai bersama oleh semua object. Ditulis langsung di badan class, di luar method mana pun.

**Instance attribute vs class attribute**

```
class Mahasiswa:
    kampus = "UNESA"     # class attribute
    jumlah = 0           # class attribute

    def __init__(self, nama):
        self.nama = nama          # instance attribute
        Mahasiswa.jumlah += 1     # ubah class attribute lewat nama class

    def ubah_nama(self, nama_baru):
        self.nama = nama_baru     # ubah attribute di luar __init__

a = Mahasiswa("Ani")
b = Mahasiswa("Budi")

print(a.nama, b.nama)
print(a.kampus, b.kampus, Mahasiswa.kampus)
print("Jumlah mahasiswa:", Mahasiswa.jumlah)
```

**Keluaran**

```
Ani Budi
UNESA UNESA UNESA
Jumlah mahasiswa: 2
```

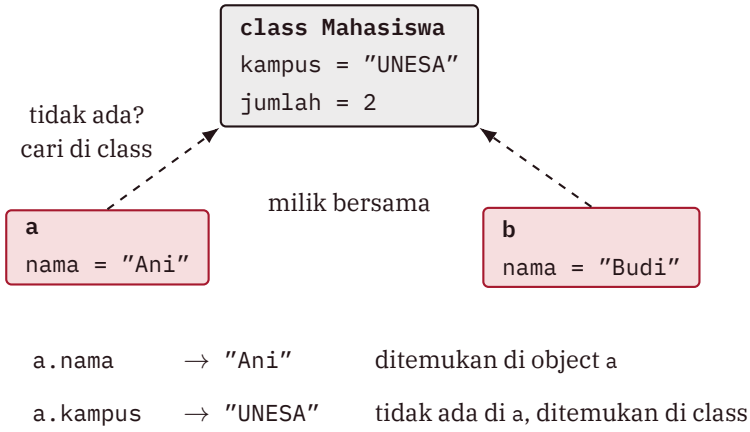
Saat kita menulis `a.kampus`, Python mencari attribute `kampus` terlebih dahulu di object `a`. Karena tidak ditemukan di sana, pencarian dilanjutkan ke class `Mahasiswa`, dan nilainya ditemukan. Urutan pencarian *object dulu, lalu class* ini (Gambar 1.8) penting untuk memahami jebakan berikut.

## Mengubah Instance Attribute setelah Object Dibuat

Method `__init__` menyiapkan nilai awal, bukan membatasi kapan attribute boleh diubah. Setelah object terbentuk, nilainya dapat diperbarui sesuai kebutuhan program. Dua cara berikut memakai class `Mahasiswa` pada contoh sebelumnya; jalankan potongan kodenya secara berurutan setelah definisi class tersebut.

### Mengubah Langsung dari Luar Class

Untuk attribute publik seperti `nama`, gunakan notasi titik diikuti penugasan nilai baru. Object yang ditulis di sebelah kiri titik menentukan data siapa yang diubah.




**Gambar 1.8.** Instance attribute tersimpan di tiap object, class attribute tersimpan di class. Python mencari attribute di object terlebih dahulu, baru kemudian di class.

**Tabel 1.4.** Perbandingan instance attribute dan class attribute.

| Aspek | Instance attribute | Class attribute |
|---|---|---|
| Pemilik | Masing-masing object | Class (dipakai bersama) |
| Tempat definisi | Di dalam method, lewat `self` | Di badan class, di luar method |
| Cara mengakses | `objek.attr` | `NamaClass.attr` atau `objek.attr` |
| Cara mengubah | `objek.attr = nilai` atau melalui instance method | `NamaClass.attr = nilai` |
| Cocok untuk | Data yang berbeda tiap object (nama, NIM, saldo) | Data yang sama untuk semua object (nama kampus, konstanta, penghitung) |

Mengubah attribute melalui nama object

```
mhs1 = Mahasiswa("Ani")
mhs2 = Mahasiswa("Budi")

print("Sebelum:", mhs1.nama, "|", mhs2.nama)
mhs1.nama = "Ani Lestari"
print("Sesudah:", mhs1.nama, "|", mhs2.nama)
```

Keluaran

```
Sebelum: Ani | Budi
Sesudah: Ani Lestari | Budi
```

Penugasan `mhs1.nama = "Ani Lestari"` mengganti nilai attribute milik `mhs1`. Nilai `mhs2.nama` tetap `"Budi"`, karena kedua nama disimpan pada object yang berbeda.

### Mengubah melalui Method di Luar `__init__`

Pada definisi class sebelumnya, `ubah_nama()` ditulis sejajar dengan `__init__`, sehingga merupakan method tersendiri di dalam class. Method itu menerima parameter `nama_baru`, lalu menjalankan `self.nama = nama_baru`. Ketika dipanggil melalui `mhs2`, `self` merujuk ke `mhs2`; karena itu, nama mahasiswa kedua yang berubah.

Mengubah attribute melalui method

```
mhs2.ubah_nama("Budi Santoso")
print(mhs1.nama)
print(mhs2.nama)
```

Keluaran

```
Ani Lestari
Budi Santoso
```

Kedua cara tersebut mengubah object yang sudah ada, tanpa membuat object baru atau menjalankan ulang `__init__`. Perbedaannya terletak pada tempat penugasannya: `mhs1.nama = ...` ditulis oleh kode pemakai class, sedangkan `self.nama = ...` ditulis di dalam method. Cara kedua juga menyediakan tempat untuk menambahkan aturan pemeriksaan sebelum perubahan dilakukan, yang akan dibahas lebih lanjut pada Bab 2 tentang encapsulation.

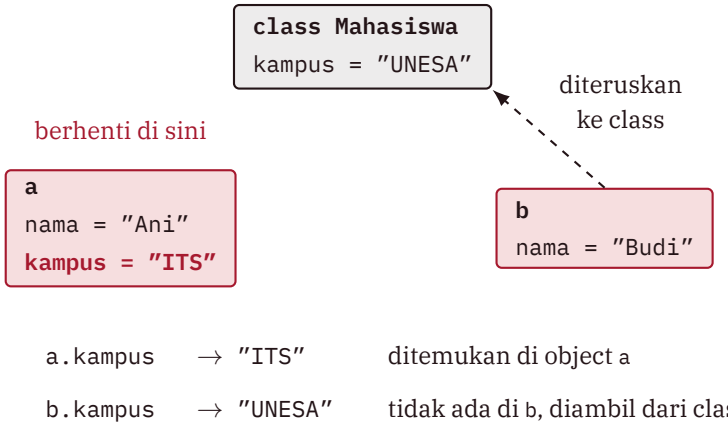
## Jebakan 1: Mengubah Class Attribute Lewat Object

Memberi nilai baru melalui object **tidak** mengubah class attribute. Yang terjadi justru Python membuat instance attribute baru dengan nama yang sama di object tersebut, sehingga class attribute "tertutupi" (*shadowing*) hanya untuk object itu.

**Shadowing class attribute**

```
a.kampus = "ITS"          # mengubah nilai kampus untuk object a saja
print(a.kampus)           # ITS
print(b.kampus)           # UNESA
print(Mahasiswa.kampus)   # UNESA
```

Gambar 1.9 memperlihatkan keadaan setelah baris `a.kampus = "ITS"` dijalankan. Karena `a` kini memiliki attribute `kampus` sendiri, pencarian berhenti di object `a` dan tidak pernah sampai ke class.




`a.kampus` → `"ITS"` ditemukan di object `a`

`b.kampus` → `"UNESA"` tidak ada di `b`, diambil dari class

**Gambar 1.9.** *Shadowing*: instance attribute `kampus` milik `a` menutupi class attribute dengan nama yang sama, tetapi hanya untuk `a`.

Karena itu, jika ingin mengubah class attribute, selalu lakukan melalui nama class: `Mahasiswa.kampus = "ITS"`.

## Jebakan 2: Class Attribute yang Mutable

Jika class attribute berupa object yang bisa diubah isinya (*mutable*) seperti list atau dictionary, perubahan isinya akan terlihat oleh object-object yang merujuk data yang sama.

**List yang tidak sengaja dipakai bersama**

```
class Kelompok:
    anggota = []                  # class attribute, dipakai bersama!

    def tambah(self, nama):
        self.anggota.append(nama)

k1 = Kelompok()
k2 = Kelompok()
k1.tambah("Ani")
print(k2.anggota)                 # k2 belum pernah memanggil tambah()
```

Keluaran

```
['Ani']
```

Keluaran `['Ani']` muncul pada `k2.anggota` karena `anggota = []` ditulis di badan class `Kelompok`, sehingga merupakan *class attribute*. Pada contoh ini, `k1` dan `k2` tidak memiliki instance attribute bernama `anggota`; keduanya mengakses satu list yang sama melalui class tersebut.

Ketika `k1.tambah("Ani")` dijalankan, `self` merujuk ke `k1`, tetapi pencarian `self.anggota` menemukan list milik `Kelompok`. Pemanggilan `append()` mengubah isi list itu, bukan membuat list baru khusus untuk `k1`. Karena `k2.anggota` juga merujuk list yang sama, pembacaannya menghasilkan `['Ani']` walaupun `k2` tidak pernah menambahkan anggota.

Perbaikannya adalah membuat list sebagai instance attribute di dalam `__init__`. Dengan demikian, setiap kali object dibuat, sebuah list kosong baru disiapkan khusus untuk object tersebut.

Perbaikan

```
class Kelompok:
    def __init__(self):
        self.anggota = []          # instance attribute, milik tiap object

    def tambah(self, nama):
        self.anggota.append(nama)

k1 = Kelompok()
k2 = Kelompok()
k1.tambah("Ani")
print(k1.anggota)
print(k2.anggota)
```

Keluaran

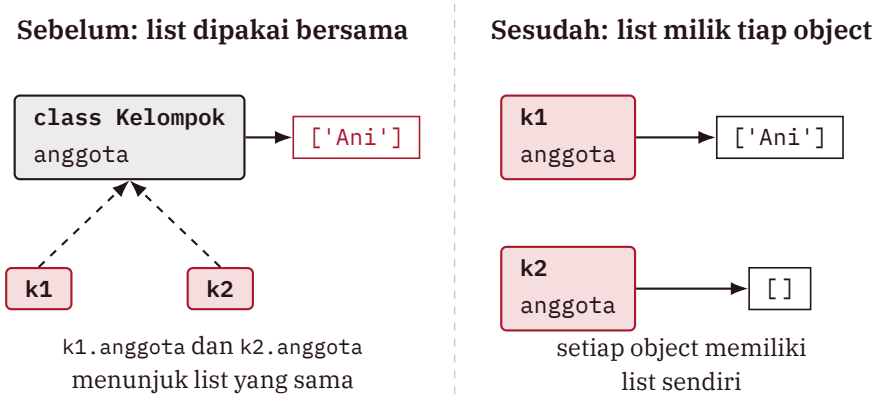
```
['Ani']
[]
```

Pada contoh perbaikan, `k1` dan `k2` dibuat kembali menggunakan definisi class yang baru. Keduanya kini memiliki list `anggota` yang berbeda. Karena hanya `k1` yang memanggil `tambah("Ani")`, list milik `k1` berisi `['Ani']`, sedangkan list milik `k2` tetap kosong, yaitu `[]`.

Perbedaan keadaan sebelum dan sesudah perbaikan dapat dilihat pada Gambar 1.10.

## 1.7 Konsep Method

Jika attribute menyimpan keadaan, maka *method* menggambarkan perilaku. Method adalah fungsi yang didefinisikan di dalam class dan bekerja dengan data milik object.




**Gambar 1.10.** Class attribute yang mutable dipakai bersama oleh semua object (kiri). Menjadikannya instance attribute memberi setiap object list miliknya sendiri (kanan).

Contohnya: rekening bisa *menerima setoran*, lampu bisa *dinyalakan*, mahasiswa bisa *memperkenalkan diri*.

Secara penulisan, method mirip fungsi biasa: diawali `def`, punya parameter, dan boleh mengembalikan nilai dengan `return`. Perbedaannya terletak pada tempat dan cara pemanggilannya.

**Tabel 1.5.** Perbedaan fungsi biasa dan method.

| Aspek | Fungsi | Method |
|---|---|---|
| Tempat definisi | Di luar class | Di dalam class |
| Cara memanggil | `fungsi(data)` | `objek.method()` |
| Akses data | Hanya lewat parameter | Langsung ke attribute object lewat `self` |

Untuk menjalankan *instance method*, kita perlu menentukan object yang datanya akan digunakan. Ada dua cara pemanggilan yang ditunjukkan pada contoh berikut.

1. **Melalui object**, dengan bentuk `objek.method()`. Contohnya adalah `ani.perkenalan()`. Python secara otomatis memberikan object `ani` sebagai argumen untuk parameter `self`, sehingga kita tidak perlu menuliskannya lagi di dalam tanda kurung.
2. **Melalui class**, dengan bentuk `NamaClass.method(objek)`. Contohnya adalah `Mahasiswa.perkenalan(ani)`. Pada bentuk ini, object `ani` harus diberikan secara eksplisit sebagai argumen pertama untuk mengisi `self`.

Kedua bentuk tersebut tetap memanggil instance method yang sama. Penggunaan nama class pada bentuk kedua tidak menjadikannya *class method*; pemanggilan itu masih memerlukan object mahasiswa yang datanya akan dibaca.

Fungsi vs method

```python
# Gaya prosedural: data dan fungsi terpisah
def perkenalan(nama, nim):
    return f"Halo, saya {nama} ({nim})"

print(perkenalan("Ani", "001"))

# Gaya OOP: data dan perilaku menyatu dalam object
class Mahasiswa:
    def __init__(self, nama, nim):
        self.nama = nama
        self.nim = nim

    def perkenalan(self):
        return f"Halo, saya {self.nama} ({self.nim})"

ani = Mahasiswa("Ani", "001")
print(ani.perkenalan())            # melalui object; self diisi otomatis
print(Mahasiswa.perkenalan(ani))   # melalui class; ani diberikan sebagai self
```

Keluaran

```
Halo, saya Ani (001)
Halo, saya Ani (001)
Halo, saya Ani (001)
```

Program lengkap di atas menghasilkan tiga baris keluaran. Baris pertama berasal dari fungsi biasa `perkenalan("Ani", "001")`, baris kedua dari method yang dipanggil melalui object, dan baris ketiga dari method yang dipanggil melalui class. Ketiganya menghasilkan teks yang sama, tetapi cara memberikan datanya berbeda.

Pada dua pemanggilan method tersebut, object `ani` sama-sama diterima sebagai `self`. Dengan demikian, `self.nama` membaca `ani.nama` dan `self.nim` membaca `ani.nim`. Bentuk `ani.perkenalan()` lebih ringkas karena pengiriman object dilakukan otomatis, sedangkan `Mahasiswa.perkenalan(ani)` memperlihatkan argumen object tersebut secara eksplisit, sesuai alur pada Gambar 1.6.

JENIS-JENIS METHOD

Python mengenal tiga jenis method: *instance method*, *class method*, dan *static method*. Bab ini fokus pada instance method, jenis yang paling sering dipakai. Dua jenis lainnya dibahas di Bab 3.

## 1.8 Instance Method

*Instance method* adalah method yang bekerja pada satu object tertentu. Ciri utamanya adalah parameter pertama `self`. Melalui `self`, instance method dapat membaca attribute,

mengubah attribute, dan memanggil method lain milik object yang sama.

### Method yang Mengubah Keadaan Object

**Class Lampu**

```
class Lampu:
    def __init__(self, lokasi):
        self.lokasi = lokasi
        self.menyala = False          # keadaan awal: mati

    def nyalakan(self):
        self.menyala = True

    def matikan(self):
        self.menyala = False

    def status(self):
        keadaan = "menyala" if self.menyala else "mati"
        return f"Lampu {self.lokasi} {keadaan}"

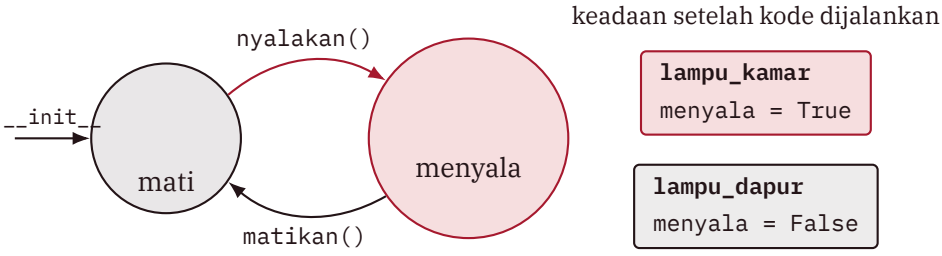
lampu_kamar = Lampu("kamar")
lampu_dapur = Lampu("dapur")

lampu_kamar.nyalakan()
print(lampu_kamar.status())
print(lampu_dapur.status())
```

**Keluaran**

```
Lampu kamar menyala
Lampu dapur mati
```

Method `nyalakan()` hanya mengubah keadaan `lampu_kamar`; `lampu_dapur` tetap mati. Inilah kekuatan `self`: kode method ditulis sekali, tetapi bekerja pada data object yang memanggilnya. Gambar 1.11 menggambarkan method sebagai "tombol" yang memindahkan object dari satu keadaan ke keadaan lain.




**Gambar 1.11.** Setiap object `Lampu` berawal dari keadaan mati. Method `nyalakan()` dan `matikan()` hanya mengubah keadaan object yang memanggilnya.

### Method dengan Parameter dan Nilai Kembali

Instance method dapat menerima parameter tambahan setelah `self` dan mengembalikan hasil perhitungan. Sebuah method juga bisa memanggil method lain di class yang sama melalui `self`.

**Class PersegiPanjang**

```python
class PersegiPanjang:
    def __init__(self, panjang, lebar):
        self.panjang = panjang
        self.lebar = lebar

    def luas(self):
        return self.panjang * self.lebar

    def keliling(self):
        return 2 * (self.panjang + self.lebar)

    def perbesar(self, faktor):
        self.panjang *= faktor
        self.lebar *= faktor

    def info(self):
        # memanggil method lain lewat self
        ukuran = f"{self.panjang} x {self.lebar}"
        return f"{ukuran}, luas {self.luas()}, keliling {self.keliling()}"

    def __str__(self):
        return self.info()        # representasi teks untuk print(pp)

pp = PersegiPanjang(4, 3)
print(pp.info())
pp.perbesar(2)
print(pp.info())
```

**Keluaran**

```
4 x 3, luas 12, keliling 14
8 x 6, luas 48, keliling 28
```

**METHOD __STR__**

Pada class yang belum menyediakan representasi teks khusus, `print(pp)` menampilkan representasi bawaan seperti `<__main__.PersegiPanjang object at 0x7f...>`. Bagian alamatnya dapat berbeda setiap kali program dijalankan. Agar hasilnya lebih bermakna, class `PersegiPanjang` di atas dilengkapi method khusus `__str__(self)` yang mengembalikan string dari `self.info()`. Method ini memungkinkan kita mencetak object secara langsung dengan

```
print(pp).
```

Contoh berikut menggunakan class `PersegiPanjang` yang sudah dilengkapi `__str__` tersebut. Object dibuat kembali dengan ukuran awal 4 dan 3 agar perubahan hasil cetaknya mudah dibandingkan.

**Mencetak object dengan print(pp)**

```
pp = PersegiPanjang(4, 3)
print(pp)                 # otomatis memakai __str__()
pp.perbesar(2)
print(pp)                 # teks mengikuti ukuran terbaru
```

**Keluaran**

```
4 x 3, luas 12, keliling 14
8 x 6, luas 48, keliling 28
```

Ketika `print(pp)` dijalankan, Python memakai `__str__` untuk memperoleh representasi string object. Method tersebut memanggil `info()`, yang membaca panjang dan lebar serta menghitung luas dan keliling. Karena itu, setelah `perbesar(2)` dijalankan, hasil cetak berikutnya menampilkan ukuran yang baru. Di dalam `__str__`, gunakan `return` untuk mengembalikan string, bukan `print()`; pencetakan dilakukan oleh pemanggilnya.

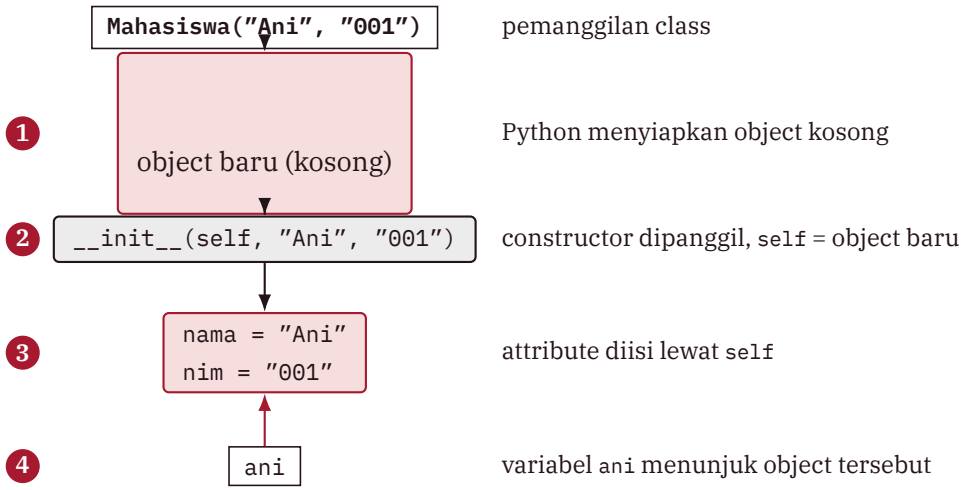
## 1.9 Membuat Object

Membuat object dilakukan dengan memanggil nama class beserta argumen yang dibutuhkan constructor. Argumen yang kita berikan akan diteruskan ke parameter `__init__` setelah `self`.

### Apa yang Terjadi Saat `Mahasiswa("Ani", "001")` Dijalankan

1. Python membuat sebuah object baru yang masih kosong dari class `Mahasiswa`.
2. Python memanggil `__init__` dengan `self` berisi object baru tadi, `nama="Ani"`, dan `nim="001"`.
3. Di dalam `__init__`, attribute `self.nama` dan `self.nim` diisi.
4. Object yang sudah terisi dikembalikan dan disimpan ke variabel di sebelah kiri tanda `=`.

Keempat langkah tersebut divisualisasikan pada Gambar 1.12 untuk perintah `ani = Mahasiswa("Ani", "001")`.




**Gambar 1.12.** Proses instansiasi langkah demi langkah.

## Parameter Default dan Argumen Kata Kunci

Seperti fungsi biasa, parameter constructor boleh memiliki nilai default sehingga argumennya tidak wajib diberikan saat object dibuat. Pada contoh berikut, parameter yang memiliki nilai default adalah `semester`, yang ditulis sebagai `semester=1`. Angka `1` digunakan jika pemanggil tidak memberikan argumen untuk `semester`. Sebaliknya, `nama` dan `nim` tidak memiliki nilai default, sehingga keduanya wajib diberikan; `self` diisi secara otomatis oleh Python.

Kita juga boleh memakai argumen kata kunci (*keyword argument*), yaitu menuliskan nama parameter beserta nilainya saat pemanggilan, misalnya `semester=5`. Nilai yang diberikan pemanggil akan digunakan sebagai pengganti nilai default.

**Constructor dengan nilai default**

```
class Mahasiswa:
    # semester memiliki nilai default 1; nama dan nim wajib diberikan
    def __init__(self, nama, nim, semester=1):
        self.nama = nama
        self.nim = nim
        self.semester = semester  # simpan nilai semester pada object

a = Mahasiswa("Ani", "001")                       # memakai default 1
b = Mahasiswa("Budi", "002", 3)                   # argumen posisi: 3
c = Mahasiswa(nim="003", nama="Citra", semester=5) # argumen kata kunci: 5

print(a.semester, b.semester, c.semester)
```

**Keluaran**

```
1 3 5
```

Dampak parameter default pada ketiga object tersebut adalah sebagai berikut.

- Pada `a = Mahasiswa("Ani", "001")`, argumen `semester` tidak diberikan. Python memakai nilai default `1`, sehingga `a.semester` bernilai `1`.
- Pada `b = Mahasiswa("Budi", "002", 3)`, argumen ketiga, yaitu `3`, mengisi parameter `semester` berdasarkan posisinya. Nilai default tidak digunakan, sehingga `b.semester` bernilai `3`.
- Pada object `c`, argumen `semester=5` mengisi parameter berdasarkan namanya. Karena argumen diberikan dengan nama parameter, urutan penulisan `nim` dan `nama` dapat ditukar. Nilai default tidak digunakan, sehingga `c.semester` bernilai `5`.

Jadi, nilai default membuat pemanggilan lebih ringkas ketika nilai awal yang disediakan sudah sesuai, tetapi tetap memungkinkan pemanggil menentukan nilai lain. Penulisan `semester=1` pada definisi method menetapkan nilai default parameter, sedangkan `self.semester = semester` menyimpan nilai parameter yang terpilih sebagai attribute object.

Jika argumen wajib tidak diberikan, misalnya `Mahasiswa("Dodi")`, Python akan menghentikan program dengan pesan `TypeError: __init__() missing 1 required positional argument: 'nim'`.

### Menyimpan Banyak Object

Dalam program nyata, kita jarang hanya membuat satu atau dua object. Object biasanya disimpan dalam list lalu diproses dengan perulangan.

**Daftar object dalam list**

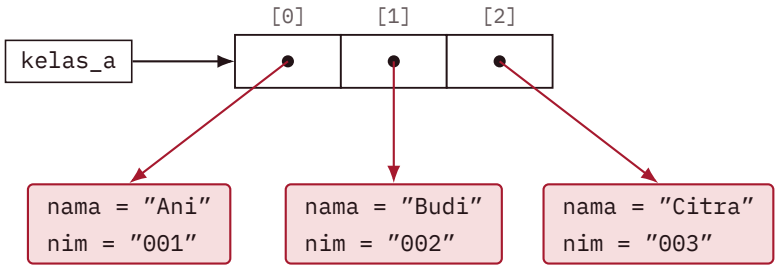
```
1  data = [("Ani", "001"), ("Budi", "002"), ("Citra", "003")]
2
3  kelas_a = []
4  for nama, nim in data:
5      kelas_a.append(Mahasiswa(nama, nim))
6
7  for mhs in kelas_a:
8      print(mhs.nim, mhs.nama)
```

**Keluaran**

```
001 Ani
002 Budi
003 Citra
```

Seperti terlihat pada Gambar 1.13, setiap elemen list sebenarnya menyimpan rujukan ke sebuah object. Saat perulangan berjalan, variabel `mhs` bergantian menunjuk object-object tersebut.

## 1.10 Pemodelan Objek dari Permasalahan Sederhana




tiap elemen list menunjuk satu object `Mahasiswa`

**Gambar 1.13.** List `kelas_a` berisi rujukan ke tiga object `Mahasiswa`.

Kemampuan terpenting dalam OOP bukanlah menghafal sintaks, melainkan *memodelkan* masalah: menentukan class apa yang dibutuhkan, data apa yang disimpan, dan perilaku apa yang dimiliki. Teknik sederhana yang bisa langsung dipakai adalah *analisis kata benda dan kata kerja*.

### Langkah Memodelkan Masalah

1. Baca deskripsi masalah dengan teliti.
2. Tandai **kata benda utama**; kandidat kuat untuk menjadi class.
3. Tandai **kata benda yang menjelaskan** kata benda utama (milik, ciri, jumlah); kandidat attribute.
4. Tandai **kata kerja**; kandidat method.
5. Tentukan aturan yang harus dijaga (misalnya "saldo tidak boleh negatif"), lalu letakkan di method yang sesuai.
6. Tuliskan class-nya, kemudian uji dengan membuat beberapa object.

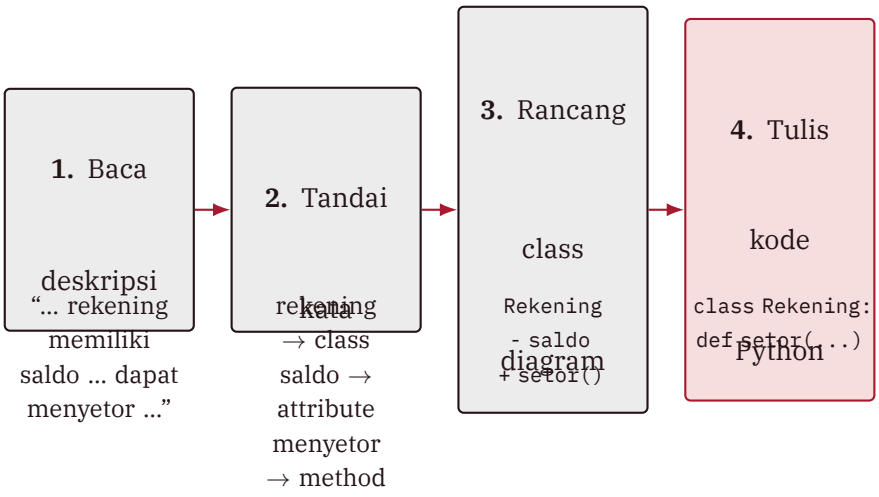
Secara garis besar, langkah-langkah tersebut membentuk alur kerja dari teks menuju kode seperti pada Gambar 1.14.

**STUDI KASUS: TABUNGAN KOPERASI MAHASISWA**

Koperasi mahasiswa ingin mencatat **rekening** tabungan anggotanya. Setiap rekening memiliki nomor rekening, nama pemilik, dan saldo. Anggota dapat *menyetor* uang, *menarik* uang selama saldonya mencukupi, dan *melihat informasi* rekening. Setoran dan penarikan harus bernilai lebih dari nol.

## Identifikasi Class, Attribute, dan Method

Hasil analisis tersebut dapat digambarkan sebagai *class diagram* sederhana. Kotak dibagi tiga: nama class, attribute, dan method. Class diagram akan dibahas lebih lengkap




**Gambar 1.14.** Alur pemodelan objek: dari deskripsi masalah, ke analisis kata, ke class diagram, lalu ke kode Python.

**Tabel 1.6.** Hasil analisis kata benda dan kata kerja.

| Temuan | Jenis | Menjadi |
|---|---|---|
| rekening | kata benda utama | class `Rekening` |
| nomor rekening, nama pemilik, saldo | kata benda penjelas | attribute `nomor`, `pemilik`, `saldo` |
| menyetor, menarik, melihat informasi | kata kerja | method `setor()`, `tarik()`, `info()` |
| saldo mencukupi, nilai lebih dari nol | aturan | pemeriksaan di dalam `setor()` dan `tarik()` |

di Bab 11.




**Gambar 1.15.** Class diagram `Rekening`.

## Implementasi

rekening.py

```python
class Rekening:
    """Rekening tabungan anggota koperasi."""

    def __init__(self, nomor, pemilik, saldo=0):
        self.nomor = nomor
        self.pemilik = pemilik
        self.saldo = saldo

    def setor(self, jumlah):
        if jumlah <= 0:
            print("Setoran harus lebih dari nol.")
            return
        self.saldo += jumlah
        print(f"Setor Rp{jumlah:,} berhasil.")

    def tarik(self, jumlah):
        if jumlah <= 0:
            print("Penarikan harus lebih dari nol.")
        elif jumlah > self.saldo:
            print("Saldo tidak mencukupi.")
        else:
            self.saldo -= jumlah
            print(f"Tarik Rp{jumlah:,} berhasil.")

    def info(self):
        return f"[{self.nomor}] {self.pemilik} - saldo Rp{self.saldo:,}"


rek_ani = Rekening("KM-001", "Ani", 100000)
rek_budi = Rekening("KM-002", "Budi")

```

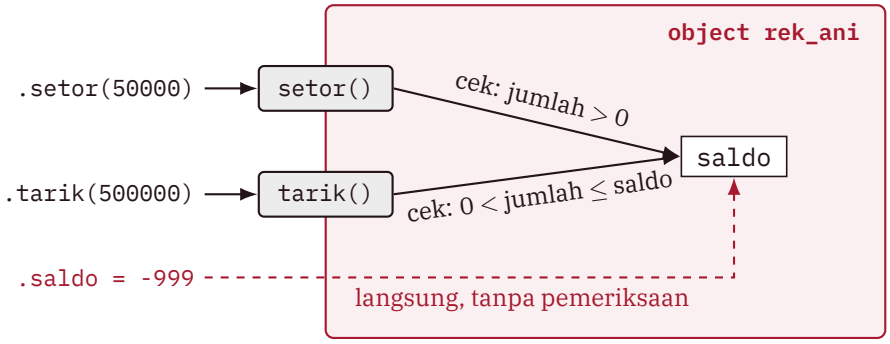
```
rek_ani.setor(50000)
rek_ani.tarik(500000)
rek_budi.setor(-1000)
rek_budi.setor(25000)

print(rek_ani.info())
print(rek_budi.info())
```

**Keluaran**

```
Setor Rp50,000 berhasil.
Saldo tidak mencukupi.
Setoran harus lebih dari nol.
Setor Rp25,000 berhasil.
[KM-001] Ani - saldo Rp150,000
[KM-002] Budi - saldo Rp25,000
```

Perhatikan bahwa aturan "saldo tidak boleh kurang dari jumlah penarikan" ditempatkan di dalam method `tarik()`. Selama perubahan saldo dilakukan melalui method, aturan itu selalu terjaga. Masalahnya, saat ini siapa pun masih bisa menulis `rek_ani.saldo = -999` dari luar class dan melewati aturan tersebut (Gambar 1.16). Cara melindungi attribute dari perubahan langsung seperti ini adalah topik Bab 2, *Visibility dan Encapsulation*.




**Gambar 1.16.** Method berperan sebagai "gerbang" yang memeriksa aturan sebelum saldo diubah. Perubahan langsung lewat attribute melewati gerbang tersebut; masalah ini diselesaikan di Bab 2.

**CIRI MODEL YANG BAIK**

- Satu class mewakili satu konsep yang jelas.
- Nama attribute dan method mudah dipahami tanpa membaca isinya.
- Aturan bisnis (validasi) berada di dalam method, bukan tersebar di luar class.
- Method kecil dan masing-masing mengerjakan satu tugas.

## 1.11 Rangkuman

- **Class** adalah cetak biru yang mendefinisikan attribute dan method; **object** (instance) adalah wujud nyata yang dibuat dari class.
- Class dibuat dengan kata kunci `class`; object dibuat dengan memanggil nama class, misalnya `Mahasiswa("Ani", "001")`.
- `__init__` adalah constructor yang otomatis dijalankan saat object dibuat untuk menyiapkan keadaan awalnya.
- `self` merujuk ke object yang sedang dipakai dan wajib menjadi parameter pertama instance method.
- **Instance attribute** milik masing-masing object; **class attribute** dipakai bersama oleh semua object.
- Mengubah class attribute lewat object justru membuat instance attribute baru (*shadowing*); class attribute yang mutable dipakai bersama oleh semua object.
- **Method** adalah fungsi di dalam class yang menggambarkan perilaku object; **instance method** bekerja pada data object lewat `self`.
- Attribute dan method diakses dengan notasi titik; `getattr`, `setattr`, dan `hasattr` dipakai untuk akses dinamis.
- Pemodelan objek dapat dimulai dengan analisis kata benda (class dan attribute) dan kata kerja (method).

## 1.12 Kuis Teoritis

### Pilihan Ganda

Pilihlah satu jawaban yang paling tepat.

1. Pernyataan yang paling tepat tentang hubungan class dan object adalah ...

   a. class adalah wujud nyata dari object
   b. object adalah cetak biru untuk membuat class
   c. class adalah cetak biru, object adalah wujud nyata yang dibuat darinya
   d. class dan object adalah dua istilah untuk hal yang sama

2. Method `__init__` dijalankan ketika ...

   a. class pertama kali didefinisikan
   b. object baru dibuat dari class
   c. object dicetak dengan `print()`
   d. program selesai dijalankan

3. Parameter `self` pada instance method merujuk ke ...

   a. class tempat method didefinisikan
   b. method itu sendiri
   c. object yang memanggil method
   d. modul utama program

4. Perhatikan kode berikut.

**Soal 4**

```
class A:
    x = 10

a1 = A()
a2 = A()
a1.x = 20
print(a2.x, A.x)
```

   Keluaran kode tersebut adalah ...

   a. `20 20`
   b. `10 10`
   c. `20 10`
   d. `10 20`

5. Attribute yang paling tepat dijadikan *class attribute* pada class `Mahasiswa` adalah ...

   a. `nim`
   b. `nama`
   c. `ipk`
   d. `nama_universitas`

6. Apa yang terjadi jika kita menulis `lampu.nyalakan` (tanpa tanda kurung)?

   a. Lampu menyala seperti biasa
   b. Terjadi `SyntaxError`
   c. Method hanya dirujuk dan tidak dijalankan
   d. Terjadi `AttributeError`

7. Perhatikan kode berikut.

**Soal 7**

```
class Tim:
    anggota = []
    def tambah(self, nama):
        self.anggota.append(nama)

```

```
t1, t2 = Tim(), Tim()
t1.tambah("Ani")
t2.tambah("Budi")
print(len(t1.anggota))
```

Keluaran kode tersebut adalah ...

a. `0`
b. `1`
c. `2`
d. Terjadi error

8. Kesalahan yang muncul ketika mengakses attribute yang tidak pernah dibuat adalah ...

a. `NameError`
b. `TypeError`
c. `ValueError`
d. `AttributeError`

9. Pada kalimat "Setiap *kendaraan* memiliki *nomor polisi* dan dapat *diparkir*", kandidat method adalah ...

a. kendaraan
b. nomor polisi
c. diparkir
d. memiliki

10. Pemanggilan `ani.perkenalan()` setara dengan ...

a. `perkenalan(ani)`
b. `Mahasiswa.perkenalan(ani)`
c. `ani.perkenalan(self)`
d. `Mahasiswa.perkenalan()`

## Uraian

1. Jelaskan perbedaan class dan object dengan analogi Anda sendiri (selain analogi yang ada di buku ini).
2. Jelaskan perbedaan instance attribute dan class attribute. Berikan masing-masing dua contoh attribute yang tepat untuk class `Produk` di sebuah toko.
3. Mengapa sebaiknya semua attribute didefinisikan di dalam `__init__`? Apa risikonya jika attribute ditambahkan dari luar class?

4. Seorang mahasiswa menulis `def hitung_luas():` di dalam sebuah class lalu memanggilnya dengan `objek.hitung_luas()`. Program gagal berjalan. Jelaskan penyebabnya dan cara memperbaikinya.
5. Dengan teknik kata benda dan kata kerja, identifikasi class, attribute, dan method dari deskripsi berikut: "Perpustakaan menyimpan data buku berupa judul, penulis, dan status ketersediaan. Buku dapat dipinjam dan dikembalikan."

## 1.13 Praktik Mandiri

Kerjakan praktik berikut secara berurutan. Simpan setiap praktik dalam file `.py` terpisah, jalankan, dan pastikan keluarannya sesuai dengan contoh.

### Praktik 1: Class Buku

Buat class `Buku` dengan ketentuan:

- Instance attribute: `judul`, `penulis`, `tahun`, dan `tersedia` (nilai awal `True`).
- Method `pinjam()`: jika buku tersedia, ubah `tersedia` menjadi `False` dan cetak pesan berhasil; jika tidak, cetak bahwa buku sedang dipinjam.
- Method `kembalikan()`: ubah `tersedia` menjadi `True`.
- Method `info()`: mengembalikan string berisi judul, penulis, tahun, dan status.

**Contoh keluaran Praktik 1**

```
Buku "Laskar Pelangi" berhasil dipinjam.
Buku "Laskar Pelangi" sedang dipinjam.
Laskar Pelangi - Andrea Hirata (2005) [dipinjam]
Laskar Pelangi - Andrea Hirata (2005) [tersedia]
```

### Praktik 2: Nilai Mahasiswa

Buat class `Mahasiswa` dengan ketentuan:

- Class attribute `jumlah_mahasiswa` yang bertambah satu setiap object baru dibuat.
- Instance attribute `nama`, `nim`, dan `daftar_nilai` (list kosong).
- Method `tambah_nilai(nilai)`: menambahkan nilai ke list hanya jika nilainya di antara 0 dan 100.
- Method `rata_rata()`: mengembalikan rata-rata nilai, atau 0 jika belum ada nilai.
- Method `predikat()`: mengembalikan `"A"` jika rata-rata $\geq 85$, `"B"` jika $\geq 70$, `"C"` jika $\geq 55$, selain itu `"D"`. Gunakan `rata_rata()` di dalamnya.

Buat minimal tiga object, isi nilainya, lalu tampilkan rata-rata, predikat, dan jumlah mahasiswa. Pastikan list nilai antarmahasiswa tidak tercampur.

Contoh keluaran Praktik 2

```
Ani   (001) rata-rata 88.33 predikat A
Budi  (002) rata-rata 72.50 predikat B
Citra (003) rata-rata 0.00 predikat D
Jumlah mahasiswa: 3
```

### Praktik 3: Pemodelan Mandiri

Pilih **salah satu** studi kasus berikut.

1. **Parkir Kampus.** Area parkir memiliki nama dan kapasitas. Kendaraan dapat masuk selama area belum penuh dan dapat keluar. Petugas dapat melihat jumlah slot yang masih kosong.
2. **Kantin.** Sebuah menu memiliki nama, harga, dan stok. Menu dapat dipesan dalam jumlah tertentu selama stok mencukupi, stok dapat ditambah, dan harga total pesanan dapat dihitung.
3. **Daftar Tugas.** Sebuah tugas memiliki judul, mata kuliah, tenggat, dan status selesai. Tugas dapat ditandai selesai dan informasinya dapat ditampilkan.

Untuk studi kasus yang dipilih, kerjakan langkah berikut:

#### Langkah Pengerjaan

1. Buat tabel analisis kata benda dan kata kerja seperti pada Subbab Pemodelan Objek.
2. Gambarkan class diagram sederhana (boleh di kertas) berisi nama class, attribute, dan method.
3. Implementasikan class tersebut di Python, termasuk validasi aturan yang disebutkan.
4. Buat minimal dua object dan uji semua method, termasuk kondisi gagal (misalnya parkir penuh atau stok habis).

KRITERIA PENILAIAN

- Ketepatan identifikasi class, attribute, dan method.
- Semua attribute didefinisikan di `__init__` dan penamaan sesuai konvensi.
- Validasi aturan berada di dalam method.
- Program berjalan tanpa error dan keluaran mudah dibaca. </invoke>

Bab 02

# Visibility dan Encapsulation

*Mengatur siapa yang boleh melihat dan mengubah data di dalam objek.*

02

DI BAB INI

- Visibility property
- Encapsulation
- Public, protected, private
- Name mangling
- Validasi lewat property

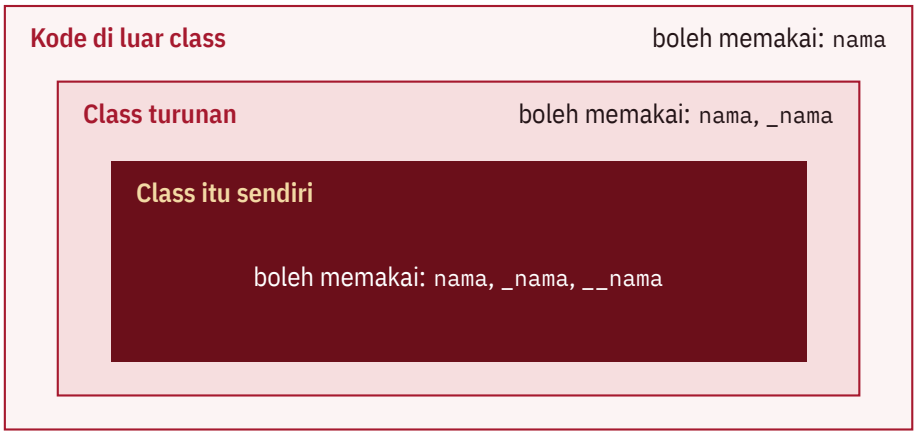
Di akhir Bab 1 kita membuat class `Rekening` yang menolak penarikan jika saldo tidak mencukupi. Aturan itu dijaga oleh method `tarik()`. Namun ada celah besar: siapa pun tetap bisa menulis `rek.saldo = -999` dari luar class, dan aturan yang sudah kita buat dengan susah payah terlewati begitu saja.

Bab ini membahas cara menutup celah tersebut. Kita akan mengenal *visibility*, yaitu tingkat keterlihatan attribute dan method, lalu menerapkan prinsip *encapsulation* agar data di dalam object hanya bisa diubah melalui jalur yang sudah kita kendalikan. Di akhir bab, kita memakai fitur *property* milik Python untuk memvalidasi data dengan sintaks yang tetap ringkas.

## 2.1 Visibility Property

*Visibility* (keterlihatan) menentukan **dari mana** sebuah attribute atau method boleh diakses. Ada tiga "wilayah" yang biasanya dibedakan:

- **di dalam class itu sendiri**, yaitu oleh method-method milik class;
- **di dalam class turunan** (*subclass*), yang akan dibahas di Bab 4;
- **di luar class**, yaitu kode lain yang memakai object tersebut.




**Gambar 2.1.** Tiga wilayah akses. Semakin ke dalam, semakin banyak nama yang boleh dipakai.

Bahasa seperti Java dan C++ memakai kata kunci khusus (`public`, `protected`, `private`) dan kompilatornya benar-benar menolak akses yang melanggar. Python mengambil pendekatan berbeda. Python tidak punya kata kunci visibility; yang ada adalah **konvensi penamaan** berupa garis bawah di depan nama.

**Tabel 2.1.** Visibility di Java/C++ dan di Python.

| Tingkat | Java/C++ | Python |
|---|---|---|
| Public | kata kunci `public` | nama biasa: `saldo` |
| Protected | kata kunci `protected` | satu garis bawah: `_saldo` |
| Private | kata kunci `private` | dua garis bawah: `__saldo` |
| Penegakan | dipaksa oleh kompilator | sebagian besar berupa kesepakatan antarprogrammer |

**"WE ARE ALL CONSENTING ADULTS"**

Komunitas Python punya semboyan "kita semua orang dewasa yang saling sepakat". Artinya, Python memercayai programmer untuk menghormati tanda garis bawah, alih-alih memasang pagar yang tidak bisa ditembus. Tanda _ dan __ adalah pesan: *"ini urusan internal, jangan disentuh dari luar"*.

## 2.2 Konsep Encapsulation

*Encapsulation* (enkapsulasi) adalah salah satu pilar utama OOP. Konsep ini memiliki dua sisi:

1. **Pembungkusan**: data (attribute) dan perilaku yang mengolahnya (method) disatukan di dalam satu class.
2. **Penyembunyian informasi** (*information hiding*): detail internal class disembunyikan, dan dunia luar hanya berinteraksi melalui antarmuka (*interface*) yang disediakan, yaitu method-method publik.

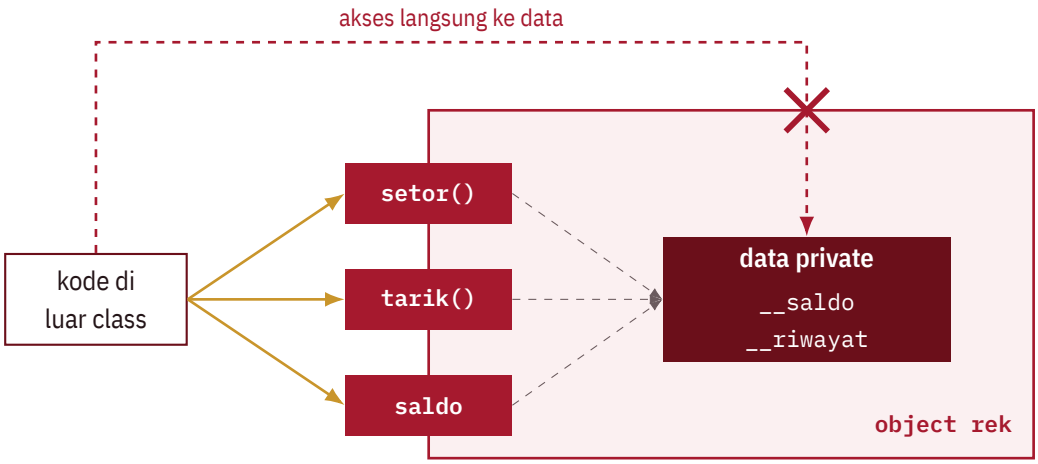
Bayangkan mesin ATM. Kita bisa menarik uang, mengecek saldo, dan mentransfer melalui tombol-tombol yang tersedia. Kita tidak bisa membuka mesin lalu mengambil uang langsung dari brankasnya. Tombol-tombol itu adalah antarmuka publik; brankas dan mekanisme di dalamnya adalah detail internal yang disembunyikan. Setiap transaksi wajib melewati tombol, sehingga aturan seperti "saldo harus cukup" selalu diperiksa.

### Implementasi Encapsulation

Rekening tanpa encapsulation

```
class Rekening:
    def __init__(self, pemilik, saldo=0):
        self.pemilik = pemilik
        self.saldo = saldo

```




**Gambar 2.2.** Encapsulation: kode luar hanya berinteraksi lewat antarmuka publik (merah) yang memeriksa aturan; akses langsung ke data private ditolak.

```
    def tarik(self, jumlah):
        if jumlah > self.saldo:
            print("Saldo tidak mencukupi.")
        else:
            self.saldo -= jumlah

rek = Rekening("Ani", 100000)
rek.tarik(500000)      # ditolak oleh method
rek.saldo = -999       # ...tetapi saldo bisa diubah langsung
rek.saldo -= 500000    # dan aturan dilewati begitu saja
print(rek.saldo)
```

**Keluaran**

```
Saldo tidak mencukupi.
-500999
```

Method `tarik()` sudah bekerja dengan benar, tetapi tidak ada gunanya jika data bisa diubah tanpa melewatinya. Encapsulation memastikan perubahan data *hanya* terjadi melalui method yang sudah memeriksa aturan.

## Manfaat Encapsulation

- **Integritas data**: keadaan object selalu valid karena setiap perubahan diperiksa.
- **Kemudahan perubahan**: cara penyimpanan data di dalam class boleh diganti tanpa mengubah kode lain yang memakai class tersebut, selama antarmuka publiknya tetap sama.
- **Kemudahan pemakaian**: pengguna class cukup mengetahui method publik, tanpa perlu memahami detail di dalamnya.

- **Kemudahan pelacakan kesalahan**: jika data menjadi salah, sumbernya hanya bisa berasal dari sedikit method, bukan dari seluruh program.

## 2.3 Public, Protected, dan Private secara Konseptual

Python mengenal tiga tingkat visibility yang dibedakan dari awalan nama.

### Public

Nama tanpa awalan garis bawah, misalnya `username`, bersifat *public*: boleh dibaca, diubah, dan dipanggil dari mana saja. Semua attribute dan method yang kita tulis di Bab 1 bersifat public.

### Protected

Nama dengan satu garis bawah, misalnya `_email`, bersifat *protected*. Maknanya: "attribute ini untuk keperluan internal class dan turunannya". Python **tidak** melarang akses dari luar; tanda ini murni kesepakatan. Editor dan alat pemeriksa kode (*linter*) biasanya memberi peringatan jika kita mengaksesnya dari luar class.

### Private

Nama dengan dua garis bawah di depan, misalnya `__password`, bersifat *private*: hanya untuk dipakai di dalam class itu sendiri. Berbeda dengan protected, Python benar-benar mempersulit akses dari luar melalui mekanisme *name mangling* (dibahas di subbab berikutnya), sehingga akses biasa akan gagal.

Tiga tingkat visibility

```python
class Akun:
    def __init__(self, username, email, password):
        self.username = username       # public
        self._email = email            # protected
        self.__password = password     # private

    def cek_password(self, teks):
        # di dalam class, attribute private boleh diakses
        return teks == self.__password

akun = Akun("ani24", "ani@mail.com", "rahasia123")
print(akun.username)
print(akun._email)                     # bisa, tetapi melanggar kesepakatan
print(akun.cek_password("rahasia123"))
print(akun.__password)                 # gagal
```

Keluaran

```
ani24
ani@mail.com
True
Traceback (most recent call last):
  ...
AttributeError: 'Akun' object has no attribute '__password'. Did you mean: '
    cek_password'?
```

Aturan yang sama berlaku untuk method. Method pembantu yang hanya dipakai di dalam class sebaiknya diberi awalan garis bawah, misalnya `_format_tanggal()` atau `__enkripsi()`.

**Tabel 2.2.** Ringkasan tingkat visibility di Python.

| Tingkat | Contoh nama | Dalam class | Dari luar | Kapan dipakai |
|---|---|---|---|---|
| Public | `nama` | boleh | boleh | antarmuka yang memang disediakan untuk pengguna class |
| Protected | `_nama` | boleh | bisa, tetapi tidak dianjurkan | detail internal yang mungkin dipakai class turunan |
| Private | `__nama` | boleh | gagal dengan cara biasa | detail internal yang benar-benar milik class itu saja |

**PROTECTED ATAU PRIVATE?**

Dalam praktik sehari-hari, banyak programmer Python lebih sering memakai satu garis bawah (`_nama`) untuk menandai data internal, dan memakai dua garis bawah hanya bila perlu menghindari bentrok nama dengan class turunan. Di buku ini kita memakai `__nama` untuk data yang wajib dilindungi agar efek name mangling terlihat jelas saat belajar.

## 2.4 Name Mangling

Mengapa `akun.__password` gagal diakses padahal attribute itu jelas dibuat di `__init__`? Jawabannya adalah *name mangling*: setiap nama di dalam class yang diawali dua garis bawah (dan tidak diakhiri dua garis bawah) otomatis diubah Python menjadi `_NamaClass__nama`.

**Melihat hasil name mangling**

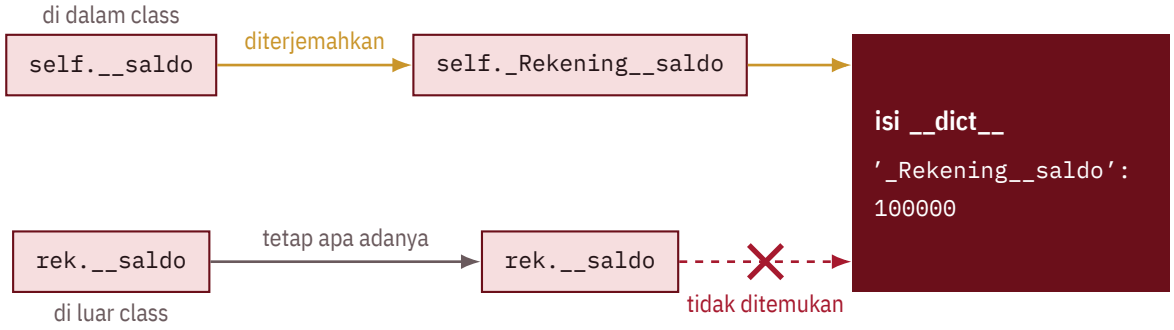
```
class Rekening:
    def __init__(self, saldo):
        self.__saldo = saldo

rek = Rekening(100000)
print(rek.__dict__)
print(rek._Rekening__saldo)     # nama aslinya setelah di-mangle
```

**Keluaran**

```
{'_Rekening__saldo': 100000}
100000
```

Attribute `__saldo` sebenarnya tersimpan dengan nama `_Rekening__saldo`. Di dalam class, Python menerjemahkan `self.__saldo` ke nama tersebut secara otomatis, sehingga method-method class tetap bisa memakainya dengan normal. Dari luar class, penerjemahan itu tidak terjadi, sehingga `rek.__saldo` tidak ditemukan.




**Gambar 2.3.** Name mangling hanya terjadi untuk kode yang ditulis di dalam class.

**NAME MANGLING BUKAN FITUR KEAMANAN**

Seperti terlihat di atas, data private masih bisa dibaca lewat `rek._Rekening__saldo`. Name mangling hanya mencegah akses *tidak sengaja*, bukan akses yang disengaja. Jangan pernah mengandalkannya untuk menyembunyikan data rahasia seperti password; data semacam itu harus dilindungi dengan cara lain, misalnya enkripsi.

### Jebakan: Menulis `__nama` dari Luar Class

Karena penerjemahan hanya terjadi di dalam class, memberi nilai ke `rek.__saldo` dari luar **tidak** mengubah saldo. Python justru membuat attribute baru bernama `__saldo` apa adanya.

Attribute baru, bukan mengubah yang private

```
rek.__saldo = 0
print(rek.__dict__)
```

Keluaran

```
{'_Rekening__saldo': 100000, '__saldo': 0}
```

### Tujuan Sebenarnya: Mencegah Bentrok Nama

Name mangling dirancang untuk melindungi attribute sebuah class agar tidak tertimpa secara tidak sengaja oleh class turunannya. Contoh berikut memakai pewarisan yang baru akan dibahas tuntas di Bab 4; cukup perhatikan bahwa class `Anak` dibangun di atas class `Induk`.

Dua attribute private dengan nama sama

```
class Induk:
    def __init__(self):
        self.__kode = "INDUK"

    def kode_induk(self):
        return self.__kode

class Anak(Induk):
    def __init__(self):
        super().__init__()
        self.__kode = "ANAK"

    def kode_anak(self):
        return self.__kode

a = Anak()
print(a.kode_induk(), a.kode_anak())
print(a.__dict__)
```

Keluaran

```
INDUK ANAK
{'_Induk__kode': 'INDUK', '_Anak__kode': 'ANAK'}
```

Karena di-mangle menjadi `_Induk__kode` dan `_Anak__kode`, kedua attribute hidup berdampingan tanpa saling menimpa. Seandainya kita memakai satu garis bawah (`_kode`), nilai milik `Induk` akan tertimpa menjadi `"ANAK"`.

YANG TIDAK DI-MANGLE

Nama yang diawali *dan* diakhiri dua garis bawah, seperti `__init__` dan `__str__`, tidak terkena name mangling. Nama-nama ini adalah *dunder method* milik Python dan memang harus bisa dipanggil dengan nama aslinya.

## 2.5 Pengendalian Akses Attribute

Setelah attribute dijadikan private, dunia luar tidak bisa lagi membacanya maupun mengubahnya secara langsung. Kita perlu menyediakan jalur resmi. Cara klasik yang juga dipakai di banyak bahasa lain adalah *getter* dan *setter*:

- **Getter**: method untuk membaca nilai, biasanya bernama `get_nama()`.
- **Setter**: method untuk mengubah nilai sekaligus memvalidasinya, biasanya bernama `set_nama(nilai)`.

Dengan mengatur getter dan setter mana yang disediakan, kita bisa menentukan pola akses setiap attribute.

**Tabel 2.3.** Pola pengendalian akses attribute.

| Pola | Yang disediakan | Contoh |
|---|---|---|
| Baca-tulis | getter dan setter dengan validasi | nama pemilik rekening |
| Hanya-baca (*read-only*) | getter saja | nomor rekening, NIM |
| Hanya-tulis (*write-only*) | setter saja | password |
| Diubah lewat aksi | getter dan method aksi, tanpa setter | saldo yang hanya berubah lewat `setor()` dan `tarik()` |

Rekening dengan getter dan setter

```
class Rekening:
    def __init__(self, pemilik, saldo_awal=0):
        if saldo_awal < 0:
            raise ValueError("Saldo awal tidak boleh negatif.")
        self.__pemilik = pemilik
        self.__saldo = saldo_awal
        self.__riwayat = []

    # getter
    def get_pemilik(self):
```

```
        return self.__pemilik

    def get_saldo(self):
        return self.__saldo

    def get_riwayat(self):
        return list(self.__riwayat)    # kirim salinan, bukan list aslinya

    # setter dengan validasi
    def set_pemilik(self, nama):
        if not nama.strip():
            raise ValueError("Nama pemilik tidak boleh kosong.")
        self.__pemilik = nama.strip()

    # method aksi: satu-satunya jalan untuk mengubah saldo
    def setor(self, jumlah):
        if jumlah <= 0:
            raise ValueError("Setoran harus lebih dari nol.")
        self.__saldo += jumlah
        self.__catat("setor", jumlah)

    def tarik(self, jumlah):
        if jumlah <= 0:
            raise ValueError("Penarikan harus lebih dari nol.")
        if jumlah > self.__saldo:
            raise ValueError("Saldo tidak mencukupi.")
        self.__saldo -= jumlah
        self.__catat("tarik", jumlah)

    # method private: hanya dipakai di dalam class
    def __catat(self, jenis, jumlah):
        self.__riwayat.append((jenis, jumlah))


rek = Rekening("Ani", 100000)
rek.setor(50000)
rek.tarik(30000)
rek.set_pemilik("Ani Lestari")
print(rek.get_pemilik(), rek.get_saldo())
print(rek.get_riwayat())
rek.tarik(999999)
```

**Keluaran**

```
Ani Lestari 120000
[('setor', 50000), ('tarik', 30000)]
Traceback (most recent call last):
```

```
  ...
ValueError: Saldo tidak mencukupi.
```

Perhatikan beberapa keputusan desain pada class di atas:

- Tidak ada `set_saldo()`. Saldo hanya boleh berubah lewat `setor()` dan `tarik()`, karena keduanya memeriksa aturan dan mencatat riwayat.
- `get_riwayat()` mengembalikan *salinan* list. Jika yang dikembalikan list aslinya, kode di luar bisa menambah atau menghapus isi riwayat dan encapsulation pun bocor.
- `__catat()` adalah method private karena pencatatan riwayat adalah detail internal, bukan layanan untuk pengguna class.

**TENTANG** RAISE VALUEERROR

Pada bab ini, data yang tidak valid ditolak dengan `raise ValueError("pesan")`. Perintah ini menghentikan program dan menampilkan pesan kesalahan, cara yang jauh lebih aman daripada diam-diam menerima data yang salah atau hanya mencetak pesan. Cara menangkap kesalahan semacam ini agar program tidak berhenti dibahas di Bab 8, *Exception Handling*.

## 2.6 Validasi Data melalui Property

Getter dan setter bekerja dengan baik, tetapi penulisannya panjang. Bandingkan dua baris berikut yang maksudnya sama, yaitu mengubah nama pemilik menjadi huruf kapital.

**Getter/setter vs sintaks attribute**

```
1  rek.set_pemilik(rek.get_pemilik().upper())   # getter/setter: sulit dibaca
2  rek.pemilik = rek.pemilik.upper()            # sintaks attribute: ringkas
```

Python menyediakan *property* untuk mendapatkan keduanya sekaligus: sintaks attribute biasa di luar, tetapi tetap dijaga oleh method di dalam.

### Decorator `@property` dan `@nama.setter`

Property dibuat dengan dua decorator. Decorator adalah penanda berawalan `@` yang ditulis di atas method; cara kerjanya akan dibahas di Bab 3. Untuk saat ini, cukup ikuti polanya.

**Membuat Property**

1. Simpan nilai sebenarnya di attribute private, misalnya `self.__ipk`.
2. Tulis method getter bernama `ipk` dan beri decorator `@property`.

3. Tulis method setter dengan nama yang **sama**, `ipk`, dan beri decorator `@ipk.setter`. Letakkan validasi di sini.

4. Di `__init__`, isi nilai lewat property (`self.ipk = ipk`), bukan langsung ke attribute private, agar validasi juga berlaku saat object dibuat.

**Property dengan validasi**

```python
class Mahasiswa:
    def __init__(self, nama, ipk):
        self.nama = nama          # melewati setter nama
        self.ipk = ipk            # melewati setter ipk

    @property
    def nama(self):
        return self.__nama

    @nama.setter
    def nama(self, nilai):
        nilai = nilai.strip()
        if not nilai:
            raise ValueError("Nama tidak boleh kosong.")
        self.__nama = nilai.title()

    @property
    def ipk(self):
        return self.__ipk

    @ipk.setter
    def ipk(self, nilai):
        if not 0 <= nilai <= 4:
            raise ValueError("IPK harus di antara 0 dan 4.")
        self.__ipk = nilai

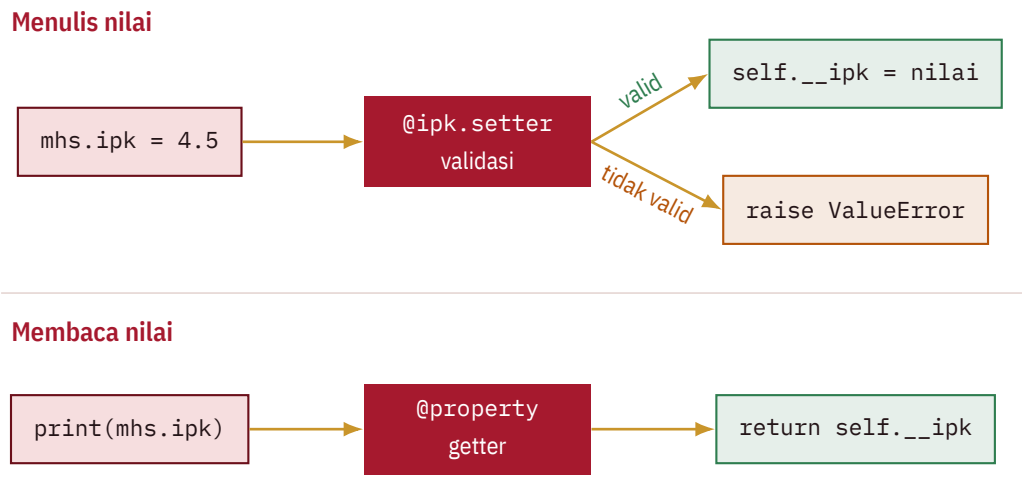
mhs = Mahasiswa("  ani lestari ", 3.5)
print(mhs.nama, mhs.ipk)     # dibaca seperti attribute biasa
mhs.ipk = 3.75               # diubah seperti attribute biasa
print(mhs.ipk)
mhs.ipk = 4.5                # ditolak oleh setter
```

**Keluaran**

```
Ani Lestari 3.5
3.75
Traceback (most recent call last):
  ...
```

```
ValueError: IPK harus di antara 0 dan 4.
```

Dari luar, `mhs.ipk` tampak seperti attribute biasa. Namun setiap kali kita menulis `mhs.ipk = …`, Python sebenarnya memanggil method setter, sehingga nilai yang tidak valid langsung ditolak. Setter juga bisa *merapikan* data, seperti menghapus spasi berlebih dan mengubah kapitalisasi nama.




**Gambar 2.4.** Di balik sintaks attribute biasa, property selalu melewatkan penulisan ke setter dan pembacaan ke getter.

**JANGAN LANGSUNG KE ATTRIBUTE PRIVATE DI `__init__`**

Jika di `__init__` kita menulis `self.__ipk = ipk`, validasi di setter dilewati dan object bisa lahir dengan IPK 9.0. Selalu isi nilai awal lewat property: `self.ipk = ipk`.

## Property Hanya-Baca dan Property Terhitung

Property yang hanya memiliki getter (tanpa setter) bersifat *read-only*. Property juga bisa *terhitung* (*computed*), yaitu nilainya tidak disimpan, melainkan dihitung dari attribute lain setiap kali dibaca.

**Property terhitung**

```
class Suhu:
    NOL_MUTLAK = -273.15

    def __init__(self, celsius):
        self.celsius = celsius

    @property
    def celsius(self):
        return self.__celsius

```

```
    @celsius.setter
    def celsius(self, nilai):
        if nilai < Suhu.NOL_MUTLAK:
            raise ValueError("Suhu tidak boleh di bawah nol mutlak.")
        self.__celsius = nilai

    @property
    def fahrenheit(self):              # terhitung dan hanya-baca
        return self.__celsius * 9 / 5 + 32

    @property
    def kelvin(self):                  # terhitung dan hanya-baca
        return self.__celsius - Suhu.NOL_MUTLAK

s = Suhu(30)
print(s.fahrenheit, s.kelvin)
s.celsius = 100
print(s.fahrenheit)                    # ikut berubah otomatis
s.fahrenheit = 50                      # tidak ada setter
```

**Keluaran**

```
86.0 303.15
212.0
Traceback (most recent call last):
  ...
AttributeError: property 'fahrenheit' of 'Suhu' object has no setter
```

Pada contoh ini, nilai `fahrenheit` dan `kelvin` dihitung dari `celsius` terbaru setiap kali dibaca dan tidak dapat diubah secara langsung.

### Deleter dengan `@nama.deleter`

Selain getter dan setter, property dapat memiliki *deleter* yang dijalankan saat kita memakai perintah `del`. Deleter jarang dipakai, tetapi berguna jika penghapusan perlu disertai tindakan lain, misalnya mencatat log.

**Deleter**

```
class Profil:
    def __init__(self, foto):
        self.__foto = foto

    @property
    def foto(self):
        return self.__foto

```

```
    @foto.deleter
    def foto(self):
        print(f"Menghapus foto {self.__foto}")
        self.__foto = "default.png"

p = Profil("ani.jpg")
del p.foto
print(p.foto)
```

**Keluaran**

```
Menghapus foto ani.jpg
default.png
```

**Tabel 2.4.** Getter/setter klasik dibandingkan property.

| Aspek | Getter/setter klasik | Property |
|---|---|---|
| Membaca | `rek.get_pemilik()` | `rek.pemilik` |
| Mengubah | `rek.set_pemilik("Ani")` | `rek.pemilik = "Ani"` |
| Validasi | di method setter | di method `@pemilik.setter` |
| Hanya-baca | tidak menyediakan setter | tidak menulis `@nama.setter` |
| Gaya | umum di Java dan C++ | gaya yang dianjurkan di Python (*Pythonic*) |

## Studi Kasus: Rekening dengan Property

Kita gabungkan semua yang sudah dipelajari untuk menyempurnakan class `Rekening`: nomor rekening hanya-baca, nama pemilik bisa diubah dengan validasi, dan saldo hanya-baca karena perubahannya wajib lewat `setor()` dan `tarik()`.

**rekening.py**

```
class Rekening:
    def __init__(self, nomor, pemilik, saldo_awal=0):
        if saldo_awal < 0:
            raise ValueError("Saldo awal tidak boleh negatif.")
        self.__nomor = nomor
        self.pemilik = pemilik            # lewat setter
        self.__saldo = saldo_awal

    @property
    def nomor(self):                      # hanya-baca
        return self.__nomor
```

```
12
13      @property
14      def pemilik(self):
15          return self.__pemilik
16
17      @pemilik.setter
18      def pemilik(self, nama):
19          nama = nama.strip()
20          if len(nama) < 3:
21              raise ValueError("Nama pemilik minimal 3 huruf.")
22          self.__pemilik = nama.title()
23
24      @property
25      def saldo(self):                        # hanya-baca
26          return self.__saldo
27
28      def setor(self, jumlah):
29          if jumlah <= 0:
30              raise ValueError("Setoran harus lebih dari nol.")
31          self.__saldo += jumlah
32
33      def tarik(self, jumlah):
34          if jumlah <= 0:
35              raise ValueError("Penarikan harus lebih dari nol.")
36          if jumlah > self.__saldo:
37              raise ValueError("Saldo tidak mencukupi.")
38          self.__saldo -= jumlah
39
40      def __str__(self):
41          return f"[{self.nomor}] {self.pemilik} - saldo Rp{self.saldo:,}"
42
43
44  rek = Rekening("KM-001", "ani lestari", 100000)
45  rek.setor(25000)
46  rek.tarik(50000)
47  print(rek)
48  rek.saldo = -999
```

**Keluaran**

```
[KM-001] Ani Lestari - saldo Rp75,000
Traceback (most recent call last):
  ...
AttributeError: property 'saldo' of 'Rekening' object has no setter
```

Celah yang kita temukan di awal bab kini tertutup. Saldo tetap mudah dibaca dengan `rek.saldo`, tetapi satu-satunya cara mengubahnya adalah melalui method yang sudah

memeriksa aturan.

## 2.7 Rangkuman

- **Visibility** menentukan dari mana attribute dan method boleh diakses. Python memakai konvensi garis bawah, bukan kata kunci.
- **Encapsulation** membungkus data dan perilaku dalam satu class sekaligus menyembunyikan detail internal di balik antarmuka publik.
- **Public** (`nama`) bebas diakses; **protected** (`_nama`) adalah kesepakatan untuk penggunaan internal dan class turunan; **private** (`__nama`) hanya untuk class itu sendiri.
- **Name mangling** mengubah `__nama` menjadi `_NamaClass__nama`. Tujuannya mencegah bentrok nama dengan class turunan, bukan sebagai fitur keamanan.
- Akses attribute private dikendalikan lewat **getter**, **setter**, atau method aksi. Pola akses bisa baca-tulis, hanya-baca, hanya-tulis, atau diubah lewat aksi.
- **Property** (`@property`, `@nama.setter`, `@nama.deleter`) memberi sintaks attribute biasa dengan validasi di dalam method.
- Isi nilai awal di `__init__` lewat property agar validasi juga berlaku saat object dibuat.

## 2.8 Kuis Teoritis

### Pilihan Ganda

Pilihlah satu jawaban yang paling tepat.

1. Dalam konvensi Python, attribute `_harga` (satu garis bawah) menandakan attribute yang bersifat ...

   a. public
   b. protected
   c. private
   d. konstan

2. Manakah yang **bukan** manfaat encapsulation?

   a. Menjaga data object tetap valid
   b. Detail internal class bisa diubah tanpa mengganggu kode pemakainya
   c. Program otomatis berjalan lebih cepat
   d. Sumber kesalahan data lebih mudah dilacak

3. Di dalam class `Produk`, attribute `self.__stok` akan disimpan dengan nama ...

   a. `__stok`

b. `_stok`
c. `__Produk_stok`
d. `_Produk__stok`

**4.** Tujuan utama name mangling adalah ...

a. mengenkripsi data rahasia
b. mencegah bentrok nama attribute dengan class turunan
c. mempercepat akses attribute
d. membuat attribute menjadi konstan

**5.** Perhatikan kode berikut.

Soal 5

```
class A:
    def __init__(self):
        self.__x = 1
    def get_x(self):
        return self.__x

a = A()
a.__x = 99
print(a.get_x())
```

Keluaran kode tersebut adalah ...

a. `1`
b. `99`
c. `AttributeError`
d. `None`

**6.** Decorator yang dipakai untuk membuat getter pada property adalah ...

a. `@getter`
b. `@property`
c. `@attribute`
d. `@nama.getter`

**7.** Sebuah class memiliki `@property def luas(self)` tanpa setter. Apa yang terjadi saat kita menjalankan `objek.luas = 10`?

a. Nilai `luas` berubah menjadi 10
b. Dibuat attribute baru bernama `luas`
c. Terjadi `AttributeError`
d. Perintah diabaikan tanpa pesan

**8.** Perhatikan kode berikut.

Soal 8

```python
class Barang:
    def __init__(self, harga):
        self.__harga = harga

    @property
    def harga(self):
        return self.__harga

    @harga.setter
    def harga(self, nilai):
        if nilai < 0:
            raise ValueError("Harga negatif")
        self.__harga = nilai

b = Barang(-500)
print(b.harga)
```

Keluaran kode tersebut adalah ...

a. `ValueError: Harga negatif`
b. `-500`
c. `0`
d. `AttributeError`

9. Nama berikut yang **tidak** terkena name mangling adalah ...

a. `__saldo`
b. `__hitung()`
c. `__str__`
d. `__data_rahasia`

10. Sebuah method `get_daftar()` mengembalikan `self.__daftar` (list asli). Masalah yang mungkin muncul adalah ...

a. list tidak bisa dibaca dari luar
b. kode di luar bisa mengubah isi list tanpa melewati validasi class
c. Python menolak mengembalikan attribute private
d. list otomatis dikosongkan

## Uraian

1. Jelaskan dua sisi dari konsep encapsulation dan berikan satu analogi dari kehidupan sehari-hari selain mesin ATM.

2. Jelaskan perbedaan protected dan private di Python. Mengapa keduanya disebut "konseptual" dan bukan pembatasan mutlak seperti di Java?

3. Seorang programmer mengaku "sudah mengamankan password" dengan menyimpannya di attribute `__password`. Apakah klaim itu benar? Jelaskan dengan mengacu pada name mangling.
4. Jelaskan mengapa nilai awal di `__init__` sebaiknya diisi lewat property (`self.ipk = ipk`) dan bukan langsung ke attribute private (`self.__ipk = ipk`).
5. Untuk class `Mahasiswa` dengan attribute NIM, nama, dan password, tentukan pola akses yang tepat (baca-tulis, hanya-baca, hanya-tulis, atau lewat aksi) untuk masing-masing attribute beserta alasannya.

## 2.9 Praktik Mandiri

Kerjakan praktik berikut secara berurutan. Simpan setiap praktik dalam file `.py` terpisah, jalankan, dan pastikan keluarannya sesuai dengan contoh.

### Praktik 1: Produk Toko

Buat class `Produk` dengan ketentuan:

- Property `kode` bersifat hanya-baca.
- Property `nama`: tidak boleh kosong; spasi di awal dan akhir dihapus.
- Property `harga`: harus lebih dari 0.
- Property `stok`: tidak boleh negatif.
- Property terhitung `nilai_persediaan` yang mengembalikan `harga * stok`.
- Method `jual(jumlah)` yang mengurangi stok jika mencukupi, dan menolak dengan `ValueError` jika tidak.
- Semua nilai awal di `__init__` wajib diisi lewat property.

**Contoh keluaran Praktik 1**

```
P-01 Buku Tulis: 40 x Rp5,000 = Rp200,000
Setelah menjual 15: stok 25, nilai Rp125,000
ValueError: Stok tidak mencukupi.
```

### Praktik 2: Akun Pengguna

Buat class `AkunPengguna` dengan ketentuan:

- Property `username`: minimal 5 karakter dan tidak boleh mengandung spasi; disimpan dalam huruf kecil.
- Password bersifat **hanya-tulis**: sediakan setter untuk menggantinya (minimal 8 karakter dan harus mengandung angka), tetapi getter selalu mengembalikan `"********"`.
- Method `login(password)` yang mengembalikan `True` jika password cocok.

- Attribute private `__gagal_login` yang bertambah setiap login gagal. Setelah 3 kali gagal, akun terkunci dan `login()` selalu mengembalikan `False`.
- Property hanya-baca `terkunci`.

**Contoh keluaran Praktik 2**

```
ani_24 ********
Login benar: True
Login salah 3x -> terkunci: True
Login benar setelah terkunci: False
```

### Praktik 3: Refaktor dengan Encapsulation

Ambil salah satu class yang Anda buat di Praktik Mandiri Bab 1 (`Buku`, `Mahasiswa`, atau hasil Praktik 3), lalu terapkan encapsulation.

#### Langkah Pengerjaan

1. Tentukan pola akses setiap attribute (baca-tulis, hanya-baca, hanya-tulis, atau lewat aksi) dan tuliskan alasannya dalam komentar.
2. Ubah attribute menjadi private, lalu sediakan property atau method aksi sesuai pola yang dipilih.
3. Pindahkan semua validasi ke setter atau method aksi; gunakan `raise ValueError` untuk data tidak valid.
4. Jika ada attribute berupa list, pastikan getter-nya mengembalikan salinan.
5. Uji dengan mencoba mengubah data secara tidak sah dari luar class dan tunjukkan bahwa percobaan tersebut ditolak.

**KRITERIA PENILAIAN**

- Ketepatan pemilihan visibility dan pola akses setiap attribute.
- Validasi berada di setter atau method aksi, dan berlaku sejak object dibuat.
- Tidak ada jalur untuk mengubah data penting tanpa validasi.
- Program berjalan, dan pengujian menunjukkan data tidak valid benar-benar ditolak.

Bab 03

# Class Method, Static Method, dan Decorator

*Method milik class, method yang berdiri sendiri, dan fungsi pembungkus fungsi.*

03

**DI BAB INI**

- Class method
- Static method
- Decorator
- Custom dan chaining decorator
- `*args` dan `**kwargs`

Semua method yang kita tulis di Bab 1 dan Bab 2 adalah *instance method*: method yang bekerja pada satu object tertentu melalui `self`. Namun tidak semua perilaku cocok ditempelkan pada satu object. Menghitung jumlah seluruh mahasiswa adalah urusan *class*, bukan urusan satu mahasiswa. Memeriksa apakah sebuah NIM formatnya benar bahkan tidak membutuhkan object maupun class sama sekali.

Untuk kebutuhan seperti itu Python menyediakan *class method* dan *static method*. Keduanya dibuat dengan menuliskan `@classmethod` atau `@staticmethod` di atas method. Tanda `@` itu, yang juga sudah kita pakai pada `@property` di Bab 2, disebut *decorator*. Pada paruh kedua bab ini kita akan membongkar cara kerja decorator dan membuat decorator sendiri.

## 3.1 Class Method

*Class method* adalah method yang terikat pada **class**, bukan pada object. Class method dibuat dengan decorator `@classmethod`, dan parameter pertamanya bukan `self` melainkan `cls`, yaitu rujukan ke class itu sendiri. Melalui `cls`, class method dapat membaca dan mengubah class attribute, serta membuat object baru dari class tersebut.

**Class method untuk data milik class**

```
class Mahasiswa:
    kampus = "UNESA"
    jumlah = 0

    def __init__(self, nama, nim):
        self.nama = nama
        self.nim = nim
        Mahasiswa.jumlah += 1

    @classmethod
    def info_kelas(cls):
        return f"{cls.__name__} di {cls.kampus}: {cls.jumlah} orang"

    @classmethod
    def ganti_kampus(cls, nama_kampus):
        cls.kampus = nama_kampus

ani = Mahasiswa("Ani", "001")
budi = Mahasiswa("Budi", "002")
```

```
20
21 print(Mahasiswa.info_kelas())          # dipanggil lewat class
22 Mahasiswa.ganti_kampus("Universitas Negeri Surabaya")
23 print(ani.info_kelas())                # lewat object pun, cls tetap class
```

**Keluaran**

```
Mahasiswa di UNESA: 2 orang
Mahasiswa di Universitas Negeri Surabaya: 2 orang
```

Class method lazimnya dipanggil lewat nama class (`Mahasiswa.info_kelas()`), karena memang tidak membutuhkan object. Jika dipanggil lewat object, Python tetap mengirim *class* dari object tersebut sebagai `cls`.

## Alternative Constructor

Kegunaan class method yang paling sering ditemui adalah sebagai *alternative constructor*: cara lain untuk membuat object dari bentuk data yang berbeda. Class hanya boleh punya satu `__init__`, tetapi boleh punya banyak class method yang masing-masing menerima format masukan berbeda, mengolahnya, lalu memanggil `cls(…)`.

**Beberapa cara membuat object Mahasiswa**

```
1  class Mahasiswa:
2      def __init__(self, nama, nim, semester=1):
3          self.nama = nama
4          self.nim = nim
5          self.semester = semester
6
7      @classmethod
8      def dari_teks(cls, teks):
9          nama, nim, semester = teks.split(";")
10         return cls(nama.strip(), nim.strip(), int(semester))
11
12     @classmethod
13     def dari_dict(cls, data):
14         return cls(data["nama"], data["nim"], data.get("semester", 1))
15
16     def __str__(self):
17         return f"{self.nim} {self.nama} (semester {self.semester})"
18
19 a = Mahasiswa("Ani", "001", 3)
20 b = Mahasiswa.dari_teks("Budi; 002; 5")
21 c = Mahasiswa.dari_dict({"nama": "Citra", "nim": "003"})
22
23 for mhs in (a, b, c):
24     print(mhs)
```

Keluaran

```
001 Ani (semester 3)
002 Budi (semester 5)
003 Citra (semester 1)
```

Nama method yang diawali `dari_…` (dalam bahasa Inggris biasanya `from_…`) membuat maksudnya langsung jelas: "buat object Mahasiswa dari teks" atau "dari dictionary".

**ALTERNATIVE CONSTRUCTOR DI PYTHON**

Pola ini banyak dipakai di pustaka bawaan Python. Contohnya `dict.fromkeys(["a", "b"], 0)` membuat dictionary dari daftar kunci, `int.from_bytes(…)` membuat bilangan dari deretan byte, dan `date.fromisoformat("2026-10-16")` dari modul `datetime` membuat tanggal dari teks.

### Mengapa `cls` dan Bukan Nama Class?

Di dalam `dari_teks()` kita menulis `cls(…)`, bukan `Mahasiswa(…)`. Perbedaannya baru terasa saat ada class turunan (Bab 4). Dengan `cls`, class method yang diwarisi akan membuat object dari class turunan tersebut, bukan dari class induknya.

`cls mengikuti class yang memanggil`

```
class MahasiswaPascasarjana(Mahasiswa):
    pass

d = MahasiswaPascasarjana.dari_teks("Dodi; 004; 1")
print(type(d).__name__)
```

Keluaran

```
MahasiswaPascasarjana
```

**GUNAKAN `cls`, BUKAN NAMA CLASS**

Di dalam class method, selalu rujuk class melalui `cls`. Menuliskan nama class secara langsung, misalnya `return Mahasiswa(…)`, membuat class turunan diam-diam menghasilkan object dari class yang salah.

## 3.2 Static Method

*Static method* adalah method yang **tidak menerima** `self` maupun `cls`. Static method dibuat dengan decorator `@staticmethod` dan pada dasarnya adalah fungsi biasa yang "dititipkan" di dalam class karena maknanya berkaitan erat dengan class tersebut. Static method tidak bisa membaca atau mengubah attribute apa pun; ia hanya bekerja dengan

parameter yang diberikan kepadanya.

Contoh yang pas adalah fungsi pemeriksa format. Memeriksa apakah sebuah teks merupakan NIM yang valid jelas berkaitan dengan class `Mahasiswa`, tetapi tidak membutuhkan data mahasiswa mana pun.

**Static method sebagai pemeriksa**

```
class Mahasiswa:
    def __init__(self, nama, nim):
        self.nama = nama
        self.nim = nim                  # lewat setter

    @property
    def nim(self):
        return self.__nim

    @nim.setter
    def nim(self, nilai):
        if not Mahasiswa.nim_valid(nilai):
            raise ValueError(f"NIM tidak valid: {nilai}")
        self.__nim = nilai

    @staticmethod
    def nim_valid(nim):
        return nim.isdigit() and len(nim) == 11

print(Mahasiswa.nim_valid("24051204084"))   # tanpa membuat object
print(Mahasiswa.nim_valid("2405-12"))
mhs = Mahasiswa("Ani", "24051204084")
print(mhs.nim)
Mahasiswa("Budi", "123")
```

**Keluaran**

```
True
False
24051204084
Traceback (most recent call last):
  ...
ValueError: NIM tidak valid: 123
```

Karena tidak bergantung pada object, `nim_valid()` bisa dipakai di mana saja: oleh setter di dalam class, atau oleh kode lain yang ingin memeriksa NIM sebelum membuat object, misalnya form pendaftaran.

Static method juga sering dipakai untuk mengelompokkan fungsi-fungsi utilitas yang saling berkaitan di bawah satu nama.

Mengelompokkan fungsi utilitas

```python
class KonversiSuhu:
    @staticmethod
    def c_ke_f(celsius):
        return celsius * 9 / 5 + 32

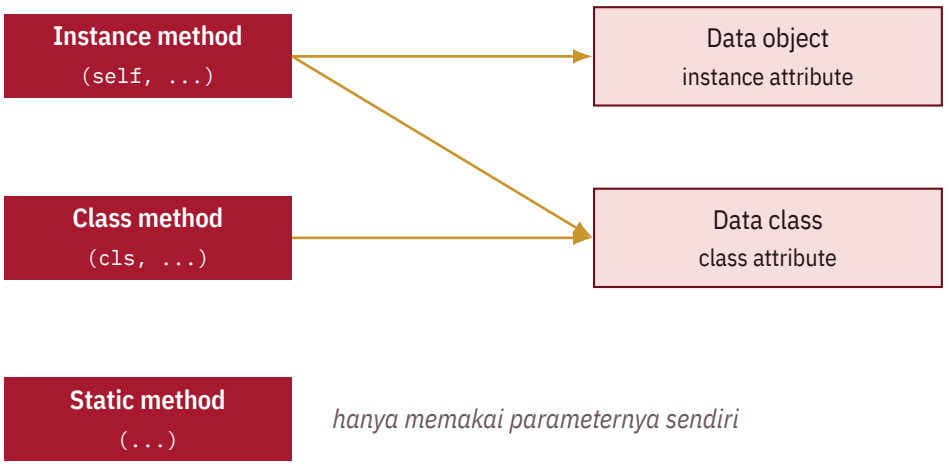
    @staticmethod
    def f_ke_c(fahrenheit):
        return (fahrenheit - 32) * 5 / 9

print(KonversiSuhu.c_ke_f(100))
print(KonversiSuhu.f_ke_c(212))
```

Keluaran

```
212.0
100.0
```

## Memilih Jenis Method




**Gambar 3.1.** Apa yang dapat diakses oleh tiap jenis method.

PERTANYAAN PENUNTUN

- Apakah method membutuhkan data object tertentu? Gunakan **instance method**.
- Apakah method hanya membutuhkan class, misalnya untuk membuat object atau mengolah class attribute? Gunakan **class method**.
- Apakah method tidak membutuhkan keduanya, tetapi maknanya sangat terkait dengan class? Gunakan **static method**.
- Jika fungsinya tidak terkait erat dengan class mana pun, tulis saja sebagai fungsi biasa di luar class.

**Tabel 3.1.** Perbandingan tiga jenis method.

| Aspek | Instance method | Class method | Static method |
|---|---|---|---|
| Decorator | tidak ada | `@classmethod` | `@staticmethod` |
| Parameter pertama | `self` (object) | `cls` (class) | tidak ada yang khusus |
| Akses instance attribute | ya | tidak | tidak |
| Akses class attribute | ya | ya | tidak |
| Biasa dipanggil lewat | object | class | class |
| Contoh kegunaan | `setor()`, `info()` | alternative constructor, penghitung | validasi format, konversi |

## 3.3 Konsep Decorator

`@property`, `@classmethod`, dan `@staticmethod` adalah *decorator bawaan* Python. Decorator adalah fungsi yang menerima sebuah fungsi, lalu mengembalikan fungsi baru yang "membungkus" fungsi asli dengan perilaku tambahan, tanpa mengubah kode fungsi asli tersebut. Untuk memahami cara kerjanya, kita perlu dua bekal terlebih dahulu.

### Bekal 1: Fungsi Adalah Object

Di Python, fungsi juga merupakan object. Akibatnya, fungsi bisa disimpan ke variabel, dikirim sebagai argumen ke fungsi lain, dan dikembalikan sebagai hasil fungsi.

**Fungsi diperlakukan seperti data**

```
def sapa(nama):
    return f"Halo, {nama}!"

salam = sapa                    # disimpan ke variabel (tanpa tanda kurung)
print(salam("Ani"))

def jalankan(fungsi, nilai):    # menerima fungsi sebagai argumen
    return fungsi(nilai)

print(jalankan(sapa, "Budi"))
print(jalankan(str.upper, "citra"))
```

Keluaran

```
Halo, Ani!
Halo, Budi!
CITRA
```

## Bekal 2: Fungsi di dalam Fungsi dan Closure

Fungsi boleh didefinisikan di dalam fungsi lain. Fungsi dalam tersebut dapat memakai variabel milik fungsi luar, dan tetap "mengingatnya" walaupun fungsi luar sudah selesai dijalankan. Fungsi dalam yang membawa ingatan seperti ini disebut *closure*.

Closure

```
def buat_pengali(n):
    def kali(x):
        return x * n            # n diingat dari fungsi luar
    return kali                 # fungsi dikembalikan sebagai hasil

kali_dua = buat_pengali(2)
kali_tiga = buat_pengali(3)
print(kali_dua(10), kali_tiga(10))
```

Keluaran

```
20 30
```

## Pola Dasar Decorator

Dengan dua bekal tersebut, kita bisa menulis decorator pertama: fungsi yang menerima fungsi `func`, mendefinisikan fungsi dalam `pembungkus` yang menjalankan kode tambahan di sekitar pemanggilan `func`, lalu mengembalikan `pembungkus`.

Membungkus fungsi secara manual

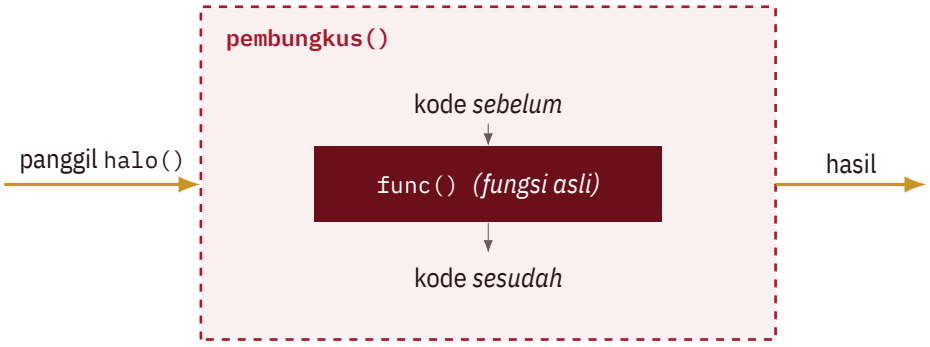
```
def dekorator(func):
    def pembungkus():
        print(">> sebelum fungsi dijalankan")
        func()
        print("<< sesudah fungsi dijalankan")
    return pembungkus

def halo():
    print("Halo, dunia!")

halo = dekorator(halo)      # nama halo kini merujuk ke pembungkus
halo()
```

Keluaran

```
>> sebelum fungsi dijalankan
Halo, dunia!
<< sesudah fungsi dijalankan
```




**Gambar 3.2.** Setelah didekorasi, pemanggilan `halo()` masuk ke `pembungkus`, yang menjalankan fungsi asli di antara kode tambahan.

## Sintaks @

Menulis `halo = dekorator(halo)` setiap kali tentu merepotkan. Python menyediakan sintaks singkat: tulis `@dekorator` tepat di atas definisi fungsi. Kedua cara berikut sepenuhnya setara.

Sintaks @

```
@dekorator
def halo():
    print("Halo, dunia!")

# sama persis dengan:
# def halo(): ...
# halo = dekorator(halo)

halo()
```

### Yang Terjadi Saat Python Membaca `@dekorator`

1. Python mendefinisikan fungsi `halo` seperti biasa.
2. Python memanggil `dekorator(halo)`. Di dalamnya, fungsi `pembungkus` dibuat dan mengingat `halo` asli sebagai `func`.
3. Nilai kembalian, yaitu `pembungkus`, disimpan dengan nama `halo`.
4. Setiap kali `halo()` dipanggil, yang sebenarnya berjalan adalah `pembungkus()`.

**DECORATOR DIJALANKAN SEKALI**

Fungsi decorator dijalankan *satu kali*, saat fungsi didefinisikan. Yang dijalankan berulang kali setiap pemanggilan adalah `pembungkus`.

## 3.4 Custom Decorator

Decorator buatan sendiri (*custom decorator*) berguna untuk menambahkan perilaku yang sama ke banyak fungsi tanpa menyalin kode: mencatat pemanggilan, mengukur waktu, memeriksa hak akses, memvalidasi argumen, dan sebagainya. Subbab ini membahas hal-hal yang perlu diperhatikan saat menulisnya.

### Jangan Lupa Mengembalikan Hasil

Pembungkus pada contoh sebelumnya memanggil `func()` tetapi membuang hasilnya. Jika fungsi asli mengembalikan nilai, pembungkus wajib meneruskannya dengan `return`.

**Meneruskan nilai kembalian**

```
def dekorator(func):
    def pembungkus():
        print("memanggil", func.__name__)
        return func()               # teruskan hasil fungsi asli
    return pembungkus

@dekorator
def angka_keberuntungan():
    return 7

print(angka_keberuntungan())
```

**Keluaran**

```
memanggil angka_keberuntungan
7
```

**PEMBUNGKUS TANPA RETURN**

Jika baris `return func()` ditulis sebagai `func()` saja, fungsi yang didekorasi akan selalu mengembalikan `None`. Kesalahan ini tidak menimbulkan pesan error sehingga sering lolos tanpa disadari.

### Decorator Pengukur Waktu dan `functools.wraps`

Contoh berikut mengukur lama sebuah fungsi berjalan. Perhatikan baris `@wraps(func)` di atas pembungkus.

ukur_waktu.py

```python
import time
from functools import wraps

def ukur_waktu(func):
    @wraps(func)
    def pembungkus():
        mulai = time.perf_counter()
        hasil = func()
        selesai = time.perf_counter()
        print(f"{func.__name__} selesai dalam {selesai - mulai:.2f} detik")
        return hasil
    return pembungkus

@ukur_waktu
def proses_berat():
    """Mensimulasikan proses yang lama."""
    time.sleep(0.5)
    return "selesai"

print(proses_berat())
print(proses_berat.__name__, "-", proses_berat.__doc__)
```

Keluaran

```
proses_berat selesai dalam 0.50 detik
selesai
proses_berat - Mensimulasikan proses yang lama.
```

Tanpa `@wraps(func)`, baris terakhir akan mencetak `pembungkus - None`, karena nama `proses_berat` kini merujuk ke fungsi `pembungkus` yang tidak punya docstring. Decorator `wraps` dari modul `functools` menyalin identitas fungsi asli (nama, docstring, dan informasi lain) ke pembungkus, sehingga fungsi yang didekorasi tetap "mengaku" sebagai dirinya sendiri. Hal ini penting untuk `help()`, pesan error, dan alat bantu debugging.

**BIASAKAN `@WRAPS`**

Jadikan `@wraps(func)` kebiasaan di setiap decorator yang Anda tulis. Mulai sekarang semua contoh di buku ini memakainya.

## Decorator dengan Argumen

Kadang decorator perlu diatur, misalnya "ulangi fungsi ini sebanyak *n* kali". Caranya adalah menambah satu lapisan fungsi lagi: fungsi terluar menerima argumen pengaturan dan mengembalikan decorator.

Decorator yang menerima argumen

```python
from functools import wraps

def ulangi(kali):                    # lapisan 1: menerima argumen
    def dekorator(func):             # lapisan 2: decorator biasa
        @wraps(func)
        def pembungkus():            # lapisan 3: pembungkus
            for _ in range(kali):
                func()
        return pembungkus
    return dekorator

@ulangi(3)
def yel():
    print("Semangat!")

yel()
```

Keluaran

```
Semangat!
Semangat!
Semangat!
```

Baris `@ulangi(3)` dibaca dalam dua tahap: `ulangi(3)` dijalankan lebih dulu dan menghasilkan `dekorator`, kemudian `dekorator` itulah yang dipakai untuk membungkus `yel`. Dengan kata lain, `yel = ulangi(3)(yel)`.

## 3.5 Chaining Decorator

Satu fungsi boleh diberi lebih dari satu decorator dengan menumpuknya. Teknik ini disebut *chaining* (merangkai) decorator.

Dua decorator bertumpuk

```python
from functools import wraps

def tebal(func):
    @wraps(func)
    def pembungkus():
        return "<b>" + func() + "</b>"
    return pembungkus

def miring(func):
    @wraps(func)
    def pembungkus():
```

```
        return "<i>" + func() + "</i>"
    return pembungkus

@tebal
@miring
def teks():
    return "Halo"

print(teks())
```

**Keluaran**

```
<b><i>Halo</i></b>
```

Tumpukan decorator dipasang **dari bawah ke atas**: decorator yang paling dekat dengan fungsi dipasang lebih dulu. Jadi kode di atas setara dengan `teks = tebal(miring(teks))`. Jika urutannya ditukar menjadi `@miring` di atas `@tebal`, hasilnya berubah menjadi `<i><b>Halo</b></i>`.

## Urutan Eksekusi

Saat fungsi dipanggil, urutannya berkebalikan dengan urutan pemasangan: eksekusi masuk dari decorator **paling atas** (lapisan terluar) menuju fungsi inti, lalu keluar kembali melewati lapisan-lapisan yang sama dalam urutan terbalik. Contoh berikut memperlihatkannya dengan jelas.

**Melacak urutan masuk dan keluar**

```
from functools import wraps

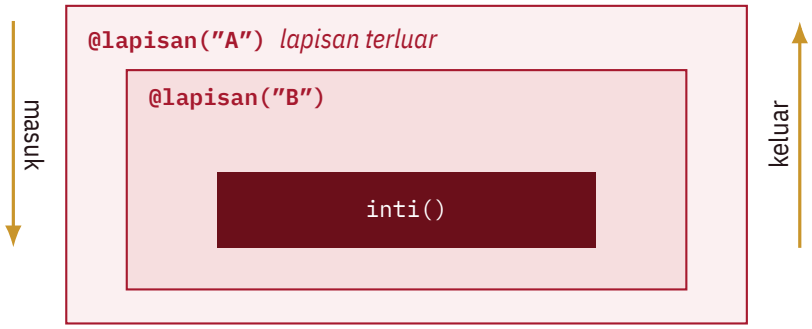
def lapisan(nama):
    def dekorator(func):
        @wraps(func)
        def pembungkus():
            print(f"masuk  {nama}")
            hasil = func()
            print(f"keluar {nama}")
            return hasil
        return pembungkus
    return dekorator

@lapisan("A")
@lapisan("B")
def inti():
    print("    inti dijalankan")

inti()
```

Keluaran

```
masuk  A
masuk  B
    inti dijalankan
keluar B
keluar A
```




**Gambar 3.3.** Decorator bertumpuk bekerja seperti lapisan bawang: pemanggilan menembus lapisan dari luar ke dalam, lalu hasilnya kembali dari dalam ke luar.

URUTAN ITU PENTING

Saat merangkai decorator, tanyakan: "pemeriksaan atau perilaku mana yang harus terjadi lebih dulu?". Misalnya, decorator yang memeriksa hak akses sebaiknya diletakkan paling atas agar pengguna tanpa izin langsung ditolak sebelum decorator lain (misalnya pencatat log) sempat bekerja.

## 3.6 `*args` dan `**kwargs` pada Decorator

Semua pembungkus yang kita tulis sejauh ini tidak menerima parameter, sehingga hanya bisa dipakai untuk fungsi tanpa parameter. Padahal decorator yang baik harus bisa membungkus fungsi *apa pun*. Kuncinya adalah `*args` dan `**kwargs`.

### Mengenal `*args` dan `**kwargs`

Pada definisi fungsi, `*args` mengumpulkan semua argumen posisi yang tersisa menjadi sebuah *tuple*, sedangkan `**kwargs` mengumpulkan semua argumen kata kunci yang tersisa menjadi sebuah *dictionary*. Nama `args` dan `kwargs` hanyalah kebiasaan; yang penting adalah tanda bintangnya.

Mengumpulkan argumen

```
1  def jumlahkan(*args):
2      print(args)
3      return sum(args)
```

```
4
5  def profil(**kwargs):
6      print(kwargs)
7
8  def campur(a, *args, **kwargs):
9      print(a, args, kwargs)
10
11 print(jumlahkan(1, 2, 3))
12 profil(nama="Ani", umur=20)
13 campur(1, 2, 3, x=4, y=5)
```

**Keluaran**

```
(1, 2, 3)
6
{'nama': 'Ani', 'umur': 20}
1 (2, 3) {'x': 4, 'y': 5}
```

Tanda bintang juga bisa dipakai ke arah sebaliknya saat *memanggil* fungsi: `*` membongkar list atau tuple menjadi argumen posisi, dan `**` membongkar dictionary menjadi argumen kata kunci.

**Tabel 3.2.** Arti tanda bintang di dua tempat.

| Tanda | Pada definisi fungsi | Pada pemanggilan fungsi |
|---|---|---|
| `*` | `def f(*args)`: kumpulkan argumen posisi menjadi tuple | `f(*daftar)`: bongkar list/tuple menjadi argumen posisi |
| `**` | `def f(**kwargs)`: kumpulkan argumen kata kunci menjadi dictionary | `f(**data)`: bongkar dictionary menjadi argumen kata kunci |

### Masalah: Pembungkus Tanpa Parameter

**Decorator lama pada fungsi berparameter**

```
1  def dekorator(func):
2      def pembungkus():
3          print("memanggil", func.__name__)
4          return func()
5      return pembungkus
6
7  @dekorator
8  def tambah(a, b):
9      return a + b
10
```

```
print(tambah(2, 3))
```

Keluaran

```
Traceback (most recent call last):
  ...
TypeError: dekorator.<locals>.pembungkus() takes 0 positional arguments but 2 were
    given
```

Argumen 2 dan 3 dikirim ke `pembungkus`, bukan ke `tambah`, dan `pembungkus` tidak menerima parameter apa pun.

### Solusi: Terima Apa Saja, Teruskan Apa Saja

Pembungkus cukup menerima `*args, **kwargs`, lalu meneruskannya utuh ke fungsi asli dengan `func(*args, **kwargs)`. Dengan pola ini, satu decorator dapat membungkus fungsi dengan jumlah dan jenis parameter berapa pun.

Decorator umum

```
from functools import wraps

def catat(func):
    @wraps(func)
    def pembungkus(*args, **kwargs):
        print(f"memanggil {func.__name__} args={args} kwargs={kwargs}")
        hasil = func(*args, **kwargs)
        print(f"{func.__name__} mengembalikan {hasil!r}")
        return hasil
    return pembungkus

@catat
def tambah(a, b):
    return a + b

@catat
def sapa(nama, salam="Halo"):
    return f"{salam}, {nama}!"

tambah(2, 3)
sapa("Ani", salam="Selamat pagi")
```

Keluaran

```
memanggil tambah args=(2, 3) kwargs={}
tambah mengembalikan 5
memanggil sapa args=('Ani',) kwargs={'salam': 'Selamat pagi'}
```

```
sapa mengembalikan 'Selamat pagi, Ani!'
```

**KERANGKA DECORATOR SIAP PAKAI**

Hampir semua decorator mengikuti kerangka yang sama. Simpan kerangka ini dan isi bagian "sebelum" dan "sesudah" sesuai kebutuhan.

Kerangka decorator

```python
from functools import wraps

def nama_decorator(func):
    @wraps(func)
    def pembungkus(*args, **kwargs):
        # ... kode sebelum ...
        hasil = func(*args, **kwargs)
        # ... kode sesudah ...
        return hasil
    return pembungkus
```

## Decorator pada Method Class

Decorator juga bisa dipasang pada method. Saat method dipanggil, `self` ikut dikirim sebagai argumen posisi pertama, sehingga ia ikut tertampung di `*args`. Jika decorator perlu memakai `self` atau argumen tertentu, parameter tersebut boleh ditulis eksplisit sebelum `*args`.

Sebagai penutup bab, kita rapikan kembali class `Rekening`. Di Bab 2, pemeriksaan "jumlah harus lebih dari nol" ditulis berulang di `setor()` dan `tarik()`. Kini pemeriksaan itu dan pencatatan riwayat kita pindahkan ke decorator, sementara class method dan static method mengisi peran masing-masing.

rekening.py

```python
from functools import wraps

def wajib_positif(func):
    @wraps(func)
    def pembungkus(self, jumlah, *args, **kwargs):
        if jumlah <= 0:
            raise ValueError(f"{func.__name__}: jumlah harus lebih dari nol.")
        return func(self, jumlah, *args, **kwargs)
    return pembungkus

def catat_transaksi(func):
    @wraps(func)
    def pembungkus(self, *args, **kwargs):
```

```
        hasil = func(self, *args, **kwargs)
        self._riwayat.append((func.__name__, *args))
        return hasil
    return pembungkus


class Rekening:
    BIAYA_ADMIN = 2500

    def __init__(self, pemilik, saldo=0):
        self.pemilik = pemilik
        self._saldo = saldo
        self._riwayat = []

    @classmethod
    def dari_teks(cls, teks):                # alternative constructor
        pemilik, saldo = teks.split(";")
        return cls(pemilik.strip(), int(saldo))

    @staticmethod
    def rupiah(angka):                       # utilitas format
        return f"Rp{angka:,}".replace(",", ".")

    @property
    def saldo(self):
        return self._saldo

    @property
    def riwayat(self):
        return list(self._riwayat)

    @catat_transaksi
    @wajib_positif
    def setor(self, jumlah):
        self._saldo += jumlah

    @catat_transaksi
    @wajib_positif
    def tarik(self, jumlah):
        total = jumlah + Rekening.BIAYA_ADMIN
        if total > self._saldo:
            raise ValueError("Saldo tidak mencukupi.")
        self._saldo -= total

    def __str__(self):
        return f"{self.pemilik}: {Rekening.rupiah(self.saldo)}"
```

```
60
61
62  rek = Rekening.dari_teks("Ani; 100000")
63  rek.setor(50000)
64  rek.tarik(20000)
65  print(rek)
66  print(rek.riwayat)
67  rek.setor(-5000)
```

Keluaran

```
Ani: Rp127.500
[('setor', 50000), ('tarik', 20000)]
Traceback (most recent call last):
  ...
ValueError: setor: jumlah harus lebih dari nol.
```

Ada beberapa hal yang patut diperhatikan dari studi kasus ini:

- `@wajib_positif` diletakkan *di bawah* `@catat_transaksi`. Karena eksekusi masuk dari atas, `catat_transaksi` memanggil `wajib_positif` terlebih dahulu, dan pencatatan baru dilakukan *setelah* transaksi berhasil. Setoran $-5000$ ditolak dan tidak pernah masuk riwayat.
- Logika validasi dan pencatatan kini ditulis sekali saja, lalu dipakai ulang oleh dua method.
- `dari_teks()` adalah class method karena tugasnya membuat object; `rupiah()` adalah static method karena hanya memformat angka tanpa memerlukan data rekening.

**MENGAPA `_RIWAYAT`, BUKAN `__RIWAYAT`?**

Decorator `catat_transaksi` ditulis *di luar* class. Ingat dari Bab 2 bahwa name mangling hanya terjadi untuk kode yang ditulis di dalam class. Seandainya attribute bernama `__riwayat`, ia tersimpan sebagai `_Rekening__riwayat`, dan decorator yang mencari `self.__riwayat` akan gagal dengan `AttributeError`. Karena itu attribute yang perlu diakses decorator memakai satu garis bawah (protected).

## 3.7 Rangkuman

- **Class method** (`@classmethod`) menerima `cls` dan bekerja pada tingkat class: mengolah class attribute dan menjadi *alternative constructor*. Selalu buat object dengan `cls(…)` agar ramah terhadap class turunan.
- **Static method** (`@staticmethod`) tidak menerima `self` maupun `cls`; cocok untuk utilitas yang terkait erat dengan class, seperti validasi format atau konversi.

- Fungsi di Python adalah object: bisa disimpan ke variabel, dikirim sebagai argumen, dan dikembalikan. Fungsi dalam yang mengingat variabel fungsi luar disebut **closure**.
- **Decorator** adalah fungsi yang menerima fungsi dan mengembalikan pembungkusnya. `@dekorator` di atas fungsi `f` setara dengan `f = dekorator(f)`.
- Pembungkus harus meneruskan nilai kembalian (`return func(…)`) dan sebaiknya diberi `@wraps(func)` agar identitas fungsi asli terjaga.
- Decorator dengan argumen membutuhkan satu lapisan fungsi tambahan: `@ulangi(3)` setara dengan `f = ulangi(3)(f)`.
- Pada **chaining**, decorator dipasang dari bawah ke atas, tetapi dieksekusi dari atas (lapisan luar) ke dalam, lalu keluar kembali.
- Pembungkus yang menerima `*args, **kwargs` dan meneruskannya dengan `func(*args, **kwargs)` dapat membungkus fungsi dan method apa pun.

## 3.8 Kuis Teoritis

### Pilihan Ganda

Pilihlah satu jawaban yang paling tepat.

1. Parameter pertama yang diterima sebuah class method adalah ...

   a. `self`, yaitu object pemanggil
   b. `cls`, yaitu class itu sendiri
   c. `args`, yaitu tuple argumen
   d. tidak ada parameter khusus

2. Sebuah method hanya mengubah teks menjadi format rupiah dan tidak membutuhkan data object maupun class. Jenis method yang paling tepat adalah ...

   a. instance method
   b. class method
   c. static method
   d. property

3. Kegunaan class method yang paling umum adalah ...

   a. mengubah instance attribute satu object
   b. menyediakan cara alternatif untuk membuat object
   c. menyembunyikan attribute dengan name mangling
   d. mempercepat eksekusi method

4. Perhatikan kode berikut.

Soal 4

```
class Toko:
    cabang = 0

    def __init__(self):
        Toko.cabang += 1

    @classmethod
    def total(cls):
        return cls.cabang

Toko(); Toko(); Toko()
print(Toko.total())
```

Keluaran kode tersebut adalah ...

a. `0`
b. `1`
c. `3`
d. `TypeError`

5. Penulisan `@log` di atas definisi fungsi `hitung` setara dengan ...

a. `log = hitung(log)`
b. `hitung = log(hitung)`
c. `hitung = log()`
d. `log(hitung())`

6. Fungsi dalam yang tetap mengingat variabel milik fungsi luarnya disebut ...

a. closure
b. constructor
c. generator
d. mangling

7. Perhatikan kode berikut.

Soal 7

```
def kurung(func):
    def pembungkus():
        return "(" + func() + ")"
    return pembungkus

def bintang(func):
    def pembungkus():
        return "*" + func() + "*"
    return pembungkus
```

```
10
11 @bintang
12 @kurung
13 def kata():
14     return "OOP"
15
16 print(kata())
```

Keluaran kode tersebut adalah ...

a. `(*OOP*)`
b. `*(OOP)*`
c. `*OOP*`
d. `(OOP)`

8. Sebuah decorator memiliki pembungkus yang memanggil `func(*args, **kwargs)` tanpa `return`. Fungsi `luas(4, 5)` yang semula mengembalikan `20` kini mengembalikan ...

a. `20`
b. `0`
c. `None`
d. `TypeError`

9. Fungsi utama `functools.wraps` pada decorator adalah ...

a. mempercepat pemanggilan fungsi
b. menyalin nama dan docstring fungsi asli ke pembungkus
c. mengizinkan decorator menerima argumen
d. mencegah fungsi dipanggil lebih dari sekali

10. Pada pembungkus `def pembungkus(*args, **kwargs)`, pemanggilan `f(1, 2, x=3)` membuat nilai `args` dan `kwargs` berturut-turut menjadi ...

a. `[1, 2]` dan `{'x': 3}`
b. `(1, 2, 3)` dan `{}`
c. `(1, 2)` dan `{'x': 3}`
d. `(1, 2)` dan `('x', 3)`

## Uraian

1. Jelaskan perbedaan instance method, class method, dan static method dari segi parameter pertama dan data yang dapat diaksesnya. Berikan satu contoh kegunaan masing-masing pada class `Produk`.

2. Mengapa di dalam alternative constructor sebaiknya ditulis `cls(…)` dan bukan nama class secara langsung? Jelaskan dengan contoh class turunan.

3. Jelaskan langkah-langkah yang terjadi ketika Python membaca kode `@ukur_waktu` di atas definisi sebuah fungsi, dan apa yang sebenarnya dijalankan ketika fungsi itu dipanggil.
4. Dua decorator, `@cek_login` dan `@catat_log`, akan dipasang pada fungsi `hapus_data()`. Tentukan urutan penumpukan yang tepat agar pengguna yang belum login tidak tercatat di log, dan jelaskan alasannya.
5. Mengapa pembungkus pada decorator sebaiknya menerima `*args, **kwargs`? Apa yang terjadi jika decorator dengan pembungkus tanpa parameter dipasang pada method sebuah class?

## 3.9 Praktik Mandiri

Kerjakan praktik berikut secara berurutan. Simpan setiap praktik dalam file `.py` terpisah, jalankan, dan pastikan keluarannya sesuai dengan contoh.

### Praktik 1: Durasi

Buat class `Durasi` yang menyimpan lama waktu dalam satuan detik.

- Constructor menerima `jam`, `menit`, dan `detik`, lalu menyimpannya sebagai total detik.
- Class method `dari_detik(total)` untuk membuat durasi dari jumlah detik.
- Class method `dari_teks(teks)` untuk membuat durasi dari teks berformat `"JJ:MM:DD"`. Tolak teks yang formatnya salah dengan `ValueError`, memakai static method di bawah.
- Static method `format_valid(teks)` yang mengembalikan `True` jika teks terdiri dari tiga bagian angka yang dipisah titik dua, dengan menit dan detik di bawah 60.
- Method `__str__` yang menampilkan durasi dalam format `JJ:MM:DD`, serta method `tambah(lain)` yang mengembalikan object `Durasi` baru hasil penjumlahan.

**Contoh keluaran Praktik 1**

```
01:30:15
00:01:40
02:05:00
01:31:55
False
ValueError: Format durasi tidak valid: 1:75:00
```

### Praktik 2: Koleksi Decorator

Buat tiga decorator berikut dalam satu file. Semuanya wajib memakai `@wraps` dan `*args, **kwargs`.

1. `@hitung_panggilan`: mencetak berapa kali fungsi sudah dipanggil setiap kali fungsi itu dijalankan.

2. `@batasi(n)`: decorator dengan argumen yang hanya mengizinkan fungsi dipanggil sebanyak *n* kali. Pemanggilan berikutnya mencetak pesan penolakan dan mengembalikan `None`.
3. `@huruf_besar`: mengubah hasil fungsi (yang berupa string) menjadi huruf kapital.

Uji ketiganya, termasuk merangkai `@batasi(2)` dan `@huruf_besar` pada satu fungsi `sapa(nama)`. Coba kedua urutan penumpukan dan jelaskan dalam komentar apakah hasilnya berbeda.

**Contoh keluaran Praktik 2**

```
[hitung] tambah dipanggil ke-1
5
[hitung] tambah dipanggil ke-2
9
HALO, ANI!
HALO, BUDI!
[batasi] sapa sudah dipanggil 2 kali, permintaan ditolak.
None
```

## Praktik 3: Toko dengan Decorator

Lanjutkan class `Produk` dari Praktik Mandiri Bab 2, lalu tambahkan fitur berikut.

### Langkah Pengerjaan

1. Tambahkan class method `dari_baris_csv(baris)` yang membuat produk dari teks seperti `"P-01,Buku Tulis,5000,40"`.
2. Tambahkan static method `rupiah(angka)` untuk menampilkan harga dengan format `Rp5.000`, dan pakai di `__str__`.
3. Tulis decorator `@wajib_positif` untuk memeriksa argumen `jumlah` pada method `jual()` dan `tambah_stok()`, lalu hapus pemeriksaan serupa dari dalam kedua method tersebut.
4. Tulis decorator `@catat` yang menyimpan setiap transaksi berhasil ke attribute protected `_log`, lalu rangkai dengan `@wajib_positif` pada urutan yang benar.
5. Buat minimal tiga produk dari daftar baris CSV menggunakan perulangan, lakukan beberapa transaksi (termasuk yang gagal), lalu tampilkan isi log.

**KRITERIA PENILAIAN**

- Ketepatan pemilihan jenis method (instance, class, atau static) untuk setiap kebutuhan.
- Decorator memakai `@wraps` dan `*args, **kwargs`, serta meneruskan nilai kembalian.

- Urutan chaining decorator tepat: transaksi yang gagal tidak tercatat di log.
- Tidak ada logika validasi yang ditulis berulang di beberapa method.

Bab 04

# Inheritance

*Membangun class baru di atas class yang sudah ada.*

04

DI BAB INI

- Konsep inheritance
- Parent dan child class
- Overriding
- Nested inheritance
- Multiple inheritance

Bayangkan kita membangun sistem informasi kampus. Ada mahasiswa, ada dosen, ada tenaga kependidikan. Ketiganya punya nama dan email, ketiganya bisa memperkenalkan diri, tetapi masing-masing juga punya data dan perilaku khas: mahasiswa punya NIM dan mengisi KRS, dosen punya NIP dan memberi nilai. Jika setiap class ditulis terpisah, bagian yang sama akan disalin berkali-kali, dan setiap perbaikan harus diulang di semua salinan.

*Inheritance* (pewarisan) menyelesaikan masalah ini. Kita menulis bagian yang sama satu kali di sebuah class umum, lalu membuat class-class khusus yang *mewarisi* semuanya dan hanya menambahkan atau mengubah bagian yang berbeda. Bab ini membahas cara kerja pewarisan, mulai dari satu tingkat, bertingkat, hingga mewarisi dari beberapa class sekaligus.

## 4.1 Konsep Inheritance

*Inheritance* adalah mekanisme untuk membuat class baru berdasarkan class yang sudah ada. Class baru otomatis memiliki seluruh attribute dan method class lama, lalu boleh menambah yang baru atau mengubah yang diwarisi. Class yang diwarisi disebut *parent class* (class induk, *base class*, *superclass*); class yang mewarisi disebut *child class* (class anak, *derived class*, *subclass*).

### Masalah yang Diselesaikan

Perhatikan dua class berikut. Hampir separuh isinya sama persis.

Duplikasi tanpa inheritance

```
class Mahasiswa:
    def __init__(self, nama, email, nim):
        self.nama = nama
        self.email = email
        self.nim = nim

    def perkenalan(self):
        return f"Halo, saya {self.nama}"

class Dosen:
    def __init__(self, nama, email, nip):
```

```
        self.nama = nama
        self.email = email
        self.nip = nip

    def perkenalan(self):
        return f"Halo, saya {self.nama}"
```

Jika suatu hari format email harus divalidasi, atau method `perkenalan()` perlu diubah, kita harus mengubah dua tempat, dan akan menjadi tiga, empat, atau lebih seiring bertambahnya jenis warga kampus. Inheritance memungkinkan bagian yang sama ditulis sekali saja.

### Hubungan is-a

Inheritance menggambarkan hubungan *is-a* ("adalah sebuah"): *mahasiswa adalah seorang warga kampus*, *dosen adalah seorang warga kampus*. Uji sederhana ini dipakai untuk memutuskan apakah inheritance tepat digunakan.

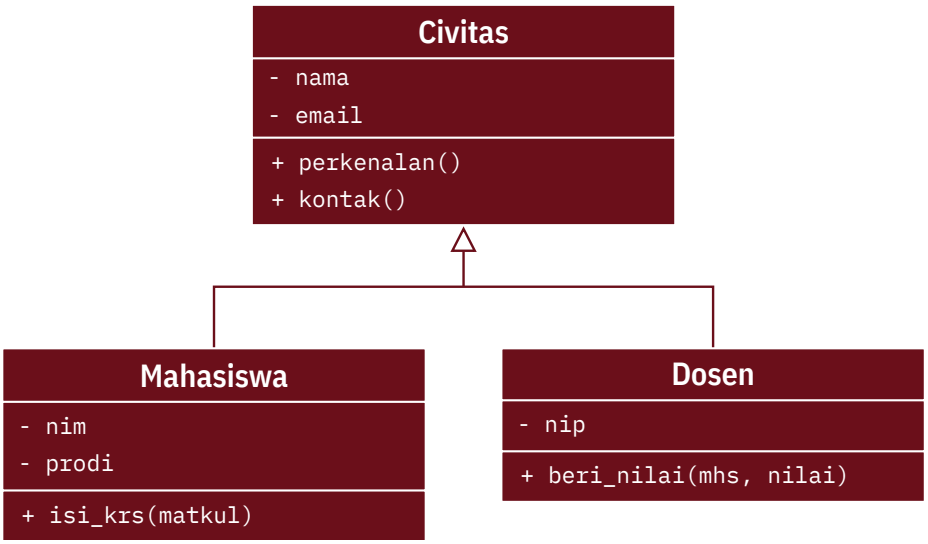
**Tabel 4.1.** Uji *is-a* untuk menentukan inheritance.

| Pernyataan | Benar? | Kesimpulan |
|---|---|---|
| Mahasiswa *adalah* warga kampus | ya | `Mahasiswa` mewarisi `Civitas` |
| Mobil *adalah* kendaraan | ya | `Mobil` mewarisi `Kendaraan` |
| Mobil *adalah* mesin | tidak | mobil *memiliki* mesin; gunakan composition (Bab 6) |
| Rekening *adalah* nasabah | tidak | rekening *dimiliki* nasabah; bukan inheritance |

### Manfaat Inheritance

- **Penggunaan ulang kode** (*reusability*): kode bersama ditulis sekali di parent.
- **Konsistensi**: perbaikan di parent otomatis berlaku untuk semua child.
- **Kemudahan pengembangan**: jenis baru cukup dibuat sebagai child yang menambahkan perbedaannya saja.
- **Pengelompokan yang alami**: object dari child class tetap bisa diperlakukan sebagai object parent, sehingga berbagai jenis object dapat diproses bersama (dasar dari *polymorphism* di Bab 6).

## 4.2 Parent/Base Class




**Gambar 4.1.** Class diagram pewarisan. Panah bersegitiga terbuka menunjuk ke parent class; child class hanya mencantumkan attribute dan method tambahannya.

*Parent class* adalah class biasa; tidak ada sintaks khusus untuk menjadikannya induk. Yang membedakan hanyalah cara kita merancangnya: parent berisi attribute dan method yang **berlaku umum** untuk semua turunannya, tanpa detail yang hanya dimiliki sebagian turunan.

**Parent class Civitas**

```
class Civitas:
    """Warga kampus secara umum."""

    def __init__(self, nama, email):
        self.nama = nama
        self.email = email

    def perkenalan(self):
        return f"Halo, saya {self.nama}"

    def kontak(self):
        return f"{self.nama} <{self.email}>"

rina = Civitas("Rina", "rina@unesa.ac.id")
print(rina.perkenalan())
print(rina.kontak())
```

**Keluaran**

```
Halo, saya Rina
Rina <rina@unesa.ac.id>
```

MERANCANG PARENT YANG BAIK

Masukkan ke parent hanya hal yang benar-benar dimiliki *semua* turunannya. Jika sebuah atribute hanya dimiliki mahasiswa (misalnya NIM), letakkan di class `Mahasiswa`, bukan di `Civitas`. Parent yang terlalu "gemuk" memaksa child membawa data yang tidak mereka butuhkan.

### Class `object` sebagai Induk Tertinggi

Walaupun `Civitas` tidak menuliskan parent apa pun, ia sebenarnya tetap mewarisi sebuah class: `object`. Di Python 3, **semua** class secara otomatis merupakan turunan `object`. Dari sanalah berasal method bawaan seperti `__str__`, `__eq__`, dan `__init__` standar yang dimiliki setiap class.

**Setiap class mewarisi object**

```
print(Civitas.__bases__)              # parent langsung dari Civitas
print(issubclass(Civitas, object))
print(rina)                            # memakai __str__ bawaan object
```

**Keluaran**

```
(<class 'object'>,)
True
<__main__.Civitas object at 0x7f3a2c1b2d50>
```

Menulis `class Civitas:` sama artinya dengan `class Civitas(object):`. Bentuk kedua sering ditemui di kode lama dari era Python 2.

## 4.3 Child/Derived Class

*Child class* dibuat dengan menuliskan nama parent di dalam tanda kurung setelah nama class. Bahkan tanpa isi apa pun, child sudah memiliki semua yang dimiliki parent.

**Child class paling sederhana**

```
class Mahasiswa(Civitas):
    pass

ani = Mahasiswa("Ani", "ani@mhs.unesa.ac.id")
print(ani.perkenalan())        # diwarisi dari Civitas
print(ani.kontak())            # diwarisi dari Civitas
```

**Keluaran**

```
Halo, saya Ani
Ani <ani@mhs.unesa.ac.id>
```

### Menambah Attribute dengan `super()`

Child biasanya membutuhkan attribute tambahan, sehingga perlu `__init__` sendiri. Agar attribute milik parent tetap disiapkan, `__init__` milik child memanggil `__init__` milik parent melalui fungsi `super()`. Fungsi `super()` mengembalikan rujukan ke parent, sehingga `super().__init__(…)` berarti "jalankan constructor milik parent untuk object ini".

**Child dengan attribute dan method tambahan**

```
class Mahasiswa(Civitas):
    def __init__(self, nama, email, nim, prodi):
        super().__init__(nama, email)    # parent menyiapkan nama dan email
        self.nim = nim                   # attribute khusus mahasiswa
        self.prodi = prodi

    def isi_krs(self, matkul):           # method khusus mahasiswa
        return f"{self.nama} mengambil {matkul}"

class Dosen(Civitas):
    def __init__(self, nama, email, nip):
        super().__init__(nama, email)
        self.nip = nip

    def beri_nilai(self, mhs, nilai):
        return f"{self.nama} memberi nilai {nilai} kepada {mhs.nama}"

ani = Mahasiswa("Ani", "ani@mhs.unesa.ac.id", "24051204001", "Kecerdasan
    Artifisial")
budi = Dosen("Budi", "budi@unesa.ac.id", "198001012005011001")

print(ani.kontak())                  # warisan Civitas
print(ani.isi_krs("PBO"))            # milik Mahasiswa
print(budi.beri_nilai(ani, "A"))     # milik Dosen
```

**Keluaran**

```
Ani <ani@mhs.unesa.ac.id>
Ani mengambil PBO
Budi memberi nilai A kepada Ani
```

**JANGAN LUPA `SUPER().__INIT__()`**

Jika child mendefinisikan `__init__` sendiri tetapi lupa memanggil `super().__init__()`, constructor parent tidak pernah dijalankan dan attribute milik parent tidak pernah dibuat. Kesalahannya baru muncul ketika attribute itu dipakai.

Akibat lupa memanggil super

```
class Tendik(Civitas):
    def __init__(self, nama, email, unit):
        self.unit = unit                # lupa super().__init__(nama, email)

sari = Tendik("Sari", "sari@unesa.ac.id", "BAAK")
print(sari.kontak())
```

Keluaran

```
Traceback (most recent call last):
  ...
AttributeError: 'Tendik' object has no attribute 'nama'
```

## Memeriksa Hubungan dengan `isinstance()` dan `issubclass()`

Object dari child class juga dianggap sebagai instance parent-nya. Fungsi `isinstance()` memeriksa hubungan object dengan class, sedangkan `issubclass()` memeriksa hubungan antar-class.

Memeriksa hubungan pewarisan

```
print(isinstance(ani, Mahasiswa))     # True
print(isinstance(ani, Civitas))       # True: mahasiswa adalah civitas
print(isinstance(ani, Dosen))         # False
print(issubclass(Dosen, Civitas))     # True
print(type(ani) == Civitas)           # False: type() hanya melihat class asli
```

**ISINSTANCE() ATAU TYPE()?**

Untuk memeriksa "apakah object ini termasuk jenis X", gunakan `isinstance()` karena ia ikut memperhitungkan pewarisan. Perbandingan `type(obj) == X` hanya benar untuk class aslinya dan akan menolak object dari child class.

## Attribute Private dan Pewarisan

Ingat kembali name mangling dari Bab 2: attribute `__saldo` di class `Akun` tersimpan sebagai `_Akun__saldo`. Akibatnya, child class **tidak bisa** mengaksesnya dengan `self.__saldo`, karena di dalam child nama tersebut diterjemahkan menjadi `_AkunPremium__saldo`. Attribute yang memang perlu dipakai oleh child sebaiknya diberi satu garis bawah (protected).

Private tidak terlihat oleh child

```
class Akun:
    def __init__(self, saldo):
```

```
        self.__saldo = saldo     # private: hanya untuk Akun
        self._poin = 10          # protected: boleh dipakai turunan

class AkunPremium(Akun):
    def info(self):
        print(f"poin {self._poin}")
        print(f"saldo {self.__saldo}")

AkunPremium(5000).info()
```

**Keluaran**

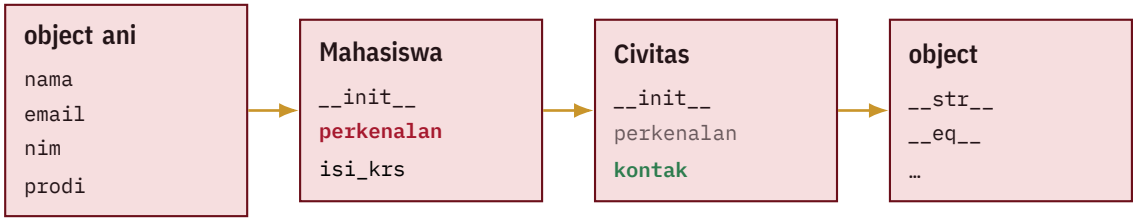
```
poin 10
Traceback (most recent call last):
  ...
AttributeError: 'AkunPremium' object has no attribute '_AkunPremium__saldo'
```

## 4.4 Konsep Overriding

*Overriding* adalah menulis ulang method warisan di child class dengan **nama yang sama**. Saat method tersebut dipanggil pada object child, versi milik child yang dijalankan, sedangkan versi parent "tertutupi". Overriding dipakai ketika perilaku umum dari parent tidak cocok, atau kurang lengkap, untuk child tertentu.

### Cara Python Mencari Method

Untuk memahami overriding, kita perlu tahu cara Python mencari attribute dan method. Ketika kita menulis `ani.perkenalan()`, Python mencari nama `perkenalan` mulai dari object itu sendiri, lalu ke class-nya, lalu ke parent, dan terus ke atas hingga `object`. Yang *pertama* ditemukan itulah yang dipakai.




`ani.perkenalan()` ditemukan di **Mahasiswa**; versi Civitas tidak pernah dicapai.

`ani.kontak()` tidak ada di Mahasiswa, sehingga pencarian berlanjut dan ditemukan di **Civitas**.

**Gambar 4.2.** Urutan pencarian method: dari object, naik ke class, lalu ke parent hingga `object`.

### Mengganti dan Memperluas Method Warisan

Ada dua gaya overriding. Pertama, **mengganti total**: child menulis perilaku yang sama sekali baru. Kedua, **memperluas**: child memanggil versi parent melalui `super()`, lalu menambahkan sesuatu.

**Dua gaya overriding**

```
class Mahasiswa(Civitas):
    def __init__(self, nama, email, nim, prodi):
        super().__init__(nama, email)
        self.nim = nim
        self.prodi = prodi

    def perkenalan(self):                          # mengganti total
        return f"Hai! Aku {self.nama}, mahasiswa {self.prodi}"

    def __str__(self):                             # override milik object
        return f"{self.nim} - {self.nama}"

class Dosen(Civitas):
    def __init__(self, nama, email, nip):
        super().__init__(nama, email)
        self.nip = nip

    def perkenalan(self):                          # memperluas
        dasar = super().perkenalan()
        return f"{dasar}, dosen dengan NIP {self.nip}"

warga = [
    Civitas("Rina", "rina@unesa.ac.id"),
    Mahasiswa("Ani", "ani@mhs.unesa.ac.id", "24051204001", "Kecerdasan
    Artifisial"),
    Dosen("Budi", "budi@unesa.ac.id", "198001012005011001"),
]
for w in warga:
    print(w.perkenalan())
print(warga[1])
```

**Keluaran**

```
Halo, saya Rina
Hai! Aku Ani, mahasiswa Kecerdasan Artifisial
Halo, saya Budi, dosen dengan NIP 198001012005011001
24051204001 - Ani
```

Perhatikan perulangan di atas: satu baris `w.perkenalan()` menghasilkan perilaku berbeda bergantung pada jenis object. Kemampuan ini adalah benih dari *polymorphism*, yang akan dibahas tuntas di Bab 6.

**JAGA BENTUK METHOD YANG DI-OVERRIDE**

Method yang di-override sebaiknya menerima parameter yang sama dan mengembalikan jenis nilai yang sama dengan versi parent. Kode yang memanggil `w.perkenalan()` tidak peduli apakah `w` mahasiswa atau dosen; jika child mengubah bentuk method-nya, kode tersebut bisa rusak.

## Studi Kasus: Penggajian Karyawan

Overriding paling terasa manfaatnya ketika method di parent memanggil method lain yang di-override oleh child. Pada contoh berikut, `slip()` hanya ditulis sekali di `Karyawan`, tetapi hasilnya menyesuaikan karena `self.hitung_gaji()` selalu memanggil versi milik class object yang sebenarnya.

penggajian.py

```
class Karyawan:
    def __init__(self, nama, gaji_pokok):
        self.nama = nama
        self.gaji_pokok = gaji_pokok

    def hitung_gaji(self):
        return self.gaji_pokok

    def slip(self):
        jenis = type(self).__name__
        return f"{self.nama:<6} {jenis:<16} Rp{self.hitung_gaji():>11,}"

class KaryawanTetap(Karyawan):
    def __init__(self, nama, gaji_pokok, tunjangan):
        super().__init__(nama, gaji_pokok)
        self.tunjangan = tunjangan

    def hitung_gaji(self):
        return super().hitung_gaji() + self.tunjangan

class KaryawanKontrak(Karyawan):
    def __init__(self, nama, tarif_jam, jam_kerja):
        super().__init__(nama, 0)
        self.tarif_jam = tarif_jam
        self.jam_kerja = jam_kerja

    def hitung_gaji(self):
        return self.tarif_jam * self.jam_kerja

daftar = [
    Karyawan("Ani", 4_000_000),
    KaryawanTetap("Budi", 5_000_000, 1_500_000),
```

```
    KaryawanKontrak("Citra", 50_000, 120),
]
for k in daftar:
    print(k.slip())
print(f"Total gaji: Rp{sum(k.hitung_gaji() for k in daftar):,}")
```

Keluaran

```
Ani    Karyawan         Rp  4,000,000
Budi   KaryawanTetap    Rp  6,500,000
Citra  KaryawanKontrak  Rp  6,000,000
Total gaji: Rp16,500,000
```

## 4.5 Nested Inheritance

*Nested inheritance* atau *multilevel inheritance* adalah pewarisan bertingkat: sebuah child class menjadi parent bagi class lain. Class di tingkat paling bawah mewarisi semua yang dimiliki seluruh leluhurnya. Setiap tingkat cukup menambahkan bagian yang menjadi ciri khasnya.

Kendaraan, Mobil, MobilListrik

```
class Kendaraan:
    def __init__(self, merek, roda):
        self.merek = merek
        self.roda = roda

    def info(self):
        return f"{self.merek}, {self.roda} roda"

    def bergerak(self):
        return "Kendaraan bergerak"

class Mobil(Kendaraan):
    def __init__(self, merek, kursi):
        super().__init__(merek, 4)          # mobil selalu beroda 4
        self.kursi = kursi

    def info(self):
        return f"{super().info()}, {self.kursi} kursi"

class MobilListrik(Mobil):
    def __init__(self, merek, kursi, baterai_kwh):
        super().__init__(merek, kursi)
        self.baterai_kwh = baterai_kwh

```

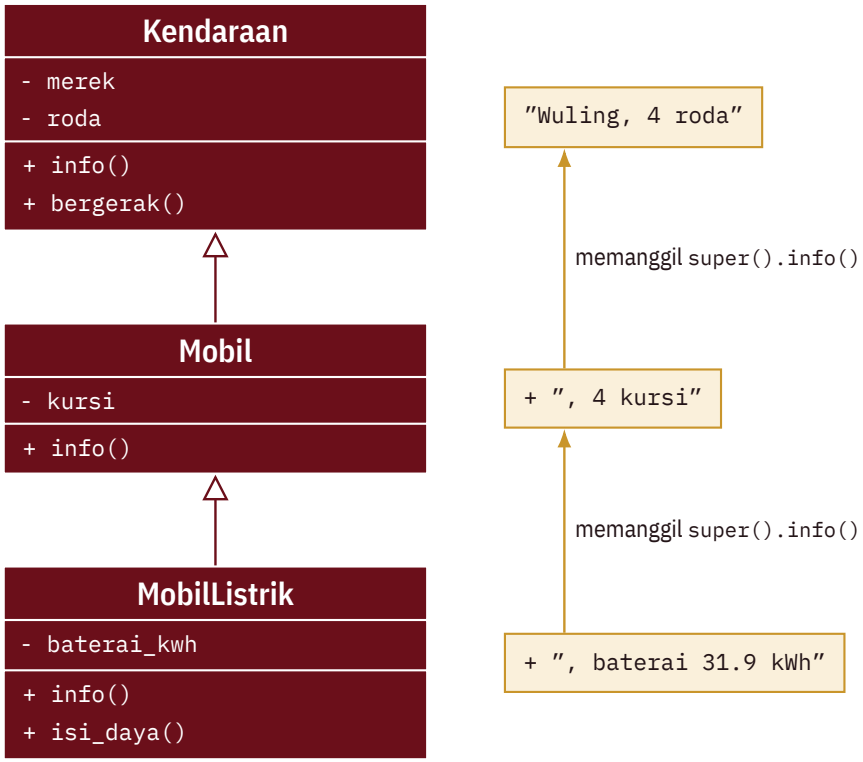
```python
    def info(self):
        return f"{super().info()}, baterai {self.baterai_kwh} kWh"

    def isi_daya(self):
        return f"{self.merek} sedang mengisi daya"

ev = MobilListrik("Wuling", 4, 31.9)
print(ev.info())
print(ev.bergerak())                    # warisan dua tingkat di atas
print(ev.isi_daya())
print(isinstance(ev, Kendaraan))
print([k.__name__ for k in MobilListrik.__mro__])
```

**Keluaran**

```
Wuling, 4 roda, 4 kursi, baterai 31.9 kWh
Kendaraan bergerak
Wuling sedang mengisi daya
True
['MobilListrik', 'Mobil', 'Kendaraan', 'object']
```




**Gambar 4.3.** Pewarisan bertingkat. Setiap tingkat memanggil `super().info()` lalu menambahkan bagiannya sendiri.

Atribut khusus `__mro__` (*Method Resolution Order*) menampilkan urutan class yang ditelusuri Python saat mencari method. Pada pewarisan bertingkat, urutannya lurus dari

bawah ke atas.

> **JANGAN TERLALU DALAM**
>
> Hierarki yang terlalu dalam (lebih dari tiga atau empat tingkat) sulit dipahami: untuk mengetahui perilaku satu method, pembaca harus menelusuri banyak class. Jika hierarki mulai memanjang, pertimbangkan apakah sebagian hubungan sebenarnya lebih cocok dimodelkan sebagai "memiliki" (composition, Bab 6).

## 4.6 Multiple Inheritance

Python mengizinkan sebuah class mewarisi **lebih dari satu** parent sekaligus. Fitur ini disebut *multiple inheritance*. Caranya, tuliskan semua parent di dalam tanda kurung, dipisahkan koma.

### Menggabungkan Kemampuan dengan Mixin

Pemakaian multiple inheritance yang paling aman adalah pola *mixin*: class kecil yang hanya menyumbang satu kemampuan (beberapa method), tidak dimaksudkan untuk dipakai sendirian, dan biasanya tidak memiliki `__init__`.

**Hewan dengan mixin kemampuan**

```
class Hewan:
    def __init__(self, nama):
        self.nama = nama

    def info(self):
        return f"{self.nama} adalah hewan"

class BisaTerbang:                      # mixin
    def terbang(self):
        return f"{self.nama} terbang di udara"

class BisaBerenang:                     # mixin
    def berenang(self):
        return f"{self.nama} berenang di air"

class Bebek(Hewan, BisaTerbang, BisaBerenang):
    pass

class Lele(Hewan, BisaBerenang):
    pass

kwek = Bebek("Kwek")
print(kwek.info())
```

```
print(kwek.terbang())
print(kwek.berenang())

lele = Lele("Lele")
print(lele.berenang())
print(hasattr(lele, "terbang"))
```

**Keluaran**

```
Kwek adalah hewan
Kwek terbang di udara
Kwek berenang di air
Lele berenang di air
False
```

Dengan mixin, setiap kemampuan ditulis sekali dan dapat "ditempelkan" ke class mana pun yang membutuhkannya. `Bebek` mendapat kemampuan terbang dan berenang, sedangkan `Lele` hanya berenang.

## Method Resolution Order (MRO)

Bagaimana jika dua parent memiliki method dengan nama yang sama? Python menentukan pemenangnya menggunakan MRO. Aturan sederhananya: **parent yang ditulis lebih kiri diperiksa lebih dulu**.

**Urutan parent menentukan pemenang**

```
class Printer:
    def mulai(self):
        return "Mencetak dokumen"

class Scanner:
    def mulai(self):
        return "Memindai dokumen"

class MesinA(Printer, Scanner):
    pass

class MesinB(Scanner, Printer):
    pass

print(MesinA().mulai())
print(MesinB().mulai())
print([k.__name__ for k in MesinA.__mro__])
```

Keluaran

```
Mencetak dokumen
Memindai dokumen
['MesinA', 'Printer', 'Scanner', 'object']
```

## Masalah Diamond

Kasus yang lebih rumit muncul ketika dua parent ternyata mewarisi leluhur yang sama, sehingga hierarkinya berbentuk belah ketupat (*diamond*). Pertanyaannya: berapa kali, dan dalam urutan apa, method leluhur dijalankan?

Diamond dan super()

```
class A:
    def halo(self):
        return "A"

class B(A):
    def halo(self):
        return "B -> " + super().halo()

class C(A):
    def halo(self):
        return "C -> " + super().halo()

class D(B, C):
    def halo(self):
        return "D -> " + super().halo()

print(D().halo())
print([k.__name__ for k in D.__mro__])
```

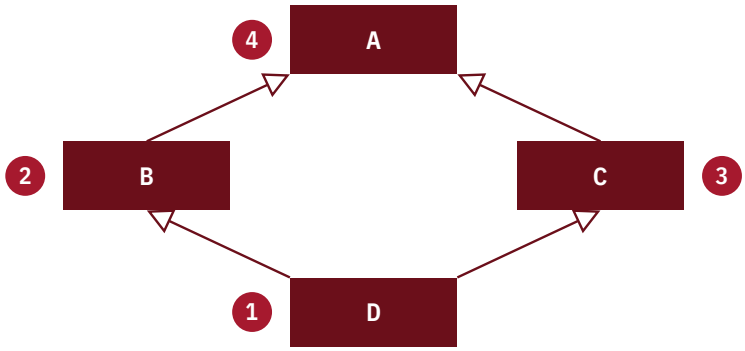
Keluaran

```
D -> B -> C -> A
['D', 'B', 'C', 'A', 'object']
```

Ada dua hal penting dari contoh ini. Pertama, `A` hanya dijalankan **sekali**, bukan dua kali. Kedua, `super()` di dalam `B` ternyata memanggil `C`, bukan `A`. Ini karena `super()` tidak berarti "parent saya", melainkan "class *berikutnya* dalam MRO object ini". Python menyusun MRO dengan algoritma bernama *C3 linearization*, yang menjamin setiap child selalu muncul sebelum parent-nya dan urutan parent dari kiri ke kanan tetap dihormati.

GUNAKAN DENGAN HEMAT

Multiple inheritance mudah menimbulkan kebingungan, terutama bila beberapa parent memiliki




Angka menunjukkan urutan MRO. `super()` di dalam `B` menuju **C**, bukan langsung ke A, dan A hanya dikunjungi sekali.

**Gambar 4.4.** Pewarisan berbentuk diamond dan urutan MRO-nya.

`__init__` dengan parameter berbeda. Pedoman praktisnya:

- Gunakan multiple inheritance terutama untuk *mixin* kecil tanpa `__init__`.
- Letakkan parent utama di posisi paling kiri, mixin di sebelah kanannya.
- Jika kebutuhannya adalah "memiliki" sesuatu, gunakan composition (Bab 6), bukan multiple inheritance.

## 4.7 Rangkuman

- **Inheritance** membuat class baru (child) yang mewarisi seluruh attribute dan method class lain (parent). Gunakan bila hubungan *is-a* terpenuhi.
- **Parent class** berisi hal-hal yang berlaku umum. Semua class di Python 3 pada akhirnya mewarisi `object`.
- **Child class** ditulis `class Anak(Induk):`. Jika child punya `__init__` sendiri, panggil `super().__init__(…)` agar attribute parent ikut disiapkan.
- `isinstance()` ikut memperhitungkan pewarisan; object child juga merupakan instance parent. Attribute private parent tidak dapat diakses child dengan nama biasa.
- **Overriding** menulis ulang method warisan dengan nama yang sama. Child dapat mengganti total atau memperluas dengan `super().method()`. Python memakai method pertama yang ditemukan dari class object ke atas.
- **Nested inheritance** membentuk rantai pewarisan bertingkat; urutan pencariannya dapat dilihat di `__mro__`.
- **Multiple inheritance** mewarisi beberapa parent sekaligus. Parent paling kiri diperiksa lebih dulu, `super()` mengikuti MRO, dan pola paling aman adalah *mixin*.

## 4.8 Kuis Teoritis

## Pilihan Ganda

Pilihlah satu jawaban yang paling tepat.

1. Hubungan antar-class yang paling tepat dimodelkan dengan inheritance adalah ...

   a. mobil dan roda
   b. kucing dan hewan
   c. mahasiswa dan KRS
   d. rekening dan nasabah

2. Penulisan yang benar untuk membuat class `Kucing` sebagai turunan `Hewan` adalah ...

   a. `class Kucing extends Hewan:`
   b. `class Kucing: Hewan`
   c. `class Kucing(Hewan):`
   d. `class Hewan(Kucing):`

3. Fungsi pemanggilan `super().__init__(nama)` di dalam constructor child adalah ...

   a. membuat object parent yang terpisah
   b. menjalankan constructor parent untuk object yang sedang dibuat
   c. menghapus attribute milik parent
   d. mengubah child menjadi parent

4. Perhatikan kode berikut.

Soal 4

```python
class Hewan:
    def suara(self):
        return "..."

class Kucing(Hewan):
    def suara(self):
        return "Meong"

class Anggora(Kucing):
    pass

print(Anggora().suara())
```

Keluaran kode tersebut adalah ...

   a. `...`
   b. `Meong`
   c. `None`
   d. `AttributeError`

5. Sebuah child class mendefinisikan `__init__` tanpa memanggil `super().__init__()`. Akibatnya adalah ...

   a. program langsung error saat class didefinisikan
   b. attribute yang dibuat di constructor parent tidak ada pada object child
   c. Python otomatis memanggil constructor parent
   d. semua method parent tidak bisa dipakai

6. Diketahui `class Mobil(Kendaraan)` dan `m = Mobil()`. Pernyataan yang **salah** adalah ...

   a. `isinstance(m, Mobil)` bernilai `True`
   b. `isinstance(m, Kendaraan)` bernilai `True`
   c. `type(m) == Kendaraan` bernilai `True`
   d. `issubclass(Mobil, object)` bernilai `True`

7. Class `Laptop(Perangkat, BisaWiFi)` memiliki MRO yang dimulai dengan ...

   a. `Laptop, Perangkat, BisaWiFi`
   b. `Laptop, BisaWiFi, Perangkat`
   c. `Perangkat, BisaWiFi, Laptop`
   d. `object, Perangkat, Laptop`

8. Perhatikan kode berikut.

Soal 8

```python
class A:
    def f(self):
        return "A"

class B(A):
    def f(self):
        return "B" + super().f()

class C(A):
    def f(self):
        return "C" + super().f()

class D(C, B):
    def f(self):
        return "D" + super().f()

print(D().f())
```

   Keluaran kode tersebut adalah ...

   a. `DBCA`
   b. `DCBA`

c. `DCA`
d. `DCABA`

9. Pada method yang di-override, cara memanggil versi milik parent adalah ...

   a. `parent.method()`
   b. `self.method()`
   c. `super().method()`
   d. `base.method(self)`

10. Pernyataan yang tepat tentang *mixin* adalah ...

    a. class utama yang menjadi induk seluruh hierarki
    b. class kecil yang menyumbang kemampuan tertentu dan tidak dipakai sendirian
    c. class yang wajib memiliki `__init__` dengan banyak parameter
    d. nama lain dari class `object`

### Uraian

1. Jelaskan hubungan *is-a* dan berikan dua pasang class yang tepat dimodelkan dengan inheritance serta dua pasang yang tidak tepat, beserta alasannya.
2. Jelaskan perbedaan antara overriding yang *mengganti total* dan yang *memperluas* method parent. Kapan masing-masing sebaiknya digunakan?
3. Pada studi kasus penggajian, method `slip()` hanya ditulis di class `Karyawan`. Jelaskan mengapa hasilnya tetap berbeda untuk karyawan tetap dan karyawan kontrak.
4. Mengapa attribute `self.__saldo` milik parent tidak bisa diakses dari child class, sedangkan `self._poin` bisa? Kaitkan jawaban Anda dengan name mangling.
5. Jelaskan apa yang dimaksud dengan masalah diamond dan bagaimana MRO serta `super()` di Python menanganinya.

## 4.9 Praktik Mandiri

Kerjakan praktik berikut secara berurutan. Simpan setiap praktik dalam file `.py` terpisah, jalankan, dan pastikan keluarannya sesuai dengan contoh.

### Praktik 1: Hierarki Bangun Datar

Buat hierarki class berikut:

- `BangunDatar` dengan attribute `nama`, method `luas()` dan `keliling()` yang mengembalikan `0`, serta method `info()` yang menampilkan nama, luas, dan keliling dengan memanggil kedua method tersebut.
- `PersegiPanjang(BangunDatar)` dan `Lingkaran(BangunDatar)` yang meng-override `luas()` dan `keliling()`.

- `Persegi(PersegiPanjang)` yang constructor-nya hanya menerima `sisi`, lalu memanggil constructor `PersegiPanjang` dengan panjang dan lebar yang sama. `Persegi` tidak boleh menulis ulang `luas()` maupun `keliling()`.

Simpan beberapa bangun datar dalam satu list, tampilkan `info()` masing-masing, lalu cetak MRO class `Persegi`.

**Contoh keluaran Praktik 1**

```
Persegi Panjang  luas   24.00  keliling  20.00
Lingkaran        luas   78.54  keliling  31.42
Persegi          luas   25.00  keliling  20.00
['Persegi', 'PersegiPanjang', 'BangunDatar', 'object']
```

### Praktik 2: Koleksi Perpustakaan

Buat class `Koleksi` dengan attribute `judul` dan `tahun`, serta method `lama_pinjam()` (default 7 hari), `denda_per_hari()` (default 1000), dan `hitung_denda(hari_terlambat)`. Kemudian buat turunannya:

- `Buku`: menambah attribute `penulis`; lama pinjam 14 hari.
- `Majalah`: menambah attribute `edisi`; lama pinjam 3 hari, denda 500 per hari.
- `BukuReferensi(Buku)`: tidak boleh dipinjam, sehingga `lama_pinjam()` mengembalikan `0`, dan denda 5000 per hari.

Setiap class meng-override `__str__` dengan memperluas versi parent memakai `super()`. Method `hitung_denda()` hanya boleh ditulis di `Koleksi`.

**Contoh keluaran Praktik 2**

```
[Buku] Laskar Pelangi (2005) - Andrea Hirata | pinjam 14 hari
[Majalah] Bobo (2026) - edisi 41 | pinjam 3 hari
[BukuReferensi] KBBI (2016) - Badan Bahasa | tidak dapat dipinjam
Denda terlambat 4 hari: Rp4,000 | Rp2,000 | Rp20,000
```

### Praktik 3: Perangkat Pintar dengan Mixin

Rancang class untuk perangkat elektronik pintar menggunakan multiple inheritance.

#### Langkah Pengerjaan

1. Buat class utama `Perangkat` dengan attribute `nama` dan method `nyalakan()`.
2. Buat minimal tiga mixin tanpa `__init__`, misalnya `BisaWiFi`, `BisaBluetooth`, dan `BisaPutarMusik`, masing-masing dengan satu atau dua method.
3. Buat class `Smartphone`, `SpeakerPintar`, dan `Laptop` yang menggabungkan `Perangkat` dengan mixin yang sesuai. Tentukan sendiri kombinasinya dan jelaskan alasannya dalam

komentar.

4. Tambahkan method `status()` di `Perangkat` dan di setiap mixin yang memanggil `super().status()` lalu menambahkan informasinya sendiri. Cetak hasil `status()` dan `__mro__` untuk ketiga class, lalu jelaskan urutan yang muncul.
5. Tunjukkan dengan `hasattr()` bahwa perangkat yang tidak memakai mixin tertentu memang tidak memiliki kemampuan tersebut.

**KRITERIA PENILAIAN**

- Ketepatan hierarki dan hubungan *is-a* pada setiap praktik.
- Pemakaian `super()` pada constructor dan method yang diperluas.
- Tidak ada method yang ditulis ulang di child jika versi parent sudah cukup.
- Penjelasan urutan MRO pada Praktik 3 sesuai dengan hasil yang dicetak program.

Bab 05

# Abstract Method, Data Class, dan Typing

*Kontrak, class pembawa data, dan cara Python menilai sebuah tipe.*

05

DI BAB INI

- Abstract method
- Data class
- Attribute mutability
- Duck typing
- Structural typing

Di Praktik Mandiri Bab 4, method `luas()` pada class `BangunDatar` mengembalikan `0`. Cara itu menyimpan masalah: jika seseorang membuat class `Segitiga` lalu lupa meng-override `luas()`, program tidak error, tetapi diam-diam menghasilkan luas nol. Kita butuh cara untuk *mewajibkan* child menyediakan method tertentu.

Bab ini membahas lima topik yang saling berkaitan. *Abstract method* dipakai untuk membuat kontrak yang wajib dipenuhi child class. *Data class* menghemat penulisan class yang tugas utamanya membawa data. *Mutability* menjelaskan mengapa sebagian data bisa berubah "dari jauh". Terakhir, *duck typing* dan *structural typing* menjelaskan cara Python menilai apakah sebuah object cocok dipakai di suatu tempat.

## 5.1 Abstract Method

*Abstract method* adalah method yang hanya dideklarasikan namanya di parent, tanpa implementasi yang berarti, dan **wajib** diimplementasikan oleh setiap child class. Class yang memiliki setidaknya satu abstract method disebut *abstract class*. Abstract class tidak bisa dibuat object-nya secara langsung; ia hanya berfungsi sebagai *kontrak* atau cetakan bagi turunannya.

### Masalah: Lupa Meng-override

Parent dengan nilai default yang menyesatkan

```
class BangunDatar:
    def luas(self):
        return 0                    # nilai "darurat"

class Segitiga(BangunDatar):
    def __init__(self, alas, tinggi):
        self.alas = alas
        self.tinggi = tinggi
    # lupa menulis luas()

print(Segitiga(10, 6).luas())
```

Keluaran

```
0
```

Tidak ada pesan kesalahan sama sekali, padahal hasilnya jelas salah. Kesalahan seperti ini bisa lolos hingga ke tangan pengguna.

### Modul `abc`: `ABC` dan `@abstractmethod`

Python menyediakan modul `abc` (*Abstract Base Classes*). Sebuah class menjadi abstract class bila mewarisi `ABC`, dan method-nya ditandai sebagai abstract dengan decorator `@abstractmethod`.

**Abstract class BangunDatar**

```
from abc import ABC, abstractmethod
import math

class BangunDatar(ABC):
    def __init__(self, nama):
        self.nama = nama

    @abstractmethod
    def luas(self):
        """Wajib diimplementasikan child."""

    @abstractmethod
    def keliling(self):
        """Wajib diimplementasikan child."""

    def info(self):                        # method biasa (konkret)
        return f"{self.nama}: luas {self.luas():.2f}, keliling {self.keliling()
    :.2f}"

class Persegi(BangunDatar):
    def __init__(self, sisi):
        super().__init__("Persegi")
        self.sisi = sisi

    def luas(self):
        return self.sisi ** 2

    def keliling(self):
        return 4 * self.sisi

class Lingkaran(BangunDatar):
    def __init__(self, r):
        super().__init__("Lingkaran")
        self.r = r

    def luas(self):
        return math.pi * self.r ** 2
```

```
37
38      def keliling(self):
39          return 2 * math.pi * self.r
40
41  for b in [Persegi(5), Lingkaran(7)]:
42      print(b.info())
```

**Keluaran**

```
Persegi: luas 25.00, keliling 20.00
Lingkaran: luas 153.94, keliling 43.98
```

Kini Python menjaga kontrak tersebut. Abstract class tidak bisa dibuat object-nya, dan child yang belum mengimplementasikan *semua* abstract method juga ditolak.

**Kontrak yang dilanggar**

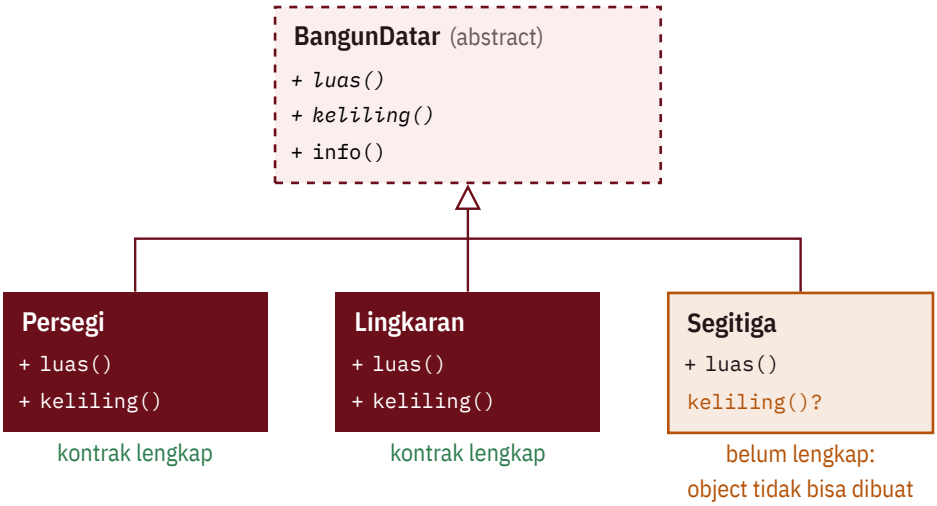
```
1  class Segitiga(BangunDatar):
2      def __init__(self, alas, tinggi):
3          super().__init__("Segitiga")
4          self.alas = alas
5          self.tinggi = tinggi
6
7      def luas(self):
8          return 0.5 * self.alas * self.tinggi
9      # keliling() belum ditulis
10
11 BangunDatar("Bebas")     # gagal: abstract class
12 Segitiga(10, 6)          # gagal: kontrak belum lengkap
```

**Keluaran**

```
TypeError: Can't instantiate abstract class BangunDatar without an implementation
    for abstract methods 'keliling', 'luas'
TypeError: Can't instantiate abstract class Segitiga without an implementation for
     abstract method 'keliling'
```

Setiap baris di atas sebenarnya menghentikan program; keduanya ditampilkan bersamaan agar mudah dibandingkan. Yang penting, kesalahan kini muncul *saat object dibuat*, jauh sebelum nilai yang salah sempat dipakai.

Perhatikan bahwa abstract class boleh memiliki method biasa seperti `info()` dan constructor. Method konkret tersebut bahkan boleh memanggil abstract method (`self.luas()`), karena dijamin setiap object yang berhasil dibuat sudah mengimplementasikannya.




**Gambar 5.1.** Abstract class sebagai kontrak. Method bercetak miring wajib diimplementasikan setiap child; `info()` adalah method konkret yang langsung diwarisi.

## Abstract Property

Decorator `@property` dan `@abstractmethod` dapat digabung untuk mewajibkan child menyediakan sebuah property. Urutannya: `@property` di atas, `@abstractmethod` di bawah.

**Abstract property**

```python
from abc import ABC, abstractmethod

class Kendaraan(ABC):
    @property
    @abstractmethod
    def jumlah_roda(self):
        ...

class Motor(Kendaraan):
    @property
    def jumlah_roda(self):
        return 2

print(Motor().jumlah_roda)
```

**Keluaran**

```
2
```

### ABSTRACT CLASS DAN INTERFACE

Bahasa seperti Java memiliki konsep *interface*, yaitu kontrak berisi daftar method tanpa implementasi. Di Python, abstract class yang seluruh method-nya abstract memainkan peran yang

sama. Badan abstract method cukup diisi docstring atau `...` (*ellipsis*).

## 5.2 Data Class

Banyak class yang tugas utamanya hanya *membawa data*: koordinat titik, data mahasiswa, item keranjang belanja, dan sebagainya. Untuk class seperti itu, kita hampir selalu menulis tiga method yang sama polanya: `__init__` untuk menyimpan data, `__repr__` agar object mudah dicetak, dan `__eq__` agar dua object berisi data sama dianggap sama.

**Class pembawa data yang ditulis manual**

```
class Titik:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return f"Titik(x={self.x}, y={self.y})"

    def __eq__(self, lain):
        if not isinstance(lain, Titik):
            return NotImplemented
        return (self.x, self.y) == (lain.x, lain.y)
```

Lima belas baris hanya untuk menyimpan dua angka. Modul `dataclasses` menyediakan decorator `@dataclass` yang menuliskan method-method tersebut secara otomatis berdasarkan daftar *field* yang kita deklarasikan.

### Decorator `@dataclass`

Field ditulis sebagai class attribute yang diberi *type hint* (anotasi tipe, dibahas lebih lanjut di Subbab Structural Typing). Dari daftar field itulah `@dataclass` membangkitkan `__init__`, `__repr__`, dan `__eq__`.

**Titik sebagai data class**

```
from dataclasses import dataclass

@dataclass
class Titik:
    x: float
    y: float

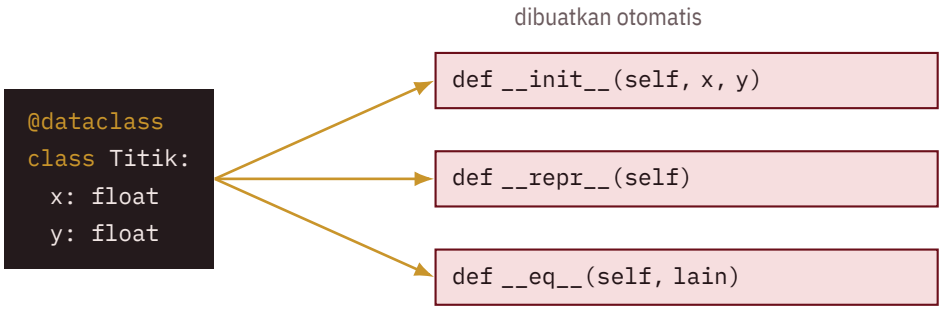
    def jarak_ke_pusat(self):              # method biasa tetap boleh
        return (self.x ** 2 + self.y ** 2) ** 0.5

```

```
a = Titik(3, 4)
b = Titik(3, 4)
print(a)
print(a == b)
print(a.jarak_ke_pusat())
```

**Keluaran**

```
Titik(x=3, y=4)
True
5.0
```




**Gambar 5.2.** Dari daftar field, `@dataclass` membangkitkan method-method pembawa data secara otomatis.

## Nilai Default dan `field()`

Field boleh diberi nilai default. Seperti parameter fungsi, field ber-default harus ditulis setelah field tanpa default. Untuk default berupa object *mutable* seperti list atau dictionary, Python melarang penulisan langsung; gunakan `field(default_factory=…)` agar setiap object mendapat salinan baru. Alasannya dibahas di subbab Attribute Mutability.

**Nilai default**

```
from dataclasses import dataclass, field

@dataclass
class Mahasiswa:
    nama: str
    nim: str
    semester: int = 1
    nilai: list = field(default_factory=list)   # list baru untuk tiap object

ani = Mahasiswa("Ani", "001")
budi = Mahasiswa("Budi", "002", semester=3)
ani.nilai.append(90)
print(ani)
print(budi)
```

Keluaran

```
Mahasiswa(nama='Ani', nim='001', semester=1, nilai=[90])
Mahasiswa(nama='Budi', nim='002', semester=3, nilai=[])
```

Jika kita menulis `nilai: list = []`, Python langsung menolak saat class didefinisikan dengan pesan `ValueError: mutable default <class 'list'> for field nilai is not allowed: use default_factory`.

### Opsi `order` dan `frozen`

Decorator `@dataclass` menerima beberapa opsi. Dua yang paling sering dipakai:

- `order=True`: membangkitkan <, <=, >, >=, sehingga object bisa diurutkan. Perbandingan dilakukan field demi field sesuai urutan deklarasi.
- `frozen=True`: membuat object *tidak bisa diubah* setelah dibuat. Setiap upaya mengubah field menghasilkan `FrozenInstanceError`.

order dan frozen

```
from dataclasses import dataclass, field

@dataclass(order=True)
class Peserta:
    skor: int
    nama: str = field(compare=False)      # tidak ikut dibandingkan

@dataclass(frozen=True)
class Koordinat:
    lintang: float
    bujur: float

hasil = [Peserta(78, "Ani"), Peserta(92, "Budi"), Peserta(85, "Citra")]
for p in sorted(hasil, reverse=True):
    print(p.skor, p.nama)

unesa = Koordinat(-7.30, 112.67)
unesa.lintang = 0
```

Keluaran

```
92 Budi
85 Citra
78 Ani
Traceback (most recent call last):
  ...
dataclasses.FrozenInstanceError: cannot assign to field 'lintang'
```

### Validasi dengan `__post_init__`

Karena `__init__` dibuat otomatis, validasi dan field turunan ditulis di method khusus `__post_init__`, yang dijalankan tepat setelah `__init__` selesai. Field yang nilainya dihitung (bukan diberikan pengguna) ditandai `field(init=False)`.

**Validasi dan field turunan**

```python
from dataclasses import dataclass, field

@dataclass
class ItemBelanja:
    nama: str
    harga: int
    jumlah: int = 1
    subtotal: int = field(init=False)

    def __post_init__(self):
        if self.harga <= 0 or self.jumlah <= 0:
            raise ValueError("Harga dan jumlah harus lebih dari nol.")
        self.subtotal = self.harga * self.jumlah

print(ItemBelanja("Pulpen", 3500, 4))
ItemBelanja("Buku", -1)
```

**Keluaran**

```
ItemBelanja(nama='Pulpen', harga=3500, jumlah=4, subtotal=14000)
Traceback (most recent call last):
  ...
ValueError: Harga dan jumlah harus lebih dari nol.
```

**Tabel 5.1.** Ringkasan fitur data class.

| Penulisan | Hasilnya |
|---|---|
| `@dataclass` | `__init__`, `__repr__`, `__eq__` dibuat otomatis |
| `@dataclass(order=True)` | ditambah operator perbandingan sehingga bisa diurutkan |
| `@dataclass(frozen=True)` | object tidak bisa diubah setelah dibuat |
| `field(default_factory=list)` | default mutable yang aman, baru untuk tiap object |
| `field(compare=False)` | field tidak ikut perbandingan == dan pengurutan |
| `field(init=False)` | field tidak menjadi parameter constructor |
| `__post_init__` | tempat validasi dan menghitung field turunan |

**KAPAN MEMAKAI DATA CLASS?**

Gunakan data class untuk class yang *intinya adalah data*, dengan sedikit perilaku. Untuk class yang kaya aturan dan perlu encapsulation ketat, misalnya `Rekening` dengan saldo yang hanya boleh berubah lewat `setor()` dan `tarik()`, class biasa dengan property sering kali lebih tepat.

## 5.3 Attribute Mutability

*Mutability* adalah sifat apakah isi sebuah object dapat diubah setelah object itu dibuat. Object *mutable* dapat diubah isinya; object *immutable* tidak dapat diubah, sehingga setiap "perubahan" sebenarnya menghasilkan object baru.

**Tabel 5.2.** Tipe bawaan berdasarkan mutability.

| Sifat | Tipe |
|---|---|
| Immutable | `int`, `float`, `bool`, `str`, `tuple`, `frozenset`, data class dengan `frozen=True` |
| Mutable | `list`, `dict`, `set`, sebagian besar object dari class buatan sendiri |

### Variabel Adalah Label, Bukan Kotak

Untuk memahami mutability, kita perlu meluruskan satu anggapan. Variabel di Python bukan "kotak" yang berisi nilai, melainkan *label* yang ditempelkan pada sebuah object. Pernyataan `b = a` tidak menyalin object; ia hanya menempelkan label kedua pada object yang sama. Kondisi ini disebut *aliasing*.

**Dua label, satu object**

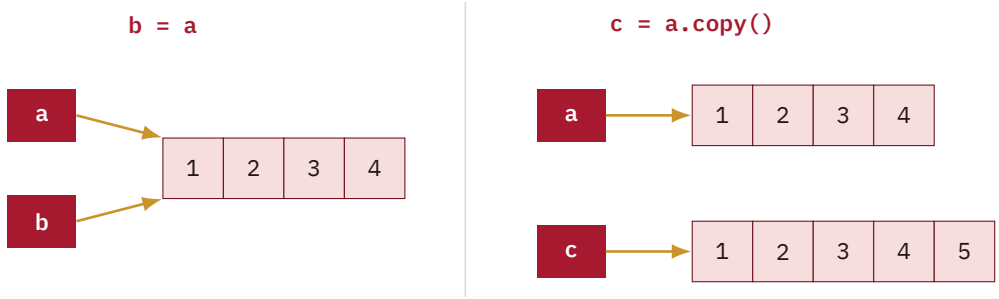
```
a = [1, 2, 3]
b = a              # b menunjuk object yang sama
b.append(4)
print(a)
print(a is b)

c = a.copy()       # object baru dengan isi sama
c.append(5)
print(a, c)
print(a is c)
```

**Keluaran**

```
[1, 2, 3, 4]
True
[1, 2, 3, 4] [1, 2, 3, 4, 5]
```

```
False
```




**Gambar 5.3.** Kiri: `a` dan `b` adalah dua label pada satu list, sehingga perubahan lewat `b` terlihat lewat `a`. Kanan: `copy()` membuat list baru yang terpisah.

Untuk object immutable, aliasing tidak berbahaya. Operasi seperti `y += 1` pada sebuah `int` tidak mengubah object lama, melainkan membuat object baru dan memindahkan label `y` ke sana, sehingga variabel lain yang menunjuk object lama tidak terpengaruh.

### Jebakan: Default Argument Mutable

Nilai default parameter fungsi dibuat **satu kali**, saat fungsi didefinisikan, bukan setiap kali fungsi dipanggil. Jika nilai default itu mutable, semua pemanggilan memakai object yang sama. Di dalam class, akibatnya semua object diam-diam berbagi satu list.

**Bug default mutable**

```
class Keranjang:
    def __init__(self, pemilik, isi=[]):     # BAHAYA
        self.pemilik = pemilik
        self.isi = isi

    def tambah(self, barang):
        self.isi.append(barang)

k1 = Keranjang("Ani")
k2 = Keranjang("Budi")
k1.tambah("Buku")
print(k2.isi)            # keranjang Budi ikut berisi buku!
```

**Keluaran**

```
['Buku']
```

Perbaikan yang lazim adalah memakai `None` sebagai default, lalu membuat list baru di dalam fungsi.

Perbaikan

```
class Keranjang:
    def __init__(self, pemilik, isi=None):
        self.pemilik = pemilik
        self.isi = [] if isi is None else list(isi)
```

Perhatikan pemakaian `list(isi)`: walaupun pemanggil memberikan list miliknya sendiri, keranjang menyimpan *salinannya*. Dengan begitu, perubahan pada list milik pemanggil tidak ikut mengubah isi keranjang, dan sebaliknya. Ini prinsip yang sama dengan getter yang mengembalikan salinan di Bab 2.

### Salinan Dangkal dan Salinan Dalam

Method `copy()` dan fungsi `copy.copy()` membuat *salinan dangkal* (*shallow copy*): wadah luarnya baru, tetapi isinya masih object yang sama. Untuk struktur bersarang, seperti list berisi list, gunakan `copy.deepcopy()` yang menyalin seluruh isi hingga ke dalam.

shallow copy vs deep copy

```
import copy

kelas = [["Ani", 90], ["Budi", 85]]
dangkal = copy.copy(kelas)
dalam = copy.deepcopy(kelas)

kelas[0][1] = 0          # ubah nilai Ani di data asli
print(dangkal[0])        # ikut berubah: isi masih dipakai bersama
print(dalam[0])          # aman: seluruh isi ikut disalin
```

Keluaran

```
['Ani', 0]
['Ani', 90]
```

**FROZEN TIDAK BERARTI MEMBEKU SELURUHNYA**

Data class dengan `frozen=True` hanya mencegah field *diganti*. Jika sebuah field berisi list, isi list itu tetap bisa diubah dengan `append()`. Untuk data yang benar-benar tidak boleh berubah, gunakan tipe immutable di dalamnya, misalnya `tuple` sebagai pengganti `list`.

## 5.4 Duck Typing

*"Jika ia berjalan seperti bebek dan bersuara seperti bebek, maka ia adalah bebek."*

*Duck typing* adalah gaya Python dalam menilai object: yang penting bukan *class apa* object itu, melainkan *apa yang bisa dilakukannya*. Selama object memiliki method atau

attribute yang dibutuhkan, ia boleh dipakai, walaupun tidak ada hubungan pewarisan sama sekali.

**Tiga class tanpa hubungan pewarisan**

```
class Bebek:
    def suara(self):
        return "Kwek kwek"

class Robot:
    def suara(self):
        return "Bip bup"

class Radio:
    def suara(self):
        return "Siaran berita pukul tujuh"

def bunyikan(benda):
    print(benda.suara())        # tidak peduli class-nya

for x in [Bebek(), Robot(), Radio()]:
    bunyikan(x)
```

**Keluaran**

```
Kwek kwek
Bip bup
Siaran berita pukul tujuh
```

Fungsi `bunyikan()` tidak memeriksa tipe dan tidak mensyaratkan parent tertentu. Ia hanya "mempercayai" bahwa benda yang diberikan bisa bersuara.

## Duck Typing di Pustaka Bawaan

Banyak fitur Python bekerja berdasarkan duck typing. Fungsi `len()` bekerja pada object apa pun yang memiliki method `__len__`, dan perulangan `for` bekerja pada object apa pun yang memiliki `__iter__`. Dengan menambahkan dua method tersebut, class buatan kita langsung bisa dipakai seperti list.

**Class sendiri yang berperilaku seperti koleksi**

```
class Playlist:
    def __init__(self, nama, lagu):
        self.nama = nama
        self._lagu = list(lagu)

    def __len__(self):
        return len(self._lagu)

```

```
    def __iter__(self):
        return iter(self._lagu)

pagi = Playlist("Pagi", ["Lagu A", "Lagu B", "Lagu C"])
print(len(pagi))
for judul in pagi:
    print("-", judul)
print("Lagu B" in pagi)
```

Keluaran

```
3
- Lagu A
- Lagu B
- Lagu C
True
```

## EAFP vs LBYL

Ada dua gaya menghadapi object yang mungkin tidak punya kemampuan yang dibutuhkan:

- **LBYL** (*Look Before You Leap*): periksa dulu, baru pakai, misalnya dengan `hasattr()`.
- **EAFP** (*Easier to Ask Forgiveness than Permission*): langsung pakai, lalu tangani kesalahannya jika gagal. Gaya ini dianggap lebih *Pythonic*.

Dua gaya

```
class Batu:
    pass

def bunyikan_lbyl(benda):
    if hasattr(benda, "suara"):
        print(benda.suara())
    else:
        print("(diam)")

def bunyikan_eafp(benda):
    try:
        print(benda.suara())
    except AttributeError:
        print("(diam)")

bunyikan_lbyl(Batu())
bunyikan_eafp(Batu())
```

Keluaran

```
(diam)
(diam)
```

Blok `try–except` dibahas lengkap di Bab 8. Untuk saat ini, cukup pahami bahwa duck typing memberi keleluasaan besar, dengan satu kelemahan: kesalahan baru ketahuan saat program *berjalan*. Subbab berikutnya membahas cara mendeteksinya lebih awal.

## 5.5 Structural Typing

Ada dua cara sebuah bahasa menilai kecocokan tipe:

- **Nominal typing**: object cocok bila class-nya *dinyatakan* sebagai turunan tipe yang diminta. Contohnya `Persegi` cocok sebagai `BangunDatar` karena ditulis `class Persegi(BangunDatar)`. Abstract class dari subbab pertama bekerja dengan cara ini.
- **Structural typing**: object cocok bila *bentuknya* sesuai, yaitu memiliki method dan attribute yang diminta, tanpa perlu menyatakan hubungan pewarisan.

Structural typing adalah versi duck typing yang dapat diperiksa oleh alat bantu sebelum program dijalankan. Untuk itu kita perlu mengenal *type hint* terlebih dahulu.

### Type Hint Dasar

*Type hint* adalah anotasi yang menyatakan tipe yang diharapkan untuk parameter, nilai kembalian, dan variabel. Kita sudah memakainya pada field data class.

Type hint

```python
def luas_persegi_panjang(panjang: float, lebar: float) -> float:
    return panjang * lebar

def cari_mahasiswa(nim: str, data: dict[str, str]) -> str | None:
    return data.get(nim)

nilai: list[int] = [90, 85, 78]

print(luas_persegi_panjang(4, 2.5))
print(cari_mahasiswa("002", {"001": "Ani", "002": "Budi"}))
print(luas_persegi_panjang("ab", 3))   # tetap jalan: hint tidak dipaksakan
```

Keluaran

```
10.0
Budi
ababab
```

Baris terakhir menunjukkan hal penting: **Python tidak memeriksa type hint saat program berjalan**. Type hint adalah dokumentasi yang dapat dibaca manusia, editor (untuk saran otomatis dan peringatan), serta alat pemeriksa statis seperti `mypy`.

**Tabel 5.3.** Penulisan type hint yang sering dipakai.

| Type hint | Artinya |
|---|---|
| `int`, `str`, `float`, `bool` | tipe dasar |
| `list[int]` | list berisi bilangan bulat |
| `dict[str, float]` | dictionary dengan kunci string dan nilai float |
| `tuple[int, str]` | tuple dua elemen: int lalu str |
| `str \| None` | string atau `None` (Python 3.10 ke atas) |
| `-> None` | fungsi tidak mengembalikan nilai |
| `Mahasiswa` | object dari class `Mahasiswa` atau turunannya |

### `typing.Protocol`

`Protocol` dari modul `typing` memungkinkan kita mendefinisikan sebuah "bentuk" yang diharapkan. Class lain dianggap memenuhi protocol tersebut **tanpa perlu mewarisinya**, selama memiliki method dengan nama dan bentuk yang sama.

suara.py

```
from typing import Protocol

class BisaBersuara(Protocol):
    def suara(self) -> str: ...

class Bebek:                        # tidak mewarisi BisaBersuara
    def suara(self) -> str:
        return "Kwek kwek"

class Robot:
    def suara(self) -> str:
        return "Bip bup"

class Batu:
    pass

def bunyikan(benda: BisaBersuara) -> None:
    print(benda.suara())

bunyikan(Bebek())
bunyikan(Robot())
```

```
22 bunyikan(Batu())                    # akan ditandai oleh mypy
```

Saat dijalankan biasa, dua baris pertama berhasil dan baris ketiga gagal dengan `AttributeError`. Namun dengan pemeriksa statis `mypy`, kesalahan itu sudah terdeteksi *sebelum* program dijalankan.

**Hasil pip install mypy, lalu mypy suara.py**

```
suara.py:22: error: Argument 1 to "bunyikan" has incompatible type "Batu";
    expected "BisaBersuara"  [arg-type]
suara.py:22: note: "Batu" is missing following "BisaBersuara" protocol member:
suara.py:22: note:     suara
Found 1 error in 1 file (checked 1 source file)
```

**TENTANG KELUARAN MYPY**

`mypy` adalah alat terpisah yang dipasang dengan `pip install mypy`. Susunan pesannya dapat sedikit berbeda antarversi, tetapi intinya sama: nama class, protocol yang diharapkan, dan member yang belum ada. Editor seperti VS Code (dengan ekstensi Python/Pylance) menampilkan peringatan serupa langsung di layar sebagai garis bawah berwarna.

## Protocol yang Dapat Diperiksa Saat Program Berjalan

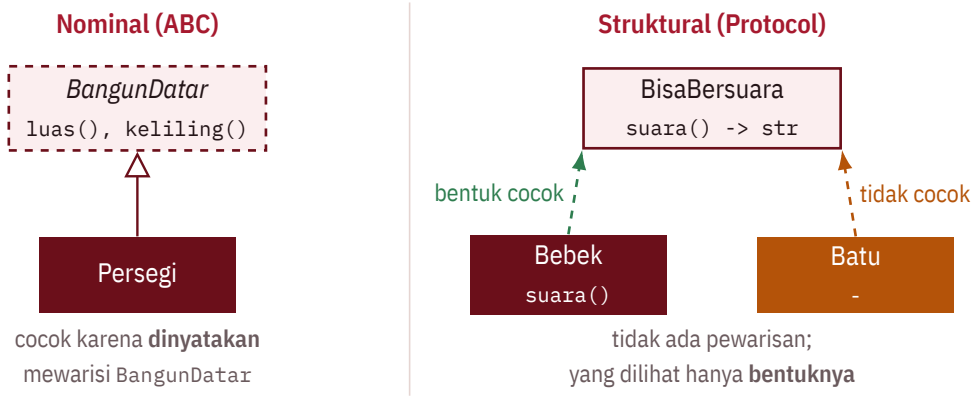
Secara bawaan, protocol hanya berguna bagi pemeriksa statis. Jika ingin memakainya dengan `isinstance()`, tambahkan decorator `@runtime_checkable`. Pemeriksaan ini hanya melihat *keberadaan* method, bukan parameter maupun tipe kembaliannya.

**runtime_checkable**

```
1  from typing import Protocol, runtime_checkable
2
3  @runtime_checkable
4  class BisaBersuara(Protocol):
5      def suara(self) -> str: ...
6
7  class Bebek:
8      def suara(self) -> str:
9          return "Kwek kwek"
10
11 class Batu:
12     pass
13
14 print(isinstance(Bebek(), BisaBersuara))
15 print(isinstance(Batu(), BisaBersuara))
```

Keluaran

```
True
False
```




**Gambar 5.4.** Nominal typing menilai dari pernyataan pewarisan; structural typing menilai dari bentuk object.

**Tabel 5.4.** Abstract class dibandingkan Protocol.

| Aspek | Abstract class (`ABC`) | `Protocol` |
|---|---|---|
| Jenis typing | nominal | struktural |
| Wajib mewarisi? | ya | tidak |
| Kapan dicek | saat object dibuat (program berjalan) | oleh pemeriksa statis sebelum program berjalan |
| Boleh berisi method konkret | ya, dan diwarisi child | boleh, tetapi tidak diwarisi class yang hanya "cocok bentuk" |
| Cocok untuk | hierarki class milik sendiri yang berbagi kode | menerima object dari mana saja, termasuk class pustaka lain yang tidak bisa diubah |

## 5.6 Rangkuman

- **Abstract class** (mewarisi `ABC`) berisi **abstract method** (`@abstractmethod`) yang wajib di-implementasikan child. Abstract class dan child yang kontraknya belum lengkap tidak bisa dibuat object-nya.
- **Data class** (`@dataclass`) membangkitkan `__init__`, `__repr__`, dan `__eq__` dari daftar field. Opsi penting: `order`, `frozen`, `field(default_factory=…)`, dan `__post_init__`.
- **Mutable** berarti isi object dapat diubah. Variabel adalah label; `b = a` tidak menyalin object. Hindari default argument mutable dan bedakan salinan dangkal dari salinan dalam.

- **Duck typing**: object dinilai dari kemampuannya, bukan class-nya. Method seperti `__len__` dan `__iter__` membuat class sendiri bekerja dengan `len()` dan `for`.
- **Structural typing** dengan `typing.Protocol` memformalkan duck typing sehingga bisa diperiksa oleh `mypy`. Type hint tidak dipaksakan saat program berjalan.

## 5.7 Kuis Teoritis

### Pilihan Ganda

Pilihlah satu jawaban yang paling tepat.

1. Apa yang terjadi bila kita mencoba membuat object dari class yang mewarisi `ABC` dan masih memiliki abstract method?

   a. Object berhasil dibuat, abstract method mengembalikan `None`
   b. Terjadi `TypeError`
   c. Terjadi `AttributeError` saat method dipanggil
   d. Python otomatis mengisi method tersebut

2. Pernyataan yang benar tentang abstract class adalah ...

   a. semua method-nya harus abstract
   b. tidak boleh memiliki constructor
   c. boleh memiliki method konkret yang memanggil abstract method
   d. hanya boleh diwarisi oleh satu child class

3. Method yang **tidak** dibuat otomatis oleh `@dataclass` tanpa opsi tambahan adalah ...

   a. `__init__`
   b. `__repr__`
   c. `__eq__`
   d. `__lt__`

4. Cara yang benar memberi default list kosong pada field data class adalah ...

   a. `nilai: list = []`
   b. `nilai: list = list`
   c. `nilai: list = field(default_factory=list)`
   d. `nilai: list = None[]`

5. Perhatikan kode berikut.

Soal 5

```
x = [1, 2]
y = x
y += [3]
```

```
z = x.copy()
z.append(4)
print(x)
```

Keluaran kode tersebut adalah ...

a. `[1, 2]`
b. `[1, 2, 3]`
c. `[1, 2, 3, 4]`
d. `[1, 2, 4]`

**6.** Tipe berikut yang bersifat *immutable* adalah ...

a. `list`
b. `dict`
c. `set`
d. `tuple`

**7.** Perhatikan kode berikut.

Soal 7

```
def tambah(item, wadah=[]):
    wadah.append(item)
    return wadah

tambah("a")
print(tambah("b"))
```

Keluaran kode tersebut adalah ...

a. `['b']`
b. `['a', 'b']`
c. `['a']`
d. `[]`

**8.** Agar object dari class buatan sendiri dapat dipakai dengan fungsi `len()`, class tersebut perlu memiliki method ...

a. `__size__`
b. `__len__`
c. `__count__`
d. `length()`

**9.** Apa yang terjadi saat fungsi `def f(x: int) -> int` dipanggil dengan `f("teks")` lalu dijalankan biasa dengan `python`?

a. Python menolak dengan `TypeError` sebelum fungsi dijalankan

b. Python otomatis mengubah `"teks"` menjadi bilangan
c. Fungsi tetap dijalankan; type hint tidak dipaksakan saat runtime
d. Program tidak bisa dijalankan sama sekali

10. Perbedaan utama `Protocol` dibandingkan abstract class adalah ...

a. class yang cocok dengan protocol tidak perlu mewarisinya
b. protocol tidak boleh memiliki method
c. protocol hanya bisa dipakai untuk data class
d. protocol memaksa pengecekan saat object dibuat

### Uraian

1. Jelaskan mengapa memberi nilai default `return 0` pada method parent lebih berbahaya daripada menjadikannya abstract method. Berikan contoh kasus selain bangun datar.
2. Bandingkan penulisan class pembawa data secara manual dengan `@dataclass`. Sebutkan dua situasi ketika data class *kurang* tepat digunakan.
3. Jelaskan apa yang dimaksud "variabel adalah label, bukan kotak" dan kaitkan dengan bug default argument mutable pada class `Keranjang`.
4. Jelaskan perbedaan salinan dangkal dan salinan dalam. Pada kondisi apa keduanya memberikan hasil yang sama?
5. Jelaskan hubungan antara duck typing dan structural typing. Mengapa `Protocol` berguna jika Python sendiri sudah mendukung duck typing?

## 5.8 Praktik Mandiri

Kerjakan praktik berikut secara berurutan. Simpan setiap praktik dalam file `.py` terpisah, jalankan, dan pastikan keluarannya sesuai dengan contoh.

### Praktik 1: Metode Pembayaran

Buat abstract class `Pembayaran` dengan ketentuan:

- Abstract property `nama_metode` dan abstract method `biaya_admin(jumlah)`.
- Method konkret `struk(jumlah)` yang menampilkan nama metode, jumlah, biaya admin, dan total, dengan memanggil kedua anggota abstract tersebut.
- Tiga child: `Tunai` (tanpa biaya), `TransferBank` (biaya tetap Rp2.500), dan `EWallet` (biaya 1% dari jumlah, minimal Rp1.000).
- Buat juga class `QRIS(Pembayaran)` yang sengaja hanya mengimplementasikan `nama_metode`, lalu tunjukkan pesan kesalahan yang muncul saat membuat objectnya.

Contoh keluaran Praktik 1

```
Tunai         Rp50.000 + admin Rp0     = Rp50.000
Transfer Bank Rp50.000 + admin Rp2.500 = Rp52.500
E-Wallet      Rp50.000 + admin Rp1.000 = Rp51.000
E-Wallet      Rp250.000 + admin Rp2.500 = Rp252.500
TypeError: Can't instantiate abstract class QRIS without an implementation for
    abstract method 'biaya_admin'
```

### Praktik 2: Papan Peringkat dengan Data Class

- Buat data class `Peserta` dengan field `nama`, `asal`, dan `skor_soal` (list berisi skor tiap soal, default list kosong yang aman).
- Tambahkan field `total` yang dihitung di `__post_init__` dan tidak menjadi parameter constructor. Tolak skor di luar rentang 0–100 dengan `ValueError`.
- Buat data class `frozen` bernama `Lomba` berisi `nama` dan `tahun`, lalu tunjukkan bahwa field-nya tidak bisa diubah.
- Urutkan peserta berdasarkan `total` dari terbesar, lalu tampilkan peringkatnya. Gunakan `sorted(…, key=…)` atau opsi `order=True` dengan `field(compare=False)`; jelaskan pilihan Anda dalam komentar.

Contoh keluaran Praktik 2

```
Lomba(nama='Olimpiade Pemrograman', tahun=2026)
1. Citra  (UNESA)  total 275
2. Ani    (ITS)    total 260
3. Budi   (UNAIR)  total 245
FrozenInstanceError: cannot assign to field 'tahun'
```

### Praktik 3: Ekspor Laporan dengan Protocol

#### Langkah Pengerjaan

1. Definisikan `Protocol` bernama `BisaDiekspor` dengan method `ekspor(self) -> str` dan attribute `judul: str`.
2. Buat tiga class yang **tidak saling mewarisi**: `LaporanNilai`, `GrafikKehadiran`, dan `TabelJadwal`. Masing-masing mengimplementasikan `ekspor()` dengan format berbeda (misalnya teks, CSV, atau baris tabel).
3. Buat class `Arsip` yang menyimpan banyak object `BisaDiekspor`, mengimplementasikan `__len__` dan `__iter__`, serta memiliki method `ekspor_semua()`. Pastikan constructor-nya aman dari jebakan default mutable.
4. Tambahkan satu class `Catatan` yang tidak memiliki `ekspor()`, lalu masukkan ke arsip.

> Jalankan `mypy` pada file Anda dan salin pesan kesalahannya ke dalam komentar. Jika tidak bisa memasang `mypy`, gunakan `@runtime_checkable` dan `isinstance()` untuk menolak object yang tidak sesuai.

**KRITERIA PENILAIAN**

- Kontrak abstract class dan protocol dirancang dengan tepat dan benar-benar ditegakkan.
- Data class memakai default mutable yang aman serta validasi di `__post_init__`.
- Tidak ada data yang tanpa sengaja dipakai bersama antar-object.
- Type hint ditulis pada semua parameter dan nilai kembalian di Praktik 3.

Bab 06

# Polymorphism dan Class Relationship

*Satu antarmuka dengan banyak bentuk, dan cara class saling terhubung.*

06

DI BAB INI

- Polymorphism
- Overriding dan overloading
- Association, aggregation, composition
- Memilih relationship

Kita sudah dua kali bertemu "benih" polymorphism. Di Bab 4, satu baris `w.perkenalan()` menghasilkan kalimat berbeda untuk mahasiswa dan dosen. Di Bab 5, fungsi `bunyikan()` bisa dipakai untuk bebek, robot, maupun radio. Bab ini merangkum fenomena tersebut menjadi pilar OOP keempat: *polymorphism*.

Paruh kedua bab membahas pertanyaan yang berbeda: selain hubungan *is-a* lewat inheritance, bagaimana class-class lain saling terhubung? Mahasiswa *punya* dosen pembimbing, program studi *memiliki* dosen, dan mobil *tersusun dari* mesin. Ketiga hubungan itu terlihat mirip, tetapi berbeda dalam hal kepemilikan dan umur object. Kita akan mempelajari *association*, *aggregation*, dan *composition*, lalu cara memilih di antara ketiganya.

## 6.1 Konsep Polymorphism

*Polymorphism* berasal dari bahasa Yunani: *poly* (banyak) dan *morph* (bentuk). Dalam OOP, polymorphism berarti **satu antarmuka yang sama dapat dipakai untuk object dengan bentuk berbeda**, dan setiap object merespons dengan caranya sendiri. Kode pemanggil cukup tahu "apa" yang diminta, tanpa perlu tahu "siapa" yang mengerjakannya.

### Polymorphism di Sekitar Kita

Tanpa disadari, kita sudah memakai polymorphism sejak awal belajar Python.

Polymorphism pada fungsi dan operator bawaan

```
print(len("kampus"), len([1, 2, 3]), len({"a": 1, "b": 2}))
print(3 + 4)                  # penjumlahan bilangan
print("Py" + "thon")          # penyambungan teks
print([1, 2] + [3])           # penggabungan list
```

Keluaran

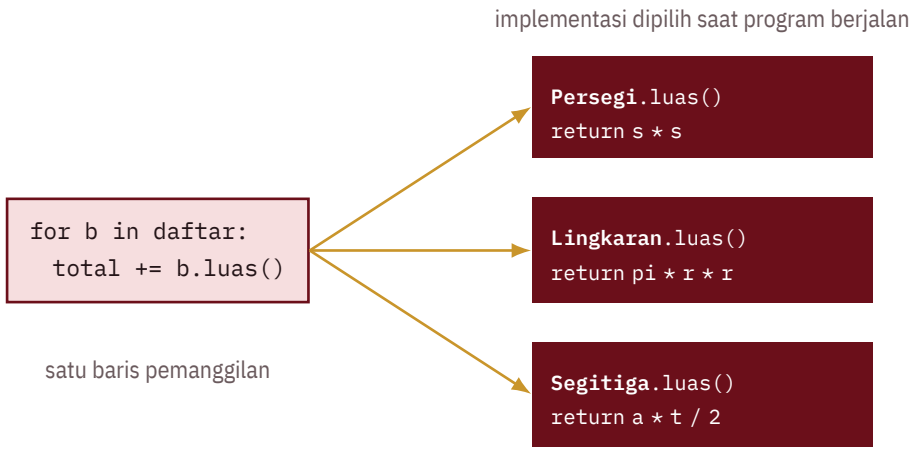
```
6 3 2
7
Python
[1, 2, 3]
```

Fungsi `len()` dan operator `+` adalah satu antarmuka, tetapi perilakunya menyesuaikan jenis object yang diberikan. String menghitung huruf, list menghitung elemen; angka dijumlahkan, teks disambung.

### Bentuk-Bentuk Polymorphism

**Tabel 6.1.** Bentuk polymorphism di Python.

| Bentuk | Cara kerja | Dibahas di |
|---|---|---|
| Lewat overriding | class turunan meng-override method parent; pemanggil memakai tipe parent | Subbab Method Overriding |
| Lewat duck typing | class tanpa hubungan pewarisan memiliki method bernama sama | Bab 5 |
| Overloading | satu nama fungsi atau operator menerima jenis argumen berbeda | Subbab Method Overloading |




**Gambar 6.1.** Polymorphism: satu pemanggilan `b.luas()` dijalankan oleh implementasi yang sesuai dengan jenis object `b`.

### Mengapa Polymorphism Penting?

Polymorphism memungkinkan kita menulis kode yang bekerja dengan *jenis umum* ("bangun datar", "notifikasi", "pembayaran"), bukan dengan setiap jenis khusus. Akibatnya, jenis baru bisa ditambahkan **tanpa mengubah** kode yang sudah ada. Prinsip ini dikenal sebagai *open/closed principle*: kode terbuka untuk dikembangkan, tetapi tertutup untuk diubah.

## 6.2 Method Overriding

Bentuk polymorphism yang paling umum dalam OOP diwujudkan lewat overriding. Parent (sering kali abstract class) mendefinisikan antarmuka; setiap child meng-override method dengan implementasinya sendiri; kode pemanggil hanya berbicara dengan antarmuka parent.

### Tanpa Polymorphism: Rantai `if-elif`

Untuk melihat manfaatnya, perhatikan dulu cara yang *tidak* memakai polymorphism. Fungsi berikut memeriksa jenis notifikasi satu per satu.

Gaya if-elif berdasarkan jenis

```
def kirim(jenis, tujuan, pesan):
    if jenis == "email":
        print(f"[EMAIL ke {tujuan}] {pesan}")
    elif jenis == "sms":
        print(f"[SMS ke {tujuan}] {pesan[:20]}...")
    elif jenis == "wa":
        print(f"[WA ke {tujuan}] {pesan}")
    # setiap jenis baru = menambah cabang di sini
```

Setiap kali ada jenis notifikasi baru, fungsi ini harus dibuka dan diubah. Jika logika serupa tersebar di banyak fungsi, semuanya harus diubah, dan satu saja yang terlewat akan menjadi bug.

### Dengan Polymorphism

notifikasi.py

```
from abc import ABC, abstractmethod

class Notifikasi(ABC):
    def __init__(self, tujuan):
        self.tujuan = tujuan

    @abstractmethod
    def kirim(self, pesan):
        ...

class Email(Notifikasi):
    def kirim(self, pesan):
        print(f"[EMAIL ke {self.tujuan}] {pesan}")

class SMS(Notifikasi):
    def kirim(self, pesan):
        print(f"[SMS ke {self.tujuan}] {pesan[:20]}...")   # SMS dipotong

class WhatsApp(Notifikasi):
    def kirim(self, pesan):
        print(f"[WA ke {self.tujuan}] {pesan}")

def siarkan(daftar, pesan):              # tidak tahu dan tidak peduli jenisnya
    for n in daftar:
        n.kirim(pesan)
```

```
26
27 penerima = [
28     Email("ani@mail.com"),
29     SMS("081234567890"),
30     WhatsApp("081298765432"),
31 ]
32 siarkan(penerima, "Kuliah PBO hari ini pindah ke ruang C01.04.03")
```

**Keluaran**

```
[EMAIL ke ani@mail.com] Kuliah PBO hari ini pindah ke ruang C01.04.03
[SMS ke 081234567890] Kuliah PBO hari ini ...
[WA ke 081298765432] Kuliah PBO hari ini pindah ke ruang C01.04.03
```

Kini, jika kampus ingin menambah notifikasi Telegram, kita cukup menulis class baru. Fungsi `siarkan()` tidak disentuh sama sekali.

**Menambah jenis baru tanpa mengubah kode lama**

```
1 class Telegram(Notifikasi):
2     def kirim(self, pesan):
3         print(f"[TELEGRAM @{self.tujuan}] {pesan}")
4
5 siarkan([Telegram("ani_unesa")], "UTS dimulai pukul 08.00")
```

**Keluaran**

```
[TELEGRAM @ani_unesa] UTS dimulai pukul 08.00
```

**TANDA PERLU POLYMORPHISM**

Jika Anda menemukan rantai `if-elif` yang memeriksa *jenis* sesuatu (`if jenis == "..."` atau `if isinstance(x, ...)`) dan setiap cabang melakukan versi berbeda dari tindakan yang sama, itu tanda kuat bahwa setiap cabang sebaiknya menjadi class tersendiri dengan method yang di-override.

## 6.3 Method Overloading secara Konseptual

*Method overloading* adalah kemampuan mendefinisikan beberapa method dengan **nama sama** tetapi **parameter berbeda** (jumlah atau tipe), lalu bahasa memilih versi yang tepat berdasarkan argumen yang diberikan. Java dan C++ mendukungnya secara langsung, misalnya `luas(int sisi)` dan `luas(int panjang, int lebar)` boleh berdampingan dalam satu class.

## Python Tidak Mendukung Overloading Langsung

Di Python, nama method hanyalah label. Mendefinisikan method dengan nama yang sama untuk kedua kalinya akan *menimpa* definisi pertama.

**Definisi kedua menimpa yang pertama**

```
class Kalkulator:
    def luas(self, sisi):
        return sisi * sisi

    def luas(self, panjang, lebar):     # menimpa versi di atas
        return panjang * lebar

k = Kalkulator()
print(k.luas(4, 5))
print(k.luas(4))
```

**Keluaran**

```
20
Traceback (most recent call last):
  ...
TypeError: Kalkulator.luas() missing 1 required positional argument: 'lebar'
```

## Meniru Overloading

Walaupun tidak didukung langsung, efek overloading dapat dicapai dengan beberapa cara.

**Parameter default dan *args**

```
class Kalkulator:
    def luas(self, panjang, lebar=None):
        if lebar is None:                # satu argumen: persegi
            return panjang * panjang
        return panjang * lebar           # dua argumen: persegi panjang

    def jumlah(self, *angka):            # berapa pun banyak argumennya
        return sum(angka)

k = Kalkulator()
print(k.luas(4), k.luas(4, 5))
print(k.jumlah(1, 2), k.jumlah(1, 2, 3, 4))
```

**Keluaran**

```
16 20
3 10
```

Untuk memilih perilaku berdasarkan *tipe* argumen, pustaka standar menyediakan `functools.singledispatchmethod`. Decorator ini memilih implementasi berdasarkan tipe argumen pertama setelah `self`.

**Overloading berdasarkan tipe**

```
from functools import singledispatchmethod

class Pencetak:
    @singledispatchmethod
    def cetak(self, data):
        print("Data:", data)

    @cetak.register
    def _(self, data: list):
        print("Daftar:", ", ".join(map(str, data)))

    @cetak.register
    def _(self, data: dict):
        for k, v in data.items():
            print(f"  {k} = {v}")

p = Pencetak()
p.cetak(42)
p.cetak(["Ani", "Budi"])
p.cetak({"nama": "Citra", "ipk": 3.8})
```

**Keluaran**

```
Data: 42
Daftar: Ani, Budi
  nama = Citra
  ipk = 3.8
```

## Operator Overloading

Bentuk overloading yang paling berguna di Python adalah *operator overloading*: menentukan arti operator seperti +, ==, atau < untuk class buatan sendiri, dengan cara mengimplementasikan *dunder method* yang sesuai.

**Class Uang dengan operator overloading**

```
class Uang:
    def __init__(self, nominal):
        self.nominal = nominal

    def __add__(self, lain):              # a + b
        return Uang(self.nominal + lain.nominal)
```

```
    def __mul__(self, faktor):            # a * 3
        return Uang(self.nominal * faktor)

    def __eq__(self, lain):               # a == b
        return self.nominal == lain.nominal

    def __lt__(self, lain):               # a < b (juga dipakai sorted)
        return self.nominal < lain.nominal

    def __str__(self):
        return f"Rp{self.nominal:,}".replace(",", ".")

kopi = Uang(18000)
roti = Uang(12500)
print(kopi + roti)
print(kopi * 3)
print(kopi == Uang(18000), roti < kopi)
print([str(u) for u in sorted([kopi, roti, Uang(5000)])])
```

**Keluaran**

```
Rp30.500
Rp54.000
True True
['Rp5.000', 'Rp12.500', 'Rp18.000']
```

**Tabel 6.2.** Beberapa operator dan dunder method pasangannya.

| Operator | Dunder method | Contoh pemakaian |
|---|---|---|
| `a + b` | `__add__` | menjumlahkan uang, vektor, durasi |
| `a - b` | `__sub__` | selisih tanggal atau koordinat |
| `a * b` | `__mul__` | mengalikan uang dengan jumlah barang |
| `a == b` | `__eq__` | membandingkan isi dua object |
| `a < b` | `__lt__` | mengurutkan dengan `sorted()` |
| `len(a)` | `__len__` | jumlah isi koleksi (Bab 5) |
| `str(a)` | `__str__` | teks untuk `print()` |

JAGA MAKNA OPERATOR

Overload operator hanya jika artinya *jelas dan wajar*. `Uang + Uang` masuk akal; `Mahasiswa + Mahasiswa` tidak. Operator dengan arti yang mengejutkan membuat kode sulit dibaca dan rawan disalahpahami.

## 6.4 Association

Selain hubungan *is-a* yang diwujudkan dengan inheritance, class dapat saling terhubung karena object-nya *mengenal* atau *memakai* object lain. Hubungan seperti ini diwujudkan dengan menyimpan rujukan ke object lain sebagai attribute. Ada tiga tingkat kekuatan hubungan: association, aggregation, dan composition.

*Association* adalah hubungan paling longgar: dua object saling mengenal dan berinteraksi, tetapi **tidak ada yang memiliki** yang lain. Keduanya dibuat secara terpisah dan memiliki umur sendiri-sendiri. Contohnya mahasiswa dan dosen pembimbing: mahasiswa *mengenal* pembimbingnya, tetapi dosen bukan bagian dari mahasiswa, dan keduanya tetap ada walaupun hubungan bimbingan berakhir.

Association dua arah

```python
class Dosen:
    def __init__(self, nama):
        self.nama = nama
        self.bimbingan = []                # daftar mahasiswa yang dikenal

class Mahasiswa:
    def __init__(self, nama):
        self.nama = nama
        self.pembimbing = None             # belum punya pembimbing

    def tetapkan_pembimbing(self, dosen):
        if self.pembimbing is not None:    # lepas dari pembimbing lama
            self.pembimbing.bimbingan.remove(self)
        self.pembimbing = dosen
        dosen.bimbingan.append(self)

pak_budi = Dosen("Budi")
bu_rina = Dosen("Rina")
ani = Mahasiswa("Ani")
dodi = Mahasiswa("Dodi")

ani.tetapkan_pembimbing(pak_budi)
dodi.tetapkan_pembimbing(pak_budi)
dodi.tetapkan_pembimbing(bu_rina)          # ganti pembimbing

print(ani.pembimbing.nama, dodi.pembimbing.nama)
```

```
print([m.nama for m in pak_budi.bimbingan])
print([m.nama for m in bu_rina.bimbingan])
```

Keluaran

```
Budi Rina
['Ani']
['Dodi']
```

Ciri association terlihat jelas: object `Dosen` dan `Mahasiswa` dibuat terpisah, dihubungkan belakangan, dan hubungannya bisa diganti kapan saja. Karena hubungannya dua arah, method `tetapkan_pembimbing()` menjaga agar kedua sisi selalu konsisten.

**DEPENDENCY: HUBUNGAN SESAAT**

Ada pula hubungan yang lebih longgar lagi, yaitu *dependency*: sebuah object hanya *memakai* object lain sesaat, misalnya sebagai parameter method, tanpa menyimpannya sebagai attribute. Contohnya `beri_nilai(self, mhs, nilai)` pada class `Dosen` di Bab 4.

## 6.5 Aggregation

*Aggregation* adalah hubungan *keseluruhan–bagian* (*whole–part*, "memiliki") yang **lemah**: object keseluruhan mengumpulkan object-object bagian, tetapi bagian-bagian itu **dibuat di luar** dan **tetap bisa hidup** tanpa keseluruhannya. Satu bagian bahkan boleh menjadi milik beberapa keseluruhan sekaligus.

Contohnya program studi dan dosen. Program studi *memiliki* dosen-dosen, tetapi dosen sudah ada sebelum bergabung, bisa pindah ke prodi lain, dan tidak ikut lenyap jika prodinya ditutup.

Aggregation

```
class Dosen:
    def __init__(self, nama):
        self.nama = nama

class ProgramStudi:
    def __init__(self, nama):
        self.nama = nama
        self._dosen = []                    # kumpulan bagian

    def tambah_dosen(self, dosen):          # bagian dibuat di luar
        self._dosen.append(dosen)

    def keluarkan_dosen(self, dosen):
        self._dosen.remove(dosen)
```

```
15
16      def daftar_dosen(self):
17          return [d.nama for d in self._dosen]
18
19  rina = Dosen("Rina")                          # dosen ada lebih dulu
20  budi = Dosen("Budi")
21
22  ka = ProgramStudi("Kecerdasan Artifisial")
23  ti = ProgramStudi("Teknik Informatika")
24  ka.tambah_dosen(rina)
25  ka.tambah_dosen(budi)
26  ti.tambah_dosen(budi)                         # satu dosen di dua prodi
27
28  print(ka.daftar_dosen(), ti.daftar_dosen())
29  del ka                                        # prodi dihapus...
30  print(rina.nama, "masih ada")                 # ...dosennya tetap ada
```

**Keluaran**

```
['Rina', 'Budi'] ['Budi']
Rina masih ada
```

## 6.6 Composition

*Composition* adalah hubungan keseluruhan–bagian yang **kuat**: bagian **dibuat oleh** keseluruhan, **dimiliki secara eksklusif**, dan **ikut berakhir** bersama keseluruhannya. Bagian tidak punya arti jika berdiri sendiri dan tidak dipakai bersama object lain.

Contohnya mobil dan mesinnya. Dalam model kita, mesin dibuat ketika mobil dibuat, terpasang hanya di mobil itu, dan tidak pernah diserahkan ke object lain.

**Composition**

```
1   class Mesin:
2       def __init__(self, cc):
3           self.cc = cc
4           self.menyala = False
5
6       def hidupkan(self):
7           self.menyala = True
8           return f"Mesin {self.cc} cc menyala"
9
10  class Roda:
11      def __init__(self, posisi):
12          self.posisi = posisi
13
```

```
class Mobil:
    def __init__(self, merek, cc):
        self.merek = merek
        self._mesin = Mesin(cc)                          # dibuat di dalam
        self._roda = [Roda(p) for p in ("KD", "KB", "KiD", "KiB")]

    def nyalakan(self):                                  # delegasi ke bagian
        return f"{self.merek}: {self._mesin.hidupkan()}"

    def jumlah_roda(self):
        return len(self._roda)

avanza = Mobil("Avanza", 1300)
print(avanza.nyalakan())
print(avanza.jumlah_roda(), "roda")
```

Keluaran

```
Avanza: Mesin 1300 cc menyala
4 roda
```

Perhatikan perbedaannya dengan aggregation. Pemakai class `Mobil` tidak pernah membuat `Mesin` maupun `Roda` sendiri; mereka cukup membuat mobil. Bagian-bagian disimpan sebagai attribute protected, dan pekerjaan diteruskan kepada bagian yang tepat. Teknik meneruskan pekerjaan ini disebut *delegasi*. Ketika object `avanza` tidak lagi dipakai, mesin dan rodanya ikut tidak terpakai, karena tidak ada object lain yang merujuk kepadanya.

## Composition over Inheritance

Pedoman desain yang terkenal berbunyi: *"utamakan composition daripada inheritance"*. Inheritance mewariskan *seluruh* antarmuka parent, termasuk bagian yang tidak diinginkan. Composition hanya membuka apa yang memang kita sediakan.

Misalkan kita ingin membuat struktur data *stack* (tumpukan), yang hanya boleh menambah dan mengambil elemen dari atas.

Stack dengan inheritance: terlalu banyak yang diwarisi

```
class StackWaris(list):
    def push(self, x):
        self.append(x)

s = StackWaris()
s.push(1)
s.push(2)
s.insert(0, 99)          # ikut diwarisi dari list: aturan stack dilanggar!
```

```
print(s)
```

Keluaran

```
[99, 1, 2]
```

Stack dengan composition: hanya membuka yang perlu

```
class Stack:
    def __init__(self):
        self._data = []                # list sebagai bagian internal

    def push(self, x):
        self._data.append(x)

    def pop(self):
        return self._data.pop()

    def __len__(self):
        return len(self._data)

s = Stack()
s.push(1)
s.push(2)
print(s.pop(), len(s))
print(hasattr(s, "insert"))           # tidak ada jalan pintas
```

Keluaran

```
2 1
False
```

Stack bukanlah list (uji *is-a* gagal); stack *menggunakan* list untuk menyimpan datanya. Composition mencerminkan hubungan itu dengan tepat.

## 6.7 Class Relationship

Kita kini mengenal lima jenis hubungan antar-class. Tabel dan gambar berikut merangkumnya, termasuk cara menggambarkannya dalam *class diagram* UML yang akan kita pakai lagi di Bab 11.

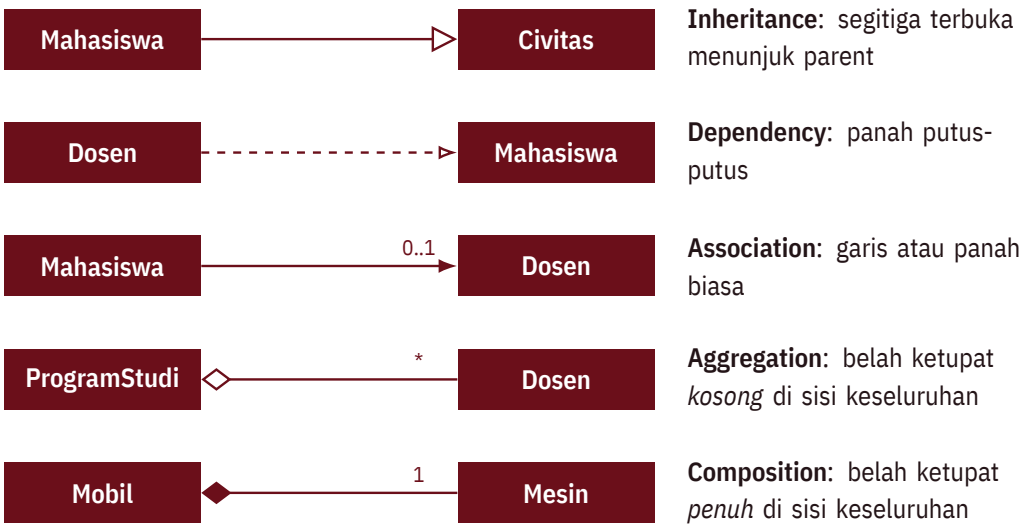
### Multiplicity

Angka kecil di ujung garis hubungan disebut *multiplicity*, yaitu berapa banyak object di ujung tersebut yang terhubung dengan satu object di ujung lainnya.

Dalam kode Python, multiplicity tercermin dari bentuk attribute: `1` atau `0..1` biasanya

**Tabel 6.3.** Ringkasan jenis hubungan antar-class.

| Hubungan | Dibaca | Ciri utama | Contoh |
|---|---|---|---|
| Inheritance | *adalah* | child mewarisi antarmuka dan kode parent | Mahasiswa–Civitas |
| Dependency | *memakai sesaat* | object lain hanya lewat parameter, tidak disimpan | Dosen–Mahasiswa pada `beri_nilai()` |
| Association | *mengenal* | disimpan sebagai attribute; umur masing-masing; tanpa kepemilikan | Mahasiswa–Dosen pembimbing |
| Aggregation | *memiliki (lemah)* | bagian dibuat di luar, bisa dipakai bersama, tetap hidup tanpa keseluruhan | ProgramStudi–Dosen |
| Composition | *tersusun dari* | bagian dibuat di dalam, eksklusif, berakhir bersama keseluruhan | Mobil–Mesin |




**Gambar 6.2.** Notasi UML untuk lima jenis hubungan. Angka di ujung garis adalah *multiplicity*.

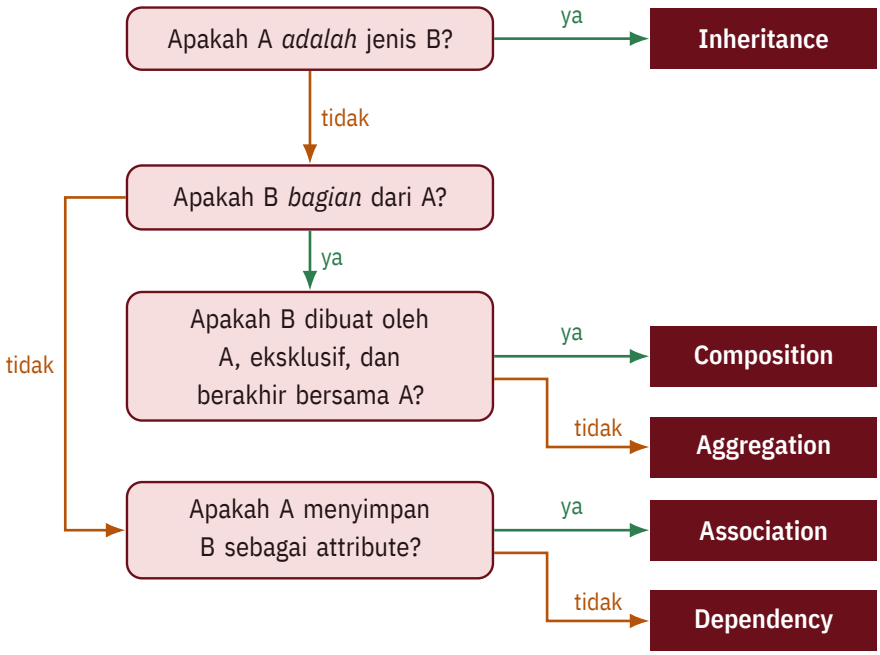
**Tabel 6.4.** Notasi multiplicity.

| Notasi | Arti | Contoh |
|---|---|---|
| `1` | tepat satu | mobil memiliki tepat satu mesin |
| `0..1` | nol atau satu | mahasiswa memiliki nol atau satu pembimbing |
| `*` atau `0..*` | nol atau lebih | prodi memiliki nol atau lebih dosen |
| `1..*` | minimal satu | pesanan berisi minimal satu item |

berupa satu rujukan (atau `None`), sedangkan `*` dan `1..*` berupa list atau koleksi lain.

## 6.8 Pemilihan Relationship Berdasarkan Kebutuhan Sistem

Satu pasang konsep yang sama bisa dimodelkan dengan hubungan berbeda, *tergantung kebutuhan sistem*. Mesin, misalnya, adalah bagian mobil yang dibuat bersama mobil (composition) dalam aplikasi rental. Namun di aplikasi bengkel, mesin bisa dilepas, diperbaiki, lalu dipasang di mobil lain (aggregation). Karena itu, pilihan relationship harus berangkat dari pertanyaan tentang *perilaku sistem*, bukan sekadar dari benda di dunia nyata.




**Gambar 6.3.** Pertanyaan penuntun untuk memilih jenis hubungan antara class A dan class B.

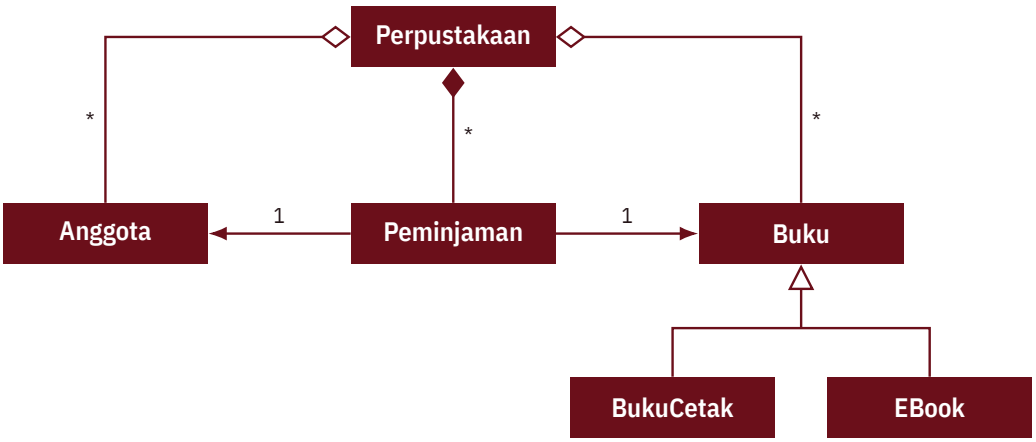
**Tabel 6.5.** Contoh pemilihan relationship pada beberapa sistem.

| Pasangan | Hubungan | Alasan berdasarkan kebutuhan |
|---|---|---|
| Pesanan–ItemPesanan | composition | item hanya bermakna di dalam pesanannya dan dihapus bersama pesanan |
| Keranjang–Produk | aggregation | produk ada di katalog sebelum dan sesudah dimasukkan ke keranjang |
| Pelanggan–Pesanan | association | pelanggan mengenal riwayat pesanannya; keduanya punya umur sendiri |
| Kasir–Struk | dependency | kasir hanya mencetak struk saat transaksi, tidak menyimpannya |
| AdminToko–Pengguna | inheritance | admin *adalah* pengguna dengan hak tambahan |

### Studi Kasus: Perpustakaan Kampus

Kita rancang sistem peminjaman perpustakaan kampus dengan kebutuhan berikut:

- Perpustakaan menyimpan banyak buku dan banyak anggota. Buku dan anggota didata terlebih dahulu, lalu didaftarkan ke perpustakaan → **aggregation**.
- Setiap peminjaman mencatat siapa meminjam buku apa → `Peminjaman` **berasosiasi** dengan `Anggota` dan `Buku`.
- Catatan peminjaman dibuat oleh perpustakaan dan hanya bermakna di dalamnya → **composition**.
- Ada buku cetak dan e-book dengan lama pinjam berbeda → **inheritance** dan **polymorphism**.




**Gambar 6.4.** Class diagram studi kasus perpustakaan: aggregation (belah ketupat kosong), composition (belah ketupat penuh), association (panah), dan inheritance (segitiga).

perpustakaan.py

```python
from abc import ABC, abstractmethod

class Buku(ABC):
    def __init__(self, judul):
        self.judul = judul
        self.dipinjam = False

    @abstractmethod
    def lama_pinjam(self):
        ...

class BukuCetak(Buku):
    def lama_pinjam(self):
        return 14

class EBook(Buku):
```

```
    def lama_pinjam(self):
        return 7

class Anggota:
    def __init__(self, nama):
        self.nama = nama

class Peminjaman:                                   # association
    def __init__(self, anggota, buku):
        self.anggota = anggota
        self.buku = buku

    def __str__(self):
        return (f"{self.anggota.nama} meminjam '{self.buku.judul}' "
                f"selama {self.buku.lama_pinjam()} hari")

class Perpustakaan:
    def __init__(self, nama):
        self.nama = nama
        self._buku = []                             # aggregation
        self._anggota = []                          # aggregation
        self._catatan = []                          # composition

    def tambah_buku(self, *buku):                  # bagian dari luar
        self._buku.extend(buku)

    def tambah_anggota(self, *anggota):
        self._anggota.extend(anggota)

    def pinjamkan(self, anggota, buku):
        if buku.dipinjam:
            return f"'{buku.judul}' sedang dipinjam"
        buku.dipinjam = True
        catatan = Peminjaman(anggota, buku)        # dibuat di dalam
        self._catatan.append(catatan)
        return str(catatan)

    def jumlah_peminjaman(self):
        return len(self._catatan)

ani, budi = Anggota("Ani"), Anggota("Budi")
laskar = BukuCetak("Laskar Pelangi")
pbo = EBook("Dasar PBO dengan Python")

perpus = Perpustakaan("Perpustakaan UNESA")
perpus.tambah_anggota(ani, budi)
```

```
63 perpus.tambah_buku(laskar, pbo)
64 print(perpus.pinjamkan(ani, laskar))
65 print(perpus.pinjamkan(budi, pbo))
66 print(perpus.pinjamkan(budi, laskar))
67 print("Total peminjaman:", perpus.jumlah_peminjaman())
```

**Keluaran**

```
Ani meminjam 'Laskar Pelangi' selama 14 hari
Budi meminjam 'Dasar PBO dengan Python' selama 7 hari
'Laskar Pelangi' sedang dipinjam
Total peminjaman: 2
```

Studi kasus ini memadukan semua yang telah dipelajari: inheritance dan polymorphism pada `Buku`, aggregation pada daftar buku dan anggota, composition pada catatan peminjaman, serta association antara peminjaman, anggota, dan buku.

**SAAT RAGU**

Jika ragu antara aggregation dan composition, tanyakan: *"Jika object keseluruhan dihapus, apakah bagian ini masih dibutuhkan sistem?"* Jika masih dibutuhkan, pilih aggregation. Jika ragu antara inheritance dan composition, pilih composition. Composition lebih mudah diubah di kemudian hari.

## 6.9 Rangkuman

- **Polymorphism**: satu antarmuka dipakai untuk object berbentuk berbeda, dan setiap object merespons dengan caranya sendiri. Polymorphism memungkinkan jenis baru ditambahkan tanpa mengubah kode pemanggil.
- Polymorphism lewat **overriding**: parent mendefinisikan antarmuka, child mengimplementasikannya, dan pemanggil hanya mengenal parent. Rantai `if-elif` berdasarkan jenis adalah tanda polymorphism dibutuhkan.
- **Method overloading** tidak didukung langsung di Python (definisi kedua menimpa yang pertama), tetapi dapat ditiru dengan parameter default, `*args`, dan `singledispatchmethod`. **Operator overloading** dilakukan dengan dunder method seperti `__add__` dan `__eq__`.
- **Association**: object saling mengenal tanpa kepemilikan, dengan umur masing-masing. **Dependency**: hanya dipakai sesaat lewat parameter.
- **Aggregation**: keseluruhan–bagian yang lemah; bagian dibuat di luar, bisa dipakai bersama, dan tetap hidup tanpa keseluruhan.
- **Composition**: keseluruhan–bagian yang kuat; bagian dibuat di dalam, eksklusif, dan berakhir bersama keseluruhan. Utamakan composition daripada inheritance bila uji *is-a* meragukan.

- Pilihan relationship bergantung pada **kebutuhan sistem**, bukan hanya pada benda di dunia nyata.

## 6.10 Kuis Teoritis

### Pilihan Ganda

Pilihlah satu jawaban yang paling tepat.

1. Pengertian polymorphism yang paling tepat adalah ...

   a. satu class memiliki banyak attribute
   b. satu antarmuka dapat dipakai untuk object berbeda yang merespons dengan caranya masing-masing
   c. satu object dapat dibuat dari banyak class
   d. satu method dapat dipanggil berkali-kali

2. Perhatikan kode berikut.

   Soal 2

   ```
   class Hewan:
       def suara(self):
           return "..."

   class Kucing(Hewan):
       def suara(self):
           return "Meong"

   class Sapi(Hewan):
       def suara(self):
           return "Mooo"

   print("-".join(h.suara() for h in [Sapi(), Hewan(), Kucing()]))
   ```

   Keluaran kode tersebut adalah ...

   a. `Mooo-Meong-...`
   b. `...-...-...`
   c. `Mooo-...-Meong`
   d. `Meong-Mooo-...`

3. Apa yang terjadi jika sebuah class Python mendefinisikan dua method bernama `hitung` dengan jumlah parameter berbeda?

   a. Python memilih method yang sesuai jumlah argumen
   b. Terjadi `SyntaxError`
   c. Definisi kedua menimpa definisi pertama

d. Kedua method digabung otomatis

**4.** Agar ekspresi `a + b` bekerja untuk object dari class buatan sendiri, class tersebut perlu mengimplementasikan ...

a. `__plus__`
b. `__add__`
c. `__sum__`
d. `__concat__`

**5.** Hubungan antara *Pesanan* dan *ItemPesanan*, di mana item dibuat oleh pesanan dan dihapus bersama pesanan, paling tepat dimodelkan sebagai ...

a. inheritance
b. association
c. aggregation
d. composition

**6.** Ciri yang membedakan aggregation dari composition adalah ...

a. pada aggregation, bagian tetap dapat hidup tanpa keseluruhan
b. pada aggregation, bagian selalu dibuat di dalam constructor keseluruhan
c. aggregation hanya berlaku untuk satu object bagian
d. aggregation memakai inheritance

**7.** Perhatikan kode berikut.

Soal 7

```python
class Baterai:
    def __init__(self, mah):
        self.mah = mah

class Ponsel:
    def __init__(self, merek, baterai):
        self.merek = merek
        self.baterai = baterai

b = Baterai(5000)
p1 = Ponsel("A", b)
p2 = Ponsel("B", b)
```

Hubungan `Ponsel–Baterai` pada kode di atas paling tepat disebut ...

a. composition, karena baterai adalah bagian ponsel
b. aggregation, karena baterai dibuat di luar dan dipakai bersama
c. inheritance, karena ponsel memiliki baterai
d. dependency, karena baterai hanya parameter

8. Notasi UML belah ketupat *penuh* di sisi class keseluruhan menyatakan hubungan ...

   a. association
   b. aggregation
   c. composition
   d. dependency

9. Multiplicity `0..1` pada ujung class `Dosen` dalam hubungan Mahasiswa–Dosen pembimbing berarti ...

   a. setiap dosen membimbing tepat satu mahasiswa
   b. setiap mahasiswa memiliki nol atau satu dosen pembimbing
   c. setiap mahasiswa wajib memiliki banyak pembimbing
   d. dosen tidak boleh membimbing mahasiswa

10. Alasan class `Stack` sebaiknya memakai composition terhadap `list` daripada mewarisi `list` adalah ...

    a. composition membuat program lebih cepat
    b. inheritance dari `list` tidak diizinkan Python
    c. inheritance ikut mewariskan method seperti `insert` yang melanggar aturan stack
    d. composition otomatis membuat stack immutable

### Uraian

1. Jelaskan bagaimana polymorphism membantu menerapkan prinsip *open/closed*. Gunakan contoh selain notifikasi.
2. Python tidak mendukung method overloading secara langsung. Jelaskan alasannya dan tiga cara untuk meniru efek overloading.
3. Jelaskan perbedaan association, aggregation, dan composition dari segi kepemilikan dan umur object, masing-masing dengan satu contoh dari lingkungan kampus.
4. Berikan satu pasang class yang dapat dimodelkan sebagai composition pada satu sistem, tetapi sebagai aggregation pada sistem lain. Jelaskan kebutuhan sistem yang membedakannya.
5. Jelaskan maksud pedoman “utamakan composition daripada inheritance”. Kapan inheritance tetap merupakan pilihan yang lebih tepat?

## 6.11 Praktik Mandiri

Kerjakan praktik berikut secara berurutan. Simpan setiap praktik dalam file `.py` terpisah, jalankan, dan pastikan keluarannya sesuai dengan contoh.

### Praktik 1: Arena Pertarungan

- Buat abstract class `Karakter` dengan attribute `nama` dan `hp`, abstract method `serang(lawan)`, serta method konkret `terima_serangan(damage)` dan `masih_hidup()`.
- Buat tiga turunan: `Prajurit` (damage tetap 15), `Penyihir` (damage 25, tetapi hanya bisa menyerang jika mana masih ada; setiap serangan mengurangi mana 10), dan `Pemanah` (damage 10, menyerang dua kali).
- Implementasikan `__lt__` agar karakter bisa diurutkan berdasarkan HP, dan `__str__` untuk menampilkan status.
- Tulis fungsi `ronde(tim_a, tim_b)` yang membuat setiap karakter hidup di tim A menyerang karakter hidup pertama di tim B. Fungsi ini **tidak boleh** memeriksa jenis karakter.

**Contoh keluaran Praktik 1**

```
Arka (Prajurit) menyerang Zed: -15
Luna (Penyihir) menyerang Zed: -25
Zed (Pemanah) menyerang Arka: -10 -10
Urutan HP: Zed 60 | Arka 80 | Luna 70 -> Zed, Luna, Arka
```

### Praktik 2: Pecahan

Buat class `Pecahan` yang mendukung operator overloading:

- Constructor menerima pembilang dan penyebut, menolak penyebut nol, dan otomatis menyederhanakan pecahan (gunakan `math.gcd`).
- Operator `+`, `-`, `*`, `==`, dan `<`.
- `__str__` menampilkan `a/b`, atau hanya `a` jika penyebutnya 1.
- Method `tambah()` yang meniru overloading: dapat menerima satu `Pecahan`, satu bilangan bulat, atau beberapa argumen sekaligus.

**Contoh keluaran Praktik 2**

```
1/2 + 1/3 = 5/6
3/4 - 1/4 = 1/2
2/3 * 3/4 = 1/2
2/4 == 1/2: True
sorted: 1/4, 1/3, 1/2
1/2 + 2 + 1/4 + 1/4 = 3
```

### Praktik 3: Sistem Akademik Mini

Rancang dan implementasikan sistem akademik dengan class `Universitas`, `Fakultas`, `ProgramStudi`, `Dosen`, `Mahasiswa`, `MataKuliah`, dan `KRS`.

### Langkah Pengerjaan

1. Tentukan jenis hubungan untuk setiap pasangan class yang saling terkait, lengkap dengan multiplicity dan alasan berdasarkan kebutuhan sistem. Tuliskan dalam bentuk tabel di komentar program.
2. Gambar class diagram sederhana (boleh di kertas atau aplikasi diagram) dengan notasi UML yang benar.
3. Implementasikan hubungan tersebut: composition dibuat di dalam constructor, aggregation menerima object dari luar, dan association disimpan sebagai rujukan.
4. Tunjukkan bukti perilaku setiap hubungan, misalnya dosen tetap ada setelah sebuah prodi dihapus (aggregation), atau KRS ikut tidak terpakai saat mahasiswa dihapus (composition).
5. Tambahkan polymorphism: dua jenis mata kuliah (misalnya `MataKuliahTeori` dan `MataKuliahPraktikum`) dengan cara hitung bobot nilai berbeda, lalu hitung IPK mahasiswa tanpa memeriksa jenis mata kuliah.

**KRITERIA PENILAIAN**

- Polymorphism diterapkan tanpa pemeriksaan jenis (`if-elif`) di kode pemanggil.
- Operator overloading memiliki makna yang wajar dan konsisten.
- Setiap relationship dipilih dengan alasan yang jelas dan diimplementasikan sesuai cirinya.
- Class diagram sesuai dengan kode yang ditulis.

Bab 07

# Modular Programming

*Memecah program menjadi bagian yang mudah dipakai ulang dan dirawat.*

07

DI BAB INI

- Modular programming
- Module dan import
- Package
- Struktur folder project
- Reusability dan maintainability

Studi kasus perpustakaan di akhir Bab 6 sudah mencapai sekitar tujuh puluh baris dalam satu file `perpustakaan.py`. Ukuran itu masih nyaman dibaca. Sekarang bayangkan tiga bab ke depan: Bab 8 menambahkan class-class exception, Bab 9 menambahkan kode untuk menyimpan data ke file, dan Bab 10 menambahkan jendela GUI. Jika semuanya ditumpuk di file yang sama, file itu akan mencapai ratusan baris, dan mencari satu method berarti menggulir layar berkali-kali. Masalahnya bertambah ketika program dikerjakan bertiga, karena setiap orang mengubah file yang sama dan perubahan mereka saling bertabrakan.

Bab ini membahas *modular programming*, yaitu cara memecah program menjadi beberapa file yang masing-masing punya tugas jelas. Kita akan mengenal *module* dan *package*, berbagai bentuk perintah `import`, serta struktur folder proyek yang rapi. Di tengah bab, kode perpustakaan dari Bab 6 kita pecah menjadi sebuah package yang akan terus dikembangkan di bab-bab berikutnya.

## 7.1 Konsep Modular Programming

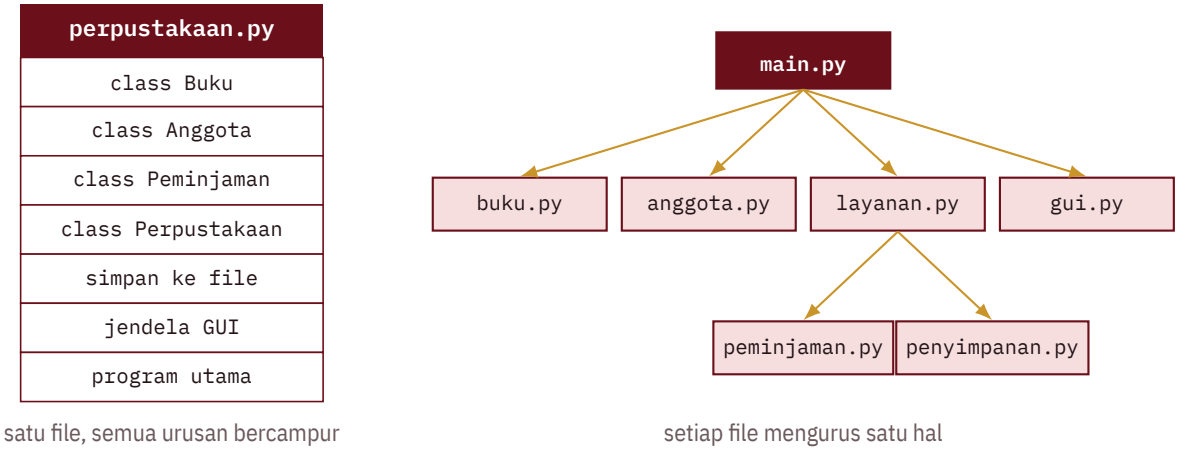
*Modular programming* adalah cara menyusun program dari bagian-bagian kecil yang disebut *modul*. Setiap modul berisi kode untuk satu urusan, misalnya urusan data buku atau urusan menyimpan file. Modul-modul itu saling memakai melalui perintah `import`.

Contoh yang paling dekat adalah buku ini sendiri. Buku ini ditulis dengan LaTeX, dan naskahnya tidak disimpan dalam satu file raksasa. File utama `main.tex` hanya berisi pengaturan dan daftar bab, sedangkan setiap bab disimpan di file sendiri di folder `bab/`. Penulis Bab 8 bisa bekerja tanpa membuka file Bab 2, dan salah ketik di satu bab mudah dilacak karena letaknya jelas. Program Python dipecah dengan cara yang sama.

### Mengapa Perlu Dipecah?

Program satu file bekerja dengan baik selama ukurannya kecil. Masalah baru terasa ketika program tumbuh:

- **Sulit dicari.** Untuk memperbaiki cara menghitung lama pinjam, kita harus menemukan beberapa baris di antara ratusan baris lain yang tidak berhubungan.
- **Sulit dikerjakan bersama.** Dua orang yang mengubah file yang sama pada waktu bersamaan harus menggabungkan perubahan secara manual.
- **Sulit dipakai ulang.** Fungsi format rupiah yang bagus di program perpustakaan tidak bisa dipakai program lain tanpa menyalin kodenya.




**Gambar 7.1.** Program yang sama dalam satu file (kiri) dan dipecah menjadi beberapa module (kanan). Panah menunjukkan arah `import`: file di pangkal panah memakai file di ujungnya.

Dengan memecah program, setiap masalah di atas mengecil. Bagian akhir bab ini membahas dua manfaat terpenting secara khusus, yaitu *reusability* dan *maintainability*.

## 7.2 Module

Di Python, setiap file `.py` adalah sebuah *module*. Nama module sama dengan nama file tanpa akhiran `.py`. Kita sudah memakai beberapa module bawaan, misalnya `functools` di Bab 3 dan `math` di Bab 5. Membuat module sendiri tidak memerlukan langkah khusus: cukup tulis sebuah file Python, lalu impor dari file lain di folder yang sama.

kampus_util.py

```
"""Fungsi bantu yang dipakai di banyak program kampus."""

KAMPUS = "Universitas Negeri Surabaya"

def format_rupiah(angka):
    return f"Rp{angka:,}".replace(",", ".")

def nim_valid(nim):
    return nim.isdigit() and len(nim) == 11
```

main.py

```
import kampus_util

print(kampus_util.KAMPUS)
print(kampus_util.format_rupiah(2500000))
print(kampus_util.nim_valid("24051204084"))
print(kampus_util.nim_valid("2405-ABC"))
```

Keluaran

```
Universitas Negeri Surabaya
Rp2.500.000
True
False
```

Perintah `import kampus_util` mencari file `kampus_util.py`, menjalankan isinya, lalu menyediakan sebuah *object module* bernama `kampus_util`. Semua nama yang dibuat di file itu, baik variabel, fungsi, maupun class, menjadi attribute object tersebut. Karena itu kita mengaksesnya dengan titik, persis seperti mengakses attribute object di Bab 1.

Module juga sebuah object

```
import kampus_util

print(type(kampus_util))
print(kampus_util.__name__)
print([n for n in dir(kampus_util) if not n.startswith("_")])
```

Keluaran

```
<class 'module'>
kampus_util
['KAMPUS', 'format_rupiah', 'nim_valid']
```

## Module Hanya Dimuat Sekali

Python mengingat setiap module yang sudah diimpor. Jika module yang sama diimpor lagi, entah dari file yang sama atau dari file lain, Python tidak menjalankan isinya untuk kedua kalinya. Module yang sudah ada langsung dipakai ulang.

salam.py

```
print("Module salam sedang dimuat...")

def halo(nama):
    return f"Halo, {nama}!"
```

main.py

```
import salam
import salam                    # impor kedua: salam.py tidak dijalankan lagi

print(salam.halo("Ani"))
```

Keluaran

```
Module salam sedang dimuat...
Halo, Ani!
```

Perilaku ini membuat import murah untuk diulang. Lima module boleh sama-sama mengimpor `kampus_util`, dan isi `kampus_util.py` tetap hanya dijalankan satu kali. Daftar module yang sudah dimuat tersimpan di dictionary `sys.modules`.

### Blok `if __name__ == "__main__"`

Saat menulis module, kita sering ingin mencoba fungsinya langsung di file yang sama. Cara yang terasa wajar adalah menambahkan beberapa `print` di bagian bawah file. Akibatnya kurang menyenangkan: baris uji itu ikut berjalan setiap kali module diimpor oleh program lain.

kampus_util.py

```python
"""Fungsi bantu yang dipakai di banyak program kampus."""

KAMPUS = "Universitas Negeri Surabaya"

def format_rupiah(angka):
    return f"Rp{angka:,}".replace(",", ".")

def nim_valid(nim):
    return nim.isdigit() and len(nim) == 11

print("Uji:", format_rupiah(1500), nim_valid("123"))      # baris uji
```

main.py

```python
import kampus_util

print(kampus_util.format_rupiah(2500000))
```

Keluaran

```
Uji: Rp1.500 False
Rp2.500.000
```

Baris `Uji:` tidak diminta oleh `main.py`, tetapi tetap muncul. Untuk membedakan "dijalankan langsung" dari "diimpor", Python menyediakan variabel khusus `__name__` di setiap module. Nilainya `"__main__"` jika file itu yang dijalankan dengan perintah `python`, dan berisi nama module jika file itu diimpor.

Dengan memeriksa nilai tersebut, kode uji hanya berjalan ketika module dijalankan langsung.

**Tabel 7.1.** Nilai `__name__` pada file `kampus_util.py`.

| Cara file dipakai | Nilai `__name__` | Blok `if` dijalankan? |
|---|---|---|
| dijalankan langsung: `python kampus_util.py` | `"__main__"` | ya |
| diimpor oleh `main.py`: `import kampus_util` | `"kampus_util"` | tidak |

**kampus_util.py**

```python
"""Fungsi bantu yang dipakai di banyak program kampus."""

KAMPUS = "Universitas Negeri Surabaya"

def format_rupiah(angka):
    return f"Rp{angka:,}".replace(",", ".")

def nim_valid(nim):
    return nim.isdigit() and len(nim) == 11

if __name__ == "__main__":              # hanya saat dijalankan langsung
    print("Uji:", format_rupiah(1500), nim_valid("123"))
```

**python kampus_util.py**

```
Uji: Rp1.500 False
```

**main.py**

```python
import kampus_util

print(kampus_util.format_rupiah(2500000))
```

**python main.py**

```
Rp2.500.000
```

KEBIASAAN BAIK

Biasakan setiap module diakhiri blok `if __name__ == "__main__":` yang berisi contoh pemakaian atau uji singkat. Module tetap bersih ketika diimpor, dan siapa pun yang membuka file itu langsung melihat cara memakainya.

**DI MANA PYTHON MENCARI MODULE?**

Saat menemui `import kampus_util`, Python memeriksa daftar folder di `sys.path` secara berurutan. Urutannya kira-kira: folder tempat file yang dijalankan berada, folder di variabel lingkungan `PYTHONPATH`, folder pustaka standar, lalu folder `site-packages` tempat library hasil `pip install`. File pertama yang cocok akan dipakai.

**JANGAN MENAMAI FILE SEPERTI MODULE BAWAAN**

Karena folder program diperiksa paling awal, file buatan sendiri bernama `random.py`, `math.py`, atau `json.py` akan *menutupi* module bawaan yang bernama sama. Perintah `import random` lalu memuat file Anda, dan pemanggilan `random.randint()` gagal dengan `AttributeError`. Python 3.13 ke atas bahkan menambahkan saran untuk mengganti nama file tersebut di pesan kesalahannya. Gunakan nama yang menggambarkan isi, misalnya `undian.py`.

## 7.3 Import

Perintah `import` punya beberapa bentuk. Semuanya memuat module dengan cara yang sama; perbedaannya terletak pada *nama* yang tersedia setelah perintah itu dijalankan.

**Tabel 7.2.** Bentuk-bentuk perintah `import`.

| Bentuk | Nama yang tersedia | Cara memakai |
|---|---|---|
| `import math` | `math` | `math.sqrt(16)` |
| `import kampus_util as ku` | `ku` | `ku.nim_valid(nim)` |
| `from math import sqrt, pi` | `sqrt` dan `pi` | `sqrt(16)`, `pi` |
| `from kampus_util import format_rupiah as rp` | `rp` | `rp(5000)` |
| `from math import *` | semua nama publik di `math` | tidak disarankan |

**Beberapa bentuk import dalam satu program**

```
import math
import kampus_util as ku
from math import sqrt, pi
from kampus_util import format_rupiah as rp

print(math.floor(3.7), sqrt(16), round(pi, 2))
print(ku.nim_valid("24051204084"))
print(rp(18000))
```

Keluaran

```
3 4.0 3.14
True
Rp18.000
```

## Memilih Bentuk Import

Bentuk `import module` membuat asal setiap nama terlihat jelas. Siapa pun yang membaca `kampus_util.format_rupiah(5000)` langsung tahu fungsi itu berasal dari mana. Bentuk `from module import nama` lebih ringkas dan cocok untuk nama yang sering dipakai di satu file. Alias dengan `as` berguna untuk nama module yang panjang. Pustaka analisis data seperti NumPy dan pandas, misalnya, hampir selalu diimpor sebagai `np` dan `pd`.

## Bentrok Nama

Masalah muncul ketika dua module memiliki nama yang sama dan keduanya diimpor dengan `from … import`. Nama yang diimpor belakangan menimpa yang lebih dulu tanpa peringatan apa pun, mirip dengan definisi method kedua yang menimpa definisi pertama di Bab 6.

toko_util.py

```python
def format_rupiah(angka):
    return "IDR " + str(angka)
```

main.py

```python
from kampus_util import *
from toko_util import *          # format_rupiah milik kampus_util tertimpa

print(format_rupiah(18000))
```

Keluaran

```
IDR 18000
```

Dengan `import kampus_util` dan `import toko_util`, kedua fungsi bisa dipakai berdampingan sebagai `kampus_util.format_rupiah` dan `toko_util.format_rupiah`. Nama module berfungsi sebagai *namespace* yang memisahkan nama-nama yang kebetulan sama.

**HINDARI** `FROM MODULE IMPORT *`

Bentuk bintang mengimpor semua nama publik sekaligus. Akibatnya pembaca tidak bisa tahu asal sebuah nama, nama yang sudah ada bisa tertimpa diam-diam seperti contoh di atas, dan editor kode kesulitan memberi saran. Tuliskan nama yang dibutuhkan satu per satu.

### Module Bawaan dan Library dari Luar

Python dipasang bersama ratusan module siap pakai yang disebut *pustaka standar* (*standard library*). Beberapa di antaranya akan kita pakai di bab-bab berikutnya.

**Tabel 7.3.** Beberapa module pustaka standar yang dipakai di buku ini.

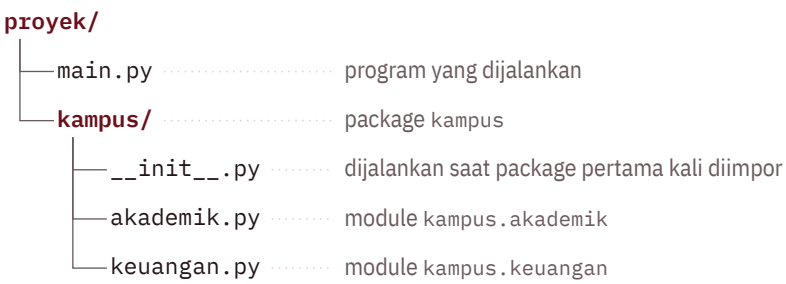
| Module | Kegunaan | Dipakai di |
|---|---|---|
| `abc` | membuat abstract class dan abstract method | Bab 5, 6 |
| `dataclasses` | membuat data class | Bab 5, 9 |
| `functools` | `wraps`, `singledispatchmethod` | Bab 3, 6 |
| `pathlib` | mengolah path file dan folder | Bab 9 |
| `csv`, `json` | membaca dan menulis file CSV dan JSON | Bab 9 |
| `tkinter` | membuat jendela GUI | Bab 10 |

Module di luar pustaka standar dipasang dengan `pip`, misalnya `pip install requests`. Setelah terpasang, module tersebut diimpor dengan cara yang sama persis seperti module bawaan. Agar orang lain bisa menjalankan proyek kita, daftar library luar yang dibutuhkan biasanya dicatat di file `requirements.txt`, yang akan kita lihat di subbab struktur folder.

## 7.4 Package

Ketika jumlah module bertambah, module-module yang berhubungan dikumpulkan dalam satu folder. Folder berisi module yang dapat diimpor ini disebut *package*. Sebuah folder menjadi package jika di dalamnya terdapat file `__init__.py`, walaupun file itu kosong.

Sebagai contoh, fungsi-fungsi bantu kampus kita pisahkan menjadi dua urusan, yaitu akademik dan keuangan, lalu dikumpulkan dalam package `kampus`.




**Gambar 7.2.** Struktur folder package `kampus`. Nama module di dalam package ditulis dengan titik: `kampus.akademik` berarti file `akademik.py` di folder `kampus`.

kampus/__init__.py

```python
# boleh kosong: keberadaan file ini sudah cukup
```

kampus/keuangan.py

```python
def format_rupiah(angka):
    return f"Rp{angka:,}".replace(",", ".")
```

kampus/akademik.py

```python
from .keuangan import format_rupiah

def nilai_huruf(angka):
    if angka >= 85:
        return "A"
    if angka >= 70:
        return "B"
    if angka >= 55:
        return "C"
    return "D"

def beasiswa(angka):
    besar = 3000000 if nilai_huruf(angka) == "A" else 0
    return format_rupiah(besar)

if __name__ == "__main__":
    print(nilai_huruf(90), beasiswa(90))
```

main.py

```python
import kampus.akademik
from kampus import keuangan
from kampus.akademik import beasiswa

print(kampus.akademik.nilai_huruf(60))
print(keuangan.format_rupiah(1250000))
print(beasiswa(91))
```

Keluaran

```
C
Rp1.250.000
Rp3.000.000
```

## Relative Import

Perhatikan baris pertama di `akademik.py`, yaitu `from .keuangan import format_rupiah`. Bentuk ini disebut *relative import*. Titik di depan nama module berarti "dari package yang sama dengan file ini". Dua titik (`..`) berarti package satu tingkat di atasnya. Bentuk lengkapnya, `from kampus.keuangan import format_rupiah`, disebut *absolute import*. Keduanya benar. Relative import membuat module di dalam package tidak bergantung pada nama package, sehingga package masih berfungsi jika suatu saat folder `kampus` diganti namanya.

Ada satu jebakan. Relative import hanya berlaku jika file dimuat *sebagai bagian dari package*. Jika `akademik.py` dijalankan langsung sebagai skrip, Python tidak tahu package induknya.

**Terminal**

```
$ python kampus/akademik.py
Traceback (most recent call last):
  ...
ImportError: attempted relative import with no known parent package
$ python -m kampus.akademik
A Rp3.000.000
```

**MENJALANKAN MODULE DI DALAM PACKAGE**

Gunakan `python -m nama_package.nama_module` dari folder proyek, tanpa akhiran `.py`. Opsi `-m` memuat file sebagai module di dalam package, sehingga relative import berjalan dan blok `if __name__ == "__main__"` tetap dijalankan.

## Mengatur Isi `__init__.py`

File `__init__.py` dijalankan satu kali saat package pertama kali diimpor. Selain menandai folder sebagai package, file ini sering dipakai untuk membuka nama-nama penting langsung di tingkat package. Pemakai package cukup menulis `import kampus` tanpa perlu tahu nama-nama module di dalamnya.

**kampus/__init__.py**

```python
from .akademik import beasiswa, nilai_huruf
from .keuangan import format_rupiah

__all__ = ["beasiswa", "nilai_huruf", "format_rupiah"]
```

**main.py**

```python
import kampus

print(kampus.nilai_huruf(75), kampus.format_rupiah(5000))
```

Keluaran

```
B Rp5.000
```

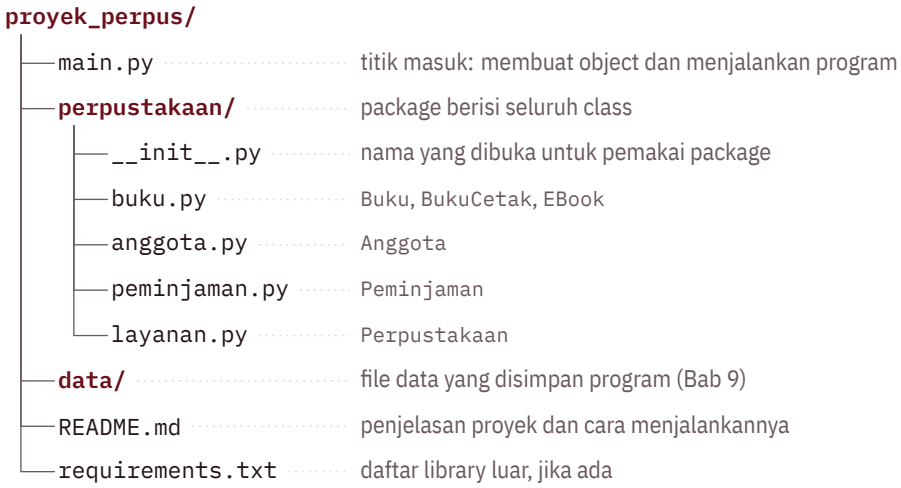
Variabel `__all__` berisi daftar nama yang dianggap publik. Daftar ini menentukan nama apa saja yang ikut terbawa oleh `from kampus import *`, dan sekaligus menjadi dokumentasi singkat tentang apa yang ditawarkan package.

**FOLDER TANPA `__INIT__.PY`**

Sejak Python 3.3, folder tanpa `__init__.py` pun dapat diimpor sebagai *namespace package*. Fitur ini dibuat untuk kasus khusus, yaitu satu package yang tersebar di beberapa lokasi. Untuk proyek biasa, tetap sertakan `__init__.py` agar jelas bahwa folder itu sebuah package.

## 7.5 Struktur Folder Project

Tidak ada satu struktur folder yang wajib untuk proyek Python. Namun proyek kecil hingga menengah, termasuk proyek akhir mata kuliah ini, umumnya mengikuti pola yang sama: satu file titik masuk di folder teratas, kode program di dalam package, dan data dipisahkan dari kode.




**Gambar 7.3.** Struktur folder yang disarankan untuk proyek perpustakaan.

Beberapa aturan sederhana membuat struktur ini tetap rapi seiring proyek bertambah besar:

- **Satu titik masuk.** `main.py` sebaiknya pendek. Isinya hanya membuat object yang dibutuhkan lalu menjalankan program. Definisi class tidak ditulis di sini.
- **Satu urusan per module.** Nama file harus bisa menjawab pertanyaan "apa isi file ini?". Nama seperti `coba2.py` atau `lain_lain.py` adalah tanda pembagiannya belum jelas.
- **Data terpisah dari kode.** File yang dibaca dan ditulis program disimpan di folder `data/`, bukan bercampur dengan file `.py`.

- **Nama module huruf kecil.** Gunakan huruf kecil dan garis bawah, tanpa spasi atau tanda hubung. Nama `data-buku.py` tidak bisa diimpor karena tanda - dibaca sebagai pengurangan.

**JALANKAN DARI FOLDER PROYEK**

Buka terminal di folder `proyek_perpus`, lalu jalankan `python main.py`. Folder tempat program dijalankan memengaruhi tempat Python mencari module, dan di Bab 9 juga memengaruhi lokasi file data. Menjalankan program dari folder lain adalah penyebab umum pesan `ModuleNotFoundError` dan `FileNotFoundError`.

## 7.6 Pemisahan Class ke dalam Module

Sekarang kita terapkan semuanya pada kode perpustakaan dari Bab 6. Tujuannya adalah memecah satu file menjadi package tanpa mengubah perilaku program sedikit pun.

### Memecah `perpustakaan.py` menjadi Package

1. Kelompokkan class berdasarkan urusannya: buku beserta turunannya, anggota, catatan peminjaman, dan layanan perpustakaan.
2. Buat folder `perpustakaan/` beserta file `__init__.py`.
3. Pindahkan setiap kelompok ke module sendiri. Tambahkan import yang dibutuhkan oleh module itu saja.
4. Atur `__init__.py` agar class yang dipakai dari luar bisa diimpor langsung dari package.
5. Tulis ulang `main.py` sehingga isinya hanya membuat object dan menjalankan program, lalu bandingkan keluarannya dengan versi satu file.

**perpustakaan/buku.py**

```
from abc import ABC, abstractmethod

class Buku(ABC):
    def __init__(self, judul):
        self.judul = judul
        self.dipinjam = False

    @abstractmethod
    def lama_pinjam(self):
        ...

```

```
class BukuCetak(Buku):
    def lama_pinjam(self):
        return 14

class EBook(Buku):
    def lama_pinjam(self):
        return 7
```

perpustakaan/anggota.py

```
class Anggota:
    def __init__(self, nama):
        self.nama = nama
```

perpustakaan/peminjaman.py

```
class Peminjaman:
    def __init__(self, anggota, buku):
        self.anggota = anggota
        self.buku = buku

    def __str__(self):
        return (f"{self.anggota.nama} meminjam '{self.buku.judul}' "
                f"selama {self.buku.lama_pinjam()} hari")
```

Perhatikan bahwa `peminjaman.py` tidak mengimpor `Anggota` maupun `Buku`. Class `Peminjaman` hanya memakai attribute `nama`, `judul`, dan method `lama_pinjam()` dari object yang diterimanya. Berkat duck typing (Bab 5), ia tidak perlu tahu class asli object tersebut. Semakin sedikit import, semakin sedikit pula ketergantungan antarmodule.

perpustakaan/layanan.py

```
from .peminjaman import Peminjaman

class Perpustakaan:
    def __init__(self, nama):
        self.nama = nama
        self._buku = []
        self._anggota = []
        self._catatan = []

    def tambah_buku(self, *buku):
        self._buku.extend(buku)

    def tambah_anggota(self, *anggota):
        self._anggota.extend(anggota)

```

```
    def pinjamkan(self, anggota, buku):
        if buku.dipinjam:
            return f"'{buku.judul}' sedang dipinjam"
        buku.dipinjam = True
        catatan = Peminjaman(anggota, buku)        # composition
        self._catatan.append(catatan)
        return str(catatan)

    def jumlah_peminjaman(self):
        return len(self._catatan)
```

perpustakaan/__init__.py

```
from .anggota import Anggota
from .buku import Buku, BukuCetak, EBook
from .layanan import Perpustakaan

__all__ = ["Anggota", "Buku", "BukuCetak", "EBook", "Perpustakaan"]
```

main.py

```
from perpustakaan import Anggota, BukuCetak, EBook, Perpustakaan

ani, budi = Anggota("Ani"), Anggota("Budi")
laskar = BukuCetak("Laskar Pelangi")
pbo = EBook("Dasar PBO dengan Python")

perpus = Perpustakaan("Perpustakaan UNESA")
perpus.tambah_anggota(ani, budi)
perpus.tambah_buku(laskar, pbo)
print(perpus.pinjamkan(ani, laskar))
print(perpus.pinjamkan(budi, pbo))
print(perpus.pinjamkan(budi, laskar))
print("Total peminjaman:", perpus.jumlah_peminjaman())
```

Keluaran

```
Ani meminjam 'Laskar Pelangi' selama 14 hari
Budi meminjam 'Dasar PBO dengan Python' selama 7 hari
'Laskar Pelangi' sedang dipinjam
Total peminjaman: 2
```

Keluarannya sama persis dengan versi satu file di Bab 6. Perubahan seperti ini, yaitu menata ulang struktur kode tanpa mengubah perilakunya, disebut *refactoring*. Membandingkan keluaran sebelum dan sesudah adalah cara paling sederhana untuk memastikan refactoring tidak merusak apa pun.

### Import Melingkar

Saat memecah class ke beberapa module, ada satu kesalahan yang sering terjadi: dua module saling mengimpor. Misalkan class `Dosen` membuat object `Mahasiswa` bimbingan, sementara class `Mahasiswa` memeriksa bahwa pembimbingnya benar-benar sebuah `Dosen`.

**lingkar/dosen.py**

```
from lingkar.mahasiswa import Mahasiswa

class Dosen:
    def __init__(self, nama):
        self.nama = nama

    def bimbing(self, nama_mhs):
        return Mahasiswa(nama_mhs, self)
```

**lingkar/mahasiswa.py**

```
from lingkar.dosen import Dosen

class Mahasiswa:
    def __init__(self, nama, pembimbing):
        if not isinstance(pembimbing, Dosen):
            raise TypeError("Pembimbing harus seorang Dosen.")
        self.nama = nama
        self.pembimbing = pembimbing
```

**main.py**

```
from lingkar.dosen import Dosen

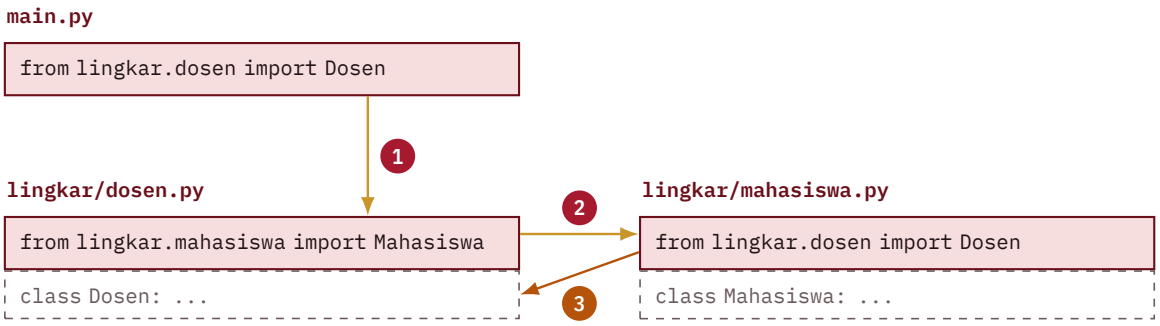
ani = Dosen("Rina").bimbing("Ani")
print(ani.nama, "dibimbing", ani.pembimbing.nama)
```

**Keluaran**

```
Traceback (most recent call last):
  ...
ImportError: cannot import name 'Dosen' from partially initialized module 'lingkar.
    dosen' (most likely due to a circular import)
```

Pesan kesalahannya menyebut *partially initialized module*, yaitu module yang baru setengah dimuat. Gambar berikut memperlihatkan urutan kejadiannya.

Cara mengatasinya bukan dengan mengutak-atik urutan baris import, melainkan dengan memperbaiki rancangan. Ketergantungan antarmodule sebaiknya *mengalir satu arah*. Ada dua jalan yang umum:




Langkah 3 meminta `Dosen`, padahal `dosen.py` baru berjalan sampai baris pertama. Class `Dosen` belum pernah dibuat, sehingga import gagal.

**Gambar 7.4.** Urutan kejadian import melingkar. Kotak berwarna sudah dijalankan; kotak putus-putus belum dijalankan.

- **Hapus ketergantungan yang tidak perlu.** Apakah `Mahasiswa` benar-benar harus tahu class `Dosen`? Yang dibutuhkan hanyalah pembimbing yang punya `nama`. Dengan duck typing, import itu bisa dibuang.
- **Pindahkan bagian bersama ke module ketiga.** Jika kedua module memang membutuhkan sesuatu yang sama, letakkan bagian itu di module baru yang diimpor oleh keduanya, sementara keduanya tidak saling mengimpor.

`lingkar/mahasiswa.py`

```
class Mahasiswa:                        # tidak lagi mengimpor dosen.py
    def __init__(self, nama, pembimbing):
        if not hasattr(pembimbing, "nama"):
            raise TypeError("Pembimbing harus memiliki nama.")
        self.nama = nama
        self.pembimbing = pembimbing
```

`main.py`

```
from lingkar.dosen import Dosen

ani = Dosen("Rina").bimbing("Ani")
print(ani.nama, "dibimbing", ani.pembimbing.nama)
```

`Keluaran`

```
Ani dibimbing Rina
```

Sekarang hanya `dosen.py` yang mengimpor `mahasiswa.py`, tidak sebaliknya. Import melingkar hilang, dan class `Mahasiswa` malah menjadi lebih mudah dipakai ulang karena tidak lagi terikat pada class `Dosen`.

## 7.7 Reusability

*Reusability* adalah kemampuan sebuah kode untuk dipakai ulang di tempat lain tanpa disalin. Module adalah satuan pemakaian ulang yang paling alami di Python. Package `kampus` yang kita buat tadi, misalnya, bisa langsung dipakai oleh dua program yang sama sekali berbeda.

**kantin.py**

```
from kampus import format_rupiah

pesanan = {"Nasi goreng": 15000, "Es teh": 4000, "Kerupuk": 2000}
for menu, harga in pesanan.items():
    print(f"{menu:<12}{format_rupiah(harga):>10}")
print(f"{'Total':<12}{format_rupiah(sum(pesanan.values())):>10}")
```

**Keluaran**

```
Nasi goreng   Rp15.000
Es teh         Rp4.000
Kerupuk        Rp2.000
Total         Rp21.000
```

**laporan_beasiswa.py**

```
from kampus import beasiswa, nilai_huruf

for nama, nilai in [("Ani", 91), ("Budi", 72), ("Citra", 86)]:
    print(f"{nama:<6} {nilai_huruf(nilai)}  {beasiswa(nilai)}")
```

**Keluaran**

```
Ani    A  Rp3.000.000
Budi   B  Rp0
Citra  A  Rp3.000.000
```

Kedua program tidak berisi satu baris pun kode format rupiah. Keduanya memakai fungsi yang sama dari package `kampus`.

### Docstring dan `help()`

Module yang dipakai ulang akan dibaca oleh orang yang tidak menulisnya. Karena itu module perlu menjelaskan dirinya sendiri. Caranya adalah dengan *docstring*, yaitu teks di antara tiga tanda kutip yang ditulis sebagai baris pertama module, class, atau fungsi. Docstring bisa dibaca oleh fungsi bawaan `help()` dan ditampilkan oleh editor kode saat kursor diarahkan ke nama fungsi.

kampus/keuangan.py

```python
"""Fungsi bantu urusan keuangan kampus."""

def format_rupiah(angka):
    """Ubah angka menjadi teks rupiah, misalnya 18000 menjadi 'Rp18.000'."""
    return f"Rp{angka:,}".replace(",", ".")
```

Membaca docstring

```python
from kampus.keuangan import format_rupiah

help(format_rupiah)
```

Keluaran

```
Help on function format_rupiah in module kampus.keuangan:

format_rupiah(angka)
    Ubah angka menjadi teks rupiah, misalnya 18000 menjadi 'Rp18.000'.
```

## Ciri Module yang Mudah Dipakai Ulang

Tidak semua module sama mudahnya untuk dipakai ulang. Bandingkan dua fungsi berikut.

Mencetak atau mengembalikan nilai

```python
def tampilkan_total(daftar_harga):          # sulit dipakai ulang
    print("Total:", sum(daftar_harga))

def hitung_total(daftar_harga):             # mudah dipakai ulang
    return sum(daftar_harga)
```

Fungsi pertama hanya berguna untuk program terminal. Program GUI di Bab 10 butuh angka total untuk ditampilkan di sebuah label, dan program di Bab 9 butuh angka itu untuk ditulis ke file. Keduanya tidak bisa memakai `tampilkan_total()`. Fungsi kedua mengembalikan nilai dan membiarkan pemanggilnya memutuskan apa yang dilakukan dengan nilai itu. Dari contoh ini kita bisa menyusun beberapa ciri module yang baik:

- mengembalikan nilai, bukan mencetak sendiri atau meminta `input()` sendiri;
- tidak bergantung pada variabel global milik program tertentu, sehingga semua yang dibutuhkan diterima lewat parameter;
- punya nama dan docstring yang menjelaskan isinya;
- menyimpan kode uji di dalam blok `if __name__ == "__main__"`.

## 7.8 Maintainability

*Maintainability* adalah seberapa mudah sebuah program diperbaiki dan diubah setelah selesai ditulis. Program jarang selesai sekali jadi. Aturan kampus berubah, pengguna meminta fitur baru, dan bug ditemukan berbulan-bulan kemudian. Program yang mudah dirawat adalah program yang perubahannya cukup dilakukan di *satu tempat*.

Misalkan kampus menambahkan nilai AB untuk rentang 80 sampai 84. Dengan package `kampus`, perubahan cukup dilakukan pada fungsi `nilai_huruf()` di `akademik.py`. Program laporan beasiswa dan program lain yang memakai fungsi itu langsung mengikuti aturan baru tanpa disentuh. Seandainya logika nilai huruf disalin ke lima program, kelimanya harus diubah, dan cukup satu yang terlewat untuk membuat dua program memberi nilai berbeda bagi mahasiswa yang sama.

**Tabel 7.4.** Gejala kode yang sulit dirawat dan cara memperbaikinya.

| Gejala | Penyebab | Perbaikan |
|---|---|---|
| File sangat panjang dan sulit dicari | terlalu banyak urusan dalam satu module | pecah per urusan |
| Satu perubahan kecil memaksa mengubah banyak file | logika yang sama disalin ke banyak tempat | pindahkan ke satu fungsi di module bersama |
| Import melingkar | dua module saling bergantung | jadikan ketergantungan satu arah |
| Nama file tidak menjelaskan isi | pembagian module belum dipikirkan | beri nama sesuai urusan |
| Fungsi sulit diuji tanpa menjalankan seluruh program | logika bercampur dengan `print` dan `input` | pisahkan logika dari tampilan |

### Konvensi Penamaan

Kode yang mengikuti konvensi bersama lebih mudah dibaca oleh orang lain, termasuk oleh diri kita sendiri beberapa bulan kemudian. Python memiliki pedoman gaya resmi bernama PEP 8. Aturan penamaannya yang terpenting dirangkum pada tabel berikut, dan semua contoh di buku ini mengikutinya.

PECAH SECUKUPNYA

Memecah program juga bisa berlebihan. Sepuluh file yang masing-masing berisi lima baris sama sulitnya dibaca dengan satu file berisi lima ratus baris. Pecah sebuah module ketika isinya mulai mengurus lebih dari satu hal, bukan sebelumnya.

**Tabel 7.5.** Konvensi penamaan menurut PEP 8.

| Jenis nama | Gaya penulisan | Contoh |
|---|---|---|
| Module dan package | huruf kecil, garis bawah | `kampus_util`, `perpustakaan` |
| Class | setiap kata diawali kapital | `BukuCetak`, `ProgramStudi` |
| Fungsi, method, variabel | huruf kecil, garis bawah | `format_rupiah`, `lama_pinjam` |
| Konstanta | huruf kapital semua | `KAMPUS`, `NOL_MUTLAK` |
| Anggota internal | diawali satu garis bawah | `_catatan`, `_buku` |

## 7.9 Rangkuman

- **Modular programming** menyusun program dari bagian-bagian kecil yang masing-masing mengurus satu hal, sehingga program lebih mudah dicari, dikerjakan bersama, dipakai ulang, dan dirawat.
- Setiap file `.py` adalah **module**. Module diimpor dengan `import`, dijalankan sekali saja, dan isinya diakses sebagai attribute object module.
- Variabel `__name__` bernilai `"__main__"` saat file dijalankan langsung. Blok `if __name__ == "__main__"` dipakai untuk kode uji yang tidak boleh ikut berjalan saat module diimpor.
- `import module` menjaga asal nama tetap jelas, `from module import nama` lebih ringkas, dan `as` memberi alias. Hindari `from module import *` karena dapat menimpa nama secara diam-diam.
- **Package** adalah folder berisi module dan file `__init__.py`. Di dalam package, **relative import** seperti `from .buku import Buku` merujuk module di package yang sama. Module di dalam package dijalankan dengan `python -m`.
- Struktur proyek yang baik memiliki satu titik masuk yang pendek, satu urusan per module, dan data yang terpisah dari kode.
- **Import melingkar** terjadi ketika dua module saling mengimpor. Atasi dengan membuat ketergantungan mengalir satu arah.
- **Reusability** tercapai jika module mengembalikan nilai, tidak bergantung pada variabel global, dan berdokumentasi. **Maintainability** tercapai jika setiap perubahan cukup dilakukan di satu tempat.

## 7.10 Kuis Teoritis

## Pilihan Ganda

Pilihlah satu jawaban yang paling tepat.

1. Pernyataan yang paling tepat tentang module di Python adalah ...

   a. module harus didaftarkan terlebih dahulu sebelum bisa diimpor
   b. setiap file `.py` dapat diimpor sebagai module
   c. module hanya boleh berisi fungsi, tidak boleh berisi class
   d. module harus disimpan di folder pustaka standar

2. Perhatikan dua file berikut, lalu `main.py` dijalankan.

**sapa.py**

```
def sapa():
    return "Halo"

print("A")
if __name__ == "__main__":
    print("B")
```

**main.py**

```
import sapa
print(sapa.sapa())
```

   Keluaran program tersebut adalah ...

   a. `A`, `B`, lalu `Halo`
   b. `A` lalu `Halo`
   c. `Halo` saja
   d. `B` lalu `Halo`

3. Setelah perintah `from math import sqrt`, cara memanggil fungsi akar kuadrat yang benar adalah ...

   a. `math.sqrt(9)`
   b. `sqrt(9)`
   c. `math(sqrt(9))`
   d. `import.sqrt(9)`

4. File `pagi.py` berisi fungsi `salam()` yang mengembalikan `"Selamat pagi"`, dan file `malam.py` berisi fungsi `salam()` yang mengembalikan `"Selamat malam"`. Apa keluaran program berikut?

Soal 4

```
from pagi import *
from malam import *
print(salam())
```

a. `Selamat pagi`
b. `Selamat malam`
c. `Selamat pagi` lalu `Selamat malam`
d. `ImportError` karena nama bentrok

5. Fungsi file `__init__.py` di dalam sebuah folder adalah ...

a. menandai folder sebagai package dan dijalankan saat package pertama kali diimpor
b. menyimpan constructor semua class di dalam package
c. mencegah folder diimpor dari luar
d. menjalankan semua module di dalam folder secara otomatis

6. Di dalam file `perpustakaan/layanan.py`, baris `from .peminjaman import Peminjaman` berarti ...

a. mengimpor `Peminjaman` dari module `peminjaman` di package yang sama
b. mengimpor `Peminjaman` dari folder induk proyek
c. mengimpor semua class di package `perpustakaan`
d. mengimpor `Peminjaman` dari pustaka standar

7. Pesan `ImportError: attempted relative import with no known parent package` paling mungkin muncul ketika ...

a. file yang berisi relative import dijalankan langsung dengan `python folder/file.py`
b. package tidak memiliki variabel `__all__`
c. nama module mengandung huruf kapital
d. module yang sama diimpor dua kali

8. File `hitung.py` berisi dua baris: `print("memuat")` dan `jumlah = 0`. Apa keluaran program berikut?

Soal 8

```
import hitung
import hitung
hitung.jumlah += 1
import hitung
print(hitung.jumlah)
```

a. `memuat` tiga kali, lalu `1`

b. `memuat` sekali, lalu `1`
c. `memuat` sekali, lalu `0`
d. `memuat` tiga kali, lalu `0`

9. Seorang mahasiswa menyimpan tugasnya dengan nama `random.py`, lalu menulis `import random` di file lain pada folder yang sama. Apa yang terjadi?

   a. Python tetap mengimpor module `random` bawaan
   b. Python mengimpor file `random.py` buatannya, sehingga fungsi seperti `randint` tidak ditemukan
   c. Python menampilkan `SyntaxError`
   d. Python menggabungkan isi kedua module

10. Dua module saling mengimpor di baris pertamanya sehingga terjadi import melingkar. Perbaikan yang paling tepat adalah ...

    a. menuliskan perintah import yang sama dua kali
    b. membuat ketergantungan mengalir satu arah atau memindahkan bagian bersama ke module ketiga
    c. mengganti semua import menjadi `from … import *`
    d. menggabungkan seluruh program kembali ke satu file

### Uraian

1. Sebutkan tiga masalah yang muncul ketika seluruh program disimpan dalam satu file besar, dan jelaskan bagaimana modular programming mengatasi masing-masing masalah tersebut.
2. Jelaskan perbedaan module dan package. Berikan contoh struktur folder sebuah package yang berisi minimal dua module, beserta cara mengimpor masing-masing module.
3. Jelaskan kegunaan blok `if __name__ == "__main__"`. Berapa nilai `__name__` ketika file dijalankan langsung, dan berapa nilainya ketika file diimpor?
4. Bandingkan `import module` dengan `from module import nama`. Kapan masing-masing lebih tepat dipakai, dan mengapa `from module import *` sebaiknya dihindari?
5. Jelaskan apa yang dimaksud import melingkar, bagaimana urutan kejadiannya sampai muncul pesan *partially initialized module*, dan dua cara mengatasinya.

## 7.11 Praktik Mandiri

Kerjakan praktik berikut secara berurutan. Karena bab ini membahas cara memecah program, setiap praktik terdiri atas beberapa file. Simpan file-file setiap praktik dalam satu folder proyek tersendiri, jalankan dari folder tersebut, dan pastikan keluarannya sesuai dengan contoh.

### Praktik 1: Module Konversi

- Buat module `konversi.py` berisi fungsi `celsius_ke_fahrenheit(c)`, `km_ke_mil(km)`, dan `kg_ke_pon(kg)`. Bulatkan setiap hasil menjadi dua angka di belakang koma.
- Simpan faktor pengali sebagai konstanta `FAKTOR_MIL = 0.621371` dan `FAKTOR_PON = 2.20462`, lalu tulis docstring untuk module dan setiap fungsi.
- Di blok `if __name__ == "__main__"`, uji setiap fungsi dengan `assert`, lalu cetak `Semua uji lulus`.
- Buat `main.py` yang memakai dua bentuk import berbeda. Pastikan pesan uji tidak muncul ketika `main.py` dijalankan.

**Contoh keluaran Praktik 1**

```
$ python konversi.py
Semua uji lulus
$ python main.py
30 C = 86.0 F
42.195 km = 26.22 mil
65 kg = 143.3 pon
```

### Praktik 2: Package Toko

Ambil class `Produk` beserta decorator `@wajib_positif` dan `@catat` dari Praktik 3 Bab 3, lalu susun menjadi package.

- Buat package `toko` dengan tiga module: `produk.py` untuk class `Produk`, `dekorator.py` untuk kedua decorator, dan `uang.py` untuk fungsi `rupiah()`.
- Module `produk.py` memakai relative import untuk mengambil decorator dan fungsi `rupiah()`.
- Atur `__init__.py` beserta `__all__` agar `main.py` cukup menulis `from toko import Produk`.
- Di `main.py`, buat tiga produk dari daftar baris CSV, lakukan beberapa transaksi, lalu tampilkan produk dan isi log.

**Contoh keluaran Praktik 2**

```
P-01 Buku Tulis  Rp5.000  stok 38
P-02 Pulpen      Rp3.500  stok 150
P-03 Penggaris   Rp7.000  stok 15
Log: ['jual P-01 x2', 'tambah_stok P-02 x50']
```

### Praktik 3: Sistem Akademik Bermodul

Susun ulang hasil Praktik 3 Bab 6 (Sistem Akademik Mini) menjadi proyek bermodul.

### Langkah Pengerjaan

1. Kelompokkan class berdasarkan urusannya, lalu tulis rancangan pembagian module dalam bentuk tabel di file `README.md`. Contohnya `institusi.py` untuk `Universitas`, `Fakultas`, dan `ProgramStudi`; `orang.py` untuk `Dosen` dan `Mahasiswa`; serta `perkuliahan.py` untuk mata kuliah dan `KRS`.
2. Buat struktur folder proyek dengan `main.py` di folder teratas dan package `akademik/` berisi module-module tersebut.
3. Gunakan relative import di dalam package, lalu atur `__init__.py` dengan `__all__` sehingga `main.py` cukup mengimpor dari `akademik`.
4. Gambar diagram ketergantungan antarmodule (panah dari module yang mengimpor ke module yang diimpor), lalu pastikan tidak ada panah yang membentuk lingkaran.
5. Beri setiap module docstring dan blok `if __name__ == "__main__"` berisi uji singkat dengan `assert`. Jalankan setiap uji dengan `python -m akademik.nama_module`.

**KRITERIA PENILAIAN**

- Setiap module mengurus satu hal dan namanya menjelaskan isinya.
- Tidak ada import melingkar dan tidak ada `from … import *`.
- `main.py` hanya membuat object dan menjalankan program, tanpa definisi class.
- Keluaran program sama dengan versi satu file dari Bab 6.

Bab 08

# Exception Handling

*Program yang baik tidak berhenti hanya karena satu kesalahan.*

08

DI BAB INI

- Error dan exception
- `try–except`
- `else`, `finally`, `raise`
- Custom exception
- Exception dalam class

Di Bab 2, class `Rekening` menolak penarikan yang melebihi saldo dengan `raise ValueError`. Penolakan itu benar, tetapi akibatnya terlalu keras: baris `rek.tarik(999999)` menghentikan seluruh program. Bayangkan mesin ATM yang mati total hanya karena seorang nasabah salah mengetik nominal. Nasabah berikutnya ikut tidak bisa memakai mesin itu, padahal kesalahannya bukan milik mereka. Saat itu kita menunda pembahasan cara menangkap kesalahan semacam ini sampai Bab 8.

Bab ini membahas *exception handling*, yaitu cara program mengenali kesalahan saat berjalan, menanganinya, lalu melanjutkan pekerjaan. Kita akan mempelajari jenis-jenis exception bawaan Python, blok `try–except` beserta `else` dan `finally`, cara membuat exception sendiri, dan cara menempatkan penanganan kesalahan dengan tepat di dalam program berorientasi objek.

## 8.1 Error dan Exception

Kesalahan dalam program Python terbagi menjadi dua kelompok besar. Kelompok pertama adalah *syntax error*: kode melanggar aturan penulisan Python, misalnya lupa titik dua atau kurung tidak ditutup. Python memeriksa sintaks seluruh file sebelum menjalankan baris pertama, sehingga program dengan syntax error tidak berjalan sama sekali.

**Syntax error: tidak ada baris yang dijalankan**

```
print("mulai")
if nilai > 80
    print("lulus")
```

**Keluaran**

```
  File "nilai.py", line 2
    if nilai > 80
                ^
SyntaxError: expected ':'
```

Perhatikan bahwa kata `mulai` tidak tercetak, walaupun baris `print` itu sendiri benar. Syntax error harus diperbaiki dengan mengubah kode. Kesalahan jenis ini tidak bisa "ditangani" saat program berjalan.

Kelompok kedua adalah *exception*: kode sudah benar secara sintaks, tetapi saat

dijalankan menemui keadaan yang tidak bisa dikerjakan. Pembagian dengan nol, key yang tidak ada di dictionary, dan file yang tidak ditemukan termasuk kelompok ini. Baris-baris sebelum kesalahan tetap berjalan normal, lalu program berhenti tepat di baris yang bermasalah.

**Exception: program berhenti di tengah jalan**

```
print("mulai")
nilai = {"Ani": 88, "Budi": 75}
print(nilai["Ani"])
print(nilai["Citra"])
print("selesai")
```

**Keluaran**

```
mulai
88
Traceback (most recent call last):
  ...
KeyError: 'Citra'
```

Kata `selesai` tidak pernah tercetak. Exception inilah yang bisa ditangkap dan ditangani, dan inilah pokok bahasan bab ini.

## Membaca Traceback

Saat exception tidak ditangani, Python mencetak *traceback*: laporan tentang jalur pemanggilan fungsi yang dilalui program sampai kesalahan terjadi. Traceback sering terlihat menakutkan bagi pemula, padahal susunannya selalu sama. Perhatikan program berikut, yang memanggil fungsi di dalam fungsi.

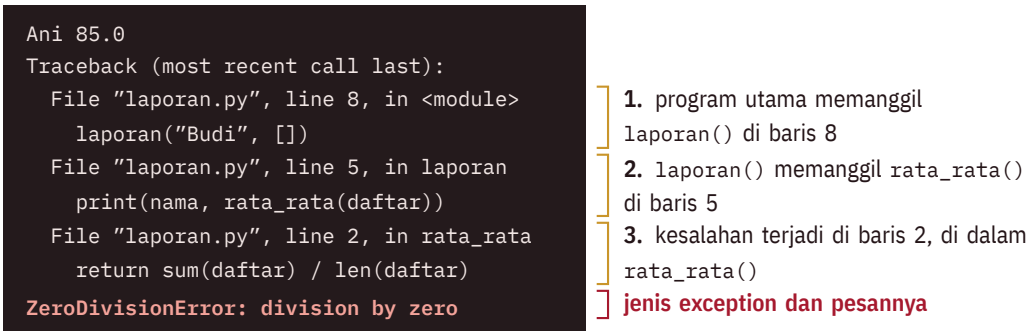
**laporan.py**

```
def rata_rata(daftar):
    return sum(daftar) / len(daftar)

def laporan(nama, daftar):
    print(nama, rata_rata(daftar))

laporan("Ani", [80, 90])
laporan("Budi", [])
```

Traceback disusun dari pemanggilan paling awal di atas sampai tempat kesalahan di bawah. Karena itu cara membacanya justru dari bawah. Baris terakhir menyebut *jenis* exception (`ZeroDivisionError`) dan pesannya. Dua baris di atasnya menunjukkan file, nomor baris, dan isi baris tempat kesalahan terjadi. Baris-baris di atasnya lagi menjelaskan bagaimana program sampai ke sana. Python 3.11 ke atas juga menambahkan baris berisi tanda ~ dan ^ di bawah setiap baris kode untuk menunjuk




**Gambar 8.1.** Bagian-bagian traceback. Mulailah membaca dari baris terakhir: jenis dan pesan kesalahan, lalu baris tempat kesalahan terjadi tepat di atasnya.

bagian yang bermasalah.

Di buku ini, traceback pada blok keluaran disingkat dengan `...` agar fokus pada baris terakhir. Saat menjalankan program sendiri, Anda akan melihat versi lengkapnya.

## 8.2 Jenis Exception

Python memiliki puluhan jenis exception bawaan. Nama setiap jenis menjelaskan kesalahannya, sehingga dengan mengenali beberapa yang paling sering muncul, sebagian besar traceback bisa dipahami dalam sekali baca.

**Tabel 8.1.** Exception bawaan yang paling sering ditemui.

| Exception | Muncul ketika | Contoh pemicu |
|---|---|---|
| `ValueError` | tipe nilainya benar, tetapi isinya tidak masuk akal | `int("dua")` |
| `TypeError` | operasi diberikan tipe data yang salah | `"5" + 3` |
| `ZeroDivisionError` | pembagian dengan nol | `10 / 0` |
| `IndexError` | indeks di luar jangkauan list | `[1, 2][5]` |
| `KeyError` | key tidak ada di dictionary | `{"a": 1}["b"]` |
| `AttributeError` | object tidak memiliki attribute atau method tersebut | `"teks".panjang` |
| `NameError` | nama variabel atau fungsi belum dibuat | `print(nilia)` |
| `FileNotFoundError` | file yang dibuka tidak ada | `open("tidak_ada.txt")` |
| `ModuleNotFoundError` | module yang diimpor tidak ditemukan (Bab 7) | `import kampus_utl` |

### Exception adalah Object

Sejak Bab 2 kita menulis `raise ValueError("pesan")`. Bentuk tulisan itu sama persis dengan membuat object: `ValueError` adalah sebuah class, dan `ValueError("pesan")` membuat object dari class tersebut. Pesan yang kita berikan disimpan sebagai attribute `args`.

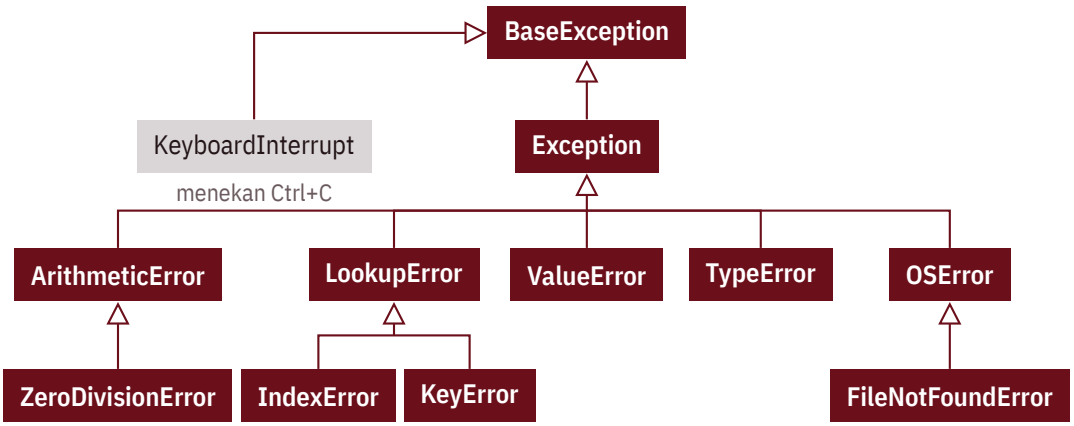
**Exception adalah object dari sebuah class**

```
e = ValueError("NIM tidak valid")
print(type(e), e.args)
print(isinstance(e, Exception))
print(ZeroDivisionError.__mro__)
```

**Keluaran**

```
<class 'ValueError'> ('NIM tidak valid',)
True
(<class 'ZeroDivisionError'>, <class 'ArithmeticError'>, <class 'Exception'>, <
    class 'BaseException'>, <class 'object'>)
```

Baris terakhir menampilkan MRO yang kita kenal di Bab 4. Class-class exception bawaan tersusun dalam hierarki pewarisan: `ZeroDivisionError` adalah turunan `ArithmeticError`, yang merupakan turunan `Exception`, dan seterusnya. Hierarki ini akan sangat berguna ketika kita menangkap beberapa jenis exception sekaligus.




**Gambar 8.2.** Sebagian hierarki class exception bawaan. Segitiga terbuka menunjuk parent, sama seperti notasi inheritance di Bab 6. `KeyboardInterrupt` sengaja tidak diturunkan dari `Exception` karena ia bukan kesalahan program.

## 8.3 `try-except`

Untuk menangkap exception, kode yang mungkin gagal ditempatkan di dalam blok `try`, dan cara menanganinya ditulis di blok `except`.

Bentuk dasar

```
try:
    ...                     # kode yang mungkin gagal
except JenisException:
    ...                     # dijalankan jika JenisException terjadi
```

Contoh paling umum adalah mengubah teks menjadi angka. Teks yang datang dari pengguna atau dari file tidak selalu berisi angka, dan `int()` akan melempar `ValueError` jika teksnya tidak bisa diubah.

Menangkap ValueError

```
def ke_angka(teks):
    try:
        return int(teks)
    except ValueError:
        print(f"'{teks}' bukan bilangan bulat, dianggap 0")
        return 0

for teks in ["85", "tujuh", "90", ""]:
    print("hasil:", ke_angka(teks))
```

Keluaran

```
hasil: 85
'tujuh' bukan bilangan bulat, dianggap 0
hasil: 0
hasil: 90
'' bukan bilangan bulat, dianggap 0
hasil: 0
```

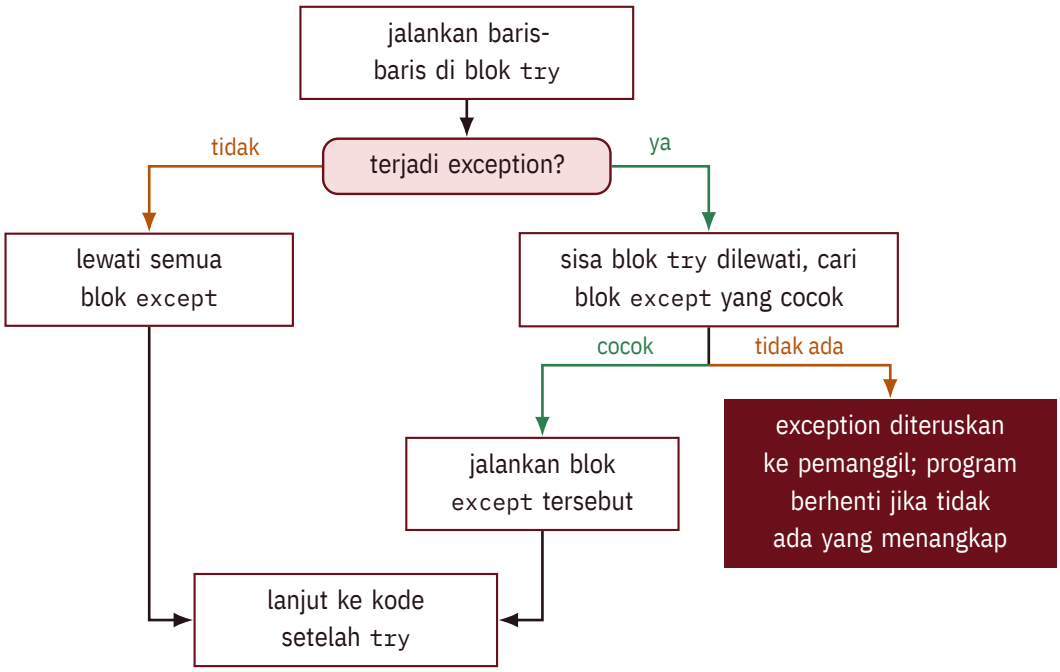
Program tidak berhenti. Setiap teks yang gagal diubah ditangani di blok `except`, lalu perulangan berlanjut ke teks berikutnya. Gambar berikut merangkum alur eksekusinya.

## Membaca Pesan Exception dengan as

Kata kunci `as` menyimpan object exception ke sebuah variabel. Dari variabel itu kita bisa membaca pesan dan jenis kesalahannya.

Menyimpan exception ke variabel

```
try:
    nilai = int("delapan puluh")
except ValueError as e:
    print("Gagal:", e)
    print("Jenis:", type(e).__name__)
```




**Gambar 8.3.** Alur eksekusi blok `try–except`.

**Keluaran**

```
Gagal: invalid literal for int() with base 10: 'delapan puluh'
Jenis: ValueError
```

## Meminta Masukan sampai Benar

Pola yang sangat sering dipakai adalah meminta masukan berulang kali sampai pengguna memberikan nilai yang valid. Agar keluarannya bisa ditampilkan di buku, masukan pengguna pada contoh berikut ditiru dengan sebuah list. Di program sungguhan, baris `next(masukan)` diganti dengan `input("Semester: ")`.

**Validasi masukan berulang**

```
masukan = iter(["", "abc", "-5", "3"])      # pengganti input() untuk contoh

def baca_semester():
    while True:
        teks = next(masukan)
        try:
            semester = int(teks)
        except ValueError:
            print(f"  '{teks}' bukan angka, ulangi.")
            continue
        if 1 <= semester <= 14:
            return semester
        print(f"  {semester} di luar rentang 1-14, ulangi.")

```

```
print("Semester:", baca_semester())
```

Keluaran

```
'' bukan angka, ulangi.
'abc' bukan angka, ulangi.
-5 di luar rentang 1-14, ulangi.
Semester: 3
```

Perhatikan bahwa pemeriksaan rentang ditulis dengan `if` biasa, bukan dengan exception. Teks `"-5"` berhasil diubah menjadi angka sehingga tidak ada exception yang terjadi. Nilainya saja yang tidak memenuhi aturan. Exception dipakai untuk keadaan yang membuat sebuah operasi *tidak bisa* dikerjakan. Aturan biasa cukup diperiksa dengan `if`.

### Jangan Menelan Semua Kesalahan

Blok `except` tanpa nama exception akan menangkap *semua* jenis kesalahan. Bentuk ini terlihat praktis, tetapi menyembunyikan bug.

Kesalahan yang tersembunyi

```
def hitung_rata(daftar):
    try:
        return sum(daftar) / len(daftr)      # salah ketik: daftr
    except:
        return 0

print(hitung_rata([80, 90, 100]))
```

Keluaran

```
0
```

Rata-rata nilai 80, 90, dan 100 jelas bukan 0. Salah ketik `daftr` memicu `NameError`, dan blok `except` menelannya tanpa jejak. Program terlihat berjalan dengan baik, padahal hasilnya salah. Seandainya blok itu ditulis `except ZeroDivisionError:`, salah ketik tersebut langsung muncul sebagai traceback dan bisa diperbaiki dalam hitungan detik.

**SEBUTKAN JENIS EXCEPTION**

Selalu tuliskan jenis exception yang memang Anda harapkan, dan lakukan sesuatu yang berarti di dalam blok `except`. Bentuk `except:` tanpa nama, apalagi `except: pass`, juga ikut menangkap `KeyboardInterrupt`, sehingga program bahkan tidak bisa dihentikan dengan Ctrl+C.

## 8.4 Multiple Exception

Satu blok kode bisa gagal dengan beberapa cara. Setiap cara kegagalan biasanya membutuhkan penanganan yang berbeda, sehingga kita menulis beberapa blok `except` berurutan.

**Beberapa blok except**

```
nilai = {"Ani": [80, 90], "Budi": [], "Citra": [70, "85"]}

def rata_rata(nama):
    try:
        daftar = nilai[nama]
        return sum(daftar) / len(daftar)
    except KeyError:
        return f"{nama} tidak terdaftar"
    except ZeroDivisionError:
        return f"{nama} belum punya nilai"
    except TypeError:
        return f"data nilai {nama} rusak"

for nama in ["Ani", "Budi", "Citra", "Dodi"]:
    print(rata_rata(nama))
```

**Keluaran**

```
85.0
Budi belum punya nilai
data nilai Citra rusak
Dodi tidak terdaftar
```

Data milik Citra berisi teks `"85"` di antara angka, sehingga `sum()` gagal dengan `TypeError`. Setiap jenis kegagalan mendapat pesan yang tepat, dan pemanggil tidak perlu tahu detail apa yang salah di dalam fungsi.

Jika beberapa jenis exception ditangani dengan cara yang sama, jenis-jenis itu bisa dikumpulkan dalam sebuah tuple pada satu blok `except`.

**Satu blok except untuk beberapa jenis**

```
jadwal = {"PBO": ["Senin", "Rabu"]}

def hari_ke(matkul, indeks):
    try:
        return jadwal[matkul][indeks]
    except (KeyError, IndexError) as e:
        return f"tidak ditemukan ({type(e).__name__})"

print(hari_ke("PBO", 1))
print(hari_ke("PBO", 5))
print(hari_ke("Kalkulus", 0))
```

Keluaran

```
Rabu
tidak ditemukan (IndexError)
tidak ditemukan (KeyError)
```

### Urutan Blok `except`

Python memeriksa blok `except` dari atas ke bawah dan menjalankan blok *pertama* yang cocok. Sebuah blok dianggap cocok jika exception yang terjadi adalah object dari class tersebut *atau turunannya*, sama seperti cara kerja `isinstance()` di Bab 4. Akibatnya, blok untuk class parent yang diletakkan lebih dulu akan "mencuri" semua exception turunannya.

Blok umum menutupi blok khusus

```
def bagi(a, b):
    try:
        return a / b
    except ArithmeticError:
        return "kesalahan aritmetika"
    except ZeroDivisionError:            # tidak akan pernah tercapai
        return "pembagi tidak boleh nol"

print(bagi(10, 0))
```

Keluaran

```
kesalahan aritmetika
```

`ZeroDivisionError` adalah turunan `ArithmeticError`, sehingga blok pertama sudah cocok dan blok kedua tidak pernah dijalankan. Menukar urutan kedua blok membuat pesan yang lebih tepat muncul.

**DARI KHUSUS KE UMUM**

Susun blok `except` dari jenis yang paling khusus ke yang paling umum, seperti menyusun syarat `if–elif` dari yang paling sempit. Jika ragu, lihat kembali gambar hierarki exception di subbab sebelumnya.

## 8.5 `else`, `finally`, dan `raise`

Blok `try` dapat dilengkapi dua blok lain: `else` dan `finally`. Bersama perintah `raise`, keduanya melengkapi alat yang kita butuhkan untuk menangani kesalahan.

### Blok `else`

Blok `else` dijalankan hanya jika blok `try` selesai *tanpa* exception. Kode di dalamnya sebenarnya bisa saja ditaruh di akhir blok `try`, tetapi memisahkannya punya keuntungan: blok `try` tetap pendek dan hanya berisi baris yang memang kita duga bisa gagal. Dengan begitu, blok `except` tidak ikut menangkap kesalahan dari baris lain yang tidak kita duga.

**Blok else**

```
def terima_nilai(teks):
    try:
        angka = int(teks)
    except ValueError:
        print(f"'{teks}' ditolak")
    else:                                   # hanya jika tidak ada exception
        huruf = "A" if angka >= 85 else "B"
        print(f"{angka} diterima, nilai huruf {huruf}")

terima_nilai("90")
terima_nilai("sembilan")
```

**Keluaran**

```
90 diterima, nilai huruf A
'sembilan' ditolak
```

### Blok `finally`

Blok `finally` *selalu* dijalankan, baik ada exception maupun tidak, bahkan ketika blok `try` keluar lebih awal dengan `return` atau ketika exception tidak tertangkap. Karena itu `finally` dipakai untuk pekerjaan beres-beres, misalnya menutup file yang sudah dibuka. Pekerjaan inilah yang akan kita temui lagi di Bab 9.

**Blok finally selalu dijalankan**

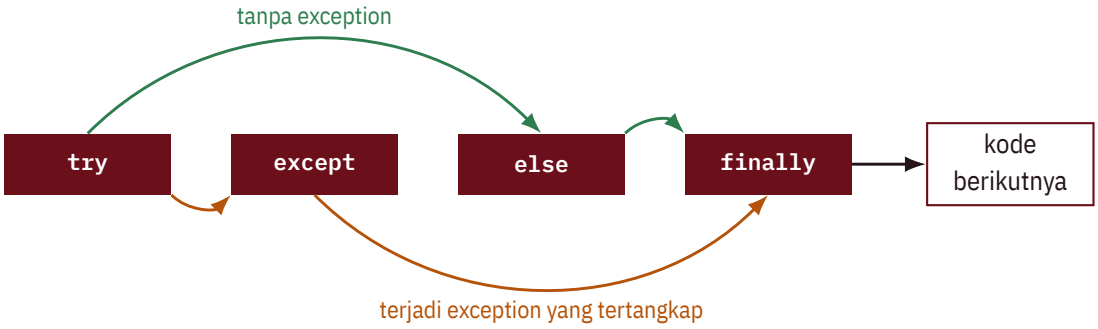
```
def proses(teks):
    print(f"Memproses '{teks}'")
    try:
        return 100 / int(teks)
    except ValueError:
        return "bukan angka"
    finally:
        print("  finally: beres-beres selesai")

print(proses("4"))
print(proses("x"))
print(proses("0"))
```

Keluaran

```
Memproses '4'
  finally: beres-beres selesai
25.0
Memproses 'x'
  finally: beres-beres selesai
bukan angka
Memproses '0'
  finally: beres-beres selesai
Traceback (most recent call last):
  ...
ZeroDivisionError: division by zero
```

Ada tiga hal menarik pada keluaran di atas. Pertama, baris `finally` tercetak *sebelum* nilai kembaliannya, karena `finally` dijalankan tepat sebelum fungsi benar-benar keluar. Kedua, pada teks `"x"` exception ditangkap, dan `finally` tetap berjalan. Ketiga, pada teks `"0"` terjadi `ZeroDivisionError` yang tidak ditangkap oleh blok mana pun. Blok `finally` tetap dijalankan lebih dulu, baru kemudian program berhenti.




**Gambar 8.4.** Dua jalur melalui `try`, `except`, `else`, dan `finally`. Jalur atas (hijau) dilalui jika tidak ada exception; jalur bawah (jingga) dilalui jika exception tertangkap. Kedua jalur selalu melewati `finally`.

## Perintah `raise`

Perintah `raise` sudah kita pakai sejak Bab 2 untuk melempar exception baru. Di dalam blok `except`, `raise` tanpa argumen punya arti khusus: lempar ulang exception yang sedang ditangani. Bentuk ini berguna ketika sebuah fungsi perlu mencatat kesalahan, tetapi keputusan untuk menanganinya tetap diserahkan kepada pemanggil.

Mencatat lalu melempar ulang

```
1  def muat_nilai(teks):
2      try:
3          return int(teks)
4      except ValueError:
5          print(f"[log] data rusak: {teks!r}")
6          raise                                # teruskan ke pemanggil
```

```
7
8 print(muat_nilai("85"))
9 print(muat_nilai("8S"))
```

Keluaran

```
85
[log] data rusak: '8S'
Traceback (most recent call last):
  ...
ValueError: invalid literal for int() with base 10: '8S'
```

**Tabel 8.2.** Ringkasan blok penanganan exception.

| Blok | Kapan dijalankan | Kegunaan |
|---|---|---|
| `try` | selalu, sampai selesai atau sampai terjadi exception | berisi baris yang mungkin gagal |
| `except` | jika terjadi exception yang jenisnya cocok | menangani kesalahan |
| `else` | jika blok `try` selesai tanpa exception | lanjutan pekerjaan yang hanya boleh terjadi jika berhasil |
| `finally` | selalu, dalam keadaan apa pun | beres-beres: menutup file, melepas sumber daya |
| `raise` | saat perintah itu dijalankan | melempar exception baru, atau melempar ulang di dalam `except` |

## 8.6 Custom Exception

Class `Rekening` di Bab 2 melempar `ValueError` untuk dua kesalahan yang sangat berbeda: jumlah setoran yang tidak positif dan saldo yang tidak mencukupi. Bagi pemanggil, keduanya tampak sama. Satu-satunya cara membedakannya adalah membandingkan teks pesan, misalnya `if str(e) == "Saldo tidak mencukupi."`. Cara ini rapuh: begitu pesannya diubah sedikit, pemeriksaan itu diam-diam berhenti bekerja.

Solusinya adalah membuat class exception sendiri, atau *custom exception*. Caranya sama dengan membuat class turunan biasa: wariskan dari `Exception` atau dari turunannya.

Hierarki exception untuk rekening

```
def rupiah(angka):
    return f"Rp{angka:,}".replace(",", ".")

class RekeningError(Exception):
    """Induk semua kesalahan yang berkaitan dengan rekening."""

class JumlahTidakValidError(RekeningError):
    def __init__(self, jumlah):
        super().__init__(f"Jumlah harus lebih dari nol, bukan {jumlah}.")
        self.jumlah = jumlah

class SaldoTidakCukupError(RekeningError):
    def __init__(self, saldo, diminta):
        super().__init__(f"Saldo {rupiah(saldo)} tidak cukup "
                         f"untuk menarik {rupiah(diminta)}.")
        self.saldo = saldo
        self.diminta = diminta

    @property
    def kekurangan(self):
        return self.diminta - self.saldo
```

Ada beberapa keputusan desain pada kode di atas:

- `RekeningError` tidak berisi apa pun selain docstring. Class ini ada untuk menjadi induk bersama, sehingga pemanggil bisa menangkap *semua* kesalahan rekening dengan satu blok `except RekeningError`.
- Setiap turunan memanggil `super().__init__(pesan)` agar pesan tetap tampil saat exception dicetak atau muncul di traceback.
- `SaldoTidakCukupError` membawa data: saldo, jumlah yang diminta, dan property `kekurangan`. Pemanggil bisa memakai data ini tanpa mengurai teks pesan.

Memakai custom exception

```
try:
    raise SaldoTidakCukupError(100000, 250000)
except SaldoTidakCukupError as e:
    print(e)
    print("Kurang:", rupiah(e.kekurangan))
    print(isinstance(e, RekeningError), isinstance(e, Exception))
```

Keluaran

```
Saldo Rp100.000 tidak cukup untuk menarik Rp250.000.
Kurang: Rp150.000
```

```
True True
```

**AKHIRAN ERROR**

Pedoman gaya PEP 8 menyarankan nama exception diakhiri kata `Error` jika exception itu menyatakan kesalahan, seperti `ValueError` dan `KeyError`. Akhiran ini membuat pembaca langsung tahu bahwa sebuah class adalah exception, bukan class biasa.

### Menerjemahkan Exception dengan `raise … from`

Kadang sebuah fungsi menangkap exception tingkat rendah, misalnya `ValueError` dari `int()`, lalu melempar exception yang lebih bermakna bagi pemanggilnya. Bentuk `raise ExceptionBaru(…) from e` melakukannya sambil tetap menyimpan exception asli sebagai penyebab. Jika exception baru itu tidak tertangkap, traceback akan menampilkan keduanya dengan keterangan *The above exception was the direct cause of the following exception*, sehingga sumber masalah tetap mudah dilacak.

**Menerjemahkan ValueError menjadi exception sendiri**

```
class NilaiTidakValidError(Exception):
    pass

def parse_nilai(teks):
    try:
        angka = int(teks)
    except ValueError as e:
        pesan = f"Nilai '{teks}' bukan bilangan bulat."
        raise NilaiTidakValidError(pesan) from e
    if not 0 <= angka <= 100:
        raise NilaiTidakValidError(f"Nilai {angka} di luar rentang 0-100.")
    return angka

for teks in ["88", "110", "A+"]:
    try:
        print("diterima:", parse_nilai(teks))
    except NilaiTidakValidError as e:
        print("ditolak :", e)
```

**Keluaran**

```
diterima: 88
ditolak : Nilai 110 di luar rentang 0-100.
ditolak : Nilai 'A+' bukan bilangan bulat.
```

Pemanggil `parse_nilai()` cukup mengenal satu jenis exception, apa pun penyebab kegagalannya. Bagaimana nilai itu diurai di dalam fungsi tetap menjadi urusan fungsi

itu sendiri, persis seperti semangat encapsulation di Bab 2.

## 8.7 Exception Handling dalam Class

Sekarang kita perbaiki class `Rekening` versi property dari Bab 2 agar memakai custom exception, lalu membangun sebuah ATM sederhana yang tetap berjalan walaupun ada transaksi yang gagal.

**Rekening yang melempar custom exception**

```
class Rekening:
    # validasi pemilik dan saldo awal seperti di Bab 2 tidak ditulis ulang
    def __init__(self, nomor, pemilik, saldo_awal=0):
        self.__nomor = nomor
        self.__pemilik = pemilik
        self.__saldo = saldo_awal

    @property
    def nomor(self):
        return self.__nomor

    @property
    def saldo(self):
        return self.__saldo

    def setor(self, jumlah):
        if jumlah <= 0:
            raise JumlahTidakValidError(jumlah)
        self.__saldo += jumlah

    def tarik(self, jumlah):
        if jumlah <= 0:
            raise JumlahTidakValidError(jumlah)
        if jumlah > self.__saldo:
            raise SaldoTidakCukupError(self.__saldo, jumlah)
        self.__saldo -= jumlah
```

Class `Rekening` tidak mencetak apa pun dan tidak menangkap exception apa pun. Tugasnya hanya menjaga aturan, lalu melempar exception yang tepat ketika aturan dilanggar. Menampilkan pesan kepada pengguna adalah tugas bagian lain, yaitu ATM.

**ATM yang tetap berjalan**

```
class ATM:
    def __init__(self, rekening):
        self._rek = rekening

```

```
    def _jalankan(self, jenis, jumlah):
        if jenis == "setor":
            self._rek.setor(jumlah)
        else:
            self._rek.tarik(jumlah)

    def proses(self, daftar_transaksi):
        for jenis, jumlah in daftar_transaksi:
            baris = f"{jenis:<5} {rupiah(jumlah):>11}"
            try:
                self._jalankan(jenis, jumlah)
            except SaldoTidakCukupError as e:
                print(f"GAGAL {baris}  kurang {rupiah(e.kekurangan)}")
            except RekeningError as e:
                print(f"GAGAL {baris}  {e}")
            else:
                print(f"OK    {baris}  saldo {rupiah(self._rek.saldo)}")

atm = ATM(Rekening("KM-001", "Ani Lestari", 100000))
atm.proses([
    ("setor", 50000),
    ("tarik", 30000),
    ("tarik", 500000),
    ("setor", -10000),
    ("tarik", 120000),
])
```

**Keluaran**

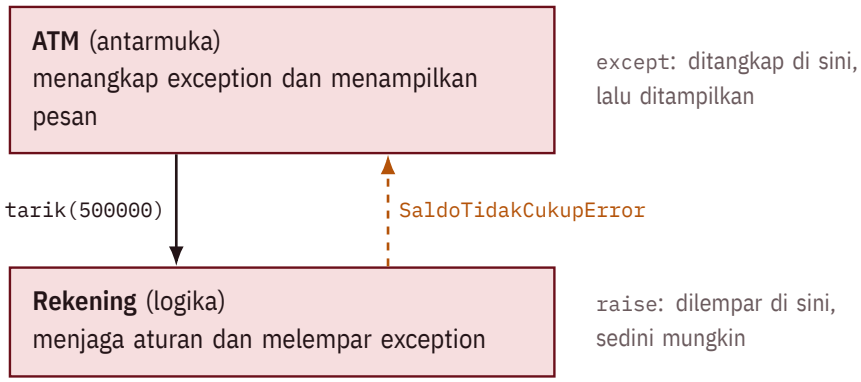
```
OK    setor    Rp50.000  saldo Rp150.000
OK    tarik    Rp30.000  saldo Rp120.000
GAGAL tarik   Rp500.000  kurang Rp380.000
GAGAL setor   Rp-10.000  Jumlah harus lebih dari nol, bukan -10000.
OK    tarik   Rp120.000  saldo Rp0
```

Kelima transaksi diproses sampai selesai. Dua transaksi yang gagal mendapat pesan yang sesuai, dan transaksi terakhir tetap dijalankan. Bandingkan dengan Bab 2, ketika satu penarikan yang gagal menghentikan seluruh program.

Kode ATM juga memanfaatkan hierarki exception. Exception `SaldoTidakCukupError` ditangkap secara khusus di blok pertama karena ATM ingin memakai attribute `kekurangan`. Blok kedua menangkap sisa kesalahan rekening lewat class induknya. Jika kelak ditambahkan exception baru, misalnya `RekeningDiblokirError` yang mewarisi `RekeningError`, ATM sudah bisa menanganinya tanpa diubah sedikit pun.

## Di Mana Exception Ditangkap?

Pembagian tugas pada gambar di atas dapat dirumuskan menjadi dua pedoman:




**Gambar 8.5.** Pembagian tugas: class logika melempar exception, lapisan antarmuka menangkap dan menampilkannya.

- **Lempar sedini mungkin.** Exception dilempar di tempat yang paling tahu aturannya, yaitu method di dalam class, tepat saat data yang salah masuk. Semakin lama data salah dibiarkan beredar, semakin jauh jarak antara tempat kesalahan muncul dan sumber aslinya.
- **Tangkap di tempat yang bisa bertindak.** Exception ditangkap di tempat yang bisa melakukan sesuatu yang berarti, seperti menampilkan pesan, meminta masukan ulang, atau memakai nilai cadangan. Biasanya tempat itu adalah lapisan antarmuka: menu terminal, ATM, atau jendela GUI di Bab 10.

Pedoman ini juga menjelaskan mengapa class `Rekening` tidak boleh mencetak pesan sendiri. Jika `tarik()` berisi `print("Saldo kurang")`, class itu hanya cocok untuk program terminal. Dengan melempar exception, class yang sama bisa dipakai oleh program terminal, oleh GUI, maupun oleh program yang mencatat transaksi ke file. Ini adalah prinsip reusability dari Bab 7 dalam bentuk lain.

## EAFP dalam Class

Di Bab 5 kita mengenal dua gaya: LBYL (*periksa dulu, baru pakai*) dan EAFP (*coba dulu, tangani jika gagal*). Kode ATM di atas memakai gaya EAFP. Untuk melihat alasannya, bandingkan kedua gaya dari sisi pemanggil.

**LBYL dan EAFP pada rekening**

```
rek = Rekening("KM-002", "Budi Santoso", 60000)

# LBYL: aturan saldo ikut ditulis di sisi pemanggil
if rek.saldo >= 50000:
    rek.tarik(50000)

# EAFP: aturan hanya ada di dalam Rekening.tarik()
try:
    rek.tarik(50000)
except SaldoTidakCukupError as e:
```

```
11      print("Ditolak:", e)
```

Keluaran

```
Ditolak: Saldo Rp10.000 tidak cukup untuk menarik Rp50.000.
```

Misalkan bank menambahkan aturan baru: saldo minimal Rp50.000 harus selalu tersisa di rekening. Dengan gaya EAFP, cukup method `tarik()` yang diubah, dan semua pemanggil langsung mengikuti aturan baru. Dengan gaya LBYL, setiap pemanggil yang menulis `if rek.saldo >= jumlah` kini memeriksa aturan yang sudah usang, dan semuanya harus dicari lalu diubah satu per satu.

**SATU ATURAN, SATU TEMPAT**

Jika sebuah aturan sudah dijaga oleh method di dalam class, jangan menyalinnya ke kode pemanggil dalam bentuk `if`. Panggil method-nya, lalu tangani exception yang mungkin dilempar.

## 8.8 Rangkuman

- **Syntax error** terdeteksi sebelum program berjalan dan harus diperbaiki dengan mengubah kode. **Exception** terjadi saat program berjalan dan dapat ditangkap.
- **Traceback** dibaca dari bawah: jenis dan pesan exception, lalu baris tempat kesalahan terjadi, lalu jalur pemanggilan yang menuju ke sana.
- Exception adalah **object** dari class yang tersusun dalam hierarki pewarisan, dengan `Exception` sebagai induk dari hampir semua kesalahan program.
- Blok **`try–except`** menangkap exception agar program tetap berjalan. Sebutkan jenis exception secara spesifik; `except:` tanpa nama menyembunyikan bug.
- Beberapa blok `except` diperiksa dari atas ke bawah, dan yang pertama cocok yang dijalankan. Susun dari jenis paling khusus ke paling umum.
- **`else`** dijalankan jika `try` berhasil tanpa exception. **`finally`** selalu dijalankan dan dipakai untuk beres-beres. **`raise`** tanpa argumen melempar ulang exception yang sedang ditangani, dan `raise … from e` menerjemahkan exception sambil menyimpan penyebab aslinya.
- **Custom exception** dibuat dengan mewarisi `Exception`. Induk bersama seperti `RekeningError` memudahkan penangkapan sekelompok kesalahan, dan attribute tambahan membawa data tentang kesalahan itu.
- Di dalam program OOP, class logika **melempar** exception sedini mungkin, sedangkan lapisan antarmuka **menangkap** dan menampilkannya.

## 8.9 Kuis Teoritis

### Pilihan Ganda

Pilihlah satu jawaban yang paling tepat.

1. Perbedaan utama syntax error dan exception adalah ...

   a. syntax error hanya terjadi di dalam class, exception terjadi di luar class
   b. syntax error terdeteksi sebelum program berjalan, exception terjadi saat program berjalan
   c. syntax error dapat ditangkap dengan `try`, exception tidak dapat ditangkap
   d. keduanya sama, hanya berbeda nama

2. Perhatikan kode berikut.

   Soal 2

   ```
   try:
       print("A")
       x = 1 / 0
       print("B")
   except ZeroDivisionError:
       print("C")
   else:
       print("D")
   finally:
       print("E")
   ```

   Keluaran kode tersebut adalah ...

   a. `A B C E`
   b. `A C E`
   c. `A C D E`
   d. `A E`

3. Pemanggilan `int("tiga")` akan melempar exception berjenis ...

   a. `TypeError`
   b. `NameError`
   c. `ValueError`
   d. `KeyError`

4. Perhatikan kode berikut.

   Soal 4

   ```
   try:
       [1, 2, 3][10]
   ```

```
except LookupError:
    print("lookup")
except IndexError:
    print("index")
```

Keluaran kode tersebut adalah ...

a. `index`
b. `lookup`
c. `lookup` lalu `index`
d. traceback `IndexError`

**5.** Bentuk `except: pass` dianggap berbahaya karena ...

a. membuat program berjalan lebih lambat
b. menangkap dan menyembunyikan semua kesalahan, termasuk bug seperti salah ketik
c. hanya bisa dipakai di dalam fungsi
d. menyebabkan `SyntaxError`

**6.** Sebuah custom exception dibuat dengan cara ...

a. mendefinisikan fungsi bernama `exception()`
b. membuat class yang mewarisi `Exception` atau turunannya
c. memberi decorator `@exception` pada sebuah class
d. menulis `raise` di dalam constructor

**7.** Perhatikan kode berikut.

Soal 7

```
class AppError(Exception):
    pass

class DataError(AppError):
    pass

try:
    raise DataError("rusak")
except AppError as e:
    print(type(e).__name__, e)
```

Keluaran kode tersebut adalah ...

a. `AppError rusak`
b. `DataError rusak`
c. traceback `DataError`, karena yang ditangkap `AppError`

d. `Exception rusak`

8. Pernyataan yang benar tentang blok `finally` adalah ...

   a. hanya dijalankan jika terjadi exception
   b. hanya dijalankan jika tidak terjadi exception
   c. selalu dijalankan, termasuk ketika `try` keluar dengan `return`
   d. tidak dijalankan jika exception tidak tertangkap

9. Perintah `raise` tanpa argumen yang ditulis di dalam blok `except` berfungsi untuk ...

   a. menghentikan program tanpa pesan
   b. melempar ulang exception yang sedang ditangani
   c. membuat exception baru berjenis `Exception`
   d. mengabaikan exception yang sedang ditangani

10. Pada program ATM, pembagian tugas yang paling tepat antara class `Rekening` dan class `ATM` adalah ...

    a. `Rekening` mencetak pesan kesalahan, `ATM` tidak perlu tahu ada kesalahan
    b. `Rekening` melempar exception, `ATM` menangkap dan menampilkan pesannya
    c. keduanya sama-sama menangkap semua exception dengan `except:`
    d. `ATM` memeriksa saldo sendiri sehingga `Rekening` tidak perlu melempar exception

### Uraian

1. Jelaskan perbedaan syntax error dan exception, masing-masing dengan satu contoh. Bagaimana cara membaca sebuah traceback?
2. Jelaskan urutan eksekusi blok `try`, `except`, `else`, dan `finally` pada dua keadaan: ketika terjadi exception yang tertangkap dan ketika tidak terjadi exception.
3. Mengapa urutan blok `except` penting? Kaitkan jawaban Anda dengan hierarki pewarisan class exception.
4. Kapan sebaiknya kita membuat custom exception? Jelaskan keuntungannya dibandingkan selalu memakai `ValueError` dengan pesan yang berbeda-beda.
5. Jelaskan pedoman "lempar sedini mungkin, tangkap di tempat yang bisa bertindak" dengan contoh class `Rekening` dan `ATM`. Kaitkan dengan gaya EAFP.

## 8.10 Praktik Mandiri

Kerjakan praktik berikut secara berurutan. Simpan setiap praktik dalam file `.py` terpisah, jalankan, dan pastikan keluarannya sesuai dengan contoh.

### Praktik 1: Validator Nilai

- Buat custom exception `NilaiTidakValidError`.
- Buat fungsi `baca_nilai(teks)` yang mengembalikan bilangan bulat 0–100. Jika teks bukan bilangan bulat, terjemahkan `ValueError` menjadi `NilaiTidakValidError` dengan `raise … from`. Jika di luar rentang, lempar `NilaiTidakValidError` langsung.
- Proses daftar masukan `["90", " 75 ", "seratus", "-3", "100", "", "88.5"]`. Tampilkan setiap hasil, lalu tampilkan jumlah masukan yang diterima dan rata-ratanya dengan dua angka di belakang koma.
- Fungsi `baca_nilai()` tidak boleh mencetak apa pun.

**Contoh keluaran Praktik 1**

```
diterima: 90
diterima: 75
ditolak : 'seratus' bukan bilangan bulat
ditolak : -3 di luar rentang 0-100
diterima: 100
ditolak : '' bukan bilangan bulat
ditolak : '88.5' bukan bilangan bulat
Diterima 3 dari 7 masukan, rata-rata 88.33
```

### Praktik 2: Gudang Alat Tulis

- Buat `GudangError` sebagai induk dengan tiga turunan: `BarangTidakAdaError` yang menyimpan `barang`; `StokKurangError` yang menyimpan `barang`, `sisa`, dan `diminta`; serta `JumlahTidakValidError` yang menyimpan `jumlah`.
- Buat class `Gudang` yang menyimpan stok dalam dictionary protected, dengan method `tambah(barang, jumlah)` dan `ambil(barang, jumlah)` yang melempar exception di atas sesuai keadaan.
- Buat fungsi `proses_pesanan(gudang, daftar)` yang memproses daftar pesanan berbentuk `(aksi, barang, jumlah)`, tetap berjalan ketika ada pesanan yang gagal, dan memakai blok `else` untuk mencetak sisa stok.
- Gunakan attribute `sisa` pada `StokKurangError` untuk menyusun pesan, bukan teks pesan exception.

**Contoh keluaran Praktik 2**

```
OK    ambil  kertas  x20  sisa 80
GAGAL ambil  tinta   x5   stok tinta hanya 3
GAGAL ambil  stapler x1   barang 'stapler' tidak ada di gudang
GAGAL tambah kertas  x0   jumlah harus lebih dari nol
OK    tambah tinta   x10  sisa 13
```

### Praktik 3: Perpustakaan Tahan Kesalahan

Kembangkan package `perpustakaan` dari Bab 7 agar memakai exception.

#### Langkah Pengerjaan

1. Tambahkan module `perpustakaan/kesalahan.py` berisi `PerpustakaanError` sebagai induk, dengan turunan `BukuSedangDipinjamError`, `BukuTidakTerdaftarError`, dan `AnggotaTidakTerdaftarError`. Setiap exception menyimpan object atau nama yang terkait sebagai attribute.
2. Ubah method `pinjamkan()` agar melempar exception, bukan mengembalikan teks `"sedang dipinjam"`. Periksa juga bahwa anggota dan buku sudah didaftarkan ke perpustakaan.
3. Tambahkan method `kembalikan(anggota, buku)` yang melempar exception jika buku tersebut tidak sedang dipinjam oleh anggota itu.
4. Di `main.py`, proses daftar permintaan pinjam dan kembali. Tangkap exception di `main.py`, tampilkan pesan yang jelas, dan lanjutkan ke permintaan berikutnya.
5. Akhiri program dengan ringkasan jumlah permintaan yang berhasil dan yang gagal, yang dicetak di blok `finally` sehingga tetap tampil walaupun terjadi kesalahan tak terduga.

KRITERIA PENILAIAN

- Class di dalam package tidak mencetak apa pun dan tidak menangkap exception yang seharusnya ditangani pemanggil.
- Tidak ada `except:` tanpa nama exception, dan urutan blok `except` dari khusus ke umum.
- Custom exception membawa data yang relevan dan dipakai oleh pemanggil.
- Program tetap berjalan sampai permintaan terakhir walaupun ada permintaan yang gagal.

Bab 09

# File Handling

*Menyimpan data objek agar tidak hilang ketika program ditutup.*

09

DI BAB INI

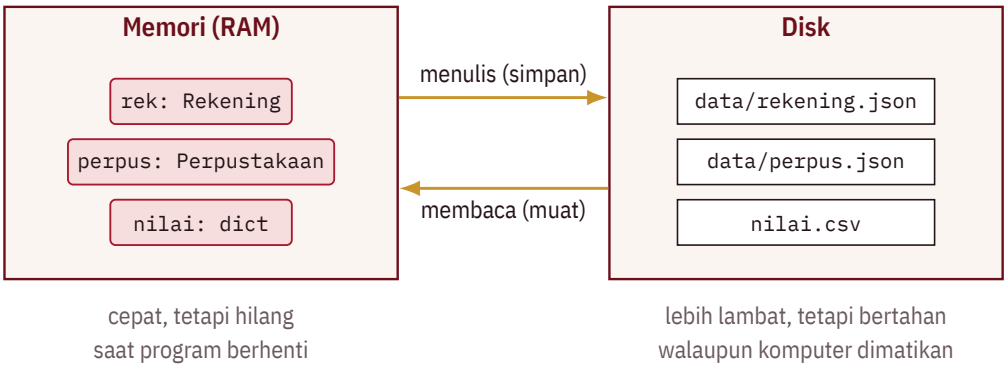
- Read dan write file
- Context manager
- File CSV dan JSON
- Serialisasi data
- Integrasi dengan object

Semua program yang kita tulis sejauh ini punya satu kelemahan yang sama: datanya hilang begitu program selesai. ATM di Bab 8 memproses lima transaksi, tetapi ketika program dijalankan lagi, saldo kembali ke Rp100.000. Perpustakaan di Bab 7 lupa siapa meminjam buku apa. Semua object itu hidup di memori komputer, dan memori dikosongkan ketika program berhenti.

Agar data bertahan, data harus ditulis ke tempat penyimpanan permanen, yaitu file di disk. Bab ini membahas cara membaca dan menulis file dengan `open()`, *context manager* `with` yang menjamin file selalu ditutup, serta dua format file data yang paling umum, yaitu CSV dan JSON. Setelah itu kita belajar mengubah object menjadi data yang bisa disimpan lalu mengembalikannya menjadi object. Di akhir bab, perpustakaan kita akhirnya bisa mengingat data peminjaman dari satu kali program dijalankan ke kali berikutnya.

## 9.1 Konsep File Handling

Komputer menyimpan data di dua tempat yang sifatnya sangat berbeda. *Memori* (RAM) sangat cepat, dan di sinilah semua variabel dan object kita berada selama program berjalan. Namun isi memori hilang ketika program berhenti. *Disk* (SSD atau harddisk) jauh lebih lambat, tetapi isinya bertahan walaupun komputer dimatikan. *File handling* adalah kegiatan memindahkan data di antara keduanya: *menulis* data dari memori ke file dan *membaca* data dari file ke memori.




**Gambar 9.1.** File handling memindahkan data di antara memori yang cepat tetapi sementara dan disk yang lambat tetapi permanen.

### File Teks dan File Biner

Isi semua file sebenarnya hanyalah deretan byte. Yang membedakan adalah cara byte itu ditafsirkan. *File teks* berisi karakter yang bisa dibaca manusia, misalnya file `.txt`, `.csv`, `.json`, dan bahkan file `.py` kita sendiri. *File biner* berisi byte yang hanya bermakna bagi program tertentu, misalnya gambar `.png`, dokumen `.pdf`, atau lembar kerja `.xlsx`. Bab ini hanya membahas file teks, karena itulah bentuk yang paling sering dipakai untuk menyimpan data aplikasi sederhana.

### Path File

Lokasi sebuah file ditulis sebagai *path*. Path *absolut* ditulis lengkap dari akar disk, misalnya `C:\Users\ani\proyek\nilai.csv` di Windows atau `/home/ani/proyek/nilai.csv` di Linux. Path *relatif* ditulis dari folder tempat program dijalankan, misalnya `nilai.csv` atau `data/perpus.json`. Program sebaiknya memakai path relatif agar tetap berjalan ketika folder proyek dipindahkan ke komputer lain. Inilah alasan tips di Bab 7 menyarankan agar program selalu dijalankan dari folder proyek.

**MODULE** `PATHLIB`

Pustaka standar menyediakan class `Path` di module `pathlib` untuk mengolah path sebagai object. Operator `/` dipakai untuk menyambung bagian path, dan Python otomatis memakai pemisah yang benar untuk setiap sistem operasi. Contohnya, `Path("data") / "perpus.json"` menghasilkan `data/perpus.json` di Linux dan `data\perpus.json` di Windows. Object `Path` juga punya method yang praktis, seperti `exists()` untuk memeriksa keberadaan file dan `mkdir()` untuk membuat folder.

## 9.2 Read dan Write File dengan `open()`

Fungsi bawaan `open()` membuka file dan mengembalikan sebuah *object file*. Melalui object inilah kita membaca atau menulis isi file. Argumen kedua `open()` menentukan *mode*, yaitu untuk apa file dibuka.

### Menulis File

Menulis file teks

```
f = open("catatan.txt", "w", encoding="utf-8")
f.write("Kuliah PBO hari Senin\n")
f.write("Tugas Bab 9 dikumpulkan Jumat\n")
f.close()
```

**Tabel 9.1.** Mode pembukaan file.

| Mode | Arti | Jika file sudah ada / belum ada |
|---|---|---|
| `"r"` | membaca (bawaan) | dibaca dari awal / `FileNotFoundError` |
| `"w"` | menulis | **isi lama dihapus** / file baru dibuat |
| `"a"` | menambah di akhir | ditambah setelah isi lama / file baru dibuat |
| `"x"` | membuat file baru | `FileExistsError` / file baru dibuat |
| `"b"` | mode biner, digabung dengan mode lain (`"rb"`, `"wb"`) | isi dibaca sebagai byte, bukan teks |

**isi catatan.txt**

```
Kuliah PBO hari Senin
Tugas Bab 9 dikumpulkan Jumat
```

Berbeda dengan `print()`, method `write()` tidak menambahkan baris baru secara otomatis. Karena itu setiap teks diakhiri `\n`. Method `close()` di akhir menutup file. Pentingnya langkah ini akan kita bahas sebentar lagi.

**SELALU TULISKAN `ENCODING="UTF-8"`**

Tanpa argumen `encoding`, Python memakai encoding bawaan sistem operasi, dan setiap sistem bisa berbeda. File yang ditulis di Linux lalu dibaca di Windows dapat menampilkan huruf yang rusak, terutama huruf beraksen seperti pada nama *André*. Menuliskan `encoding="utf-8"` di setiap `open()` untuk file teks membuat program berperilaku sama di semua komputer.

## Membaca File

Ada beberapa cara membaca isi file. Method `read()` mengambil seluruh isi file sebagai satu string.

**Membaca seluruh isi file**

```
1 f = open("catatan.txt", encoding="utf-8")
2 isi = f.read()
3 f.close()
4 print(repr(isi))
```

**Keluaran**

```
'Kuliah PBO hari Senin\nTugas Bab 9 dikumpulkan Jumat\n'
```

Fungsi `repr()` memperlihatkan karakter `\n` di akhir setiap baris. Karakter ini ikut

terbaca, sehingga saat membaca per baris kita biasanya membuangnya dengan `strip()`. Cara membaca per baris yang paling umum adalah memakai object file langsung di dalam `for`.

**Membaca per baris**

```
f = open("catatan.txt", encoding="utf-8")
for nomor, baris in enumerate(f, start=1):
    print(nomor, baris.strip())
f.close()
```

**Keluaran**

```
1 Kuliah PBO hari Senin
2 Tugas Bab 9 dikumpulkan Jumat
```

**Tabel 9.2.** Cara membaca isi file teks.

| Cara | Hasil | Cocok untuk |
|---|---|---|
| `f.read()` | seluruh isi sebagai satu string | file kecil |
| `f.readline()` | satu baris berikutnya | membaca baris judul, lalu sisanya |
| `f.readlines()` | list berisi semua baris | file kecil yang perlu diolah per baris |
| `for baris in f:` | satu baris per putaran | file sebesar apa pun, karena hanya satu baris yang ada di memori |

## Menambah Isi File

Mode `"a"` (*append*) menambahkan teks di akhir file tanpa menghapus isi yang sudah ada.

**Menambah baris di akhir file**

```
f = open("catatan.txt", "a", encoding="utf-8")
f.write("Praktikum dipindah ke ruang C01\n")
f.close()
```

**isi catatan.txt**

```
Kuliah PBO hari Senin
Tugas Bab 9 dikumpulkan Jumat
Praktikum dipindah ke ruang C01
```

**MODE "W" MENGOSONGKAN FILE SEKETIKA**

File yang dibuka dengan mode "w" langsung dikosongkan saat itu juga, bahkan sebelum `write()` pertama dijalankan. Salah memilih "w" padahal maksudnya "a" berarti kehilangan seluruh isi file lama. Periksa kembali mode setiap kali menulis ke file yang berisi data penting.

## File yang Tidak Ada

Membuka file yang tidak ada dengan mode baca melempar `FileNotFoundError`. Exception ini ditangani dengan cara yang sudah kita pelajari di Bab 8.

**Menangani file yang tidak ada**

```
try:
    f = open("jadwal.txt", encoding="utf-8")
except FileNotFoundError as e:
    print("File tidak ditemukan:", e.filename)
```

**Keluaran**

```
File tidak ditemukan: jadwal.txt
```

## Mengapa File Harus Ditutup?

Selama file terbuka, sistem operasi menyediakan sumber daya untuknya. Lebih dari itu, data yang ditulis dengan `write()` sering ditampung dulu di memori dan baru benar-benar dikirim ke disk ketika file ditutup. File yang lupa ditutup bisa membuat sebagian data tidak pernah tersimpan, dan di Windows file itu juga tidak bisa dibuka oleh program lain.

Menulis `close()` di akhir saja belum cukup. Jika terjadi exception di antara `open()` dan `close()`, baris `close()` tidak pernah dijalankan.

**Exception membuat close() terlewat**

```
nilai = [90, 85, "A", 70]
f = open("rata.txt", "w", encoding="utf-8")
try:
    f.write(f"Rata-rata: {sum(nilai) / len(nilai)}\n")
    f.close()
except TypeError as e:
    print("Gagal:", e)
print("File sudah ditutup?", f.closed)
```

**Keluaran**

```
Gagal: unsupported operand type(s) for +: 'int' and 'str'
File sudah ditutup? False
```

Solusinya sudah kita kenal dari Bab 8: letakkan `close()` di blok `finally`, karena blok itu selalu dijalankan.

**Menutup file di blok finally**

```
f = open("rata.txt", "w", encoding="utf-8")
try:
    f.write(f"Rata-rata: {sum(nilai) / len(nilai)}\n")
except TypeError as e:
    print("Gagal:", e)
finally:
    f.close()
print("File sudah ditutup?", f.closed)
```

**Keluaran**

```
Gagal: unsupported operand type(s) for +: 'int' and 'str'
File sudah ditutup? True
```

Cara ini benar, tetapi panjang. Untungnya Python menyediakan cara yang jauh lebih ringkas untuk hasil yang sama.

## 9.3 Context Manager

Pernyataan `with` membuka file dan menjamin file itu ditutup ketika blok `with` selesai, baik selesai dengan normal maupun karena exception. Object yang bisa dipakai bersama `with` disebut *context manager*.

**Membaca file dengan with**

```
with open("catatan.txt", encoding="utf-8") as f:
    for baris in f:
        print(baris.strip())

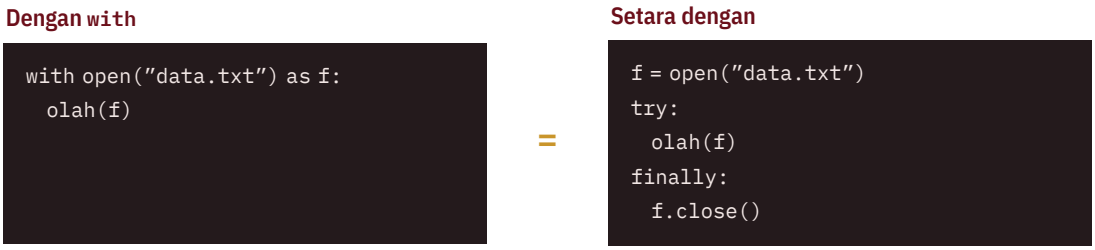
print("Di luar blok with, file ditutup?", f.closed)
```

**Keluaran**

```
Kuliah PBO hari Senin
Tugas Bab 9 dikumpulkan Jumat
Praktikum dipindah ke ruang C01
Di luar blok with, file ditutup? True
```

Kita tidak menulis `close()` sama sekali, tetapi file sudah tertutup begitu program keluar dari blok yang menjorok ke dalam. Gambar berikut memperlihatkan bahwa `with` hanyalah cara singkat untuk menulis pola `try–finally`.

Satu pernyataan `with` dapat membuka beberapa file sekaligus. Contoh berikut mem-




**Gambar 9.2.** Pernyataan `with` setara dengan pola `try–finally`, tetapi jauh lebih ringkas dan tidak mungkin lupa menutup file.

baca `catatan.txt` dan menulis salinannya yang diberi nomor baris.

**Dua file dalam satu with**

```
with open("catatan.txt", encoding="utf-8") as sumber, \
     open("catatan_bernomor.txt", "w", encoding="utf-8") as tujuan:
    for nomor, baris in enumerate(sumber, start=1):
        tujuan.write(f"{nomor}. {baris}")
```

**isi catatan_bernomor.txt**

```
1. Kuliah PBO hari Senin
2. Tugas Bab 9 dikumpulkan Jumat
3. Praktikum dipindah ke ruang C01
```

**GUNAKAN WITH UNTUK SETIAP FILE**

Mulai sekarang, buka file hanya dengan `with open(…) as f:`. Cara `open()` lalu `close()` di awal bab hanya dipakai untuk menjelaskan apa yang terjadi di balik layar.

## Membuat Context Manager Sendiri

Pernyataan `with` tidak hanya untuk file. Object apa pun bisa menjadi context manager jika class-nya memiliki dua dunder method: `__enter__` dan `__exit__`. Ini adalah protokol lain yang serupa dengan `__len__` di Bab 5 dan `__add__` di Bab 6: Python memanggil method itu secara otomatis pada saat yang tepat.

**Context manager buatan sendiri**

```
class Sesi:
    def __init__(self, nama):
        self.nama = nama

    def __enter__(self):                        # saat masuk blok with
        print(f"[{self.nama}] mulai")
        return self

```

```
    def __exit__(self, jenis, nilai, tb):       # saat keluar blok with
        if jenis is None:
            print(f"[{self.nama}] selesai tanpa masalah")
        else:
            print(f"[{self.nama}] berhenti karena {jenis.__name__}: {nilai}")
        return True                             # exception sudah ditangani

with Sesi("impor nilai"):
    print("  memproses data...")

with Sesi("hitung rata-rata"):
    print("  memproses data...")
    hasil = 10 / 0
    print("  baris ini tidak dijalankan")

print("program tetap berjalan")
```

**Keluaran**

```
[impor nilai] mulai
  memproses data...
[impor nilai] selesai tanpa masalah
[hitung rata-rata] mulai
  memproses data...
[hitung rata-rata] berhenti karena ZeroDivisionError: division by zero
program tetap berjalan
```

Method `__enter__` dijalankan ketika program masuk ke blok `with`, dan nilai kembalian-nya diberikan ke variabel setelah kata `as`. Method `__exit__` dijalankan ketika program keluar dari blok, dalam keadaan apa pun. Tiga parameternya berisi jenis exception, object exception, dan traceback-nya. Jika blok selesai tanpa exception, ketiganya bernilai `None`.

**NILAI KEMBALIAN `__EXIT__`**

Jika `__exit__` mengembalikan `True`, exception dianggap sudah ditangani dan tidak diteruskan, sama seperti `except` yang menelan kesalahan. Lakukan ini hanya jika memang disengaja. Object file mengembalikan `False`, sehingga exception tetap diteruskan setelah file ditutup.

## 9.4 File CSV

*CSV* (*Comma-Separated Values*) adalah format untuk data berbentuk tabel. Setiap baris file adalah satu baris tabel, dan nilai-nilai di dalamnya dipisahkan koma. Baris pertama biasanya berisi nama kolom. File CSV bisa dibuka langsung oleh Excel dan Google Sheets, sehingga format ini sangat umum untuk data nilai, daftar hadir, dan laporan transaksi.

Python menyediakan module `csv` untuk membaca dan menulis format ini.

**Menulis file CSV**

```
import csv

data = [
    ["nim", "nama", "uts", "uas"],
    ["24051204001", "Ani", 85, 90],
    ["24051204002", "Budi", 70, 78],
    ["24051204003", "Citra", 92, 88],
]
with open("nilai.csv", "w", newline="", encoding="utf-8") as f:
    csv.writer(f).writerows(data)
```

**isi nilai.csv**

```
nim,nama,uts,uas
24051204001,Ani,85,90
24051204002,Budi,70,78
24051204003,Citra,92,88
```

**ARGUMEN `NEWLINE=""`**

Module `csv` mengatur sendiri karakter akhir baris. Jika file dibuka tanpa `newline=""`, Python ikut menambahkan karakter akhir baris miliknya, dan di Windows hasilnya adalah baris kosong di antara setiap baris data. Selalu sertakan `newline=""` saat membuka file CSV, baik untuk membaca maupun menulis.

**Membaca file CSV**

```
with open("nilai.csv", newline="", encoding="utf-8") as f:
    pembaca = csv.reader(f)
    judul = next(pembaca)                    # baris pertama: nama kolom
    print(judul)
    for baris in pembaca:
        print(baris)
```

**Keluaran**

```
['nim', 'nama', 'uts', 'uas']
['24051204001', 'Ani', '85', '90']
['24051204002', 'Budi', '70', '78']
['24051204003', 'Citra', '92', '88']
```

Perhatikan tanda kutip pada `'85'` dan `'90'`. File CSV tidak menyimpan informasi tipe data, sehingga *semua* nilai yang dibaca berupa string. Angka harus diubah sendiri dengan `int()` atau `float()` sebelum dihitung.

### `DictReader` dan `DictWriter`

Mengakses kolom dengan indeks seperti `baris[2]` mudah keliru, apalagi jika urutan kolom berubah. Class `csv.DictReader` membaca setiap baris sebagai dictionary dengan nama kolom sebagai key, dan `csv.DictWriter` menulis dictionary menjadi baris CSV. Contoh berikut membaca `nilai.csv`, menghitung nilai akhir (40% UTS dan 60% UAS), lalu menulis rekapnya ke file CSV baru.

**Membaca dan menulis CSV dengan dictionary**

```
with open("nilai.csv", newline="", encoding="utf-8") as masuk, \
     open("rekap.csv", "w", newline="", encoding="utf-8") as keluar:
    penulis = csv.DictWriter(keluar, fieldnames=["nama", "akhir", "lulus"])
    penulis.writeheader()
    for baris in csv.DictReader(masuk):
        akhir = round(0.4 * int(baris["uts"]) + 0.6 * int(baris["uas"]), 1)
        penulis.writerow({"nama": baris["nama"], "akhir": akhir,
                          "lulus": akhir >= 75})
```

**isi rekap.csv**

```
nama,akhir,lulus
Ani,88.0,True
Budi,74.8,False
Citra,89.6,True
```

### Mengapa Tidak Memakai `split(",")` Saja?

Di Praktik 3 Bab 3, baris CSV seperti `"P-01,Buku Tulis,5000,40"` dipecah dengan `split(",")`. Cara itu bekerja selama tidak ada nilai yang mengandung koma. Format CSV mengizinkan nilai yang mengandung koma asalkan diapit tanda kutip, dan pada nilai seperti itu `split()` menghasilkan potongan yang salah.

**split() dibandingkan module csv**

```
baris = 'P-04,"Pensil 2B, isi 12",24000,10'
print(baris.split(","))
print(next(csv.reader([baris])))
```

**Keluaran**

```
['P-04', '"Pensil 2B', ' isi 12"', '24000', '10']
['P-04', 'Pensil 2B, isi 12', '24000', '10']
```

Module `csv` memahami aturan tanda kutip, sehingga nama produk tetap utuh. Saat menulis, module ini juga otomatis menambahkan tanda kutip pada nilai yang membutuhkannya.

## 9.5 File JSON

CSV cocok untuk tabel, tetapi tidak semua data berbentuk tabel. Data seorang mahasiswa, misalnya, berisi nama dan IPK, tetapi juga *daftar* mata kuliah yang diambil. *JSON* (*JavaScript Object Notation*) adalah format teks untuk data bersarang seperti ini. Bentuknya sangat mirip dengan dictionary dan list di Python, dan hampir semua bahasa pemrograman serta layanan web memakainya untuk bertukar data.

**Menulis file JSON**

```python
import json

mhs = {
    "nim": "24051204084",
    "nama": "Ani",
    "aktif": True,
    "ipk": 3.72,
    "matkul": ["PBO", "Statistika", "Basis Data"],
    "pembimbing": None,
}
with open("mhs.json", "w", encoding="utf-8") as f:
    json.dump(mhs, f, indent=2)
```

**isi mhs.json**

```
{
  "nim": "24051204084",
  "nama": "Ani",
  "aktif": true,
  "ipk": 3.72,
  "matkul": [
    "PBO",
    "Statistika",
    "Basis Data"
  ],
  "pembimbing": null
}
```

Argumen `indent=2` membuat file tersusun rapi dengan indentasi dua spasi. Tanpa argumen itu, seluruh data ditulis dalam satu baris panjang. Perhatikan juga bahwa `True` ditulis sebagai `true` dan `None` sebagai `null`, mengikuti aturan penulisan JSON.

**Membaca file JSON**

```python
with open("mhs.json", encoding="utf-8") as f:
    data = json.load(f)

print(type(data), data["matkul"][0], data["aktif"])
```

Keluaran

```
<class 'dict'> PBO True
```

Berbeda dengan CSV, angka dan nilai boolean di JSON dibaca kembali dengan tipe yang benar, sehingga `data["ipk"]` langsung bisa dihitung tanpa `float()`.

Module `json` punya dua pasang fungsi. Fungsi `dump()` dan `load()` bekerja dengan file, sedangkan `dumps()` dan `loads()` bekerja dengan string. Huruf `s` di akhir nama berarti *string*. Pasangan kedua berguna ketika data JSON tidak berasal dari file, misalnya dari layanan web.

dumps dan loads

```
1  teks = json.dumps({"kota": "Surabaya", "suhu": 31})
2  print(teks, type(teks))
3  print(json.loads(teks)["suhu"] + 1)
```

Keluaran

```
{"kota": "Surabaya", "suhu": 31} <class 'str'>
32
```

## Tipe Data Python dan JSON

Tidak semua tipe data Python punya padanan di JSON. Tabel berikut memperlihatkan apa yang terjadi pada setiap tipe ketika disimpan lalu dibaca kembali.

**Tabel 9.3.** Padanan tipe data Python dan JSON.

| Python | JSON | Dibaca kembali sebagai |
|---|---|---|
| `dict` | object `{…}` | `dict`, tetapi key *selalu* string |
| `list` | array `[…]` | `list` |
| `tuple` | array `[…]` | `list`, bukan tuple |
| `str` | string | `str` |
| `int`, `float` | number | `int`, `float` |
| `True`, `False` | `true`, `false` | `bool` |
| `None` | `null` | `None` |
| `set`, object buatan sendiri | tidak ada padanan | `TypeError` saat disimpan |

Data yang berubah atau ditolak

```
asli = {"jadwal": ("Senin", "Rabu"), 1: "satu"}
print(json.loads(json.dumps(asli)))
json.dumps({"pbo", "python"})
```

Keluaran

```
{'jadwal': ['Senin', 'Rabu'], '1': 'satu'}
Traceback (most recent call last):
  ...
TypeError: Object of type set is not JSON serializable
```

Tuple kembali sebagai list, key angka `1` kembali sebagai string `'1'`, dan set ditolak sama sekali. Ketiga hal ini sering menjadi sumber bug yang sulit dilacak, karena data terlihat tersimpan dengan baik dan masalahnya baru muncul ketika data dibaca kembali.

**HURUF NON-ASCII**

Secara bawaan, `json.dump()` menulis huruf di luar ASCII sebagai kode, misalnya huruf é menjadi `\u00e9`. File tetap benar, tetapi sulit dibaca manusia. Tambahkan argumen `ensure_ascii=False` agar huruf tersebut ditulis apa adanya.

**Tabel 9.4.** Perbandingan CSV dan JSON.

| Aspek | CSV | JSON |
|---|---|---|
| Bentuk data | tabel: baris dan kolom | bersarang: dictionary di dalam list, dan sebaliknya |
| Tipe data | semua dibaca sebagai string | string, angka, boolean, `null`, list, dictionary |
| Dibuka dengan | Excel, Google Sheets, editor teks | editor teks, hampir semua bahasa pemrograman |
| Cocok untuk | data nilai, daftar hadir, laporan transaksi | data aplikasi, konfigurasi, data object |

## 9.6 Serialisasi Data Sederhana

Sejauh ini kita menyimpan dictionary dan list. Bagaimana dengan object buatan sendiri?

**Object tidak bisa langsung disimpan**

```
class Buku:
    def __init__(self, judul, penulis, tahun):
        self.judul = judul
        self.penulis = penulis
        self.tahun = tahun

json.dumps(Buku("Laskar Pelangi", "Andrea Hirata", 2005))
```

**Keluaran**

```
Traceback (most recent call last):
  ...
TypeError: Object of type Buku is not JSON serializable
```

Module `json` tidak tahu attribute mana yang perlu disimpan dan bagaimana membuat object itu kembali. Proses mengubah object menjadi bentuk yang bisa disimpan atau dikirim disebut *serialisasi*. Kebalikannya, mengubah data simpanan kembali menjadi object, disebut *deserialisasi*. Untuk object buatan sendiri, kedua proses itu harus kita sediakan sendiri.

Pola yang paling sederhana adalah memberi class dua method: `ke_dict()` untuk mengubah object menjadi dictionary, dan class method `dari_dict()` untuk membuat object dari dictionary. Method kedua adalah *alternative constructor* yang kita pelajari di Bab 3.

**Serialisasi dengan ke_dict dan dari_dict**

```
class Buku:
    def __init__(self, judul, penulis, tahun):
        self.judul = judul
        self.penulis = penulis
        self.tahun = tahun

    def ke_dict(self):
        return {"judul": self.judul, "penulis": self.penulis,
                "tahun": self.tahun}

    @classmethod
    def dari_dict(cls, data):
        return cls(data["judul"], data["penulis"], data["tahun"])

    def __str__(self):
        return f"{self.judul} ({self.tahun}), {self.penulis}"

koleksi = [Buku("Laskar Pelangi", "Andrea Hirata", 2005),
           Buku("Bumi Manusia", "Pramoedya Ananta Toer", 1980)]

with open("koleksi.json", "w", encoding="utf-8") as f:
```

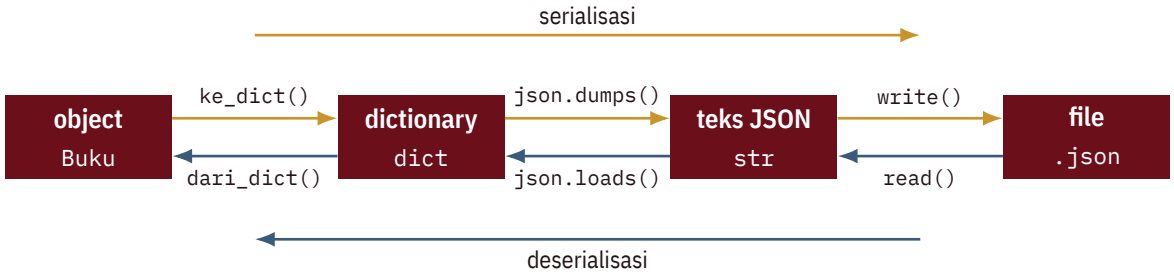
```
    json.dump([b.ke_dict() for b in koleksi], f, indent=2)

with open("koleksi.json", encoding="utf-8") as f:
    dimuat = [Buku.dari_dict(d) for d in json.load(f)]

for buku in dimuat:
    print(buku)
```

**Keluaran**

```
Laskar Pelangi (2005), Andrea Hirata
Bumi Manusia (1980), Pramoedya Ananta Toer
```




**Gambar 9.3.** Tahapan serialisasi (kiri ke kanan) dan deserialisasi (kanan ke kiri). Fungsi `json.dump()` dan `json.load()` menggabungkan dua tahap terakhir sekaligus.

Mengapa `dari_dict()` dibuat sebagai class method? Karena method ini *membuat* object, sementara object-nya belum ada saat method dipanggil. Pemakaian `cls` sebagai ganti nama class juga membuat class turunan, misalnya `BukuCetak`, otomatis menghasilkan object dari class turunan itu, seperti yang dibahas di Bab 3.

## Serialisasi Data Class

Untuk data class dari Bab 5, kita tidak perlu menulis `ke_dict()` sendiri. Fungsi `asdict()` dari module `dataclasses` mengubah object data class menjadi dictionary. Kebalikannya cukup ditulis dengan `**`, yang membuka dictionary menjadi argumen kata kunci seperti `**kwargs` di Bab 3.

**asdict dan pembukaan dictionary**

```
from dataclasses import dataclass, asdict

@dataclass
class Mahasiswa:
    nim: str
    nama: str
    ipk: float

teks = json.dumps(asdict(Mahasiswa("24051204084", "Ani", 3.72)))
```

```
print(teks)
print(Mahasiswa(**json.loads(teks)))
```

Keluaran

```
{"nim": "24051204084", "nama": "Ani", "ipk": 3.72}
Mahasiswa(nim='24051204084', nama='Ani', ipk=3.72)
```

**BAGAIMANA DENGAN PICKLE?**

Python juga punya module `pickle` yang bisa menyimpan hampir semua object tanpa perlu `ke_dict()`. Namun file pickle tidak bisa dibaca manusia dan hanya bisa dibuka oleh Python. Yang lebih berbahaya, membuka file pickle dapat menjalankan kode apa pun yang disisipkan di dalamnya, sehingga file pickle dari sumber yang tidak dipercaya tidak boleh dibuka. Untuk data aplikasi sederhana, JSON adalah pilihan yang lebih aman dan lebih mudah diperiksa.

## 9.7 Integrasi File Handling dengan Object

Kini kita gabungkan semuanya agar perpustakaan bisa mengingat data peminjaman. Agar fokus pada penyimpanan, model perpustakaan di subbab ini disederhanakan: peminjam dicatat dengan namanya saja.

Satu keputusan desain penting diambil sejak awal: class `Perpustakaan` *tidak* membuka file sendiri. Urusan file diserahkan kepada object terpisah yang disebut *penyimpanan*, dengan dua method saja: `muat()` dan `simpan()`. Dalam struktur proyek Bab 7, class-class ini menempati module `penyimpanan.py`.

penyimpanan.py

```
import json
from pathlib import Path
from typing import Protocol

class DataRusakError(Exception):
    pass

class Penyimpanan(Protocol):
    def muat(self) -> list[dict]: ...
    def simpan(self, data: list[dict]) -> None: ...

class PenyimpananJSON:
    def __init__(self, path):
        self.path = Path(path)

    def muat(self):
```

```
        try:
            with open(self.path, encoding="utf-8") as f:
                return json.load(f)
        except FileNotFoundError:
            return []                               # belum pernah disimpan
        except json.JSONDecodeError as e:
            raise DataRusakError(f"{self.path} rusak: {e}") from e

    def simpan(self, data):
        self.path.parent.mkdir(parents=True, exist_ok=True)
        with open(self.path, "w", encoding="utf-8") as f:
            json.dump(data, f, indent=2)

class PenyimpananMemori:
    """Penyimpanan untuk uji coba: data hanya disimpan di memori."""
    def __init__(self, data=None):
        self.data = data or []

    def muat(self):
        return list(self.data)

    def simpan(self, data):
        self.data = list(data)
```

perpus.py

```
from penyimpanan import Penyimpanan

class BukuSedangDipinjamError(Exception):
    pass

class Buku:
    def __init__(self, kode, judul, peminjam=None):
        self.kode = kode
        self.judul = judul
        self.peminjam = peminjam

    def ke_dict(self):
        return {"kode": self.kode, "judul": self.judul,
                "peminjam": self.peminjam}

    @classmethod
    def dari_dict(cls, data):
        return cls(data["kode"], data["judul"], data["peminjam"])

class Perpustakaan:
    def __init__(self, penyimpanan: Penyimpanan):
```

```
        self._penyimpanan = penyimpanan
        self._buku = {}
        for data in penyimpanan.muat():
            buku = Buku.dari_dict(data)
            self._buku[buku.kode] = buku

    def jumlah_buku(self):
        return len(self._buku)

    def tambah_buku(self, kode, judul):
        if kode not in self._buku:                  # abaikan jika sudah ada
            self._buku[kode] = Buku(kode, judul)

    def pinjamkan(self, kode, nama):
        buku = self._buku[kode]
        if buku.peminjam is not None:
            raise BukuSedangDipinjamError(
                f"'{buku.judul}' sedang dipinjam {buku.peminjam}")
        buku.peminjam = nama

    def simpan(self):
        self._penyimpanan.simpan([b.ke_dict() for b in self._buku.values()])

    def laporan(self):
        for b in self._buku.values():
            status = f"dipinjam {b.peminjam}" if b.peminjam else "tersedia"
            print(f"  {b.kode}  {b.judul:<24} {status}")
```

Program utama berikut dijalankan dua kali berturut-turut. Isinya sama persis, tetapi keluarannya berbeda, karena pada kali kedua program memuat data yang disimpan oleh kali pertama.

main.py

```
from penyimpanan import PenyimpananJSON
from perpus import BukuSedangDipinjamError, Perpustakaan

perpus = Perpustakaan(PenyimpananJSON("data/perpus.json"))
print("Dimuat dari file:", perpus.jumlah_buku(), "buku")

perpus.tambah_buku("B01", "Laskar Pelangi")
perpus.tambah_buku("B02", "Bumi Manusia")
perpus.tambah_buku("B03", "Dasar PBO dengan Python")
try:
    perpus.pinjamkan("B01", "Ani")
    print("Ani meminjam B01")
except BukuSedangDipinjamError as e:
```

```
        print("Ditolak:", e)

    perpus.simpan()
    perpus.laporan()
```

**python main.py (pertama kali)**

```
Dimuat dari file: 0 buku
Ani meminjam B01
  B01  Laskar Pelangi           dipinjam Ani
  B02  Bumi Manusia             tersedia
  B03  Dasar PBO dengan Python  tersedia
```

**python main.py (kedua kali)**

```
Dimuat dari file: 3 buku
Ditolak: 'Laskar Pelangi' sedang dipinjam Ani
  B01  Laskar Pelangi           dipinjam Ani
  B02  Bumi Manusia             tersedia
  B03  Dasar PBO dengan Python  tersedia
```

**isi data/perpus.json**

```
[
  {
    "kode": "B01",
    "judul": "Laskar Pelangi",
    "peminjam": "Ani"
  },
  {
    "kode": "B02",
    "judul": "Bumi Manusia",
    "peminjam": null
  },
  {
    "kode": "B03",
    "judul": "Dasar PBO dengan Python",
    "peminjam": null
  }
]
```

Pada kali pertama, file `data/perpus.json` belum ada. Method `muat()` menangkap `FileNotFoundError` dan mengembalikan list kosong, sehingga perpustakaan dimulai tanpa buku. Folder `data/` pun dibuat otomatis oleh `mkdir()` saat pertama kali menyimpan. Pada kali kedua, ketiga buku beserta status peminjamannya dimuat dari file, sehingga permintaan pinjam yang sama ditolak. Perpustakaan kita akhirnya punya ingatan.

## File yang Rusak

Method `muat()` memperlakukan dua keadaan dengan cara yang berbeda. File yang *belum ada* dianggap wajar dan menghasilkan list kosong. File yang *rusak*, misalnya terpotong karena listrik padam saat penyimpanan, justru dilaporkan dengan `DataRusakError`.

**Memuat file JSON yang terpotong**

```
from penyimpanan import PenyimpananJSON

with open("rusak.json", "w", encoding="utf-8") as f:
    f.write('[{"kode": "B01", "judul": "Laskar')      # isi file terpotong

PenyimpananJSON("rusak.json").muat()
```

**Keluaran**

```
Traceback (most recent call last):
  ...
penyimpanan.DataRusakError: rusak.json rusak: Unterminated string starting at:
    line 1 column 27 (char 26)
```

Mengapa file rusak tidak ikut dianggap kosong saja? Karena program akan menyimpan kembali datanya. Jika file rusak diperlakukan sebagai list kosong, pemanggilan `simpan()` berikutnya akan menimpa file itu dengan daftar kosong, dan data yang sebenarnya masih bisa diselamatkan hilang untuk selamanya. Kesalahan yang berisiko menghilangkan data harus dilaporkan dengan jelas, bukan disembunyikan.

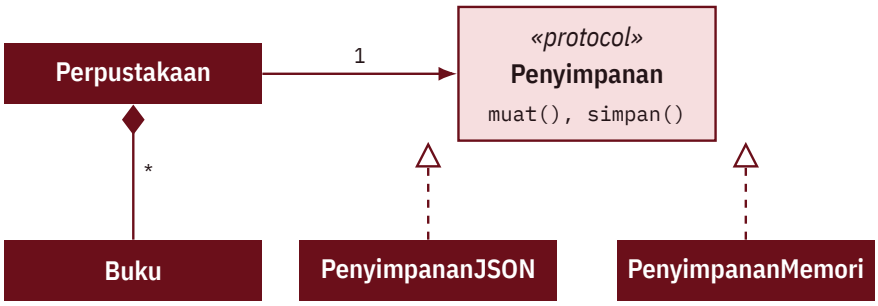
**MENYIMPAN DENGAN AMAN**

Untuk aplikasi sungguhan, tulislah data ke file sementara lebih dulu, misalnya `perpus.json.tmp`, lalu ganti file lama dengan `os.replace()`. Penggantian ini terjadi dalam satu langkah, sehingga file data tidak pernah tertinggal dalam keadaan setengah tertulis walaupun program berhenti di tengah penyimpanan.

## Penyimpanan yang Dapat Ditukar

Class `Perpustakaan` hanya tahu bahwa penyimpanannya punya method `muat()` dan `simpan()`, sesuai `Protocol` `Penyimpanan` (Bab 5). Ia tidak tahu apakah data berasal dari file JSON, file CSV, basis data, atau sekadar list di memori. Inilah polymorphism dari Bab 6 yang diterapkan pada urusan penyimpanan.

Manfaatnya langsung terasa saat menguji program. Dengan `PenyimpananMemori`, perilaku perpustakaan bisa diperiksa tanpa membuat atau mengubah file apa pun.




**Gambar 9.4.** `Perpustakaan` tersusun dari banyak `Buku` (composition) dan mengenal satu penyimpanan lewat protocol `Penyimpanan`. Panah putus-putus bersegitiga menyatakan bahwa sebuah class memenuhi protocol tanpa pewarisan.

**Menguji perpustakaan tanpa file**

```
from penyimpanan import PenyimpananMemori
from perpus import Perpustakaan

data_awal = [{"kode": "X01", "judul": "Buku Uji", "peminjam": None}]
memori = PenyimpananMemori(data_awal)
uji = Perpustakaan(memori)
uji.pinjamkan("X01", "Budi")
uji.simpan()
print(memori.data)
```

**Keluaran**

```
[{'kode': 'X01', 'judul': 'Buku Uji', 'peminjam': 'Budi'}]
```

Menambah format penyimpanan baru juga tidak mengubah `Perpustakaan` sedikit pun. Cukup tulis class baru yang punya `muat()` dan `simpan()`, lalu berikan object-nya saat membuat perpustakaan. Praktik 3 di akhir bab meminta Anda membuat versi CSV.

## 9.8 Rangkuman

- Data di memori hilang saat program berhenti. **File handling** menulis data ke disk agar bertahan, lalu membacanya kembali saat dibutuhkan.
- `open(path, mode, encoding="utf-8")` membuka file. Mode `"r"` membaca, `"w"` menulis dan *mengosongkan* file lama, `"a"` menambah di akhir, dan `"x"` hanya membuat file baru.
- Isi file dibaca dengan `read()`, `readline()`, `readlines()`, atau perulangan `for baris in f`. Setiap baris yang dibaca masih membawa karakter `\n`.
- File harus selalu ditutup. Pernyataan **`with`** menutup file secara otomatis dalam keadaan apa pun dan setara dengan pola `try–finally`.
- Class menjadi **context manager** dengan mengimplementasikan `__enter__` dan `__exit__`.

- **CSV** cocok untuk data tabel. Gunakan module `csv` dengan `newline=""`, dan ingat bahwa semua nilai dibaca sebagai string. `DictReader` dan `DictWriter` memakai nama kolom sebagai key.
- **JSON** cocok untuk data bersarang dan mempertahankan tipe angka, boolean, dan `None`. Tuple kembali sebagai list, key dictionary selalu menjadi string, dan set ditolak.
- **Serialisasi** mengubah object menjadi data yang bisa disimpan. Pola `ke_dict()` dan class method `dari_dict()` bekerja untuk class biasa, sedangkan data class bisa memakai `asdict()`.
- Urusan file sebaiknya dipisahkan ke class penyimpanan tersendiri. File yang belum ada ditangani dengan wajar, file yang rusak dilaporkan, dan penyimpanan yang berbeda dapat ditukar berkat polymorphism.

## 9.9 Kuis Teoritis

### Pilihan Ganda

Pilihlah satu jawaban yang paling tepat.

1. File `data.txt` sudah berisi 100 baris. Apa yang terjadi jika file itu dibuka dengan `open("data.txt", "w")`?

   a. isi lama tetap ada dan teks baru ditambahkan di akhir
   b. isi lama langsung dikosongkan
   c. terjadi `FileExistsError`
   d. file hanya bisa dibaca

2. Perhatikan kode berikut.

   Soal 2

   ```
   with open("catatan.txt", encoding="utf-8") as f:
       baris_pertama = f.readline()
   print(f.closed, repr(baris_pertama[-1]))
   ```

   Keluaran kode tersebut adalah ...

   a. `False 'n'`
   b. `True '\n'`
   c. `False '\n'`
   d. `True 'i'`

3. Argumen `newline=""` dipakai saat membuka file CSV agar ...

   a. file dibuka dalam mode biner
   b. module `csv` mengatur akhir baris sendiri sehingga tidak muncul baris kosong tambahan

c. baris kosong di file otomatis dihapus
d. koma di dalam nilai diabaikan

**4.** Baris CSV `Ani,85` dibaca dengan `csv.reader` ke variabel `baris`. Apa hasil `baris[1] + baris[1]`?

a. `170`
b. `'8585'`
c. `TypeError`
d. `'85 85'`

**5.** Hasil `json.dumps({"aktif": True, "wali": None})` adalah ...

a. `{"aktif": True, "wali": None}`
b. `{"aktif": true, "wali": null}`
c. `{'aktif': true, 'wali': null}`
d. `TypeError`, karena `None` tidak bisa disimpan

**6.** Sebuah tuple `(1, 2)` disimpan ke file JSON, lalu dibaca kembali. Tipe datanya sekarang adalah ...

a. `tuple`
b. `list`
c. `str`
d. `dict`

**7.** Pemanggilan `json.dump(buku, f)` dengan `buku` sebuah object class `Buku` buatan sendiri menghasilkan `TypeError`. Perbaikan yang paling tepat adalah ...

a. membuka file dengan mode `"a"`
b. menyimpan `buku.ke_dict()` dan menyediakan `Buku.dari_dict()` untuk membacanya kembali
c. mengganti `json.dump` dengan `f.write(buku)`
d. menambahkan `indent=2`

**8.** Method `__exit__` pada sebuah context manager dipanggil ...

a. hanya jika blok `with` selesai tanpa exception
b. hanya jika terjadi exception di dalam blok `with`
c. setiap kali program keluar dari blok `with`, dalam keadaan apa pun
d. sebelum `__enter__`

**9.** Perbedaan `json.dump()` dan `json.dumps()` adalah ...

a. `dump()` menulis ke file, `dumps()` menghasilkan string
b. `dump()` untuk dictionary, `dumps()` untuk list
c. `dumps()` menulis beberapa file sekaligus

d. tidak ada perbedaan

10. Method `PenyimpananJSON.muat()` melempar `DataRusakError` untuk file yang rusak, bukan mengembalikan list kosong. Alasannya adalah ...

    a. agar program berjalan lebih cepat
    b. agar data lama tidak tertimpa daftar kosong saat `simpan()` dipanggil berikutnya
    c. karena `json.load()` tidak bisa membaca file kosong
    d. karena file rusak pasti tidak bisa dibuka dengan `open()`

### Uraian

1. Jelaskan perbedaan mode `"r"`, `"w"`, `"a"`, dan `"x"` pada `open()`. Untuk setiap mode, berikan satu contoh situasi yang tepat untuk memakainya.
2. Mengapa file harus ditutup setelah dipakai? Jelaskan mengapa `with` lebih aman daripada memanggil `close()` di akhir kode.
3. Bandingkan format CSV dan JSON. Data apa yang lebih cocok disimpan dengan CSV, dan data apa yang lebih cocok dengan JSON? Berikan masing-masing satu contoh dari lingkungan kampus.
4. Jelaskan apa yang dimaksud serialisasi dan deserialisasi. Mengapa `dari_dict()` sebaiknya dibuat sebagai class method?
5. Pada studi kasus perpustakaan, class `Perpustakaan` tidak membuka file sendiri, tetapi menerima object penyimpanan. Jelaskan dua keuntungan rancangan ini.

## 9.10 Praktik Mandiri

Kerjakan praktik berikut secara berurutan. Simpan setiap praktik dalam folder terpisah, jalankan dari folder tersebut, dan pastikan keluarannya sesuai dengan contoh. Gunakan `with` dan `encoding="utf-8"` untuk setiap file.

### Praktik 1: Buku Harian

- Buat class `BukuHarian` dengan attribute `path` dan dua method: `tulis(tanggal, teks)` yang menambahkan satu baris berformat `tanggal | teks` ke file, dan `tampilkan()` yang mencetak seluruh catatan beserta nomor urutnya.
- Jika file belum ada, `tampilkan()` mencetak `Buku harian masih kosong` tanpa traceback.
- Tolak teks kosong dengan `ValueError`, dan pastikan tidak ada baris yang tertulis ke file dalam kasus itu.
- Jalankan program dua kali dan tunjukkan bahwa catatan dari kali pertama masih ada pada kali kedua.

Contoh keluaran Praktik 1

```
Buku harian masih kosong
1. 2026-10-20 | Mulai mengerjakan tugas PBO
2. 2026-10-21 | Bab 9 selesai dibaca
Ditolak: teks catatan tidak boleh kosong
```

### Praktik 2: Dari CSV ke Laporan JSON

Buat file `nilai_pbo.csv` berisi kolom `nim`, `nama`, `tugas`, `uts`, dan `uas` untuk minimal lima mahasiswa. Sisipkan satu baris yang sengaja rusak, misalnya nilai UAS berisi `"-"`.

- Baca file dengan `csv.DictReader`. Hitung nilai akhir dengan bobot 20% tugas, 30% UTS, dan 50% UAS, lalu tentukan nilai huruf.
- Baris yang rusak dilewati dan dilaporkan, tanpa menghentikan program.
- Tulis hasilnya ke `laporan.json` dalam bentuk dictionary berisi nama kelas, jumlah mahasiswa, rata-rata kelas, dan list data setiap mahasiswa.
- Baca kembali `laporan.json` dan tampilkan mahasiswa dengan nilai tertinggi.

Contoh keluaran Praktik 2

```
Baris 4 dilewati (Dodi): nilai uas '-' bukan angka
Laporan ditulis: 4 mahasiswa, rata-rata 81.65
Nilai tertinggi: Citra (90.4, A)
```

### Praktik 3: Gudang Persisten

Kembangkan class `Gudang` dari Praktik 2 Bab 8 agar stoknya tersimpan di file.

#### Langkah Pengerjaan

1. Pisahkan urusan file dari class `Gudang`. Constructor `Gudang` menerima object penyimpanan dengan method `muat()` dan `simpan()`, seperti class `Perpustakaan` di bab ini.
2. Buat `PenyimpananJSON` dan `PenyimpananCSV` untuk gudang. Pada versi CSV, ubah kembali kolom jumlah stok menjadi `int` saat memuat, karena CSV membaca semua nilai sebagai string.
3. Tangani file yang belum ada sebagai gudang kosong, dan laporkan file yang rusak dengan custom exception.
4. Simpan stok secara otomatis setelah setiap transaksi yang berhasil, dan pastikan transaksi yang gagal tidak mengubah isi file.
5. Jalankan program dua kali dengan penyimpanan JSON, lalu dua kali dengan penyimpanan CSV. Tunjukkan bahwa keduanya menghasilkan keluaran yang sama tanpa mengubah satu

baris pun di class `Gudang`.

**KRITERIA PENILAIAN**

- Semua file dibuka dengan `with` dan `encoding="utf-8"`, serta `newline=""` untuk CSV.
- Class `Gudang` tidak membuka file sendiri, dan kedua penyimpanan dapat ditukar tanpa mengubahnya.
- File yang belum ada dan file yang rusak ditangani dengan cara yang berbeda dan tepat.
- Data yang dimuat dari CSV dikonversi ke tipe yang benar sebelum dipakai.

Bab 10

# Dasar GUI

*Dari terminal ke jendela, tombol, dan kolom isian.*

10

DI BAB INI

- Prinsip dasar GUI
- Komponen GUI
- Event handling
- Integrasi dengan class OOP
- GUI web dengan Streamlit

Semua program di buku ini berkomunikasi lewat terminal: `print()` menampilkan teks, dan `input()` menunggu pengguna mengetik. Bagi programmer, cara ini sudah cukup. Namun kebanyakan pengguna komputer terbiasa dengan jendela, tombol, dan kolom isian yang bisa diklik. Kasir kantin tidak akan mau mengetik perintah di terminal hanya untuk mencatat pesanan, dan dosen lebih suka mengisi nilai di formulir daripada menyusun list Python.

Bab ini membahas dasar-dasar *Graphical User Interface* (GUI) dengan Tkinter, library GUI yang sudah terpasang bersama Python. Kita akan mempelajari cara kerja program GUI yang berbeda dari program terminal, komponen-komponen dasarnya, cara menanggapi klik dan ketikan pengguna, serta cara menyusun program GUI dengan class. Di akhir bab, kita juga mengenal Streamlit, library untuk membuat aplikasi web yang banyak dipakai untuk memamerkan model kecerdasan artifisial. Bab 13 nanti memakai semua dasar ini untuk membangun tampilan aplikasi utuh yang dirancang di Bab 11.

## 10.1 Prinsip Dasar GUI

Perbedaan terbesar antara program terminal dan program GUI bukan pada tampilannya, melainkan pada *siapa yang menentukan urutan kejadian*. Program terminal berjalan dari atas ke bawah: program meminta nama, lalu NIM, lalu nilai, dalam urutan yang ditentukan programmer. Program GUI menampilkan semua kolom sekaligus, lalu *menunggu*. Pengguna bebas mengisi kolom mana pun lebih dulu, mengklik tombol kapan saja, atau menutup jendela. Setiap tindakan pengguna itu disebut *event*, dan program yang alurnya digerakkan oleh event disebut *event-driven*.

### Event Loop

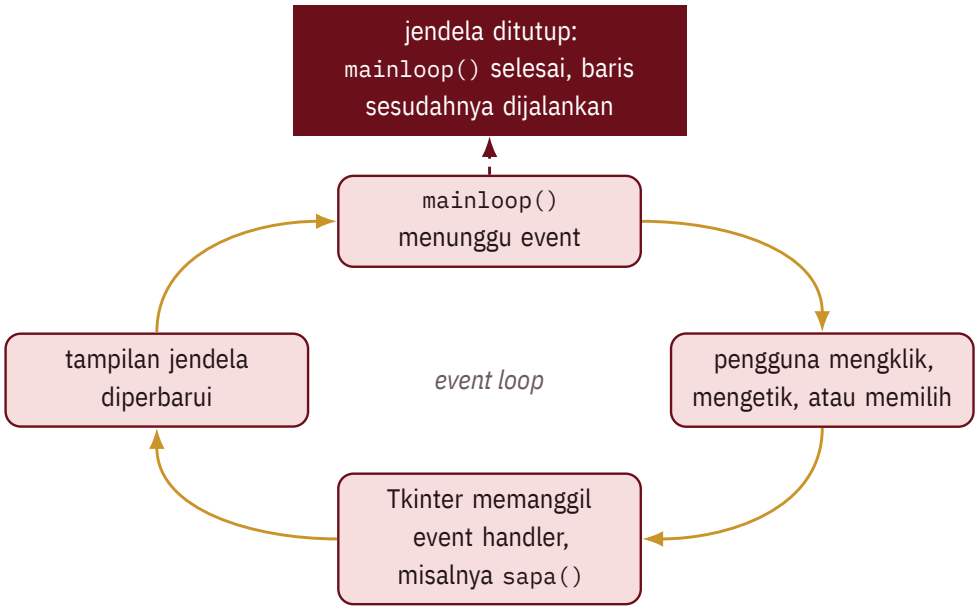
Di balik setiap program GUI terdapat sebuah perulangan yang disebut *event loop*. Perulangan ini terus menunggu event, lalu memanggil fungsi yang sudah kita daftarkan untuk menangani event tersebut. Fungsi seperti ini disebut *event handler*. Setelah event handler selesai, tampilan jendela diperbarui dan event loop kembali menunggu. Di Tkinter, event loop dijalankan oleh method `mainloop()`.

### Jendela Pertama

Program GUI paling sederhana terdiri atas empat langkah: membuat jendela, mengatur jendela, menaruh komponen di dalamnya, lalu menjalankan event loop.

**Tabel 10.1.** Perbedaan program terminal dan program GUI.

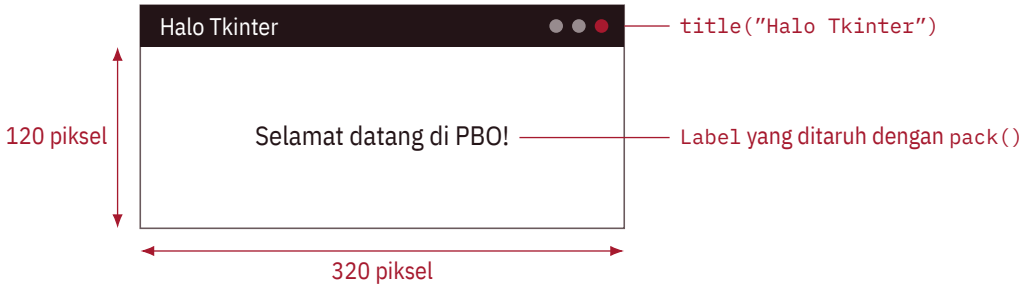
| Aspek | Program terminal | Program GUI |
|---|---|---|
| Masukan | `input()`, satu per satu dalam urutan tetap | kolom isian, tombol, dan pilihan yang bisa dipakai dalam urutan bebas |
| Keluaran | `print()` ke terminal | label, daftar, dan kotak pesan di jendela |
| Alur | ditentukan program, dari atas ke bawah | ditentukan pengguna melalui event |
| Berakhir | setelah baris terakhir dijalankan | ketika jendela ditutup |




**Gambar 10.1.** Event loop. Program GUI menghabiskan hampir seluruh waktunya untuk menunggu event, lalu menjalankan event handler yang sesuai.

Jendela pertama

```
1  import tkinter as tk
2
3  jendela = tk.Tk()                          # 1. membuat jendela utama
4  jendela.title("Halo Tkinter")              # 2. mengatur judul dan ukuran
5  jendela.geometry("320x120")
6
7  label = tk.Label(jendela, text="Selamat datang di PBO!")
8  label.pack(pady=40)                        # 3. menaruh label di jendela
9
10 jendela.mainloop()                         # 4. menjalankan event loop
```




**Gambar 10.2.** Jendela yang dihasilkan program pertama. Ukuran dari `geometry()` adalah ukuran isi jendela, belum termasuk baris judul.

Baris `import tkinter as tk` memberi alias pendek `tk`, seperti yang kita pelajari di Bab 7. Class `tk.Tk` adalah jendela utama, dan setiap program Tkinter biasanya hanya punya satu object `Tk`. Method `pack()` menaruh label di dalam jendela. Tanpa `pack()` atau cara penataan lain, label sudah dibuat tetapi tidak pernah tampil.

Baris terakhir, `mainloop()`, tidak langsung selesai. Program "tertahan" di baris itu selama jendela masih terbuka. Kode yang ditulis sesudahnya baru dijalankan setelah pengguna menutup jendela.

Kode setelah mainloop()

```
1  import tkinter as tk
2
3  jendela = tk.Tk()
4  print("sebelum mainloop")
5  jendela.mainloop()
6  print("setelah jendela ditutup")
```

Keluaran

```
sebelum mainloop
setelah jendela ditutup
```

Baris kedua pada keluaran di atas baru muncul setelah jendela ditutup. Karena itu semua komponen harus dibuat dan ditata *sebelum* `mainloop()` dipanggil.

### Ciri GUI yang Baik

Jendela yang berfungsi belum tentu nyaman dipakai. Tabel berikut merangkum beberapa prinsip sederhana yang membedakan GUI yang membingungkan dari GUI yang menyenangkan. Prinsip-prinsip ini akan kita terapkan pada contoh-contoh di bab ini.

**Tabel 10.2.** Prinsip dasar GUI yang baik.

| Prinsip | Artinya | Contoh |
|---|---|---|
| Umpan balik | setiap tindakan pengguna memberi tanda bahwa tindakan itu diterima | label berubah menjadi "Data tersimpan" |
| Label yang jelas | setiap kolom isian diberi keterangan, termasuk format yang diharapkan | "NIM (11 digit)" |
| Mencegah kesalahan | pilihan yang tidak sah dibuat tidak bisa dipilih | tombol Daftar nonaktif sampai syarat dicentang |
| Pesan kesalahan yang membantu | pesan menjelaskan apa yang salah dan cara memperbaikinya | "Nilai harus 0 sampai 100", bukan "Error" |
| Konsisten | tombol dan istilah yang sama selalu berada di tempat dan bernama sama | tombol Simpan selalu di kanan bawah |

## 10.2 Komponen GUI

Setiap komponen yang tampil di jendela, misalnya label, tombol, atau kolom isian, disebut *widget*. Di Tkinter, setiap widget adalah object dari sebuah class. Perintah `tk.Label(jendela, text="Halo")` membuat object dari class `Label`, persis seperti membuat object `Mahasiswa` di Bab 1. Argumen pertama adalah *wadah* tempat widget itu berada. Jendela berisi widget-widget, sehingga hubungan keduanya adalah composition seperti yang dibahas di Bab 6.

**Widget adalah object**

```
import tkinter as tk

jendela = tk.Tk()
label = tk.Label(jendela, text="Halo")
print(type(label).__name__, isinstance(label, tk.Widget))

label.config(text="Halo, Ani", fg="white", bg="#6B0F1A")
print(label["text"], label.cget("bg"))
```

```
print(label.master is jendela)
jendela.destroy()
```

Keluaran

```
Label True
Halo, Ani #6B0F1A
True
```

Pengaturan sebuah widget, seperti teks dan warnanya, disebut *option*. Option bisa diberikan saat widget dibuat, diubah kemudian dengan method `config()`, dan dibaca dengan `cget()` atau dengan tanda kurung siku seperti dictionary. Attribute `master` menunjuk wadah widget tersebut.

**Tabel 10.3.** Widget Tkinter yang paling sering dipakai.

| Widget | Kegunaan | Contoh pembuatan |
|---|---|---|
| `Label` | menampilkan teks yang tidak bisa diubah pengguna | `tk.Label(w, text="Nama")` |
| `Button` | tombol yang menjalankan fungsi saat diklik | `tk.Button(w, text="Simpan", command=f)` |
| `Entry` | isian teks satu baris | `tk.Entry(w, width=20)` |
| `Text` | isian teks banyak baris | `tk.Text(w, height=5)` |
| `Checkbutton` | pilihan ya atau tidak | `tk.Checkbutton(w, text="Setuju")` |
| `Radiobutton` | satu pilihan dari beberapa | `tk.Radiobutton(w, text="Pagi", value=1)` |
| `ttk.Combobox` | daftar pilihan yang terbuka ke bawah | `ttk.Combobox(w, values=[…])` |
| `Listbox` | daftar item yang bisa dipilih | `tk.Listbox(w, height=6)` |
| `Frame` | wadah untuk mengelompokkan widget | `tk.Frame(w)` |
| `messagebox` | kotak pesan (berupa fungsi, bukan widget) | `messagebox.showinfo(judul, pesan)` |

**MODULE** `TKINTER.TTK`

Tkinter memiliki dua set widget. Widget di `tkinter` adalah versi klasik yang mudah diatur warnanya. Widget di `tkinter.ttk` (*themed Tk*) mengikuti tampilan sistem operasi sehingga

terlihat lebih modern, tetapi warnanya diatur lewat *style*. Keduanya bisa dicampur dalam satu jendela. Buku ini memakai widget klasik agar contoh tetap ringkas, kecuali `ttk.Combobox` yang tidak punya versi klasik.

### Menata Widget: `pack`, `grid`, dan `place`

Widget yang sudah dibuat harus ditata di dalam wadahnya dengan salah satu dari tiga *geometry manager*.

**Tabel 10.4.** Tiga cara menata widget.

| Cara | Cara kerja | Cocok untuk |
|---|---|---|
| `pack()` | menumpuk widget berurutan dari atas ke bawah atau dari kiri ke kanan | susunan sederhana, misalnya satu kolom tombol |
| `grid()` | menempatkan widget di baris dan kolom seperti tabel | formulir dengan label di kiri dan isian di kanan |
| `place()` | menempatkan widget pada koordinat piksel tertentu | jarang dipakai, karena tidak menyesuaikan diri ketika ukuran jendela berubah |

Formulir adalah tempat `grid()` paling berguna. Setiap widget diberi nomor baris (`row`) dan kolom (`column`). Option `sticky` menentukan ke sisi mana widget menempel di dalam selnya: `"w"` (*west*) berarti kiri dan `"e"` (*east*) berarti kanan. Option `padx` dan `pady` memberi jarak di sekeliling widget. Karena jaraknya sama untuk semua widget, kedua option itu dikumpulkan dalam dictionary `jarak`, lalu dibuka menjadi argumen kata kunci dengan `**jarak`, seperti di Bab 9.

**Formulir dengan grid**

```
import tkinter as tk
from tkinter import ttk

jendela = tk.Tk()
jendela.title("Pendaftaran Seminar")
jarak = {"padx": 8, "pady": 4}          # jarak yang sama untuk semua widget

tk.Label(jendela, text="Nama").grid(row=0, column=0, sticky="w", **jarak)
tk.Entry(jendela, width=30).grid(row=0, column=1, **jarak)

tk.Label(jendela, text="NIM").grid(row=1, column=0, sticky="w", **jarak)
tk.Entry(jendela, width=30).grid(row=1, column=1, **jarak)

tk.Label(jendela, text="Prodi").grid(row=2, column=0, sticky="w", **jarak)
```

```
15 prodi = ttk.Combobox(jendela, width=28, state="readonly", values=[
16     "Kecerdasan Artifisial", "Teknik Informatika", "Sistem Informasi"])
17 prodi.grid(row=2, column=1, **jarak)
18
19 pengingat = tk.Checkbutton(jendela, text="Kirimi saya pengingat lewat email")
20 pengingat.grid(row=3, column=1, sticky="w", **jarak)
21 tk.Button(jendela, text="Daftar").grid(row=4, column=1, sticky="e", **jarak)
22
23 jendela.mainloop()
```




**Gambar 10.3.** Hasil formulir dengan `grid()`. Garis putus-putus memperlihatkan baris dan kolom yang tidak tampak di jendela sebenarnya.

**JANGAN MENCAMPUR PACK DAN GRID**

Di dalam *satu wadah yang sama*, pakailah hanya satu geometry manager. Tkinter menolak campuran keduanya dengan pesan `TclError: cannot use geometry manager grid inside … which already has slaves managed by pack`. Jika sebagian jendela lebih cocok ditata dengan `pack` dan sebagian lain dengan `grid`, kelompokkan widget ke dalam beberapa `Frame`. Setiap frame boleh memakai cara penataannya sendiri.

## Variabel Tkinter

Nilai di dalam widget bisa dihubungkan dengan object variabel khusus dari Tkinter: `StringVar`, `IntVar`, `DoubleVar`, dan `BooleanVar`. Nilai variabel ini dibaca dengan `get()` dan diubah dengan `set()`. Begitu nilainya diubah, semua widget yang terhubung ikut berubah secara otomatis.

**Variabel Tkinter**

```
1 import tkinter as tk
2
3 jendela = tk.Tk()
```

```
nama = tk.StringVar(value="Ani")
setuju = tk.BooleanVar()

isian = tk.Entry(jendela, textvariable=nama)
sapaan = tk.Label(jendela, textvariable=nama)
centang = tk.Checkbutton(jendela, text="Setuju", variable=setuju)

nama.set("Budi")                    # isian dan label ikut berubah
centang.invoke()                    # meniru klik pada kotak centang
print(isian.get(), sapaan["text"], setuju.get())
jendela.destroy()
```

**Keluaran**

```
Budi Budi True
```

Method `invoke()` menjalankan sebuah widget seolah-olah widget itu diklik. Di buku ini `invoke()` dipakai agar hasil klik bisa ditampilkan sebagai keluaran. Dalam program sungguhan, penggunalah yang mengklik.

## 10.3 Event Handling

Jendela yang hanya menampilkan widget belum bisa berbuat apa-apa. Supaya program bereaksi, kita mendaftarkan event handler untuk event tertentu. Cara paling sederhana adalah option `command` pada tombol.

**Tombol dengan event handler**

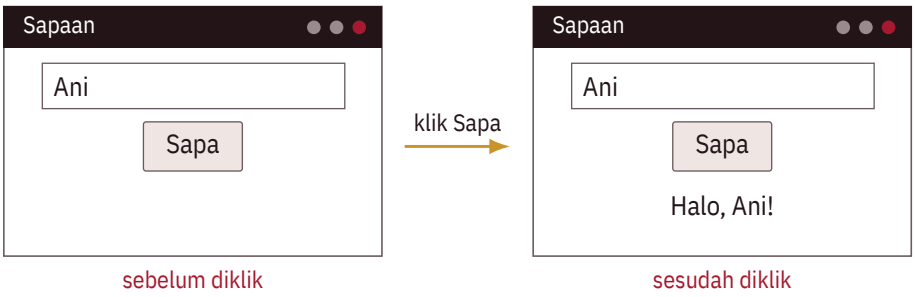
```
import tkinter as tk

def sapa():
    hasil.config(text=f"Halo, {isian.get()}!")

jendela = tk.Tk()
jendela.title("Sapaan")
isian = tk.Entry(jendela, width=24)
isian.pack(padx=10, pady=(10, 4))
tk.Button(jendela, text="Sapa", command=sapa).pack()
hasil = tk.Label(jendela, text="")
hasil.pack(pady=10)

jendela.mainloop()
```

Option `command=sapa` memberikan *fungsinya*, bukan hasil pemanggilannya. Di Bab 3 kita belajar bahwa fungsi adalah object yang bisa diberikan ke fungsi lain. Tkinter menyimpan object fungsi itu dan memanggilnya setiap kali tombol diklik. Fungsi yang




**Gambar 10.4.** Saat tombol Sapa diklik, Tkinter memanggil fungsi `sapa()`, yang membaca isian lalu mengubah teks label.

diberikan untuk dipanggil belakangan seperti ini disebut *callback*.

`COMMAND=SAPA`, **BUKAN** `COMMAND=SAPA()`

Tanda kurung setelah nama fungsi berarti "panggil sekarang". Jika ditulis `command=sapa()`, fungsi `sapa` langsung dijalankan satu kali ketika tombol *dibuat*, lalu nilai kembaliannya, yaitu `None`, yang diberikan sebagai command. Akibatnya tombol tidak melakukan apa pun ketika diklik.

**Kesalahan umum: memanggil fungsi saat membuat tombol**

```
import tkinter as tk

def simpan():
    print("data disimpan")

jendela = tk.Tk()
tombol = tk.Button(jendela, text="Simpan", command=simpan())    # salah
print("command tombol:", repr(tombol["command"]))
jendela.destroy()
```

**Keluaran**

```
data disimpan
command tombol: ''
```

Teks `data disimpan` tercetak padahal tidak ada tombol yang diklik, dan tombolnya sendiri tidak punya command.

## Event Handler dengan Argumen

Option `command` memanggil fungsinya tanpa argumen. Jika event handler membutuhkan argumen, misalnya tombol penilaian bintang satu sampai tiga, kita membungkusnya dengan `lambda`, yaitu fungsi tanpa nama yang ditulis dalam satu baris.

**Event handler dengan argumen**

```
import tkinter as tk

def pilih(bintang):
    print("Anda memberi", bintang, "bintang")

jendela = tk.Tk()
tombol = []
for bintang in range(1, 4):
    t = tk.Button(jendela, text="*" * bintang,
                  command=lambda b=bintang: pilih(b))
    t.pack(side="left")
    tombol.append(t)

tombol[0].invoke()                  # meniru klik tombol pertama
tombol[2].invoke()                  # meniru klik tombol ketiga
jendela.destroy()
```

**Keluaran**

```
Anda memberi 1 bintang
Anda memberi 3 bintang
```

Bagian `b=bintang` pada lambda tidak boleh dilupakan. Tanpa bagian itu, lambda baru membaca nilai variabel `bintang` ketika tombol diklik. Saat itu perulangan sudah lama selesai dan `bintang` bernilai 3, sehingga *semua* tombol memberi tiga bintang.

**Lambda tanpa nilai default**

```
jendela = tk.Tk()
salah = []
for bintang in range(1, 4):
    salah.append(tk.Button(jendela, command=lambda: pilih(bintang)))

salah[0].invoke()                   # tombol pertama, tetapi...
jendela.destroy()
```

**Keluaran**

```
Anda memberi 3 bintang
```

## Event Keyboard dan Mouse dengan `bind()`

Option `command` hanya tersedia pada widget seperti tombol. Untuk event lain, misalnya menekan Enter di kolom isian atau mengklik sebuah label, kita memakai method `bind()`. Event handler untuk `bind()` selalu menerima satu argumen, yaitu object *event* yang berisi keterangan tentang kejadian tersebut, misalnya tombol keyboard yang ditekan

(`event.keysym`) atau posisi mouse (`event.x` dan `event.y`).

**Tabel 10.5.** Beberapa nama event yang sering dipakai dengan `bind()`.

| Nama event | Terjadi ketika |
|---|---|
| `"<Button-1>"` | tombol kiri mouse diklik pada widget |
| `"<Double-Button-1>"` | tombol kiri mouse diklik dua kali, misalnya untuk membuka item di Listbox |
| `"<Return>"` | tombol Enter ditekan |
| `"<KeyRelease>"` | tombol keyboard apa pun dilepas |
| `"<FocusOut>"` | pengguna meninggalkan sebuah kolom isian |
| `"<Enter>"`, `"<Leave>"` | penunjuk mouse masuk ke atau keluar dari widget |

**Menanggapi ketikan dan tombol Enter**

```
import tkinter as tk

def hitung_digit(event):
    n = len(isian.get())
    info.config(text=f"{n}/11 digit", fg="#2E7D4F" if n == 11 else "#231417")

def kirim(event):
    print("NIM dikirim:", isian.get())

jendela = tk.Tk()
jendela.title("Isian NIM")
isian = tk.Entry(jendela, width=20)
isian.pack(padx=10, pady=(10, 2))
info = tk.Label(jendela, text="0/11 digit")
info.pack(pady=(0, 10))

isian.bind("<KeyRelease>", hitung_digit)      # setiap tombol keyboard dilepas
isian.bind("<Return>", kirim)                 # tombol Enter
jendela.mainloop()
```

Label `info` berubah setiap kali pengguna mengetik satu angka, dan warnanya berubah hijau begitu NIM mencapai sebelas digit. Ini contoh prinsip *umpan balik*: pengguna tahu bahwa isiannya sudah benar sebelum menekan apa pun.

### Menangani Kesalahan di Event Handler

Pengguna bisa mengetik apa saja di kolom isian, termasuk teks yang bukan angka. Exception yang terjadi di dalam event handler tidak menghentikan program. Tkinter hanya mencetak traceback ke terminal, sementara jendelanya tetap terbuka. Dari sisi pengguna, tombol terlihat tidak bekerja dan tidak ada penjelasan apa pun. Karena itu

event handler harus menangkap exception yang mungkin terjadi, lalu menyampaikannya dengan kotak pesan dari module `messagebox`.

Contoh berikut adalah konverter suhu yang memakai class `Suhu` dari Bab 2. Class itu melempar `ValueError` jika suhu berada di bawah nol mutlak.

**Konverter suhu**

```
import tkinter as tk
from tkinter import messagebox

class Suhu:                                  # dari Bab 2, diringkas
    NOL_MUTLAK = -273.15

    def __init__(self, celsius):
        if celsius < Suhu.NOL_MUTLAK:
            raise ValueError("Suhu tidak boleh di bawah nol mutlak.")
        self.celsius = celsius

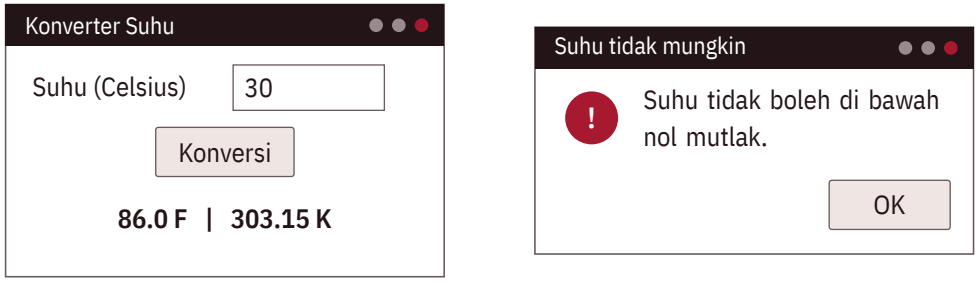
    @property
    def fahrenheit(self):
        return self.celsius * 9 / 5 + 32

    @property
    def kelvin(self):
        return self.celsius - Suhu.NOL_MUTLAK

def konversi(event=None):
    teks = isian.get().replace(",", ".")     # terima 36,5 maupun 36.5
    try:
        celsius = float(teks)
    except ValueError:
        messagebox.showerror("Masukan tidak valid",
                             "Isi suhu dengan angka, misalnya 36,5.")
        return
    try:
        suhu = Suhu(celsius)
    except ValueError as e:
        messagebox.showerror("Suhu tidak mungkin", str(e))
        return
    hasil.config(text=f"{suhu.fahrenheit:.1f} F   |   {suhu.kelvin:.2f} K")

jendela = tk.Tk()
jendela.title("Konverter Suhu")
tk.Label(jendela, text="Suhu (Celsius)").grid(row=0, column=0, padx=8, pady=8)
isian = tk.Entry(jendela, width=10)
isian.grid(row=0, column=1, padx=8)
isian.bind("<Return>", konversi)
tombol = tk.Button(jendela, text="Konversi", command=konversi)
```

```
42 tombol.grid(row=1, column=0, columnspan=2)
43 hasil = tk.Label(jendela, text="", font=("TkDefaultFont", 11, "bold"))
44 hasil.grid(row=2, column=0, columnspan=2, pady=8)
45
46 jendela.mainloop()
```




**Gambar 10.5.** Kiri: hasil konversi untuk masukan 30. Kanan: kotak pesan yang muncul jika pengguna memasukkan $-300$.

Beberapa hal pada contoh ini patut diperhatikan:

- Fungsi `konversi()` dipakai untuk dua hal: sebagai `command` tombol (dipanggil tanpa argumen) dan sebagai handler `bind()` untuk tombol Enter (dipanggil dengan object event). Parameter `event=None` membuat keduanya bisa memakai fungsi yang sama.
- Kegagalan `float()` dan pelanggaran aturan `Suhu` ditangani di dua blok terpisah. Pesan bawaan `float()`, yaitu `could not convert string to float`, tidak ramah bagi pengguna, sehingga diganti pesan yang menjelaskan cara memperbaikinya.
- Class `Suhu` tidak tahu bahwa dirinya dipakai di jendela. Ia hanya melempar exception, dan event handler yang memutuskan cara menampilkannya. Pembagian tugas ini sama dengan `Rekening` dan `ATM` di Bab 8.

## 10.4 Integrasi GUI dengan Class OOP

Konverter suhu di atas bekerja, tetapi strukturnya punya masalah yang akan membesar seiring program tumbuh. Fungsi `konversi()` memakai variabel global `isian` dan `hasil`. Jendela dengan dua puluh widget akan punya dua puluh variabel global, dan setiap fungsi bisa membaca maupun mengubah semuanya. Solusinya adalah cara yang sudah kita pelajari sejak Bab 1: bungkus data dan perilaku yang saling berkaitan ke dalam sebuah class.

### Aplikasi sebagai Class

Jendela aplikasi dijadikan class turunan `tk.Tk` (inheritance, Bab 4). Widget yang perlu diakses kembali disimpan sebagai attribute, dan event handler menjadi method.

Konverter suhu sebagai class

```
import tkinter as tk
from tkinter import messagebox

class AplikasiSuhu(tk.Tk):
    def __init__(self):
        super().__init__()                    # menyiapkan jendela Tk
        self.title("Konverter Suhu")
        label = tk.Label(self, text="Suhu (Celsius)")
        label.grid(row=0, column=0, padx=8, pady=8)
        self.isian = tk.Entry(self, width=10)
        self.isian.grid(row=0, column=1, padx=8)
        self.isian.bind("<Return>", lambda event: self.konversi())
        tombol = tk.Button(self, text="Konversi", command=self.konversi)
        tombol.grid(row=1, column=0, columnspan=2)
        self.hasil = tk.Label(self, font=("TkDefaultFont", 11, "bold"))
        self.hasil.grid(row=2, column=0, columnspan=2, pady=8)

    def konversi(self):
        try:
            suhu = Suhu(float(self.isian.get().replace(",", ".")))
        except ValueError as e:
            messagebox.showerror("Masukan tidak valid", str(e))
            return
        teks = f"{suhu.fahrenheit:.1f} F   |   {suhu.kelvin:.2f} K"
        self.hasil.config(text=teks)

if __name__ == "__main__":
    AplikasiSuhu().mainloop()
```

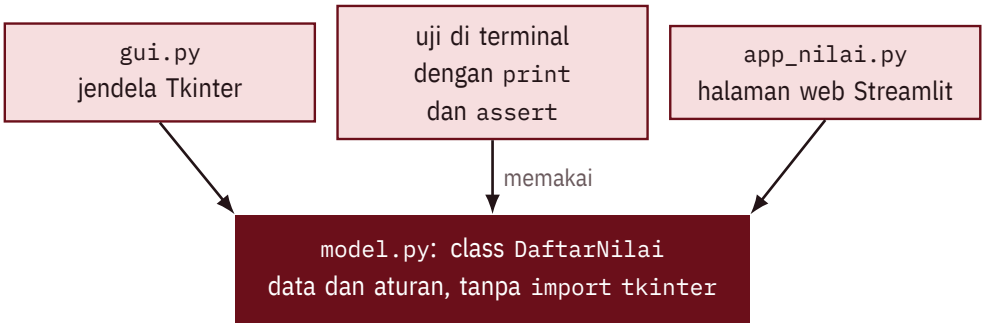
Tidak ada lagi variabel global selain nama class. Semua widget menjadi milik object aplikasi, dan method `konversi()` mengaksesnya lewat `self`. Perhatikan juga `command=self.konversi`: method yang ditulis tanpa tanda kurung adalah *bound method* yang sudah membawa `self`-nya, sehingga Tkinter cukup memanggilnya tanpa argumen. Demi ringkasnya contoh, kedua jenis kesalahan di sini ditangani dalam satu blok `except`. Pada aplikasi sungguhan, pesan `float()` sebaiknya tetap diganti seperti pada versi sebelumnya.

**MEWARISI ATAU MEMILIKI JENDELA?**

Selain mewarisi `tk.Tk`, class aplikasi juga bisa *memiliki* jendela sebagai attribute (composition), misalnya `self.jendela = tk.Tk()`. Kedua cara sama-sama umum. Mewarisi `tk.Frame` juga sering dipakai agar sebuah bagian tampilan bisa ditaruh di dalam jendela lain.

## Memisahkan Model dan Tampilan

Langkah berikutnya lebih penting lagi: memisahkan *aturan dan data* dari *tampilan*. Bagian pertama disebut *model*, dan bagian kedua disebut *view*. Model tidak boleh mengimpor `tkinter`. Ia hanya berisi class biasa yang menyimpan data, menjaga aturan, dan melempar exception. View berisi jendela dan widget, membaca masukan pengguna, memanggil model, lalu menampilkan hasilnya.




**Gambar 10.6.** Satu model dapat dipakai oleh beberapa view. Model tidak tahu apa pun tentang tampilan yang memakainya.

Pemisahan ini memberi tiga keuntungan. Model bisa diuji tanpa membuka jendela. Model bisa dipakai ulang oleh tampilan lain, seperti program terminal atau halaman web, tanpa diubah sedikit pun. Perubahan tampilan juga tidak berisiko merusak aturan. Di Bab 7, keuntungan seperti ini kita sebut reusability dan maintainability.

## Studi Kasus: Pencatat Nilai

Kita terapkan pemisahan tersebut pada aplikasi kecil untuk mencatat nilai mahasiswa. Modelnya berupa class `DaftarNilai`, yang juga menyimpan datanya ke file JSON seperti di Bab 9.

model.py

```
import json

class DaftarNilai:
    def __init__(self):
        self._nilai = {}                        # nama -> nilai

    def tambah(self, nama, nilai):
        nama = nama.strip()
        if not nama:
            raise ValueError("Nama tidak boleh kosong.")
        if not 0 <= nilai <= 100:
            raise ValueError("Nilai harus di antara 0 dan 100.")
        self._nilai[nama] = nilai

    def rata_rata(self):
```

```
        if not self._nilai:
            return 0
        return sum(self._nilai.values()) / len(self._nilai)

    def semua(self):
        return sorted(self._nilai.items())

    def simpan(self, path):
        with open(path, "w", encoding="utf-8") as f:
            json.dump(self._nilai, f, indent=2)
```

Sebelum membuat jendela apa pun, model ini bisa langsung diuji di terminal.

**Menguji model tanpa jendela**

```
from model import DaftarNilai

daftar = DaftarNilai()
daftar.tambah("Ani", 88)
daftar.tambah("Budi", 75)
print(daftar.semua(), daftar.rata_rata())
try:
    daftar.tambah("Citra", 120)
except ValueError as e:
    print("Ditolak:", e)
```

**Keluaran**

```
[('Ani', 88), ('Budi', 75)] 81.5
Ditolak: Nilai harus di antara 0 dan 100.
```

View-nya adalah class `AplikasiNilai` yang menerima object model melalui constructor.

**gui.py**

```
import tkinter as tk
from tkinter import messagebox

from model import DaftarNilai

JARAK = {"padx": 8, "pady": 4}

class AplikasiNilai(tk.Tk):
    def __init__(self, daftar):
        super().__init__()
        self.daftar = daftar
        self.title("Pencatat Nilai")
```

```
        self.resizable(False, False)

        tk.Label(self, text="Nama").grid(row=0, column=0, sticky="w", **JARAK)
        self.isian_nama = tk.Entry(self, width=24)
        self.isian_nama.grid(row=0, column=1, **JARAK)

        tk.Label(self, text="Nilai").grid(row=1, column=0, sticky="w",
                                          **JARAK)
        self.isian_nilai = tk.Entry(self, width=24)
        self.isian_nilai.grid(row=1, column=1, **JARAK)
        self.isian_nilai.bind("<Return>", lambda event: self.tambah())

        tombol = tk.Button(self, text="Tambah", command=self.tambah)
        tombol.grid(row=2, column=1, sticky="e", padx=8)

        self.tampilan = tk.Listbox(self, width=30, height=6,
                                   font="TkFixedFont")
        self.tampilan.grid(row=3, column=0, columnspan=2, padx=8, pady=8)

        self.label_rata = tk.Label(self, text="Rata-rata: -")
        self.label_rata.grid(row=4, column=0, columnspan=2, sticky="w",
                             padx=8)
        tombol = tk.Button(self, text="Simpan ke JSON", command=self.simpan)
        tombol.grid(row=4, column=1, sticky="e", padx=8, pady=8)

    def tambah(self):
        try:
            nilai = int(self.isian_nilai.get())
        except ValueError:
            messagebox.showerror("Data tidak valid",
                                 "Nilai harus berupa bilangan bulat.")
            return
        try:
            self.daftar.tambah(self.isian_nama.get(), nilai)
        except ValueError as e:
            messagebox.showerror("Data tidak valid", str(e))
            return
        self.isian_nama.delete(0, tk.END)
        self.isian_nilai.delete(0, tk.END)
        self.isian_nama.focus_set()          # siap untuk data berikutnya
        self.perbarui()

    def perbarui(self):
        self.tampilan.delete(0, tk.END)
        for nama, nilai in self.daftar.semua():
            self.tampilan.insert(tk.END, f"{nama:<24}{nilai:>4}")
```

```
59          rata = self.daftar.rata_rata()
60          self.label_rata.config(text=f"Rata-rata: {rata:.2f}")
61
62      def simpan(self):
63          self.daftar.simpan("nilai.json")
64          messagebox.showinfo("Tersimpan", "Data nilai disimpan ke nilai.json")
65
66  if __name__ == "__main__":
67      AplikasiNilai(DaftarNilai()).mainloop()
```




**Gambar 10.7.** Aplikasi Pencatat Nilai setelah tiga nilai ditambahkan.

Bandingkan isi kedua file. File `model.py` tidak tahu ada jendela, tombol, atau kotak pesan. File `gui.py` tidak tahu aturan rentang nilai. Ia hanya meneruskan masukan ke model dan menampilkan pesan exception yang diterimanya. Jika suatu saat aturan berubah, misalnya nilai boleh berupa pecahan, perubahan utamanya ada di model. Jika tampilannya ingin dirombak, model tidak perlu disentuh.

Aplikasi ini juga menerapkan prinsip GUI dari awal bab. Daftar dan rata-rata langsung diperbarui setelah setiap penambahan (umpan balik). Kolom isian dikosongkan dan kursor dikembalikan ke kolom nama sehingga data berikutnya bisa langsung diketik (mencegah kesalahan). Setiap penolakan disertai penjelasan (pesan kesalahan yang membantu).

## 10.5 GUI Berbasis Web dengan Streamlit

Tkinter membuat jendela *desktop* yang berjalan di komputer pengguna. Di bidang kecerdasan artifisial dan sains data, aplikasi lebih sering dibuat sebagai *halaman web*: demo model klasifikasi gambar, dasbor hasil analisis, atau prototipe yang dibagikan lewat tautan. Library yang paling populer untuk keperluan ini adalah *Streamlit*. Dengan Streamlit, halaman web interaktif ditulis sepenuhnya dalam Python, tanpa HTML, CSS, maupun JavaScript.

Streamlit tidak termasuk pustaka standar, sehingga perlu dipasang dengan `pip` (Bab 7).

Program Streamlit tidak dijalankan dengan `python`, melainkan dengan perintah `streamlit run`, yang lalu membuka halamannya di browser.

Terminal

```
$ pip install streamlit
$ streamlit run halo_streamlit.py

  Local URL: http://localhost:8501
```

halo_streamlit.py

```
import streamlit as st

st.title("Halo Streamlit")
nama = st.text_input("Siapa nama Anda?")
if nama:
    st.write(f"Halo, {nama}!")
```

Tidak ada `mainloop()`, tidak ada `command`, dan tidak ada event handler. Fungsi `st.text_input()` langsung *mengembalikan* isi kolom saat ini. Rahasianya terletak pada cara kerja Streamlit yang sangat berbeda dari Tkinter: **setiap kali pengguna berinteraksi, seluruh skrip dijalankan ulang dari baris pertama**. Ketika pengguna selesai mengetik nama, Streamlit menjalankan skrip dari atas lagi. Kali ini `nama` berisi teks yang diketik, sehingga baris `st.write()` ikut dijalankan.

## Object Dibuat Ulang Setiap Interaksi

Cara kerja "jalankan ulang dari atas" membawa satu jebakan penting bagi program OOP. Perhatikan percobaan pertama memakai model `DaftarNilai`.

Percobaan yang keliru

```
import streamlit as st
from model import DaftarNilai

daftar = DaftarNilai()                  # dibuat ulang pada SETIAP interaksi
nama = st.text_input("Nama")
nilai = st.number_input("Nilai", min_value=0, max_value=100, step=1)
if st.button("Tambah"):
    daftar.tambah(nama, nilai)
st.write(daftar.semua())
```

Setelah Ani ditambahkan, daftar memang menampilkan Ani. Namun begitu Budi ditambahkan, Ani menghilang. Setiap klik membuat skrip dijalankan ulang, dan baris pertama membuat object `DaftarNilai` yang baru dan kosong. Object yang lama ikut hilang bersama semua isinya.

Streamlit menyediakan `st.session_state`, yaitu wadah yang isinya *bertahan* di antara

satu kali skrip dijalankan dan berikutnya, selama tab browser pengguna masih terbuka. Object model cukup dibuat satu kali, lalu disimpan di sana.

app_nilai.py

```
import streamlit as st

from model import DaftarNilai

st.title("Pencatat Nilai")

if "daftar" not in st.session_state:          # hanya pada kunjungan pertama
    st.session_state.daftar = DaftarNilai()
daftar = st.session_state.daftar

with st.form("tambah_nilai", clear_on_submit=True):
    nama = st.text_input("Nama")
    nilai = st.number_input("Nilai", min_value=0, max_value=100, step=1)
    if st.form_submit_button("Tambah"):
        try:
            daftar.tambah(nama, nilai)
        except ValueError as e:
            st.error(str(e))

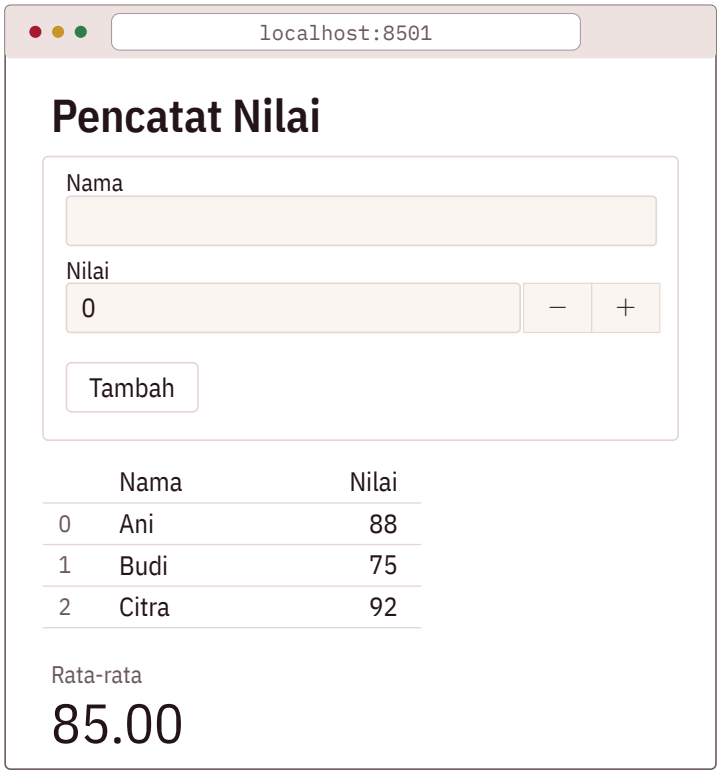
st.table([{"Nama": n, "Nilai": v} for n, v in daftar.semua()])
st.metric("Rata-rata", f"{daftar.rata_rata():.2f}")
```

Bandingkan dengan versi Tkinter. File `model.py` dipakai *tanpa diubah satu baris pun*. Inilah buah dari pemisahan model dan tampilan di subbab sebelumnya. Perhatikan juga dua hal berikut.

- `st.number_input()` dengan `min_value` dan `max_value` mencegah pengguna memasukkan nilai di luar rentang, dan selalu mengembalikan angka. Pemeriksaan `int()` yang kita tulis di versi Tkinter tidak lagi diperlukan. Model tetap menjaga aturannya sendiri, sehingga nama kosong tetap ditolak dan pesannya ditampilkan dengan `st.error()`.
- `st.form()` mengumpulkan beberapa isian agar skrip baru dijalankan ulang ketika tombol Tambah diklik, bukan setiap kali satu kolom berubah. Isian dikosongkan kembali sesudahnya karena option `clear_on_submit` bernilai `True`.

**PILIH SESUAI KEBUTUHAN**

Tidak ada yang lebih baik secara mutlak. Gunakan Tkinter jika aplikasi harus berjalan sebagai program desktop atau tanpa internet. Gunakan Streamlit jika hasil kerja perlu dipamerkan lewat browser, terutama model kecerdasan artifisial dan visualisasi data. Selama model dipisahkan dari tampilan, berpindah dari satu library ke library lain hanya berarti menulis ulang view-nya




**Gambar 10.8.** Halaman `app_nilai.py` di browser setelah tiga nilai ditambahkan. Kolom `number_input` dengan tombol − dan + hanya menerima 0 sampai 100.

**Tabel 10.6.** Perbandingan Tkinter dan Streamlit.

| Aspek | Tkinter | Streamlit |
|---|---|---|
| Bentuk aplikasi | jendela desktop | halaman web di browser |
| Pemasangan | sudah terpasang bersama Python | `pip install streamlit` |
| Cara menjalankan | `python gui.py` | `streamlit run app_nilai.py` |
| Alur program | event loop memanggil event handler | seluruh skrip dijalankan ulang setiap interaksi |
| Menyimpan keadaan | attribute object, hidup selama jendela terbuka | `st.session_state` |
| Pengaturan tata letak | sangat bebas: `grid`, `pack`, `place` | terbatas, tetapi rapi tanpa diatur |
| Cocok untuk | aplikasi kasir, alat bantu kantor, aplikasi tanpa internet | demo model AI, dasbor data, prototipe cepat |

saja.

## 10.6 Rangkuman

- Program GUI bersifat **event-driven**: alurnya ditentukan oleh event dari pengguna. Method `mainloop()` menjalankan **event loop** yang menunggu event dan memanggil **event handler**, dan kode setelahnya baru berjalan ketika jendela ditutup.
- Setiap **widget** adalah object. Option-nya diberikan saat pembuatan, diubah dengan `config()`, dan dibaca dengan `cget()`. Widget harus ditata dengan `pack()`, `grid()`, atau `place()`, dan satu wadah hanya boleh memakai satu cara penataan.
- `StringVar`, `IntVar`, `DoubleVar`, dan `BooleanVar` menghubungkan nilai widget dengan object Python.
- Option `command` menerima *fungsi*, bukan hasil pemanggilannya: `command=simpan`, bukan `command=simpan()`. Argumen diberikan lewat `lambda` dengan nilai default, misalnya `lambda b=bintang: pilih(b)`.
- Method `bind()` menghubungkan event keyboard dan mouse, misalnya `"<Return>"` dan `"<KeyRelease>"`, dengan handler yang menerima object event.
- Event handler harus menangkap exception dan menyampaikannya dengan `messagebox`, karena exception yang lolos hanya tercetak di terminal.
- Aplikasi GUI disusun sebagai **class**, dengan widget sebagai attribute dan event handler sebagai method. **Model** (data dan aturan) dipisahkan dari **view** (tampilan), sehingga model bisa diuji dan dipakai ulang tanpa jendela.
- **Streamlit** membuat halaman web dari skrip Python yang dijalankan ulang dari atas setiap kali pengguna berinteraksi. Object yang harus bertahan disimpan di `st.session_state`.

## 10.7 Kuis Teoritis

### Pilihan Ganda

Pilihlah satu jawaban yang paling tepat.

1. Program GUI disebut *event-driven* karena ...

   a. programnya selalu berjalan lebih cepat daripada program terminal
   b. alurnya ditentukan oleh event dari pengguna, seperti klik dan ketikan
   c. semua fungsinya harus diberi decorator
   d. programnya hanya bisa dijalankan di Windows

2. Pernyataan yang benar tentang `mainloop()` adalah ...

a. membuat jendela baru setiap kali dipanggil
b. menjalankan event loop, dan kode sesudahnya baru dijalankan setelah jendela ditutup
c. hanya diperlukan jika jendela berisi tombol
d. harus dipanggil sebelum widget pertama dibuat

**3.** Perhatikan kode berikut.

Soal 3

```
import tkinter as tk

def simpan():
    print("tersimpan")

jendela = tk.Tk()
tk.Button(jendela, text="Simpan", command=simpan()).pack()
jendela.mainloop()
```

Apa yang terjadi ketika program dijalankan dan tombol Simpan diklik tiga kali?

a. `tersimpan` tercetak tiga kali
b. `tersimpan` tercetak satu kali saat program dimulai, dan klik tidak mencetak apa pun
c. `tersimpan` tercetak empat kali
d. program berhenti dengan `TypeError`

**4.** Geometry manager yang paling cocok untuk formulir berisi label di kolom kiri dan isian di kolom kanan adalah ...

a. `pack()`
b. `grid()`
c. `place()`
d. `mainloop()`

**5.** Tiga tombol dibuat dalam perulangan `for i in range(1, 4)` dengan `command=lambda: pilih(i)`. Jika tombol *pertama* diklik, `pilih()` menerima argumen ...

a. 1
b. 3
c. 0
d. `None`

**6.** Untuk menjalankan sebuah fungsi ketika pengguna menekan tombol Enter di kolom isian, kita menulis ...

a. `isian.bind("<Return>", kirim)`
b. `isian.config(command=kirim)`

c. `isian.bind("Enter", kirim())`

d. `isian.pack(on_enter=kirim)`

7. Satu `Frame` berisi widget yang sudah ditata dengan `pack()`. Apa yang terjadi jika widget lain di frame yang sama ditata dengan `grid()`?

   a. widget itu ditempatkan di bawah widget lain

   b. Tkinter melempar `TclError`

   c. semua widget otomatis berpindah ke `grid()`

   d. widget itu tidak tampil, tetapi tidak ada pesan apa pun

8. Keuntungan utama memisahkan model dari view adalah ...

   a. jendela tampil lebih cepat

   b. model bisa diuji dan dipakai ulang oleh tampilan lain tanpa diubah

   c. jumlah baris kode pasti berkurang

   d. model bisa langsung mengatur warna widget

9. Pada aplikasi Streamlit, object `DaftarNilai()` yang dibuat di baris paling atas skrip kehilangan isinya setiap kali tombol diklik karena ...

   a. Streamlit menghapus semua object setiap satu menit

   b. seluruh skrip dijalankan ulang dari atas, sehingga object baru yang kosong dibuat lagi; solusinya menyimpan object di `st.session_state`

   c. object tidak boleh dipakai di Streamlit

   d. tombol Streamlit selalu memanggil `destroy()`

10. Cara yang paling tepat untuk menangani masukan tidak valid di dalam event handler Tkinter adalah ...

    a. membiarkan exception terjadi agar program berhenti

    b. menangkap exception, lalu menampilkan pesan yang jelas dengan fungsi `showerror()` dari `messagebox`

    c. mencetak pesan dengan `print()`

    d. menutup jendela dengan `destroy()`

## Uraian

1. Jelaskan perbedaan alur program terminal dan program GUI. Apa yang dimaksud event loop, dan apa peran `mainloop()` di dalamnya?

2. Jelaskan mengapa `command=simpan()` merupakan kesalahan, dan jelaskan cara yang benar untuk memberikan argumen kepada event handler dari sebuah tombol.

3. Bandingkan `pack()`, `grid()`, dan `place()`. Untuk masing-masing, berikan contoh tata letak yang cocok.

4. Jelaskan maksud pemisahan model dan view. Dengan contoh aplikasi Pencatat Nilai, tunjukkan bagian mana yang termasuk model dan bagian mana yang termasuk view.
5. Jelaskan perbedaan cara kerja Tkinter dan Streamlit. Mengapa object model di Streamlit perlu disimpan di `st.session_state`, sedangkan di Tkinter cukup disimpan sebagai attribute?

## 10.8 Praktik Mandiri

Kerjakan praktik berikut secara berurutan. Pisahkan setiap praktik menjadi `model.py` dan `gui.py`, dan pastikan `model.py` tidak mengimpor `tkinter`. Karena hasilnya berupa jendela, contoh keluaran berikut menjelaskan apa yang harus tampil untuk setiap masukan.

### Praktik 1: Kalkulator BMI

- Buat class `BMI` di `model.py` dengan constructor `(berat_kg, tinggi_cm)`, property `nilai`, dan method `kategori()`: Kurus (di bawah 18,5), Normal (18,5 sampai di bawah 25), Gemuk (25 sampai di bawah 30), dan Obesitas (30 ke atas). Tolak berat atau tinggi yang tidak positif dengan `ValueError`.
- Buat jendela dengan dua isian, tombol Hitung, dan label hasil. Tombol Enter di isian tinggi juga menjalankan perhitungan.
- Terima angka dengan koma maupun titik desimal, dan tampilkan kotak pesan yang jelas untuk setiap masukan yang tidak valid.

**Contoh hasil Praktik 1**

```
berat 60, tinggi 165    -> label: BMI 22.0 (Normal)
berat 90, tinggi 170    -> label: BMI 31.1 (Obesitas)
berat 52,5, tinggi 170  -> label: BMI 18.2 (Kurus)
berat abc               -> kotak pesan: Berat harus berupa angka, misalnya 55,5.
tinggi 0                -> kotak pesan: Tinggi harus lebih dari nol.
```

### Praktik 2: Formulir Pendaftaran

- Buat formulir dengan `grid()` berisi isian nama, isian NIM, `ttk.Combobox` program studi, dua `Radiobutton` untuk kelas Pagi atau Malam, dan `Checkbutton` persetujuan.
- Tombol Daftar dinonaktifkan (`state="disabled"`) sampai kotak persetujuan dicentang. Gunakan `BooleanVar` dan option `command` pada `Checkbutton`.
- Tampilkan jumlah digit NIM secara langsung dengan `bind("<KeyRelease>", …)`.
- Validasi data di class `Pendaftar` pada `model.py`. Pendaftar yang valid ditambahkan ke `Listbox`, dan NIM yang sudah terdaftar ditolak.

Contoh hasil Praktik 2

```
kotak persetujuan belum dicentang -> tombol Daftar tidak bisa diklik
NIM 2405120408                    -> label: 10/11 digit
NIM 24051204084 (baru)            -> Listbox: 24051204084  Ani  KA  Pagi
NIM 24051204084 (lagi)            -> kotak pesan: NIM 24051204084 sudah terdaftar.
```

### Praktik 3: ATM dengan Dua Tampilan

Buat tampilan untuk class `Rekening` dan custom exception dari Bab 8.

#### Langkah Pengerjaan

1. Salin `Rekening` dan hierarki `RekeningError` ke `model.py` tanpa mengubah satu baris pun.
2. Buat class `AplikasiATM(tk.Tk)` di `gui.py` dengan label saldo, isian nominal, tombol Setor dan Tarik, serta `Listbox` riwayat transaksi.
3. Tangkap `SaldoTidakCukupError` secara khusus dan tampilkan kekurangannya memakai attribute `kekurangan`. Tangkap kesalahan rekening lainnya lewat `RekeningError`.
4. Simpan dan muat saldo memakai class penyimpanan JSON dari Bab 9, sehingga saldo tetap ada ketika aplikasi dibuka kembali.
5. Buat `app_atm.py` dengan Streamlit yang memakai `model.py` yang sama. Simpan object rekening di `st.session_state`.

KRITERIA PENILAIAN

- `model.py` tidak mengimpor `tkinter` maupun `streamlit`, dan dipakai oleh kedua tampilan tanpa perubahan.
- Tidak ada variabel global untuk widget. Semua widget menjadi attribute dan semua event handler menjadi method.
- Setiap masukan tidak valid menghasilkan pesan yang jelas, dan tidak ada traceback yang muncul di terminal.
- Prinsip GUI diterapkan: umpan balik setelah setiap transaksi, label yang jelas, dan pencegahan kesalahan.

Bab 11

# Analisis dan Perancangan Aplikasi

*Memahami apa yang perlu dibangun sebelum menulis kode.*

11

DI BAB INI

- Analisis kebutuhan
- Identifikasi object
- Class, attribute, dan method
- Class relationship
- Class diagram sederhana

Di bab-bab sebelumnya kita sudah menulis class, menjaga data dengan encapsulation, memakai inheritance dan polymorphism, menyimpan data ke file, serta membuat tampilan GUI. Sekarang pertanyaannya berubah: jika mendapat permintaan membuat sebuah aplikasi, bagaimana kita menentukan class apa yang perlu dibuat dan bagaimana class-class itu bekerja sama?

Bab ini membahas langkah dari kebutuhan pengguna menuju rancangan aplikasi. Studi kasusnya adalah **Sistem Peminjaman Perpustakaan**, yang mengembangkan contoh perpustakaan pada Bab 6, 7, dan 9. Rancangan di bab ini menjadi acuan implementasi pada Bab 12 dan pembuatan GUI pada Bab 13. Hasil akhirnya berupa kebutuhan yang dapat diperiksa, daftar object dan class, pembagian tanggung jawab, serta diagram yang saling sesuai.

Pendekatan dari spesifikasi menuju rancangan class juga digunakan dalam proyek terpadu oleh [North, 2025]. Hubungan antara benda dalam masalah, data, perilaku, inheritance, dan composition dibahas oleh [Lutz, 2025], khususnya pada Bab 31. Contoh dan keputusan rancangan dalam bab ini disusun untuk kebutuhan perpustakaan sederhana pada modul ini.

CAPAIAN PEMBELAJARAN

Setelah mempelajari bab ini, Anda diharapkan mampu:

- menuliskan kebutuhan aplikasi beserta aturan dan kriteria penerimaannya;
- membedakan pengguna aplikasi, object, class, attribute, dan method;
- menentukan tanggung jawab class serta alasan pemilihan relasinya;
- menggambar class diagram sederhana dengan multiplicity yang tepat;
- memeriksa kesesuaian rancangan melalui skenario berhasil dan gagal.

Bekal yang digunakan adalah materi class dan object, encapsulation, relationship, exception handling, file JSON, serta pemisahan model dan tampilan dari bab sebelumnya.

## 11.1 Analisis Kebutuhan Aplikasi

*Analisis kebutuhan* adalah kegiatan memahami masalah, siapa yang menggunakan aplikasi, pekerjaan yang perlu dibantu, dan batas keberhasilan aplikasi. Hasilnya harus cukup jelas untuk dibaca pengguna maupun programmer. Urutan memahami kebutuhan

sebelum merancang class juga ditekankan dalam bagian pengembangan framework pada [Anonim, tt].

Permintaan seperti “buat aplikasi perpustakaan yang lengkap” belum cukup untuk dijadikan rancangan. Kata “lengkap” dapat berarti berbeda bagi petugas, mahasiswa, dan programmer. Permintaan itu perlu diuraikan menjadi pekerjaan yang dapat diamati, misalnya mendaftarkan buku, mencatat peminjaman, dan melihat buku yang belum dikembalikan.

## Masalah, Pengguna, dan Batas Aplikasi

Bayangkan sebuah ruang baca kampus mencatat peminjaman di buku tulis. Petugas sulit memastikan apakah sebuah buku sedang dipinjam, siapa peminjamnya, dan kapan harus dikembalikan. Ketika catatan diperbarui di beberapa tempat, status buku kadang berbeda dengan catatan peminjamannya.

Pada proyek ini, **petugas perpustakaan** menjadi pengguna aplikasi. Anggota menyampaikan permintaan kepada petugas; anggota belum mengoperasikan aplikasi sendiri. Petugas merupakan *aktor*, yaitu pihak yang berinteraksi dengan sistem. Seorang aktor tidak otomatis harus menjadi class di dalam model aplikasi.

**RUANG LINGKUP PROYEK**

- Aplikasi dipakai oleh satu petugas pada satu komputer dalam satu waktu.
- Data yang dikelola adalah buku, anggota, dan catatan peminjaman.
- Satu kode buku mewakili satu eksemplar buku cetak atau satu lisensi peminjaman e-book. Satu kode hanya dapat dipinjam oleh satu anggota pada waktu yang sama.
- Buku cetak dipinjam selama 14 hari, sedangkan e-book selama 7 hari. Angka ini merupakan aturan studi kasus, bukan ketentuan semua perpustakaan.
- Data disimpan dalam file JSON lokal. Tampilan desktop dibuat pada Bab 13.
- Versi awal belum mencakup login, pembayaran denda, pemesanan buku, penghapusan riwayat, maupun penggunaan bersama melalui jaringan.

Batas tersebut membantu menentukan pekerjaan yang akan diselesaikan pada Bab 12 dan 13. Fitur tambahan dapat dikembangkan setelah alur utama berjalan dan datanya konsisten.

## Kebutuhan Fungsional

*Kebutuhan fungsional* menyatakan pekerjaan yang harus dapat dilakukan aplikasi. Berikan kode pada setiap kebutuhan agar mudah ditelusuri ke rancangan dan pemeriksaannya.

Perhatikan bahwa F03 menyebut keadaan yang harus diperiksa. Menyediakan tombol Pinjam saja belum memenuhi kebutuhan jika tombol tersebut masih mengizinkan dua

**Tabel 11.1.** Kebutuhan fungsional Sistem Peminjaman Perpustakaan.

| Kode | Kebutuhan | Hasil yang diharapkan |
|---|---|---|
| F01 | Mendaftarkan buku | Kode, judul, dan jenis buku tercatat; kode yang sama tidak didaftarkan dua kali. |
| F02 | Mendaftarkan anggota | ID dan nama anggota tercatat; ID yang sama tidak didaftarkan dua kali. |
| F03 | Meminjamkan buku | Catatan peminjaman dibuat jika anggota dan buku terdaftar serta buku tersedia. |
| F04 | Mencatat pengembalian | Catatan peminjaman ditandai selesai tanpa menghapus riwayatnya. |
| F05 | Melihat data dan status | Petugas dapat melihat daftar buku, ketersediaannya, dan peminjaman yang masih aktif. |
| F06 | Menyimpan dan memuat | Buku, anggota, serta riwayat peminjaman tersedia kembali setelah aplikasi dibuka ulang. |

anggota meminjam kode buku yang sama secara bersamaan.

### Kebutuhan Nonfungsional

*Kebutuhan nonfungsional* menyatakan kualitas atau batas kerja aplikasi. Rumuskan dengan cara yang dapat diperiksa. Pernyataan "aplikasi harus mudah digunakan" perlu dilengkapi perilaku yang menunjukkan kemudahan tersebut.

**Tabel 11.2.** Kebutuhan nonfungsional dan cara memeriksanya.

| Kode | Kualitas | Cara pemeriksaan |
|---|---|---|
| NF01 | Konsistensi data | Peminjaman yang ditolak tidak menambah riwayat atau mengubah status buku. |
| NF02 | Pesan yang membantu | Masukan tidak valid menghasilkan penjelasan penyebab dan data yang perlu diperbaiki. |
| NF03 | Pemisahan tanggung jawab | Model dapat diperiksa dari terminal tanpa mengimpor atau membuka GUI. |
| NF04 | Penyimpanan yang dapat diperiksa | JSON memakai UTF-8; file rusak dilaporkan dan tidak diam-diam dianggap sebagai data kosong. |

Pemisahan komponen membantu kita memahami dan menguji aplikasi ketika ukurannya bertambah. Hal ini sejalan dengan penjelasan [Wilson, 2024] tentang membatasi interaksi antarkomponen agar rancangan tetap dapat dipahami.

### Aturan Bisnis

*Aturan bisnis* menentukan keadaan yang sah dalam masalah yang sedang dimodelkan. Aturan ini harus dijaga oleh model atau layanan aplikasi, sehingga tetap berlaku ketika tampilan diganti.

1. Kode buku, ID anggota, dan ID peminjaman harus unik dalam kelompok datanya masing-masing. Kode dan nama wajib terisi.
2. Peminjaman hanya boleh dilakukan untuk anggota dan buku yang sudah terdaftar.
3. Sebuah kode buku boleh memiliki banyak riwayat peminjaman, tetapi paling banyak satu peminjaman aktif pada suatu waktu.
4. Setiap catatan peminjaman menghubungkan tepat satu anggota dan satu buku.
5. Tanggal jatuh tempo ditetapkan saat peminjaman dibuat, berdasarkan tanggal pinjam dan lama pinjam jenis buku tersebut.
6. Tanggal pengembalian tidak boleh mendahului tanggal pinjam. Peminjaman yang sudah selesai tidak dapat dikembalikan untuk kedua kalinya.
7. Pengembalian mempertahankan riwayat. Data yang gagal diproses tidak boleh meninggalkan perubahan sebagian.

Aturan ketiga dapat diperiksa dengan menghitung peminjaman aktif untuk sebuah kode buku. Kita tidak perlu menyimpan flag `tersedia` lain yang harus terus diselaraskan dengan riwayat peminjaman.

### Menulis Skenario Penggunaan

Skenario menjelaskan satu pekerjaan dari sudut pandang pengguna. Tuliskan kondisi awal, langkah utama, hasil akhir, dan kemungkinan penolakan.

#### Skenario Meminjamkan Buku

1. **Kondisi awal:** anggota M001 dan buku B001 sudah terdaftar; B001 belum memiliki peminjaman aktif.
2. Petugas memilih anggota M001 dan buku B001, lalu memasukkan tanggal pinjam 27 Oktober 2026.
3. Aplikasi memeriksa keberadaan anggota, keberadaan buku, dan ketersediaannya.
4. Aplikasi meminta lama pinjam dari object buku. Karena B001 adalah buku cetak, lama pinjamnya 14 hari.
5. Aplikasi membuat catatan P001 dengan tanggal jatuh tempo 10 November 2026.
6. Petugas melihat peminjaman berhasil dan status B001 menjadi sedang dipinjam.
7. **Alur penolakan:** jika B001 masih dipinjam, aplikasi memberi pesan penolakan dan tidak

menambah catatan baru.

Skenario ini belum menentukan warna tombol atau letak tabel GUI. Yang harus disepakati lebih dahulu adalah data, aturan, dan hasil pekerjaannya. Bentuk antarmuka mengikuti alur tersebut pada Bab 13.

### Kriteria Penerimaan dan Ketertelusuran

*Kriteria penerimaan* menjelaskan bukti bahwa suatu kebutuhan sudah terpenuhi. Hubungan kebutuhan, method, dan skenario pemeriksaan disebut *ketertelusuran* (*traceability*). Dengan hubungan ini, kita dapat menunjukkan alasan sebuah method diperlukan.

**Tabel 11.3.** Contoh penelusuran kebutuhan ke rancangan dan pemeriksaan.

| Kode | Operasi yang dirancang | Kriteria penerimaan |
|---|---|---|
| F01 | `tambah_buku(buku)` | Pendaftaran ulang B001 ditolak; buku yang sudah tercatat tidak tertimpa. |
| F03 | `pinjamkan(...)` | Pinjam pertama berhasil; permintaan kedua untuk B001 yang masih aktif ditolak. |
| F04 | `kembalikan(...)` | P001 selesai, B001 tersedia lagi, dan catatan P001 tetap ada. |
| F05 | `tersedia(kode)` | Hasilnya sesuai dengan keberadaan peminjaman aktif untuk kode tersebut. |
| F06 | `simpan()` dan `muat()` | Setelah dimuat ulang, identitas, tanggal, dan hubungan peminjaman tetap sama. |

**HASIL ANALISIS KEBUTUHAN**

Sebelum melanjutkan, pastikan Anda dapat menjawab siapa pengguna aplikasi, data apa yang dikelola, operasi apa yang disediakan, aturan apa yang harus dijaga, dan bagaimana keberhasilannya diperiksa. Kebutuhan yang belum jelas perlu diperjelas sebelum diterjemahkan menjadi class.

## 11.2 Identifikasi Object

*Object* adalah satu entitas tertentu dalam model aplikasi, lengkap dengan identitas, keadaan, dan perilakunya. Mulailah dengan contoh konkret agar rancangan tidak berhenti pada nama-nama class yang masih abstrak.

Dalam sistem perpustakaan, buku berkode B001 adalah satu object, anggota M001 adalah object lain, dan catatan P001 yang menghubungkan keduanya juga dapat dimod-

elkan sebagai object. Tiga object tersebut mewakili peran yang berbeda walaupun terlibat dalam satu pekerjaan.

### Menyaring Kata Benda dari Kebutuhan

Analisis kata benda dan kata kerja dari Bab 1 dapat dipakai kembali. Namun, hasilnya adalah *kandidat*, bukan daftar class yang langsung diterima seluruhnya. Lutz menjelaskan pemetaan benda dan tindakan sebagai langkah awal rancangan, lalu menunjukkan bagaimana object bekerja sama melalui relasi [Lutz, 2025].

**Tabel 11.4.** Penyaringan kandidat dari deskripsi aplikasi.

| Kandidat | Keputusan | Alasan |
|---|---|---|
| Buku | Object domain | Memiliki kode, judul, dan aturan lama pinjam. |
| Anggota | Object domain | Memiliki identitas dan menjadi pihak yang meminjam. |
| Peminjaman | Object domain | Menyimpan hubungan anggota–buku beserta tanggal dan statusnya. |
| Judul dan tanggal | Attribute | Menjelaskan keadaan suatu entitas; belum memerlukan tanggung jawab class tersendiri. |
| Petugas | Aktor | Mengoperasikan aplikasi; versi ini belum mengelola data akun petugas. |
| JSON dan tombol | Format dan komponen tampilan | JSON adalah bentuk penyimpanan, sedangkan tombol dibahas saat merancang GUI. |

Keputusan tersebut bergantung pada kebutuhan. Jika pada pengembangan berikutnya perpustakaan harus mengelola pergantian petugas dan hak akses, data petugas mungkin memerlukan class. Pada ruang lingkup sekarang, penambahan itu belum dibutuhkan.

### Identitas dan Keadaan Object

Identitas membedakan satu entitas dari entitas lain. Dua eksemplar dengan judul sama tetap merupakan buku yang berbeda jika kodenya berbeda. Sebaliknya, buku B001 tetap buku yang sama setelah judulnya diperbaiki atau setelah dipinjam berkali-kali.

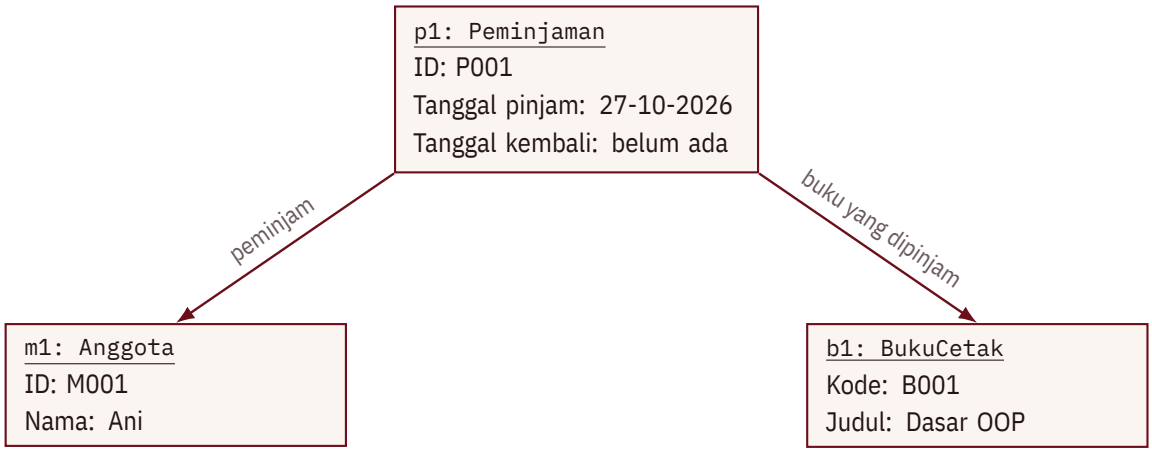
ID seperti M001 dan B001 merupakan identitas dalam aplikasi, bukan hasil fungsi `id()` Python. Nilai `id()` berkaitan dengan object selama program berjalan dan tidak digunakan sebagai identitas tetap dalam file JSON.

**Tabel 11.5.** Contoh object yang digunakan untuk memeriksa rancangan.

| Object | Calon class | Keadaan contoh |
|---|---|---|
| B001 | `BukuCetak` | Judul “Dasar OOP”, lama pinjam 14 hari. |
| B002 | `BukuCetak` | Judul “Dasar OOP”, eksemplar lain yang dapat dipinjam secara terpisah. |
| E001 | `EBook` | Judul “Dasar Python”, lama pinjam 7 hari. |
| M001 | `Anggota` | Nama Ani; menjadi peminjam pada catatan P001. |
| P001 | `Peminjaman` | Menghubungkan M001 dengan B001, dipinjam 27 Oktober 2026, belum dikembalikan. |

## Melihat Hubungan antar-Object

Gambar 11.1 memperlihatkan satu kejadian peminjaman. Diagram object menggambarkan contoh keadaan pada suatu saat, sedangkan class diagram nanti menggambarkan struktur yang dapat menghasilkan banyak object seperti itu.




**Gambar 11.1.** Satu object peminjaman merujuk satu anggota dan satu buku. Nama p1, m1, dan b1 adalah penanda object pada diagram.

Catatan P001 tidak perlu menyalin seluruh data anggota dan buku. Ia dapat menyimpan rujukan ke object yang sudah terdaftar. Jika ada dua catatan untuk M001, keduanya merujuk anggota yang sama. Hal ini perlu dipertahankan ketika data dimuat kembali dari file.

### PERIKSA DENGAN DUA CONTOH

Untuk setiap calon class, coba tuliskan dua object yang mungkin dibuat darinya. Jika yang Anda tuliskan sebenarnya hanya dua nilai seperti “Ani” dan “Budi”, periksa apakah keduanya cukup menjadi nilai attribute `nama`. Object mahasiswa atau anggota mempunyai identitas dan peran yang lebih lengkap daripada nama saja.

## 11.3 Identifikasi Class, Attribute, dan Method

Setelah contoh object ditemukan, kita mengelompokkan object yang memiliki struktur dan perilaku sejenis ke dalam class. Untuk setiap class, tentukan **tanggung jawab**, **data yang perlu diketahui**, dan **operasi yang disediakan**. Pertanyaan ini membantu mencegah satu class mengerjakan semua urusan aplikasi.

### Membagi Tanggung Jawab Class

Rancangan inti menggunakan enam class domain: `Buku`, `BukuCetak`, `EBook`, `Anggota`, `Peminjaman`, dan `Perpustakaan`. Dua class pendukung, yaitu kontrak `Penyimpanan` dan implementasi `PenyimpananJSON`, menangani batas antara model dan file.

**Tabel 11.6.** Tanggung jawab class domain.

| Class | Tanggung jawab | Batas tanggung jawab |
|---|---|---|
| `Buku` | Menyimpan identitas buku dan mendefinisikan operasi `lama_pinjam()`. | Tidak memilih peminjam atau membuka file. |
| `BukuCetak`, `EBook` | Menentukan lama pinjam sesuai jenis buku. | Tidak mengelola seluruh riwayat perpustakaan. |
| `Anggota` | Menyimpan identitas anggota yang terdaftar. | Tidak mengubah ketersediaan buku secara langsung. |
| `Peminjaman` | Menyimpan satu transaksi dan mencatat penyelesaiannya. | Tidak mengatur tombol atau kotak pesan. |
| `Perpustakaan` | Mendaftarkan entitas, memeriksa aturan lintas-object, dan mengoordinasikan transaksi. | Tidak mengatur widget atau membaca format JSON sendiri. |

**Tabel 11.7.** Tanggung jawab komponen penyimpanan.

| Komponen | Tanggung jawab |
|---|---|
| `Penyimpanan` | Kontrak dua operasi: `muat()` mengembalikan dictionary data aplikasi dan `simpan(data)` menyimpan dictionary tersebut. Kontrak dapat dinyatakan dengan `Protocol` dari Bab 5. |
| `PenyimpananJSON` | Membaca serta menulis JSON pada path yang diberikan, dan melaporkan kesalahan file. Komponen ini tidak memutuskan apakah sebuah buku boleh dipinjam. |

Pemisahan tersebut melanjutkan pola penyimpanan pada Bab 9. Payload proyek ini berupa dictionary yang menampung beberapa kelompok data, sehingga bentuknya lebih

lengkap daripada list buku pada contoh sebelumnya. Nama operasi tetap `muat()` dan `simpan(data)`.

### Menentukan Attribute dan Jenis Datanya

Pilih attribute yang diperlukan untuk menjalankan aturan dan mempertahankan keadaan aplikasi. Tipe data membantu menjelaskan nilai yang diharapkan, meskipun validasinya tetap harus ditulis saat implementasi.

- **Buku**: `kode: str` dan `judul: str`. Kode bersifat tetap setelah pendaftaran. Class turunan menentukan lama pinjam melalui method, sehingga tidak perlu menyalin percabangan jenis buku ke seluruh aplikasi.
- **Anggota**: `id_anggota: str` dan `nama: str`. ID bersifat tetap; nama harus berisi teks setelah spasi berlebih dihapus.
- **Peminjaman**: `id_pinjam: str`, rujukan `anggota` dan `buku`, serta `tanggal_pinjam`, `tanggal_jatuh_tempo`, dan `tanggal_kembali`. Dalam model, tanggal memakai `date`; tanggal kembali bernilai `None` selama peminjaman aktif.
- **Perpustakaan**: dictionary buku berdasarkan kode, dictionary anggota berdasarkan ID, koleksi catatan peminjaman, object penyimpanan, dan penomoran ID peminjaman berikutnya.

Dictionary memudahkan pencarian buku B001 dan anggota M001 berdasarkan identitasnya. Koleksi peminjaman mempertahankan transaksi yang masih aktif maupun yang sudah selesai. Bentuk koleksi ini dapat dipilih kembali ketika implementasi, selama aturan dan antarmuka yang disepakati tetap terpenuhi.

### Data Tersimpan, Data Turunan, dan Catatan Historis

Tidak semua informasi perlu disimpan sebagai attribute yang dapat diubah. Periksa dari mana sebuah nilai berasal dan apakah nilai itu harus mengikuti keadaan terkini atau mempertahankan keputusan pada masa lalu.

**Tabel 11.8.** Pemilihan data tersimpan dan data turunan.

| Informasi | Keputusan | Alasan |
|---|---|---|
| Judul buku | Disimpan | Merupakan data dasar yang tidak dapat dihitung dari catatan peminjaman. |
| Status aktif peminjaman | Diturunkan | Aktif jika `tanggal_kembali` bernilai `None`; tidak perlu flag kedua. |
| Ketersediaan buku | Diturunkan | Tersedia jika tidak ada peminjaman aktif yang merujuk kode buku tersebut. |
| Tanggal jatuh tempo | Disimpan saat transaksi dibuat | Mempertahankan kesepakatan awal meskipun aturan lama pinjam berubah kemudian. |

Untuk P001, tanggal jatuh tempo dihitung sekali saat peminjaman dibuat: 27 Oktober ditambah 14 hari menjadi 10 November 2026. Jika kebijakan buku cetak kelak berubah menjadi 10 hari, transaksi P001 tetap menggunakan tanggal yang sudah disepakati. Ini berbeda dari property terhitung yang selalu mengikuti nilai terbaru.

### Menentukan Method dan Kontraknya

Method dipilih dari tindakan yang diperlukan oleh kebutuhan aplikasi. Sebuah *kontrak method* menjelaskan masukan, keadaan yang harus terpenuhi, hasil, dan cara melaporkan kegagalan. Kontrak membantu programmer model dan programmer GUI menggunakan pengertian yang sama.

**Tabel 11.9.** Operasi utama pada class `Perpustakaan`.

| Method | Hasil | Aturan yang dijaga |
|---|---|---|
| `tambah_buku(buku)` | `None` | Kode valid dan belum terdaftar. |
| `tambah_anggota(anggota)` | `None` | ID valid dan belum terdaftar. |
| `pinjamkan(id_anggota, kode_buku, tanggal)` | Object `Peminjaman` | Anggota dan buku terdaftar; tidak ada peminjaman aktif untuk buku yang sama. |
| `kembalikan(id_pinjam, tanggal)` | `None` | Catatan ditemukan, masih aktif, dan tanggal kembali tidak mendahului tanggal pinjam. |
| `tersedia(kode_buku)` | `bool` | Kode buku terdaftar; hasil dihitung dari catatan aktif. |

Tiga method pendukung menyediakan daftar untuk tampilan: `daftar_buku()`, `daftar_anggota()`, dan `daftar_peminjaman()`. Method terakhir menerima parameter opsional `aktif_saja=False`; jika diberi nilai `True`, hasilnya hanya memuat catatan aktif. Method `simpan()` dan `muat()` mengoordinasikan pemindahan keadaan aplikasi melalui object penyimpanan. Koleksi internal tidak diberikan sebagai jalur untuk mengubah data tanpa pemeriksaan; hasil daftar dapat berupa salinan koleksi.

Untuk `pinjamkan()`, urutan rancangannya adalah mencari anggota, mencari buku, memeriksa ketersediaan, meminta lama pinjam dari buku, membuat ID dan catatan baru, lalu memasukkannya ke koleksi. Pemeriksaan dilakukan sebelum koleksi diubah. Jika salah satu syarat gagal, keadaan perpustakaan tetap seperti semula.

**LETAKKAN ATURAN PADA PEMILIKNYA**

`Peminjaman` menjaga aturan tentang tanggal dan penyelesaian satu transaksi. `Perpustakaan` menjaga aturan yang memerlukan beberapa object, seperti keberadaan anggota dan ketersedi-

aan buku. GUI meminta operasi tersebut dan menampilkan hasil atau pesan exception; GUI tidak menjadi satu-satunya tempat pemeriksaan aturan.

Kesalahan dapat dilaporkan dengan exception yang sudah dipelajari di Bab 8. Contohnya `BukuSedangDipinjamError` untuk buku yang belum tersedia dan `DataRusakError` untuk data simpanan yang tidak dapat dipakai. Nama serta hierarki exception ditetapkan ketika implementasi; penyebab kegagalannya sudah dicatat dalam kontrak ini.

## Merancang Bentuk Data Simpanan

Di dalam memori, `Peminjaman` merujuk object anggota dan buku. Di dalam JSON, hubungan itu disimpan melalui ID. Contoh berikut adalah rancangan isi file, bukan keluaran program yang sudah diimplementasikan.

Contoh rancangan data/perpustakaan.json

```
{
  "versi": 1,
  "nomor_pinjam_berikutnya": 2,
  "buku": [
    {"kode": "B001", "judul": "Dasar OOP", "jenis": "cetak"},
    {"kode": "E001", "judul": "Dasar Python", "jenis": "ebook"}
  ],
  "anggota": [
    {"id_anggota": "M001", "nama": "Ani"}
  ],
  "peminjaman": [
    {
      "id_pinjam": "P001",
      "id_anggota": "M001",
      "kode_buku": "B001",
      "tanggal_pinjam": "2026-10-27",
      "tanggal_jatuh_tempo": "2026-11-10",
      "tanggal_kembali": null
    }
  ]
}
```

Field `jenis` menentukan class buku yang dibuat saat pemuatan. Tanggal disimpan sebagai teks berformat ISO, lalu dikembalikan menjadi object `date` dalam model. Penomoran berikutnya ikut dipertahankan agar aplikasi tidak menggunakan kembali ID transaksi yang sudah tercatat.

Pemuatan dilakukan dengan membuat seluruh object buku dan anggota terlebih dahulu, kemudian menghubungkan catatan peminjaman melalui kode dan ID. Jika P001 merujuk M001 yang tidak ditemukan, data tidak boleh dianggap valid. Data juga perlu diperiksa terhadap kode ganda, tanggal yang tidak sah, dan dua peminjaman aktif untuk

buku yang sama. Pemeriksaan ini menjaga aturan yang sama seperti saat pengguna memasukkan data.

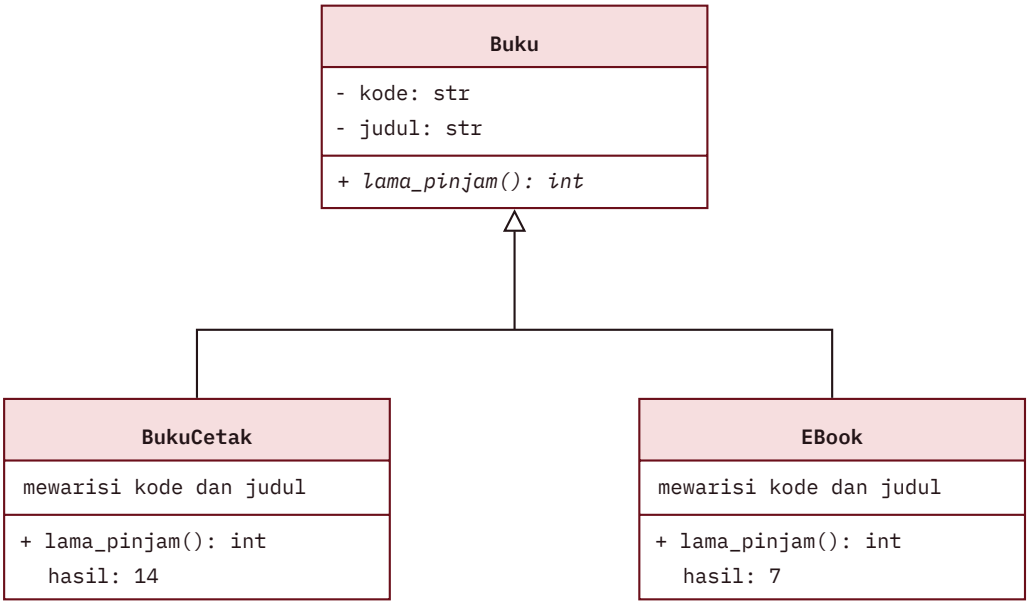
## 11.4 Class Relationship

*Class relationship* menjelaskan bagaimana object dari suatu class berhubungan dengan object class lain. Pemilihannya didasarkan pada kebutuhan dan tanggung jawab, bukan pada keinginan memakai semua jenis relasi sekaligus. Perbedaan hubungan *is-a* dan *has-a* dibahas oleh [Lutz, 2025]; pertimbangan inheritance dan composition juga dibahas oleh [North, 2025] pada Bab 5.

### Inheritance untuk Jenis Buku

`BukuCetak` dan `EBook` sama-sama merupakan jenis `Buku`. Keduanya memiliki kode serta judul, tetapi memberikan hasil berbeda untuk `lama_pinjam()`. Karena itu, hubungan ini sesuai dimodelkan dengan inheritance.

Class `Buku` direncanakan sebagai abstract class yang mendefinisikan kontrak `lama_pinjam()`. Layanan `Perpustakaan` cukup memanggil operasi tersebut pada object buku yang dipilih. Layanan tidak perlu menyalin percabangan "jika cetak maka 14, jika e-book maka 7" di setiap proses. Pemanggilan dengan antarmuka yang sama dan hasil sesuai jenis object merupakan penerapan polymorphism.




**Gambar 11.2.** Generalisasi jenis buku. Segitiga kosong mengarah ke class yang lebih umum; nama class dan operasi abstract ditulis miring.

### Association pada Catatan Peminjaman

`Peminjaman` berasosiasi dengan `Anggota` dan `Buku`. Catatan itu perlu mengetahui siapa peminjamnya dan buku apa yang dipinjam. Sebuah peminjaman bukan jenis anggota

atau jenis buku, sehingga kedua hubungan tersebut tidak dibuat dengan inheritance.

Sebaliknya, anggota dan buku tidak harus menyimpan salinan riwayat masing-masing. Pada rancangan ini, layanan perpustakaan memperoleh riwayat dari koleksi `Peminjaman` berdasarkan ID atau kode yang diperlukan. Keputusan ini mengurangi tempat yang harus diperbarui saat transaksi berlangsung.

### Aggregation dan Composition dalam Lingkup Model

Daftar buku dan anggota pada `Perpustakaan` dimodelkan sebagai *aggregation*: layanan menerima object yang didaftarkan dan mengelolanya sebagai kumpulan. Buku dan anggota tetap mempunyai identitas sendiri di luar satu transaksi peminjaman.

Koleksi `Peminjaman` dimodelkan sebagai *composition* dalam rancangan ini: setiap catatan dibuat oleh satu perpustakaan dan menjadi bagian riwayat perpustakaan tersebut. Catatan tidak dipakai sebagai milik bersama oleh dua perpustakaan. Kepemilikan ini adalah keputusan model untuk proyek satu perpustakaan, bukan kesimpulan dari letak baris pembuatan object di dalam constructor.

Menutup aplikasi tidak berarti menghapus riwayat. Object dalam memori dapat berakhir, tetapi keadaan perpustakaan disimpan ke file dan dibangun kembali saat dibuka. Siklus hidup data domain perlu dibedakan dari umur sebuah variabel atau proses Python.

### Multiplicity dan Batas Peminjaman Aktif

*Multiplicity* menunjukkan banyaknya object yang dapat terhubung pada ujung suatu relasi. Gunakan `1` untuk tepat satu, `0..1` untuk nol atau satu, dan `0..*` untuk nol atau lebih.

**Tabel 11.10.** Multiplicity pada rancangan perpustakaan.

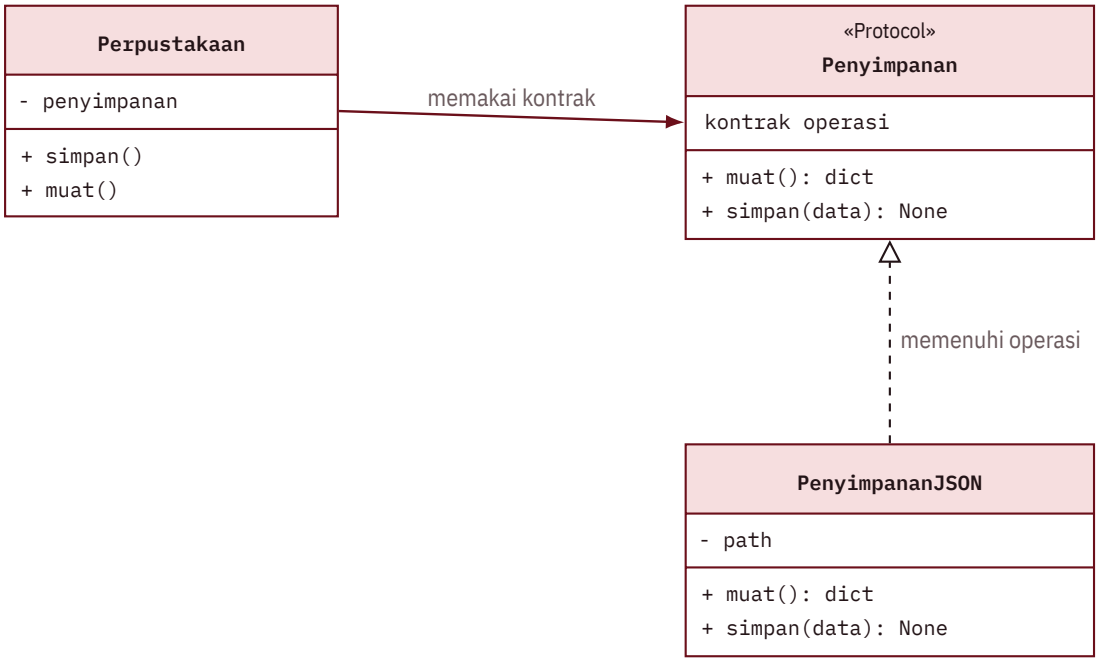
| Dilihat dari | Jumlah | Makna |
|---|---|---|
| Satu perpustakaan ke buku | `0..*` | Perpustakaan dapat belum memiliki buku atau mempunyai banyak buku terdaftar. |
| Satu perpustakaan ke anggota | `0..*` | Perpustakaan dapat memiliki banyak anggota. |
| Satu perpustakaan ke peminjaman | `0..*` | Seluruh catatan peminjaman menjadi bagian riwayatnya. |
| Satu peminjaman ke anggota | `1` | Setiap catatan mempunyai tepat satu peminjam. |
| Satu peminjaman ke buku | `1` | Setiap catatan mencatat tepat satu kode buku. |
| Satu buku ke riwayat peminjaman | `0..*` | Buku dapat dipinjam berkali-kali pada waktu berbeda. |

Relasi buku ke *seluruh riwayat* memakai `0..*`, bukan `0..1`. Batas paling banyak satu

hanya berlaku pada catatan yang masih *aktif*. Karena itu, tambahkan aturan tertulis: *untuk setiap kode buku, jumlah peminjaman aktif tidak boleh lebih dari satu*. Angka pada garis relasi saja belum menyatakan batas berdasarkan keadaan transaksi.

### Hubungan Model, Penyimpanan, dan Tampilan

`Perpustakaan` memakai kontrak `Penyimpanan`, sedangkan `PenyimpananJSON` menyediakan operasinya. Karena object penyimpanan disimpan dan digunakan oleh layanan, hubungan layanan ke kontrak dapat digambar sebagai association. Implementasi yang memenuhi kontrak digambar dengan garis realisasi putus-putus menuju kontrak.




**Gambar 11.3.** Kontrak penyimpanan memisahkan layanan dari format file. Pada Python, implementasi dapat memenuhi `Protocol` secara struktural tanpa mewarisinya secara eksplisit.

Tampilan pada Bab 13 memanggil layanan yang sama untuk mendaftarkan, meminjamkan, dan mengembalikan buku. Model tidak perlu mengimpor `tkinter`. Mengganti tata letak tombol tidak mengubah aturan peminjaman; mengganti penyimpanan pun tidak menuntut pengubahan seluruh event handler.

## 11.5 Class Diagram Sederhana

*Class diagram* menggambarkan struktur class dan hubungan di antaranya. Diagram pada Gambar 11.4 memakai notasi UML yang disederhanakan untuk menunjukkan keputusan yang akan diimplementasikan. Diagram ini dibaca bersama kebutuhan dan kontrak, bukan sebagai pengganti keduanya.

### Membaca Kotak dan Garis Relasi

Kotak class terdiri atas tiga bagian: nama class, attribute, dan operasi. Penulisan `nama: tipe` menunjukkan data yang disimpan, sedangkan `method(...)` menunjukkan operasi yang dapat dipanggil. Tanda + menandai antarmuka publik dan - menandai detail internal pada rancangan.

Tanda - pada diagram tidak menambahkan pembatasan akses secara otomatis di Python. Implementasinya tetap memakai konvensi penamaan dan property yang dipelajari pada Bab 2. Misalnya, kode disimpan sebagai data internal dan dibaca melalui property hanya-baca.

**Tabel 11.11.** Notasi hubungan yang digunakan pada diagram.

| Relasi | Notasi | Cara membaca |
|---|---|---|
| Association | Garis utuh | Object saling terhubung; angka di ujung menyatakan multiplicity. |
| Aggregation | Belah ketupat kosong | Ujung belah ketupat berada pada class pengelola kumpulan. |
| Composition | Belah ketupat terisi | Ujung belah ketupat berada pada pemilik bagian. |
| Inheritance | Segitiga kosong | Segitiga menunjuk class yang lebih umum. |
| Realisasi kontrak | Garis putus-putus dan segitiga kosong | Segitiga menunjuk kontrak yang dipenuhi implementasi. |

### Diagram Class Domain

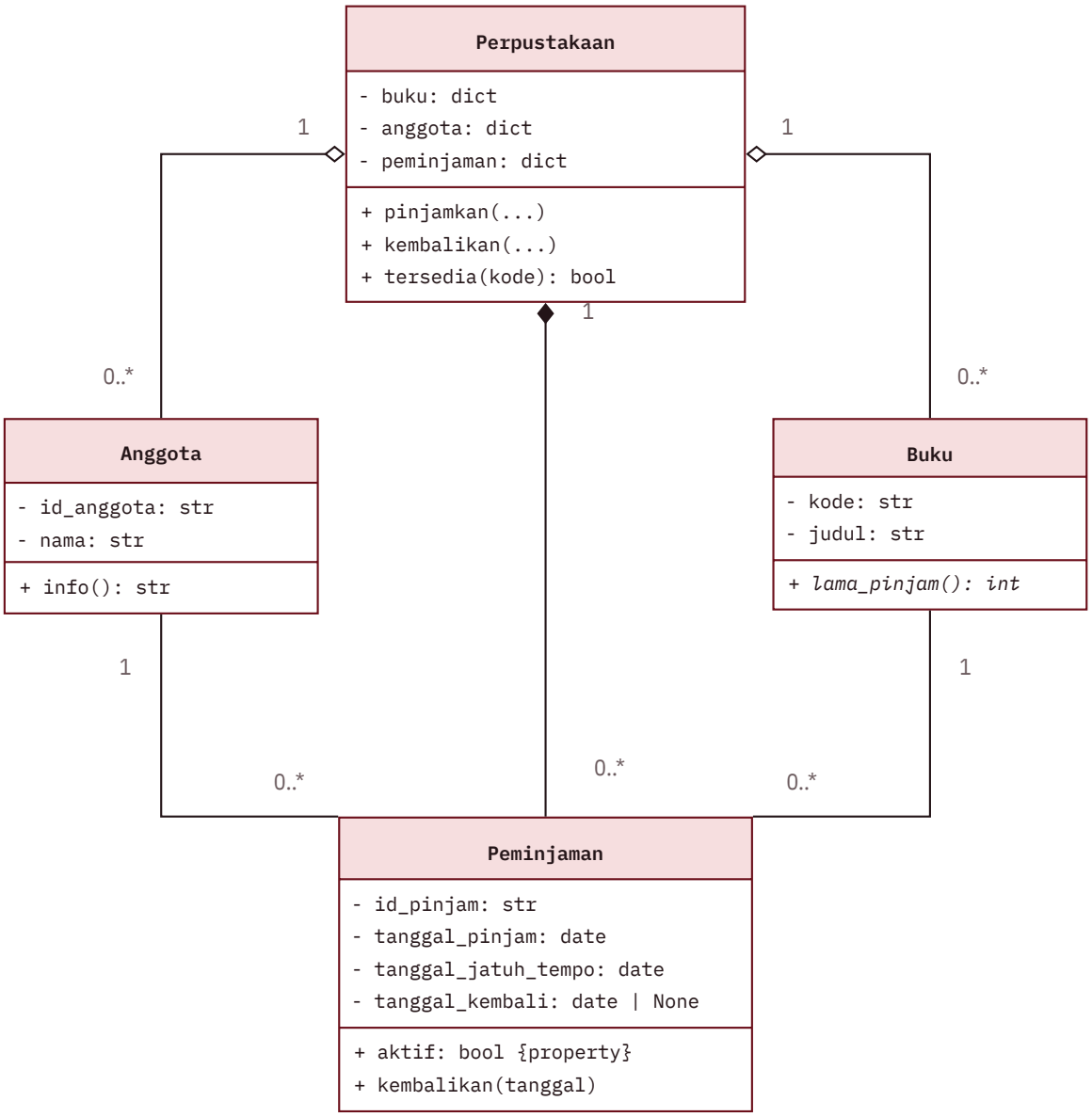
Diagram berikut menampilkan empat class utama. Turunan `BukuCetak` dan `EBook` sudah dirinci pada Gambar 11.2, sedangkan komponen file berada pada Gambar 11.3. Pemisahan gambar menjaga agar nama, attribute, dan garis relasi tetap terbaca.

Pada `Peminjaman`, rujukan `anggota` dan `buku` ditampilkan sebagai garis association sehingga tidak diulang sebagai daftar attribute pada kotaknya. Property `aktif` merupakan informasi turunan yang hanya dibaca. Nama `info()` pada `Anggota` mewakili operasi penyajian teks data anggota; operasi tersebut tidak membuka jendela atau menampilkan kotak pesan.

### Menelusuri Diagram melalui Satu Transaksi

Gunakan kembali object M001, B001, dan P001 untuk memeriksa apakah diagram mendukung kebutuhan.

1. Layanan mencari M001 pada dictionary anggota dan B001 pada dictionary buku. Jika




**Gambar 11.4.** Class diagram inti perpustakaan. Satu catatan peminjaman terhubung dengan satu anggota dan satu buku; satu buku dapat memiliki banyak catatan sepanjang riwayatnya. Operasi tambahan dijelaskan dalam kontrak method.

salah satunya tidak ditemukan, transaksi ditolak.

2. Layanan memeriksa catatan peminjaman. Jika ada catatan aktif untuk B001, transaksi ditolak tanpa mengubah koleksi.
3. Layanan memanggil `lama_pinjam()` pada B001. Object `BukuCetak` mengembalikan 14 hari.
4. Layanan membuat P001, menghubungkannya ke M001 dan B001, lalu menyimpan tanggal pinjam serta jatuh temponya.
5. Saat P001 dikembalikan, layanan meminta catatan itu memeriksa tanggal kembali dan menyelesaikan transaksi.
6. Ketersediaan B001 dihitung kembali. Karena P001 sudah tidak aktif, B001 dapat dipinjam lagi dan P001 tetap berada dalam riwayat.

**Tabel 11.12.** Perubahan keadaan yang diharapkan dari skenario rancangan.

| Keadaan | Peminjaman aktif B001 | Dampak |
|---|---|---|
| Sebelum dipinjam | Tidak ada | B001 tersedia. |
| P001 dibuat | P001 | B001 tidak tersedia untuk peminjaman lain. |
| Percobaan pinjam kedua | Tetap P001 | Permintaan ditolak; tidak ada catatan baru. |
| P001 dikembalikan | Tidak ada | B001 tersedia; riwayat P001 tetap tersimpan. |

Tabel tersebut adalah hasil yang harus dibuktikan saat implementasi, bukan laporan program yang sudah dijalankan. Jika sebuah keadaan tidak dapat dijelaskan melalui class dan method yang dirancang, perbaiki pembagian tanggung jawab sebelum menulis aplikasinya.

### Memeriksa Rancangan sebelum Implementasi

Periksa paling tidak enam keadaan: pendaftaran kode ganda, peminjaman sah, peminjaman buku yang masih aktif, peminjaman oleh anggota yang tidak terdaftar, pengembalian sah, dan pengembalian kedua untuk catatan yang sudah selesai. Tambahkan pemeriksaan simpan–muat untuk memastikan relasi tidak berubah setelah aplikasi dibuka ulang.

Pemeriksaan sejak tahap rancangan membantu menemukan kekurangan sebelum banyak kode bergantung pada keputusan tersebut. Wilson mencontohkan pengembangan melalui komponen kecil dan pemeriksaan perilaku yang dapat diulang [Wilson, 2024]. Bentuk latihan dengan syarat yang jelas dan hasil yang dapat diperiksa juga digunakan dalam kumpulan latihan [Edcorner Learning, tt].

**Dokumen yang Dibawa ke Bab 12**

1. Daftar kebutuhan F01–F06, kebutuhan kualitas, batas aplikasi, dan aturan bisnis.
2. Contoh object beserta kode, ID, dan keadaan awal yang dipakai untuk pemeriksaan.
3. Daftar class, attribute, method, serta kontrak operasi utamanya.
4. Diagram relasi dengan multiplicity dan catatan batas satu peminjaman aktif per buku.
5. Rancangan JSON dan aturan membangun kembali hubungan melalui ID.
6. Skenario berhasil dan gagal, lengkap dengan perubahan keadaan yang diharapkan.

Bab 12 akan mengubah dokumen tersebut menjadi module model dan penyimpanan yang dapat diperiksa dari terminal. Bab 13 kemudian menambahkan GUI yang memanggil model itu. Jika suatu kontrak perlu diubah saat implementasi, catat alasannya dan perbarui kebutuhan atau diagram yang berkaitan.

## 11.6 Rangkuman

- Analisis kebutuhan menjelaskan pengguna, ruang lingkup, fungsi, kualitas, aturan, dan bukti keberhasilan aplikasi.
- Kebutuhan fungsional menyatakan pekerjaan aplikasi; kebutuhan nonfungsional menyatakan kualitas atau batas kerjanya.
- Identifikasi object dimulai dari contoh konkret. Kata benda dapat menjadi kandidat object, tetapi sebagian cukup menjadi attribute, aktor, atau komponen tampilan.
- Class mengelompokkan object sejenis. Tanggung jawabnya menentukan attribute yang diperlukan dan method yang disediakan.
- Kontrak method menjelaskan masukan, hasil, syarat, serta kegagalan. Aturan lintas-object dijaga oleh layanan aplikasi.
- Data turunan seperti status aktif tidak perlu disimpan sebagai flag kedua. Data historis seperti tanggal jatuh tempo perlu mempertahankan keputusan saat transaksi dibuat.

- Inheritance digunakan untuk hubungan jenis buku; association menghubungkan peminjaman dengan anggota dan buku. Aggregation dan composition dipilih sesuai kepemilikan dalam model.
- Multiplicity pada riwayat buku adalah `0..*`; batas satu peminjaman aktif dituliskan sebagai aturan tambahan.
- Class diagram, kontrak, skenario, dan rancangan penyimpanan harus menggambarkan keputusan yang sama sebelum aplikasi diimplementasikan.

## 11.7 Kuis Teoritis

### Pilihan Ganda

Pilihlah satu jawaban yang paling tepat berdasarkan ruang lingkup studi kasus dalam bab ini.

1. Pernyataan yang termasuk kebutuhan fungsional adalah ...

   a. model dapat diperiksa tanpa membuka GUI
   b. aplikasi menolak peminjaman buku yang masih dipinjam
   c. file menggunakan UTF-8
   d. pesan kesalahan mudah dipahami

2. Dalam rancangan awal, "judul buku" paling tepat menjadi ...

   a. aktor
   b. class layanan
   c. attribute pada buku
   d. method penyimpanan

3. B001 dan B002 memiliki judul yang sama tetapi kode berbeda. Keduanya merupakan ...

   a. dua object buku yang berbeda
   b. dua nama untuk object yang pasti sama
   c. satu class dan satu method
   d. data duplikat yang pasti harus dihapus

4. Multiplicity dari satu catatan peminjaman ke buku dalam studi kasus ini adalah ...

   a. `0..*`
   b. `0..1`
   c. `1..*`
   d. `1`

5. Hubungan antara `Peminjaman` dan `Anggota` paling tepat dinyatakan sebagai ...

a. inheritance karena peminjaman adalah anggota
b. association karena peminjaman merujuk peminjamnya
c. inheritance karena kedua class memiliki ID
d. hubungan yang tidak perlu dicatat

6. Pemeriksaan bahwa sebuah buku belum mempunyai peminjaman aktif terutama menjadi tanggung jawab ...

a. layanan `Perpustakaan` yang melihat koleksi transaksi
b. warna tombol Pinjam
c. module `json` tanpa bantuan model
d. class `Anggota` tanpa melihat buku atau transaksi lain

7. Menghitung ketersediaan dari peminjaman aktif membantu ...

a. menghapus semua riwayat saat pengembalian
b. menghindari kebutuhan identitas buku
c. menghindari dua sumber status yang dapat berbeda
d. menjamin semua kebutuhan aplikasi otomatis terpenuhi

8. Bukti bahwa model terpisah dari GUI adalah ...

a. semua class berada di dalam event handler
b. setiap method model membuka kotak pesan
c. aturan hanya diperiksa ketika tombol ditekan
d. operasi peminjaman dapat diperiksa dari terminal tanpa membuka jendela

9. Cara menyimpan relasi peminjaman dalam JSON adalah ...

a. menyimpan hasil `id(object)` sebagai identitas tetap
b. menyimpan ID anggota dan kode buku, lalu menghubungkannya kembali saat dimuat
c. menyimpan gambar tampilan GUI saja
d. menyimpan object Python langsung dengan `json.dump()` tanpa konversi

10. Segitiga kosong pada hubungan `BukuCetak` dengan `Buku` menunjuk ke ...

a. object yang dibuat paling akhir
b. class turunan `BukuCetak`
c. class yang lebih umum, yaitu `Buku`
d. file JSON

## Uraian

1. Uraikan permintaan “buat aplikasi perpustakaan yang mudah dipakai” menjadi dua kebutuhan fungsional dan dua kebutuhan kualitas yang dapat diperiksa.

2. Jelaskan mengapa petugas, anggota, dan nama anggota tidak otomatis menjadi tiga class yang setara dalam ruang lingkup proyek ini.
3. Mengapa riwayat buku memakai multiplicity `0..*`, padahal sebuah buku hanya boleh memiliki satu peminjaman aktif? Berikan contoh dua transaksi pada waktu berbeda.
4. Jelaskan alasan tanggal jatuh tempo disimpan saat transaksi dibuat, sedangkan status aktif dan ketersediaan diturunkan dari data yang sudah ada.
5. Jelaskan langkah memuat P001 dari JSON agar rujukan anggota dan bukunya menunjuk object terdaftar yang benar, serta dua keadaan data yang harus ditolak.

**CEK JAWABAN PILIHAN GANDA**

1 b; 2 c; 3 a; 4 d; 5 b; 6 a; 7 c; 8 d; 9 b; 10 c. Cocokkan alasan jawaban Anda dengan kebutuhan, kontrak, dan diagram, terutama jika jawaban benar diperoleh dengan menebak.

## 11.8 Praktik Mandiri

Hasil praktik pada bab ini berupa dokumen rancangan. Tuliskan keputusan dan alasannya agar dapat dipakai sebagai acuan implementasi, baik oleh Anda sendiri maupun teman satu kelompok.

### Praktik 1: Melengkapi Dokumen Kebutuhan

Gunakan studi kasus perpustakaan dari bab ini. Pilih dua kebutuhan dari F01–F06, kemudian lengkapi masing-masing dengan aktor, kondisi awal, masukan, langkah utama, hasil berhasil, dan dua alur penolakan. Salah satu kebutuhan yang dipilih harus berupa peminjaman atau pengembalian.

Untuk setiap alur penolakan, sebutkan data mana yang harus tetap sama. Misalnya, peminjaman kedua yang ditolak tidak boleh menambah riwayat atau mengubah peminjam pada catatan yang masih aktif.

### Praktik 2: Menilai Dampak Perubahan Aturan

Perpustakaan ingin menambahkan aturan baru: satu anggota paling banyak mempunyai tiga peminjaman aktif. Aturan ini merupakan pengembangan latihan, belum menjadi batas pada rancangan dasar bab ini.

**Langkah Pengerjaan**

1. Tambahkan kebutuhan dan kriteria penerimaan untuk batas tiga peminjaman aktif.
2. Tentukan class serta method yang perlu memeriksa aturan itu, dan jelaskan data apa yang diperlukannya.

3. Jelaskan apakah aturan tersebut membutuhkan class baru. Berikan alasan, bukan hanya menambah class agar diagram tampak lebih lengkap.
4. Buat skenario peminjaman ketiga yang berhasil, peminjaman keempat yang ditolak, dan peminjaman baru setelah satu buku dikembalikan.
5. Perbarui kontrak atau catatan pada diagram yang terdampak. Tunjukkan bagian rancangan yang tetap dapat digunakan.

### Praktik 3: Merancang Peminjaman Alat Laboratorium

Rancang aplikasi peminjaman alat laboratorium dengan petugas sebagai pengguna. Setiap alat memiliki kode dan nama; mahasiswa terdaftar dapat meminjam alat yang tersedia; pengembalian mempertahankan riwayat. Tentukan sendiri dua jenis alat dengan lama pinjam berbeda dan jelaskan alasan perbedaannya.

- Tuliskan ruang lingkup, minimal lima kebutuhan fungsional, tiga kebutuhan kualitas, dan aturan bisnisnya.
- Identifikasi minimal dua contoh object untuk setiap class domain yang direncanakan.
- Susun daftar class, attribute, method, serta kontrak peminjaman dan pengembalian.
- Gambarkan class diagram dengan nama relasi dan multiplicity yang jelas. Gunakan inheritance hanya jika hubungan *is-a* dapat dijelaskan.
- Rancang data JSON yang mempertahankan hubungan antar-object melalui identitas.
- Siapkan minimal enam skenario pemeriksaan, termasuk kegagalan dan simpan–muat data.

KRITERIA PENILAIAN

- **Kebutuhan dan aturan, 25%**: jelas, dapat diperiksa, dan sesuai ruang lingkup.
- **Identifikasi object, 20%**: identitas dan keadaan konkret dibedakan dari class serta attribute.
- **Pembagian tanggung jawab, 20%**: attribute dan method mendukung kebutuhan tanpa menumpuk semua pekerjaan pada satu class.
- **Relasi dan diagram, 20%**: arah generalisasi, kepemilikan, serta multiplicity sesuai dengan penjelasan.
- **Pemeriksaan rancangan, 15%**: skenario berhasil, gagal, dan pemuatan data menghasilkan keadaan yang konsisten.

Bab 12

# Implementasi Aplikasi

*Mengubah rancangan menjadi kode yang berjalan.*

12

**DI BAB INI**

- Implementasi class
- Encapsulation
- Inheritance/composition sesuai rancangan
- Polymorphism bila diperlukan
- Module/package
- File handling
- Integrasi antar-class

Pada Bab 11 kita telah merancang Sistem Peminjaman Perpustakaan: siapa penggunanya, data apa yang diperlukan, dan aturan apa yang harus dijaga. Bab ini mengubah rancangan tersebut menjadi aplikasi Python yang dapat dijalankan melalui terminal. Model dan layanan yang dibuat di sini akan digunakan kembali untuk GUI pada Bab 13.

Kita menggunakan kasus yang sama: satu kode buku mewakili satu eksemplar buku cetak atau satu lisensi e-book. Buku cetak dipinjam selama 14 hari, sedangkan e-book selama 7 hari. Satu buku hanya boleh mempunyai satu peminjaman aktif. Riwayat yang sudah selesai tetap disimpan. Aplikasi digunakan oleh satu petugas pada satu komputer; login, denda, dan akses serentak belum termasuk ruang lingkupnya.

Penerapan class dalam proyek utuh membantu melihat hubungan antara data, aturan, dan pekerjaan pengguna, sebagaimana dibahas dalam proyek terpadu oleh [North, 2025]. Kode dalam bab ini disusun untuk rancangan perpustakaan pada Bab 11, dengan pemisahan komponen agar setiap perilaku dapat diperiksa sebelum dihubungkan ke tampilan [Wilson, 2024].

**CAPAIAN PEMBELAJARAN**

Setelah mempelajari bab ini, Anda diharapkan mampu:

- menerjemahkan rancangan class dan kontrak method menjadi kode Python;
- menjaga identitas, nilai attribute, dan perubahan status melalui encapsulation;
- menerapkan inheritance dan hubungan antar-object sesuai tanggung jawabnya;
- menggunakan polymorphism untuk variasi perilaku yang memang diperlukan;
- menyusun module dalam package dan menyimpan data beserta relasinya ke JSON;
- mengintegrasikan pendaftaran, peminjaman, pengembalian, serta simpan–muat;
- memeriksa hasil normal dan penolakan transaksi tanpa membuka GUI.

**CARA MENGIKUTI KODE DALAM BAB INI**

Gunakan Python 3.10 atau yang lebih baru. Seluruh contoh hanya memakai pustaka standar Python. Buat folder proyek baru agar tidak bercampur dengan contoh perpustakaan dari bab sebelumnya. Setiap listing yang berjudul nama file merupakan **isi lengkap file tersebut**; setiap

file hanya perlu dibuat sekali. Listing berjudul `demo_...py` adalah pemeriksaan tambahan, bukan bagian yang digabungkan ke `main.py`.
Lengkapi semua file inti sampai Subbab Integrasi Antar-Class sebelum menjalankan contoh yang mengimpor package `perpustakaan`. Urutan pembahasan mengikuti konsep; beberapa module yang diimpor baru dituliskan pada subbab berikutnya.

## 12.1 Implementasi Class

### Menerjemahkan Tanggung Jawab ke Kode

Class tidak cukup hanya mempunyai attribute yang sama dengan diagram. Method-nya harus memenuhi kontrak: menerima masukan yang sesuai, menjaga aturan, lalu menghasilkan nilai atau exception yang dapat ditangani pemanggil. Pembagian pekerjaan berikut mempertahankan rancangan pada Bab 11.

**Tabel 12.1.** Tanggung jawab komponen pada implementasi.

| Komponen | Tanggung jawab |
|---|---|
| `Buku`, turunannya | Menyimpan kode dan judul, menyediakan lama pinjam sesuai jenis buku. |
| `Anggota` | Menyimpan ID anggota dan nama yang valid. |
| `Peminjaman` | Menghubungkan anggota dengan buku, menyimpan tanggal, dan menyelesaikan satu transaksi. |
| `Perpustakaan` | Mendaftarkan object, mencari data, memeriksa ketersediaan, dan mengoordinasikan transaksi serta simpan–muat. |
| `PenyimpananJSON` | Membaca dan menulis dictionary melalui file JSON. |
| `main.py` | Menerima input terminal dan menampilkan hasil layanan. |

Kode buku dan ID anggota merupakan identitas tetap. Judul dan nama boleh diperbarui, tetapi tidak boleh kosong. Object `Peminjaman` menyimpan rujukan ke object anggota dan buku yang sudah terdaftar. Saat data disimpan, rujukan tersebut diubah menjadi ID; saat dimuat, hubungan object dibangun kembali.

### Menyiapkan Exception Aplikasi

Buat folder `perpustakaan`, lalu file `kesalahan.py` di dalamnya. Semua exception khusus mewarisi `PerpustakaanError`. Dengan demikian, antarmuka dapat menangkap kesalahan aplikasi melalui satu induk, sedangkan pemeriksaan yang lebih khusus tetap dapat membedakan penyebabnya.

perpustakaan/kesalahan.py

```python
class PerpustakaanError(Exception):
    """Induk kesalahan yang dapat ditampilkan oleh antarmuka."""


class DataTidakValidError(PerpustakaanError):
    pass


class DataTidakDitemukanError(PerpustakaanError):
    pass


class BukuSedangDipinjamError(PerpustakaanError):
    pass


class PeminjamanSelesaiError(PerpustakaanError):
    pass


class DataRusakError(PerpustakaanError):
    pass


class PenyimpananError(PerpustakaanError):
    pass
```

`DataTidakValidError` dipakai untuk masukan seperti nama kosong atau tanggal kembali yang terlalu awal. `DataTidakDitemukanError` berarti ID yang diminta belum terdaftar. `BukuSedangDipinjamError` dan `PeminjamanSelesaiError` menyatakan penolakan transaksi. `DataRusakError` menyatakan isi simpanan tidak dapat dipakai, sedangkan `PenyimpananError` menyatakan kegagalan membuka atau menulis file.

## Membuat Model Buku, Anggota, dan Peminjaman

Buat file `model.py` dalam folder yang sama. Listing berikut memuat seluruh class domain, termasuk property dan turunan buku yang akan dibahas lebih rinci pada subbab berikutnya. Model tidak memanggil `input()`, `print()`, atau komponen GUI; hasilnya dapat dipakai oleh antarmuka yang berbeda.

perpustakaan/model.py

```python
from abc import ABC, abstractmethod
from datetime import date, datetime, timedelta

from .kesalahan import DataTidakValidError, PeminjamanSelesaiError
```

```python


def teks_wajib(nilai, label):
    if not isinstance(nilai, str) or not nilai.strip():
        raise DataTidakValidError(f"{label} harus berupa teks yang terisi.")
    return nilai.strip()


def tanggal_valid(nilai):
    if not isinstance(nilai, date) or isinstance(nilai, datetime):
        raise DataTidakValidError("Tanggal harus berupa object date.")
    return nilai


class Buku(ABC):
    JENIS = ""

    def __init__(self, kode, judul):
        self._kode = teks_wajib(kode, "Kode buku")
        self.judul = judul

    @property
    def kode(self):
        return self._kode

    @property
    def judul(self):
        return self._judul

    @judul.setter
    def judul(self, nilai):
        self._judul = teks_wajib(nilai, "Judul buku")

    @abstractmethod
    def lama_pinjam(self):
        """Mengembalikan lama pinjam dalam hari."""

    def ke_data(self):
        return {"kode": self.kode, "judul": self.judul, "jenis": self.JENIS}

    def __str__(self):
        return f"[{self.kode}] {self.judul}"


class BukuCetak(Buku):
    JENIS = "cetak"
```

```
51
52      def lama_pinjam(self):
53          return 14
54
55
56  class EBook(Buku):
57      JENIS = "ebook"
58
59      def lama_pinjam(self):
60          return 7
61
62
63  class Anggota:
64      def __init__(self, id_anggota, nama):
65          self._id_anggota = teks_wajib(id_anggota, "ID anggota")
66          self.nama = nama
67
68      @property
69      def id_anggota(self):
70          return self._id_anggota
71
72      @property
73      def nama(self):
74          return self._nama
75
76      @nama.setter
77      def nama(self, nilai):
78          self._nama = teks_wajib(nilai, "Nama anggota")
79
80      def info(self):
81          return f"{self.id_anggota} - {self.nama}"
82
83      def ke_data(self):
84          return {"id_anggota": self.id_anggota, "nama": self.nama}
85
86
87  class Peminjaman:
88      def __init__(self, id_pinjam, anggota, buku, tanggal, jatuh_tempo=None):
89          if not isinstance(anggota, Anggota) or not isinstance(buku, Buku):
90              raise DataTidakValidError("Peminjaman memerlukan anggota dan buku.")

91          self._id_pinjam = teks_wajib(id_pinjam, "ID peminjaman")
92          self._anggota = anggota
93          self._buku = buku
94          self._tanggal_pinjam = tanggal_valid(tanggal)
95          if jatuh_tempo is None:
```

```
            try:
                jatuh_tempo = tanggal + timedelta(days=buku.lama_pinjam())
            except OverflowError as exc:
                raise DataTidakValidError("Tanggal di luar jangkauan.") from
    exc
        self._tanggal_jatuh_tempo = tanggal_valid(jatuh_tempo)
        if jatuh_tempo < tanggal:
            raise DataTidakValidError("Jatuh tempo mendahului tanggal pinjam.")
        self._tanggal_kembali = None

    @property
    def id_pinjam(self):
        return self._id_pinjam

    @property
    def anggota(self):
        return self._anggota

    @property
    def buku(self):
        return self._buku

    @property
    def tanggal_pinjam(self):
        return self._tanggal_pinjam

    @property
    def tanggal_jatuh_tempo(self):
        return self._tanggal_jatuh_tempo

    @property
    def tanggal_kembali(self):
        return self._tanggal_kembali

    @property
    def aktif(self):
        return self.tanggal_kembali is None

    def kembalikan(self, tanggal):
        if not self.aktif:
            raise PeminjamanSelesaiError("Peminjaman sudah selesai.")
        tanggal = tanggal_valid(tanggal)
        if tanggal < self.tanggal_pinjam:
            raise DataTidakValidError("Pengembalian mendahului tanggal pinjam.")

        self._tanggal_kembali = tanggal
```

```
140
141     def ke_data(self):
142         kembali = self.tanggal_kembali
143         return {
144             "id_pinjam": self.id_pinjam,
145             "id_anggota": self.anggota.id_anggota,
146             "kode_buku": self.buku.kode,
147             "tanggal_pinjam": self.tanggal_pinjam.isoformat(),
148             "tanggal_jatuh_tempo": self.tanggal_jatuh_tempo.isoformat(),
149             "tanggal_kembali": kembali.isoformat() if kembali else None,
150         }
```

Fungsi `teks_wajib()` memeriksa tipe dan isi teks, kemudian membuang spasi di awal serta akhir. Fungsi `tanggal_valid()` memastikan nilai berupa `date`, bukan teks atau object `datetime`. Pemeriksaan ini menjaga agar operasi tanggal di model mempunyai arti yang sama. Input dari terminal akan dikonversi dengan `date.fromisoformat()` sebelum diteruskan ke layanan.

Parameter `jatuh_tempo=None` pada constructor `Peminjaman` mempunyai dua kegunaan. Untuk transaksi baru, nilainya dihitung dari tanggal pinjam ditambah hasil `buku.lama_pinjam()`. Saat memuat riwayat dari JSON, pemanggil memberikan tanggal jatuh tempo yang sudah tersimpan. Karena itu, perubahan kebijakan lama pinjam kelak tidak mengubah kesepakatan transaksi lama.

Property `aktif` dihitung dari `tanggal_kembali is None`. Tidak ada attribute boolean kedua yang harus diperbarui bersamaan. Method `kembalikan()` memeriksa keadaan dan tanggal terlebih dahulu; tanggal kembali baru diisi jika seluruh syarat terpenuhi. Method `ke_data()` menyiapkan nilai yang bisa ditulis sebagai JSON, tanpa melakukan pekerjaan file di dalam model.

**SATU ATURAN, SATU TEMPAT PEMERIKSAAN**

Model memeriksa kebenaran satu object. Layanan memeriksa aturan yang melibatkan koleksi object. Misalnya, `Peminjaman` memeriksa tanggal kembali, tetapi hanya `Perpustakaan` yang dapat memeriksa apakah buku sudah mempunyai peminjaman aktif lain.

## 12.2 Encapsulation

### Membatasi Perubahan melalui Antarmuka Object

Encapsulation pada proyek ini menjaga agar perubahan data mengikuti aturan yang sudah ditetapkan. Property `kode` dan `id_anggota` hanya mempunyai getter sehingga tidak dapat diberi nilai melalui antarmuka publik. Ini diperlukan karena kode dan ID dipakai sebagai kunci dictionary serta penghubung riwayat. Jika identitas berubah sepihak, kunci koleksi dan data object dapat tidak sesuai.

Property `judul` dan `nama` mempunyai setter karena keduanya boleh dikoreksi. Setter memanggil validasi sebelum mengisi attribute internal; jika nilai baru ditolak, nilai lama tetap tersimpan. Tanggal dan rujukan pada peminjaman hanya dibaca melalui property. Penyelesaian transaksi dilakukan dengan method `kembalikan()`, bukan dengan mengisi tanggal kembali secara langsung.

Setelah seluruh file inti tersedia, buat file `demo_encapsulation.py` di folder utama proyek. Isi dengan contoh berikut, lalu jalankan melalui terminal.

**Perintah menjalankan demo**

```
python demo_encapsulation.py
```

**demo_encapsulation.py**

```
from datetime import date

from perpustakaan import Anggota, BukuCetak, Peminjaman
from perpustakaan.kesalahan import DataTidakValidError, PeminjamanSelesaiError


buku = BukuCetak("B001", "Dasar OOP")
buku.judul = "OOP dengan Python"
print(buku)
try:
    buku.judul = "   "
except DataTidakValidError as exc:
    print("Ditolak:", exc)
print("Judul tetap:", buku.judul)
try:
    buku.kode = "B999"
except AttributeError:
    print("Kode hanya dapat dibaca.")

ani = Anggota("M001", "Ani")
pinjam = Peminjaman("P001", ani, buku, date(2026, 10, 27))
print("Aktif:", pinjam.aktif)
pinjam.kembalikan(date(2026, 11, 2))
print("Aktif setelah kembali:", pinjam.aktif)
try:
    pinjam.kembalikan(date(2026, 11, 3))
except PeminjamanSelesaiError as exc:
    print("Ditolak:", exc)
```

Keluaran

```
[B001] OOP dengan Python
Ditolak: Judul buku harus berupa teks yang terisi.
Judul tetap: OOP dengan Python
Kode hanya dapat dibaca.
Aktif: True
Aktif setelah kembali: False
Ditolak: Peminjaman sudah selesai.
```

Judul berhasil berubah karena nilai barunya valid. Percobaan mengisi judul dengan spasi ditolak sebelum `_judul` diubah. Pemberian nilai ke `buku.kode` memicu `AttributeError` karena property tersebut tidak mempunyai setter. Pemanggilan pengembalian kedua ditolak karena transaksi sudah selesai.

### Menjaga Koleksi dan Memahami Batasnya

Layanan menyimpan buku, anggota, dan peminjaman dalam dictionary internal. Method daftar mengembalikan list baru. Menghapus isi list hasil tersebut tidak menghapus catatan di layanan. Namun, object di dalam list masih merupakan object yang sama, sehingga perubahan melalui setter yang sah tetap terlihat pada layanan.

Salinan koleksi tersebut adalah *shallow copy*, bukan salinan mendalam seluruh object. Awalan satu garis bawah juga merupakan konvensi untuk pemakaian internal, bukan mekanisme keamanan. Pemanggil yang sengaja menulis ke `_judul` atau koleksi internal dapat melewati aturan. Karena itu, kode antarmuka pada proyek ini selalu memakai property dan method publik yang disediakan.

**PEMBUATAN OBJECT DAN TRANSAKSI APLIKASI**

Pembuatan `Peminjaman` secara langsung pada contoh konsep hanya memeriksa aturan satu catatan. Dalam aplikasi, lakukan peminjaman melalui `perpus.pinjamkan(...)` agar keberadaan anggota, keberadaan buku, dan larangan peminjaman ganda ikut diperiksa. Demikian juga, lakukan pengembalian melalui layanan agar pemanggil bekerja dengan catatan yang terdaftar.

## 12.3 Inheritance/Composition sesuai Rancangan

### Inheritance untuk Jenis Buku

`BukuCetak` dan `EBook` merupakan jenis `Buku`. Keduanya memakai validasi kode, setter judul, method `ke_data()`, dan representasi teks yang diwarisi dari induknya. Perbedaannya berada pada nilai `JENIS` dan implementasi `lama_pinjam()`.

Class `Buku` menggunakan `ABC` dan `@abstractmethod`. Kita tidak membuat `Buku(...)` secara langsung karena lama pinjamnya belum ditentukan. Class turunan yang konkret wajib menyediakan implementasi method abstrak tersebut. Pemilihan inheritance didasarkan pada kesamaan kontrak dan hubungan jenis, sebagaimana pembahasan pe-

makaian class oleh [Lutz, 2025].

### Association untuk Rujukan, Composition untuk Catatan Layanan

`Peminjaman` bukan turunan `Anggota` ataupun `Buku`. Ia menyimpan rujukan ke keduanya melalui attribute internal. Hubungan ini adalah association: satu peminjaman menunjuk satu peminjam dan satu buku, sedangkan object anggota serta buku dapat dipakai oleh banyak catatan sepanjang riwayatnya.

Simpan contoh berikut sebagai `demo_relasi.py`. Operator `is` memeriksa apakah dua rujukan menunjuk object yang sama, bukan sekadar memiliki isi yang sama.

**demo_relasi.py**

```
from datetime import date

from perpustakaan import Anggota, Buku, BukuCetak, Peminjaman


ani = Anggota("M001", "Ani")
buku = BukuCetak("B001", "Dasar OOP")
pinjam = Peminjaman("P001", ani, buku, date(2026, 10, 27))

print("BukuCetak adalah Buku:", isinstance(buku, Buku))
print("Anggota yang sama:", pinjam.anggota is ani)
print("Buku yang sama:", pinjam.buku is buku)
ani.nama = "Ani Lestari"
print("Peminjam:", pinjam.anggota.nama)
```

**Keluaran**

```
BukuCetak adalah Buku: True
Anggota yang sama: True
Buku yang sama: True
Peminjam: Ani Lestari
```

Nama baru terlihat melalui `pinjam.anggota.nama` karena catatan menyimpan rujukan ke object `ani` yang sama. Nama anggota tidak digandakan di dalam peminjaman. Konsekuensinya, riwayat menampilkan nama anggota terbaru; menyimpan salinan nama pada waktu transaksi merupakan kebutuhan berbeda yang belum diterapkan pada proyek ini.

Dalam rancangan Bab 11, layanan `Perpustakaan` memiliki catatan peminjamannya sebagai composition. Implementasinya dilakukan dengan membuat `Peminjaman` di dalam `pinjamkan()` dan menyimpannya dalam dictionary layanan. Buku dan anggota didaftarkan sebagai object yang sudah dibuat, sesuai hubungan aggregation pada diagram. Composition di sini menyatakan kepemilikan dalam model aplikasi; menutup program tidak berarti menghapus riwayat yang sudah disimpan ke file.

Object penyimpanan diberikan ke constructor layanan. Class `Perpustakaan`

memakainya tanpa mewarisi class file JSON. Pemilihan hubungan ini mengikuti tanggung jawab object: layanan mengelola transaksi, sedangkan object penyimpanan mengurus file.

## 12.4 Polymorphism bila Diperlukan

Polymorphism diperlukan ketika pemanggil menjalankan operasi yang sama tetapi perilakunya dapat berbeda menurut jenis object. Pada aplikasi ini, perbedaannya nyata: buku cetak mempunyai masa pinjam 14 hari dan e-book 7 hari. Constructor `Peminjaman` cukup memanggil `buku.lama_pinjam()`, tanpa memilih rumus berdasarkan jenis buku.

Simpan dan jalankan `demo_polymorphism.py` berikut. Kedua object melewati baris pembuatan peminjaman yang sama, tetapi menghasilkan tanggal jatuh tempo berbeda.

**demo_polymorphism.py**

```
from datetime import date

from perpustakaan import Anggota, BukuCetak, EBook, Peminjaman


ani = Anggota("M001", "Ani")
koleksi = [BukuCetak("B001", "Dasar OOP"), EBook("E001", "Dasar Python")]
for nomor, buku in enumerate(koleksi, start=1):
    pinjam = Peminjaman(f"P{nomor:03d}", ani, buku, date(2026, 10, 27))
    print(buku.kode, buku.lama_pinjam(), "hari, jatuh tempo",
          pinjam.tanggal_jatuh_tempo)
```

**Keluaran**

```
B001 14 hari, jatuh tempo 2026-11-10
E001 7 hari, jatuh tempo 2026-11-03
```

Jika kelak ditambahkan jenis buku lain dengan masa pinjam berbeda, turunan baru menyediakan `lama_pinjam()`. Rumus di constructor peminjaman tetap dapat digunakan. Namun, integrasi jenis baru juga memerlukan pembaruan pilihan input dan pemetaan jenis pada pemuatan JSON. Polymorphism mengurangi percabangan pada perhitungan transaksi, bukan menghilangkan seluruh pekerjaan integrasi.

Kontrak `Penyimpanan` juga memberi peluang polymorphism: layanan memanggil `muat()` dan `simpan(data)` tanpa mengetahui cara penulisan file. Bab ini menyediakan satu implementasi, yaitu JSON. Implementasi lain dapat ditambahkan jika kebutuhan memang berubah; tidak perlu membuat banyak class penyimpanan hanya untuk memperbanyak turunan.

## 12.5 Module/Package

### Menyusun Folder dan Arah Import

Module adalah satu file Python, sedangkan package mengelompokkan module yang berkaitan. Pada proyek ini kita memakai package biasa dengan file `__init__.py`. Struktur akhirnya sebagai berikut. Folder `data` dan file JSON dibuat ketika penyimpanan pertama berhasil.

**Struktur folder proyek**

```
proyek_perpustakaan/
|-- main.py
|-- demo_integrasi.py
|-- demo_encapsulation.py
|-- demo_relasi.py
|-- demo_polymorphism.py
|-- cek_kontrak.py
|-- perpustakaan/
|   |-- __init__.py
|   |-- kesalahan.py
|   |-- model.py
|   |-- penyimpanan.py
|   `-- layanan.py
`-- data/
    `-- perpustakaan.json
```

**Tabel 12.2.** Isi file dan ketergantungannya.

| File | Isi | Memakai |
|---|---|---|
| `kesalahan.py` | Exception khusus | Exception bawaan Python |
| `model.py` | Class domain dan validasinya | Exception khusus, tanggal, ABC |
| `penyimpanan.py` | Kontrak dan adapter JSON | Exception khusus, JSON, path, file sementara |
| `layanan.py` | Koordinasi aplikasi | Model, kontrak penyimpanan, exception |
| `__init__.py` | Nama publik package | Model, layanan, penyimpanan, exception |
| `main.py` | Menu terminal | Nama publik package dan pustaka standar |

Model tidak mengimpor layanan; penyimpanan tidak mengimpor menu; layanan tidak mengimpor `main.py`. Arah import tersebut mencegah ketergantungan berputar dan memungkinkan model digunakan kembali ketika menu terminal diganti dengan

GUI. Pemisahan komponen berdasarkan pekerjaan yang jelas juga memudahkan pemeriksaan perilaku setiap komponen [Wilson, 2024].

### Menetapkan Antarmuka Package

Buat `perpustakaan/__init__.py` dengan isi berikut. File ini menyediakan nama yang paling sering dipakai oleh antarmuka. Exception khusus yang lebih rinci tetap dapat diimpor dari module `perpustakaan.kesalahan`.

**perpustakaan/__init__.py**

```
from .model import Anggota, Buku, BukuCetak, EBook, Peminjaman
from .layanan import Perpustakaan
from .penyimpanan import PenyimpananJSON
from .kesalahan import PerpustakaanError
```

Tanda titik pada `from .model import ...` berarti import relatif dari package yang sama. Sebaliknya, `main.py` memakai `from perpustakaan import ...` karena berada di luar package. Jalankan aplikasi dari folder `proyek_perpustakaan`, bukan dengan menjalankan `layanan.py` secara langsung.

**Perintah di terminal**

```
python main.py
python demo_integrasi.py
```

Jika sistem menyediakan perintah `python3` atau `py` untuk interpreter yang digunakan, sesuaikan nama perintahnya. Tidak ada perintah `pip install` yang diperlukan untuk proyek ini. Import package hanya menyiapkan class; program menu dijalankan oleh blok `if __name__ == "__main__"` dalam `main.py`.

## 12.6 File Handling

### Mengubah Object Menjadi Data JSON

JSON tidak menyimpan object Python beserta method dan rujukannya secara langsung. Karena itu, setiap model menyediakan `ke_data()` yang menghasilkan dictionary berisi teks, angka, atau `None`. Tanggal diubah menjadi teks ISO seperti `2026-10-27`. Nilai `None` menjadi `null` di JSON.

Satu data aplikasi memuat `versi`, nomor peminjaman berikutnya, daftar buku, daftar anggota, dan daftar peminjaman. Penanda versi membuat format yang belum didukung dapat ditolak secara jelas. Nomor berikutnya disimpan agar ID transaksi tetap berlanjut setelah aplikasi ditutup.

Contoh satu catatan peminjaman dalam JSON

```
{
  "id_pinjam": "P001",
  "id_anggota": "M001",
  "kode_buku": "B001",
  "tanggal_pinjam": "2026-10-27",
  "tanggal_jatuh_tempo": "2026-11-10",
  "tanggal_kembali": null
}
```

Catatan tersebut menyimpan identitas anggota dan buku, bukan salinan object atau hasil `id()`. Ketika dimuat, buku dan anggota harus dibuat terlebih dahulu. Layanan kemudian mencari object yang sesuai dengan ID pada catatan. Dengan urutan itu, peminjaman kembali terhubung ke object yang terdaftar, bukan ke salinan terpisah.

## Membuat Adapter Penyimpanan JSON

Buat `perpustakaan/penyimpanan.py` berikut. `Protocol` menyatakan bentuk operasi yang diperlukan layanan. Anotasi tersebut membantu pembacaan kode dan pemeriksaan tipe; Python tidak otomatis memeriksa kepatuhan object pada saat runtime hanya karena parameter diberi anotasi.

perpustakaan/penyimpanan.py

```
import json
import os
import tempfile
from pathlib import Path
from typing import Protocol

from .kesalahan import DataRusakError, PenyimpananError


class Penyimpanan(Protocol):
    def muat(self) -> dict: ...
    def simpan(self, data: dict) -> None: ...


class PenyimpananJSON:
    def __init__(self, path):
        self.path = Path(path)

    def muat(self):
        try:
            with self.path.open(encoding="utf-8") as sumber:
                data = json.load(sumber)
        except FileNotFoundError:
```

```
            raise  # layanan menentukan perilaku saat belum ada file
        except (json.JSONDecodeError, UnicodeError) as exc:
            raise DataRusakError("File JSON tidak dapat dibaca.") from exc
        except OSError as exc:
            raise PenyimpananError("Gagal membuka file data.") from exc
        if not isinstance(data, dict):
            raise DataRusakError("Isi utama JSON harus berupa object.")
        return data

    def simpan(self, data):
        sementara = None
        try:
            self.path.parent.mkdir(parents=True, exist_ok=True)
            with tempfile.NamedTemporaryFile(
                mode="w", encoding="utf-8", dir=self.path.parent,
                prefix=".perpustakaan-", suffix=".tmp", delete=False,
            ) as tujuan:
                sementara = Path(tujuan.name)
                json.dump(data, tujuan, ensure_ascii=False, indent=2)
            os.replace(sementara, self.path)
        except (OSError, TypeError, ValueError) as exc:
            raise PenyimpananError("Gagal menyimpan data.") from exc
        finally:
            if sementara is not None:
                try:
                    sementara.unlink(missing_ok=True)
                except OSError:
                    pass  # pembersihan tidak menutupi kegagalan utama
```

Method `muat()` bertugas membaca JSON dan memastikan nilai utamanya berupa dictionary. Pemeriksaan makna data, seperti ID anggota yang tidak ditemukan atau dua transaksi aktif untuk satu buku, menjadi tugas layanan. File yang belum ada dibiarkan menghasilkan `FileNotFoundError`; layanan akan menganggapnya sebagai pemakaian pertama. File yang ada tetapi rusak menghasilkan exception yang berbeda.

Saat menyimpan, program menulis file sementara pada folder yang sama, menutupnya, lalu memanggil `os.replace()` untuk mengganti file tujuan. Cara ini mengurangi risiko file lama terpotong ketika proses penulisan belum selesai. Pada Windows, file sementara perlu ditutup sebelum diganti. Bagian `finally` membersihkan file sementara yang tersisa tanpa menutupi penyebab kegagalan utama.

**BEDAKAN FILE BELUM ADA DAN FILE RUSAK**

File yang belum ada adalah keadaan wajar pada pemakaian pertama. File rusak harus dilaporkan agar pengguna dapat memeriksa atau memulihkan datanya. Aplikasi terminal pada bab ini berhenti jika pemuatan gagal; aplikasi tidak melanjutkan dengan data kosong lalu menimpa

simpanan lama.
Penggantian melalui file sementara bukan sistem pencadangan, bukan pengaturan akses serentak, dan tidak menjamin pemulihan dari setiap gangguan listrik atau kerusakan disk. Proyek ini tetap mengikuti batas satu petugas dan satu proses aplikasi.

### Memuat Data tanpa Mengubah Keadaan Sebagian

Layanan tidak langsung mengosongkan koleksi ketika mulai memuat file. Ia membuat keadaan sementara, memeriksa seluruh buku, anggota, dan transaksi, lalu mengganti keadaan lama hanya setelah semua data sah. Jika satu rujukan tidak ditemukan atau satu tanggal salah, proses dibatalkan dan keadaan layanan yang lama tetap ada.

Tanggal jatuh tempo dibaca dari file dan diberikan ke constructor peminjaman. Tanggal kembali yang bukan `null` diterapkan melalui `kembalikan()` agar pemeriksaan tanggal dan status tetap digunakan. Setelah itu, layanan memeriksa ID peminjaman, larangan dua catatan aktif pada satu buku, serta urutan nomor berikutnya. Implementasi lengkap proses ini berada dalam `layanan.py` pada subbab berikut.

## 12.7 Integrasi Antar-Class

### Menghubungkan Aturan melalui Layanan

Layanan menjadi pintu masuk operasi aplikasi. Menu tidak mencari sendiri transaksi aktif lalu mengubah attribute peminjaman. Menu memanggil `pinjamkan()` atau `kembalikan()` dan menangani hasilnya. Dengan begitu, GUI pada Bab 13 dapat menggunakan aturan yang sama.

Buat `perpustakaan/layanan.py` dengan isi lengkap berikut. Koleksi buku, anggota, dan peminjaman memakai dictionary dengan identitas sebagai kunci. List untuk tampilan dibuat melalui method daftar. JSON tetap memakai daftar record karena format penyimpanan tidak harus meniru bentuk koleksi internal secara persis.

perpustakaan/layanan.py

```python
from datetime import date

from .kesalahan import (
    PerpustakaanError, DataTidakValidError, DataTidakDitemukanError,
    BukuSedangDipinjamError, DataRusakError,
)
from .model import Anggota, Buku, BukuCetak, EBook, Peminjaman, teks_wajib
from .penyimpanan import Penyimpanan


class Perpustakaan:
    def __init__(self, penyimpanan: Penyimpanan):
```

```
        self._penyimpanan = penyimpanan
        self._buku = {}
        self._anggota = {}
        self._peminjaman = {}
        self._nomor_pinjam_berikutnya = 1

    def tambah_buku(self, buku):
        if not isinstance(buku, Buku):
            raise DataTidakValidError("Data harus berupa object Buku.")
        if buku.kode in self._buku:
            raise DataTidakValidError("Kode buku sudah terdaftar.")
        self._buku[buku.kode] = buku

    def tambah_anggota(self, anggota):
        if not isinstance(anggota, Anggota):
            raise DataTidakValidError("Data harus berupa object Anggota.")
        if anggota.id_anggota in self._anggota:
            raise DataTidakValidError("ID anggota sudah terdaftar.")
        self._anggota[anggota.id_anggota] = anggota

    def _cari(self, koleksi, identitas, label):
        identitas = teks_wajib(identitas, label)
        if identitas not in koleksi:
            raise DataTidakDitemukanError(f"{label} tidak ditemukan.")
        return koleksi[identitas]

    def tersedia(self, kode_buku):
        buku = self._cari(self._buku, kode_buku, "Buku")
        return not any(
            p.aktif and p.buku.kode == buku.kode
            for p in self._peminjaman.values()
        )

    def pinjamkan(self, id_anggota, kode_buku, tanggal):
        anggota = self._cari(self._anggota, id_anggota, "Anggota")
        buku = self._cari(self._buku, kode_buku, "Buku")
        if not self.tersedia(buku.kode):
            raise BukuSedangDipinjamError("Buku masih dipinjam.")
        identitas = f"P{self._nomor_pinjam_berikutnya:03d}"
        pinjam = Peminjaman(identitas, anggota, buku, tanggal)
        self._peminjaman[identitas] = pinjam
        self._nomor_pinjam_berikutnya += 1
        return pinjam

    def kembalikan(self, id_pinjam, tanggal):
        pinjam = self._cari(self._peminjaman, id_pinjam, "Peminjaman")
```

```
        pinjam.kembalikan(tanggal)

    def daftar_buku(self):
        return list(self._buku.values())

    def daftar_anggota(self):
        return list(self._anggota.values())

    def daftar_peminjaman(self, aktif_saja=False):
        return [p for p in self._peminjaman.values()
                if not aktif_saja or p.aktif]

    def simpan(self):
        data = {
            "versi": 1,
            "nomor_pinjam_berikutnya": self._nomor_pinjam_berikutnya,
            "buku": [b.ke_data() for b in self._buku.values()],
            "anggota": [a.ke_data() for a in self._anggota.values()],
            "peminjaman": [p.ke_data() for p in self._peminjaman.values()],
        }
        self._penyimpanan.simpan(data)

    def muat(self):
        try:
            data = self._penyimpanan.muat()
        except FileNotFoundError:
            return  # pertama kali dipakai: belum ada data di disk

        # Susun keadaan sementara; keadaan lama tetap utuh jika data salah.
        try:
            wajib = {"versi", "nomor_pinjam_berikutnya", "buku",
                     "anggota", "peminjaman"}
            if not isinstance(data, dict) or set(data) != wajib:
                raise DataTidakValidError("Field data tidak lengkap.")
            if type(data["versi"]) is not int or data["versi"] != 1:
                raise DataTidakValidError("Versi data tidak didukung.")
            for nama in ("buku", "anggota", "peminjaman"):
                if not isinstance(data[nama], list):
                    raise DataTidakValidError("Koleksi harus berupa list.")
            baru = Perpustakaan(self._penyimpanan)
            jenis_buku = {"cetak": BukuCetak, "ebook": EBook}
            for d in data["buku"]:
                kelas = jenis_buku[d["jenis"]]
                baru.tambah_buku(kelas(d["kode"], d["judul"]))
            for d in data["anggota"]:
                baru.tambah_anggota(Anggota(d["id_anggota"], d["nama"]))
```

```
            nomor_terbesar = 0
            for d in data["peminjaman"]:
                p = Peminjaman(
                    d["id_pinjam"], baru._anggota[d["id_anggota"]],
                    baru._buku[d["kode_buku"]],
                    date.fromisoformat(d["tanggal_pinjam"]),
                    date.fromisoformat(d["tanggal_jatuh_tempo"]),
                )
                if d["tanggal_kembali"] is not None:
                    p.kembalikan(date.fromisoformat(d["tanggal_kembali"]))
                identitas = p.id_pinjam
                if (not identitas.startswith("P")
                        or not identitas[1:].isdigit()
                        or identitas in baru._peminjaman):
                    raise DataTidakValidError("ID peminjaman tidak sah.")
                if p.aktif and not baru.tersedia(p.buku.kode):
                    raise DataTidakValidError("Buku dipinjam dua kali.")
                baru._peminjaman[identitas] = p
                nomor_terbesar = max(nomor_terbesar, int(identitas[1:]))
            nomor = data["nomor_pinjam_berikutnya"]
            if type(nomor) is not int or nomor <= nomor_terbesar:
                raise DataTidakValidError("Penomoran peminjaman tidak sah.")
            baru._nomor_pinjam_berikutnya = nomor
        except (KeyError, TypeError, ValueError, PerpustakaanError) as exc:
            raise DataRusakError("Struktur data perpustakaan tidak sah.") from
    exc

        self._buku = baru._buku
        self._anggota = baru._anggota
        self._peminjaman = baru._peminjaman
        self._nomor_pinjam_berikutnya = baru._nomor_pinjam_berikutnya
```

Method `pinjamkan()` mencari anggota dan buku yang terdaftar, memeriksa ketersediaan, lalu membentuk catatan. Penambahan ke dictionary dan kenaikan nomor dilakukan setelah constructor berhasil. Karena itu, permintaan yang ditolak tidak menambah riwayat dan tidak menghabiskan nomor transaksi.

Method `tersedia()` menghitung status dari peminjaman aktif. Setelah tanggal kembali terisi, catatan tetap berada di dictionary sebagai riwayat, tetapi tidak lagi menghalangi peminjaman berikutnya. Kita tidak perlu menyelaraskan flag `dipinjam` pada buku dengan flag `aktif` pada transaksi.

Pada `muat()`, pemetaan `jenis_buku` mengubah penanda `"cetak"` atau `"ebook"` menjadi class pembentuk object. Percabangan saat membaca format file berbeda tujuannya dari perhitungan lama pinjam. Setelah object terbentuk, transaksi kembali memakai polymorphism melalui `lama_pinjam()`.

### Membuat Menu Terminal

Buat `main.py` di folder utama proyek. Path data dibangun dari lokasi file program, sehingga tidak bergantung pada folder kerja terminal. Menu menerima input, mengubah teks tanggal menjadi `date`, dan meneruskan pekerjaan ke layanan.

main.py

```
from datetime import date
from pathlib import Path

from perpustakaan import (
    Anggota, BukuCetak, EBook, Perpustakaan,
    PenyimpananJSON, PerpustakaanError,
)


def main():
    path = Path(__file__).resolve().parent / "data" / "perpustakaan.json"
    perpus = Perpustakaan(PenyimpananJSON(path))
    try:
        perpus.muat()
    except PerpustakaanError as exc:
        print("Data tidak dimuat:", exc)
        return 1  # tidak melanjutkan dengan data kosong lalu menimpa file

    while True:
        print("\n1 Buku | 2 Tambah buku | 3 Tambah anggota")
        print("4 Pinjam | 5 Kembali | 6 Riwayat | 0 Simpan dan keluar")
        try:
            pilihan = input("Pilihan: ").strip()
            if pilihan == "1":
                for buku in perpus.daftar_buku():
                    status = "tersedia" if perpus.tersedia(buku.kode) else "
    dipinjam"
                    print(buku, "-", status)
            elif pilihan == "2":
                kode = input("Kode buku: ")
                judul = input("Judul: ")
                jenis = input("Jenis (cetak/ebook): ").strip().lower()
                kelas = {"cetak": BukuCetak, "ebook": EBook}.get(jenis)
                if kelas is None:
                    raise ValueError("Jenis harus cetak atau ebook.")
                perpus.tambah_buku(kelas(kode, judul))
                print("Buku terdaftar.")
            elif pilihan == "3":
                identitas = input("ID anggota: ")
                nama = input("Nama: ")
                perpus.tambah_anggota(Anggota(identitas, nama))
```

```
                print("Anggota terdaftar.")
            elif pilihan == "4":
                anggota = input("ID anggota: ")
                buku = input("Kode buku: ")
                tanggal = date.fromisoformat(input("Tanggal YYYY-MM-DD: ").
    strip())
                pinjam = perpus.pinjamkan(anggota, buku, tanggal)
                print("Berhasil:", pinjam.id_pinjam,
                      "jatuh tempo", pinjam.tanggal_jatuh_tempo)
            elif pilihan == "5":
                identitas = input("ID peminjaman: ")
                tanggal = date.fromisoformat(input("Tanggal YYYY-MM-DD: ").
    strip())
                perpus.kembalikan(identitas, tanggal)
                print("Pengembalian tercatat.")
            elif pilihan == "6":
                for p in perpus.daftar_peminjaman():
                    status = "aktif" if p.aktif else "selesai"
                    print(p.id_pinjam, p.anggota.nama, p.buku.kode, status)
            elif pilihan == "0":
                perpus.simpan()
                print("Data tersimpan.")
                return 0
            else:
                print("Pilihan tidak dikenal.")
        except (PerpustakaanError, ValueError) as exc:
            print("Ditolak:", exc)
        except (EOFError, KeyboardInterrupt):
            print("\nInput dihentikan; perubahan sesi ini belum disimpan.")
            return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

Jalankan `python main.py`, lalu daftarkan B001 sebagai buku cetak berjudul Dasar OOP dan M001 sebagai anggota bernama Ani. Gunakan menu Pinjam dengan tanggal `2026-10-27`. Contoh berikut merupakan cuplikan interaksi; tampilan menu yang berulang tidak dituliskan kembali.

**Cuplikan interaksi terminal**

```
Pilihan: 4
ID anggota: M001
Kode buku: B001
Tanggal YYYY-MM-DD: 2026-10-27
Berhasil: P001 jatuh tempo 2026-11-10
```

```
Pilihan: 1
[B001] Dasar OOP – dipinjam

Pilihan: 0
Data tersimpan.
```

Perubahan selama menu berjalan berada di memori. Pilihan `0` menyimpan data dan baru keluar jika penyimpanan berhasil. Jika penyimpanan ditolak, program menampilkan pesan dan kembali ke menu sehingga pengguna dapat mencoba lagi. Penghentian dengan Ctrl+C atau berakhirnya input tidak menyimpan perubahan sesi secara otomatis. Buka kembali program untuk memeriksa bahwa data yang berhasil disimpan tetap terbaca.

## Memeriksa Alur Terpadu secara Berulang

Menu berguna untuk mencoba aplikasi sebagai pengguna, tetapi pengulangan input manual mudah keliru. Buat `demo_integrasi.py` berikut untuk memeriksa satu alur tetap: daftar buku dan anggota, pinjam, tolak peminjaman kedua, kembalikan, pinjam e-book, simpan, lalu muat ulang.

Contoh memakai `TemporaryDirectory` agar setiap eksekusi dimulai dari keadaan baru. File demo berada di folder sementara dan dibersihkan saat blok selesai; file data milik `main.py` tidak digunakan.

**demo_integrasi.py**

```python
from datetime import date
from pathlib import Path
from tempfile import TemporaryDirectory

from perpustakaan import Anggota, BukuCetak, EBook, Perpustakaan,
    PenyimpananJSON
from perpustakaan.kesalahan import BukuSedangDipinjamError


with TemporaryDirectory() as folder:
    simpanan = PenyimpananJSON(Path(folder) / "perpustakaan.json")
    perpus = Perpustakaan(simpanan)
    perpus.muat()
    perpus.tambah_buku(BukuCetak("B001", "Dasar OOP"))
    perpus.tambah_buku(EBook("E001", "Dasar Python"))
    perpus.tambah_anggota(Anggota("M001", "Ani"))

    pinjam = perpus.pinjamkan("M001", "B001", date(2026, 10, 27))
    print("Pinjam:", pinjam.id_pinjam, "jatuh tempo", pinjam.
    tanggal_jatuh_tempo)
```

```
    print("B001 tersedia:", perpus.tersedia("B001"))
    try:
        perpus.pinjamkan("M001", "B001", date(2026, 10, 28))
    except BukuSedangDipinjamError as exc:
        print("Ditolak:", exc)
    print("Jumlah riwayat:", len(perpus.daftar_peminjaman()))

    perpus.kembalikan("P001", date(2026, 11, 2))
    print("Sesudah kembali:", perpus.tersedia("B001"))
    digital = perpus.pinjamkan("M001", "E001", date(2026, 10, 27))
    print("E-book jatuh tempo:", digital.tanggal_jatuh_tempo)
    perpus.simpan()

    dimuat = Perpustakaan(simpanan)
    dimuat.muat()
    print("Dimuat:", len(dimuat.daftar_buku()), "buku,",
          len(dimuat.daftar_peminjaman()), "riwayat")
    print("Status:", [(p.id_pinjam, p.aktif) for p in dimuat.daftar_peminjaman()
    ])
```

**Keluaran**

```
Pinjam: P001 jatuh tempo 2026-11-10
B001 tersedia: False
Ditolak: Buku masih dipinjam.
Jumlah riwayat: 1
Sesudah kembali: True
E-book jatuh tempo: 2026-11-03
Dimuat: 2 buku, 2 riwayat
Status: [('P001', False), ('P002', True)]
```

Jumlah riwayat tetap satu setelah percobaan peminjaman ganda, sehingga penolakan tidak membuat catatan tambahan. Setelah P001 dikembalikan, B001 tersedia kembali. Transaksi e-book berikutnya mendapat ID P002 dan jatuh tempo tujuh hari setelah tanggal pinjam. Pemuatan ulang mempertahankan kedua catatan beserta statusnya: P001 selesai, P002 aktif.

## Memeriksa Kontrak dengan Assertion

Keluaran membantu pembaca menelusuri alur, tetapi assertion dapat memeriksa syarat secara langsung. Simpan pemeriksaan berikut sebagai `cek_kontrak.py` di folder utama. Contoh ini menilai relasi setelah simpan–muat dan memastikan pengembalian yang ditolak tidak mengubah transaksi. Assertion di sini dipakai untuk pengujian; validasi masukan aplikasi tetap menggunakan exception, sebab assertion dapat dinonaktifkan oleh opsi interpreter.

cek_kontrak.py

```python
from datetime import date
from pathlib import Path
from tempfile import TemporaryDirectory

from perpustakaan import Anggota, BukuCetak, Perpustakaan, PenyimpananJSON
from perpustakaan.kesalahan import DataTidakValidError


with TemporaryDirectory() as folder:
    simpanan = PenyimpananJSON(Path(folder) / "data.json")
    perpus = Perpustakaan(simpanan)
    perpus.tambah_buku(BukuCetak("B001", "Dasar OOP"))
    perpus.tambah_anggota(Anggota("M001", "Ani"))
    pinjam = perpus.pinjamkan("M001", "B001", date(2026, 10, 27))

    try:
        perpus.kembalikan("P001", date(2026, 10, 26))
    except DataTidakValidError:
        pass
    else:
        raise AssertionError("Tanggal kembali seharusnya ditolak.")
    assert pinjam.aktif and pinjam.tanggal_kembali is None
    assert len(perpus.daftar_peminjaman()) == 1

    perpus.simpan()
    dimuat = Perpustakaan(simpanan)
    dimuat.muat()
    riwayat = dimuat.daftar_peminjaman()
    assert riwayat[0].buku is dimuat.daftar_buku()[0]
    assert riwayat[0].anggota is dimuat.daftar_anggota()[0]
    assert not dimuat.tersedia("B001")

    dimuat.kembalikan("P001", date(2026, 11, 2))
    berikut = dimuat.pinjamkan("M001", "B001", date(2026, 11, 3))
    assert berikut.id_pinjam == "P002"

print("Kontrak pengembalian, relasi, dan nomor transaksi terpenuhi.")
```

Keluaran

```
Kontrak pengembalian, relasi, dan nomor transaksi terpenuhi.
```

Pemeriksaan identitas rujukan memakai `is`: object buku pada transaksi hasil muat harus sama dengan object yang ada di daftar buku layanan hasil muat. Kesamaan judul saja belum membuktikan hubungan tersebut. Pemeriksaan ID P002 memastikan penghitung ikut disimpan, bukan kembali ke satu saat aplikasi dibuka ulang.

**Tabel 12.3.** Skenario tambahan dan hasil yang harus diperiksa.

| Percobaan | Hasil yang diharapkan |
|---|---|
| Kode buku atau ID anggota ganda | Ditolak; object yang sudah terdaftar tetap sama. |
| Anggota atau buku tidak terdaftar | Ditolak; riwayat dan nomor berikutnya tidak berubah. |
| Mengembalikan sebelum tanggal pinjam | Ditolak; transaksi tetap aktif. |
| Mengembalikan transaksi selesai | Ditolak; tanggal kembali sebelumnya tetap tersimpan. |
| File JSON berisi teks yang tidak sah | Pemuatan gagal dengan pesan; simpanan tidak ditimpa data kosong. |
| JSON sah tetapi rujukan ID hilang | Pemuatan gagal; keadaan lama di memori tetap utuh. |
| Dua catatan aktif untuk satu buku | Data ditolak saat dimuat. |
| Gagal menulis file sementara | Kesalahan penyimpanan dilaporkan; file tujuan lama belum diganti. |

**Urutan Pemeriksaan sebelum Membuat GUI**

1. Lengkapi file package, `main.py`, dan file demo pada folder yang benar.
2. Jalankan demo konsep dan cocokkan keluarannya dengan contoh pada bab ini.
3. Jalankan `python demo_integrasi.py` serta `python cek_kontrak.py`.
4. Coba menu terminal dari data baru, simpan dengan pilihan 0, lalu buka kembali.
5. Periksa skenario penolakan. Catat keadaan sebelum dan sesudah operasi yang gagal.
6. Gunakan salinan file data untuk mencoba kasus kerusakan JSON atau rujukan ID.

Setelah langkah tersebut terpenuhi, model, layanan, dan penyimpanan sudah dapat dipakai oleh GUI. Pada Bab 13, tombol dan formulir akan menggantikan input menu sambil tetap memanggil operasi layanan yang sama.

## 12.8 Rangkuman

- Implementasi menerjemahkan tanggung jawab dan kontrak rancangan menjadi perilaku yang dapat dijalankan dan diperiksa.
- Property menjaga perubahan data: identitas hanya dibaca, sedangkan judul dan nama diperbarui melalui setter yang melakukan validasi.
- Inheritance dipakai untuk jenis buku; association menghubungkan transaksi dengan anggota serta buku; layanan memiliki kumpulan catatan peminjamannya.
- Polymorphism memungkinkan perhitungan jatuh tempo memanggil operasi yang sama untuk jenis buku yang berbeda.
- Module memisahkan model, exception, penyimpanan, layanan, dan tampilan. Arah import mengikuti tanggung jawab tersebut.
- JSON menyimpan data sederhana serta ID rujukan. Hubungan object dibangun kembali setelah buku dan anggota dimuat.
- Pemuatan divalidasi pada keadaan sementara sebelum mengganti keadaan lama. Penyimpanan menulis file sementara sebelum mengganti file tujuan.
- Aturan lintas-object dijalankan melalui layanan. Kegagalan transaksi tidak boleh meninggalkan perubahan sebagian.
- Pemeriksaan integrasi perlu mencakup hasil normal, penolakan, kelanjutan ID, dan hubungan object setelah simpan–muat.

## 12.9 Kuis Teoritis

### Pilihan Ganda

Pilihlah satu jawaban yang paling tepat berdasarkan implementasi dalam bab ini.

1. Kode buku disediakan sebagai property hanya-baca karena ...

   a. judul buku tidak boleh diubah
   b. kode menjadi identitas tetap dan kunci koleksi
   c. semua attribute Python harus tidak dapat diubah
   d. JSON tidak dapat menyimpan kode berupa teks

2. Jika setter judul menerima teks berisi spasi saja, hasilnya adalah ...

   a. judul lama dihapus sebelum kesalahan dilaporkan
   b. judul diganti dengan satu spasi
   c. exception dilaporkan dan judul lama tetap ada
   d. object buku otomatis dihapus

3. Pemanggilan `buku.lama_pinjam()` pada constructor peminjaman merupakan contoh ...

   a. polymorphism
   b. pembacaan file
   c. duplikasi seluruh object
   d. penghapusan inheritance

4. Pemeriksaan bahwa B001 belum mempunyai peminjaman aktif lain dilakukan oleh ...

   a. menu saja
   b. `Anggota.info()`
   c. encoder JSON secara otomatis
   d. layanan `Perpustakaan`

5. Setelah `daftar_buku()` menghasilkan list baru, pemanggilan `clear()` pada list itu akan ...

   a. menghapus semua buku pada layanan
   b. mengosongkan list hasil tanpa menghapus dictionary layanan
   c. menghapus file JSON secara otomatis
   d. mengubah semua kode buku

6. Cara menyimpan hubungan peminjaman dengan buku dalam JSON adalah ...

   a. menyalin method buku sebagai teks kode
   b. menyimpan alamat memori Python
   c. menyimpan kode buku lalu menghubungkannya kembali saat dimuat
   d. menyimpan hanya judul dan mengabaikan identitas

7. Alasan memakai keadaan sementara saat pemuatan adalah ...

   a. mencegah kegagalan di tengah pemuatan meninggalkan data lama yang sudah berubah sebagian
   b. membuat setiap kesalahan selalu diabaikan
   c. menghindari kebutuhan validasi
   d. mengganti semua ID dengan `id()`

8. Saat buku yang masih dipinjam diminta lagi, nomor berikutnya ...

   a. selalu bertambah dua
   b. selalu kembali ke satu
   c. berubah menjadi kode anggota
   d. tetap karena catatan baru belum berhasil dibuat

9. Jika JSON ada tetapi tidak dapat dibaca, menu terminal akan ...

   a. menghapusnya tanpa memberi pesan
   b. melaporkan kegagalan dan berhenti sebelum mengubah simpanan
   c. otomatis menganggap semua buku tersedia lalu menyimpan
   d. memuat hanya baris pertama tanpa pemeriksaan

10. Penyimpanan tanggal jatuh tempo di dalam transaksi bertujuan untuk ...

    a. selalu mengikuti perubahan kebijakan terbaru pada riwayat lama
    b. mengganti tanggal pinjam setiap kali file dibuka
    c. mempertahankan kesepakatan jatuh tempo ketika transaksi dibuat
    d. menghilangkan kebutuhan tanggal kembali

## Uraian

1. Jelaskan mengapa validasi tanggal diletakkan pada `Peminjaman`, sedangkan pemeriksaan buku tersedia diletakkan pada `Perpustakaan`.
2. Telusuri apa yang berubah dan tidak berubah ketika peminjaman kedua untuk buku yang sama ditolak.
3. Jelaskan perbedaan inheritance buku dengan association antara peminjaman dan anggota. Gunakan hasil `demo_relasi.py` untuk mendukung jawaban.
4. Mengapa pemuatan JSON perlu membentuk buku dan anggota sebelum peminjaman? Apa yang harus dilakukan jika ID rujukannya tidak ditemukan?
5. Jika ditambahkan `BukuReferensi` dengan masa pinjam tiga hari, bagian mana yang harus ditambah dan bagian mana yang tetap dapat digunakan? Sertakan pemetaan JSON serta pilihan input dalam jawaban.

KUNCI PILIHAN GANDA

1 b; 2 c; 3 a; 4 d; 5 b; 6 c; 7 a; 8 d; 9 b; 10 c.

## 12.10 Praktik Mandiri

Latihan berikut melanjutkan proyek, bukan mengganti seluruh rancangan. Gunakan salinan proyek dan data khusus latihan. Bentuk latihan dengan syarat serta hasil yang dapat diperiksa membantu pembaca menilai perilaku programnya sendiri, sebagaimana pendekatan latihan pada [Edcorner Learning, tt].

### Praktik 1: Menambah Jenis Buku

Tambahkan class `BukuReferensi` dengan penanda jenis `"referensi"` dan lama pinjam tiga hari. Perbarui ekspor package, pemetaan jenis ketika memuat, serta pilihan input menu. Gunakan seluruh validasi kode dan judul yang sudah tersedia pada induk.

- Hasil wajib: peminjaman pada 27 Oktober 2026 jatuh tempo pada 30 Oktober 2026.
- Simpan, muat ulang, lalu pastikan object kembali menjadi `BukuReferensi`, bukan `BukuCetak`.
- Buktikan bahwa rumus pembuatan jatuh tempo pada `Peminjaman` tidak perlu ditambah percabangan jenis.

### Praktik 2: Menampilkan Peminjaman Terlambat

Tambahkan operasi layanan yang menerima tanggal pemeriksaan dan menghasilkan peminjaman yang masih aktif dengan tanggal jatuh tempo *sebelum* tanggal pemeriksaan. Jangan menyimpan flag `terlambat` tambahan karena status ini bergantung pada tanggal pemeriksaan.

- Hasil wajib: transaksi yang jatuh tempo 10 November belum terlambat pada 10 November, tetapi terlambat pada 11 November jika belum dikembalikan.
- Transaksi selesai tidak ditampilkan, walaupun tanggal pemeriksaan melewati jatuh temponya.
- Tambahkan menu laporan dan uji masukan tanggal yang tidak valid tanpa mengubah riwayat.

### Praktik 3: Memeriksa Kegagalan Simpan–Muat

Buat pemeriksaan terpisah menggunakan folder sementara. Siapkan data sah, lalu uji salinan data dengan ID anggota hilang, dua transaksi aktif pada satu buku, nomor berikutnya yang sudah terpakai, dan JSON yang tidak sah. Periksa exception serta keadaan layanan sebelum dan sesudah setiap percobaan.

- Hasil wajib: seluruh data salah ditolak, keadaan lama tetap ada, dan file asli tidak ditimpa oleh data kosong.
- Tambahkan kasus simpan–muat yang berhasil untuk membuktikan bahwa pemeriksaan tidak hanya menolak semua file.
- Catat langkah, hasil yang diharapkan, hasil aktual, dan penyebab jika terjadi perbedaan.

**Tabel 12.4.** Rubrik praktik implementasi.

| Aspek | Bobot | Bukti |
|---|---|---|
| Aturan dan hasil operasi | 35% | Skenario normal serta penolakan menghasilkan keadaan yang tepat. |
| Pemakaian konsep OOP | 25% | Tanggung jawab, property, relasi, dan polymorphism sesuai kebutuhan. |
| Integritas simpan–muat | 25% | Identitas, relasi, jenis object, tanggal, dan nomor transaksi bertahan. |
| Keterbacaan dan pemeriksaan | 15% | File tersusun jelas, hasil bisa diulang, dan laporan menjelaskan perilaku yang diperiksa. |

Bab 13

# Implementasi GUI

*Memberi tampilan pada aplikasi yang sudah dibangun.*

13

DI BAB INI

- Implementasi GUI
- Komponen GUI
- Event handling
- Integrasi dengan class OOP

## 13.1 Implementasi GUI

## 13.2 Komponen GUI

## 13.3 Event Handling

## 13.4 Integrasi GUI dengan Class OOP

Bab 14

# Penutup

*Merangkum perjalanan dari class pertama hingga aplikasi utuh.*

14

DI BAB INI

- Rangkuman materi
- Langkah selanjutnya

## 14.1 Rangkuman Materi

## 14.2 Langkah Selanjutnya

# Daftar Pustaka

*Sumber yang dirujuk di buku ini.*

[Anonim, tt] Anonim (t.t.). Object-oriented programming with python. Penulis, penerbit, dan tahun terbit tidak tercantum pada salinan digital yang digunakan.

[Edcorner Learning, tt] Edcorner Learning (t.t.). *Python Object Oriented Programming Exercises: Become a Pro Developer*. Penerbit tidak tercantum. Tahun terbit tidak tercantum pada salinan digital yang digunakan.

[Lutz, 2025] Lutz, M. (2025). *Learning Python: Powerful Object-Oriented Programming*. O'Reilly Media, Sebastopol, CA, 6 edition.

[North, 2025] North, M. (2025). *Object-Oriented Programming in Python: Design and Build Programs Using Classes, Objects, and the Four Pillars of OOP*, volume 1 of *Think, Code, Create – Advanced Python Programming Series*. Northward House Publishing, Brisbane, Queensland, Australia.

[Wilson, 2024] Wilson, G. (2024). *Software Design by Example: A Tool-Based Introduction with Python*. CRC Press.

kamu.py

```python
class Kamu(Mahasiswa):
  def __init__(self):
    super().__init__()
    self.semangat = 100
```




# Semua adalah object. Termasuk kamu.

*Pernah menulis kode yang makin lama makin mirip mi kusut? Buku ini mengajakmu merapikannya dengan cara berpikir berorientasi objek: dari class pertamamu, encapsulation, inheritance, dan polymorphism, sampai membangun aplikasi lengkap dengan GUI. Setiap bab punya kode yang bisa langsung dijalankan, ilustrasi, kuis, dan praktik mandiri. Siapkan kopi, buka editor, dan selamat ngoding!*

belajar_oop 80%

```
>>> isinstance(kamu, object)
True
```

PEMROGRAMAN BERORIENTASI OBJEK DENGAN PYTHON • UNIVERSITAS NEGERI SURABAYA • 2026